

Kubernetes: Tài liệu tham khảo kỹ thuật CKA hoàn chỉnh

Bản tiếng Việt · CKA Kubernetes 1.35 · cập nhật 17/09/2026. Đọc các thay đổi cho phiên bản thi.

Bao gồm CKA 2026, Tương tác container, Kustomize và Hoạt động cụm

Lưu ý của tác giả: Hướng dẫn này là tài liệu tham khảo kỹ thuật toàn diện dành cho các kỹ sư, Những người thực hành DevOps, SRE và ứng viên CKA. Mỗi lệnh bao gồm ngữ cảnh, mục đích, cờ hiệu, những cạm bẫy phổ biến và ví dụ thực tế. Các chương được sắp xếp theo tiến trình học tập hợp lý từ kiến trúc cụm thông qua các hoạt động container, gỡ lỗi, cấu hình nâng cao và chứng nhận.

Mục lục

Phần 1 - Kiến trúc và thiết lập cụm

- Chương 1 - Kiến trúc Kubernetes: Tổng quan hoàn chỉnh
- Chương 2 - Cài đặt cụm từ đầu với kubeadm
- Chương 3 - Kiến trúc Control Plane có tính sẵn sàng cao
- Chương 4 - Giao diện mở rộng: CNI, CSI và CRI

Phần 2 - Workload & Cấu hình

- Chương 5 - Tìm hiểu Pods và Container
- Chương 6 - ConfigMaps và Secrets: Tài liệu tham khảo đầy đủ
- Chương 7 - StorageClasses và Cung cấp volume động
- Chương 8 - Tự động thay đổi quy mô workload với HPA và VPA
- Chương 9 - Cơ chế tự phục hồi: probe và PodDisruptionBudget
- Chương 10 - Operator và định nghĩa tài nguyên tùy chỉnh
- Chương 11 - Gateway API: Quản lý lưu lượng Ingress hiện đại

Phần 3 - Mạng & Bảo mật

- Chương 12 - Kubernetes Services: ClusterIP, NodePort, LoadBalancer và các lựa chọn khác
- Chương 13 - Ingress: Định tuyến HTTP và chấm dứt TLS
- Chương 14 - NetworkPolicy: Kiểm soát lưu lượng và phân đoạn vi mô
- Chương 15 - RBAC: Kiểm soát truy cập dựa trên Role
- Chương 16 - Bảo mật Kubernetes: Xác thực, Bảo mật Pod và TLS

Phần 4 - Tương tác container

- 17. Chương 17 - Thực thi các lệnh bên trong container
- 18. Chương 18 - Xem và truyền trực tuyến Nhật ký container
- 19. Chương 19 - Sao chép tệp đến và từ container
- 20. Chương 20 - Kiểm tra và mô tả container
- 21. Chương 21 - Chuyển tiếp cổng tới container
- 22. Chương 22 - Giám sát tài nguyên bên trong container
- 23. Chương 23 - Loại bỏ và sửa đổi các container trong Pods

Phần 5 - Gỡ lỗi & Khả năng quan sát

- 24. Chương 24 - Tạo và xóa container thử nghiệm tại chỗ
- 25. Chương 25 - Kiểm tra mạng từ các container tạm thời
- 26. Chương 26 - CoreDNS: Kiểm tra, chẩn đoán và xác minh phân giải DNS
- 27. Chương 27 - Gỡ lỗi nâng cao với các container tạm thời
- 28. Chương 28 - Các lệnh nhận biết namespace và quản lý bối cảnh

Phần 6 - Cấu hình nâng cao & CKA

- 29. Chương 29 - kubectl Kustomize: Quản lý cấu hình tại Scal
- 30. Chương 30 - Chứng nhận CKA 2026: Nghiên cứu hoàn chỉnh và Tham khảo lệnh

Phụ lục

- Phụ lục A - Bảng tham khảo nhanh
- Phụ lục B - Tham chiếu tốc độ lệnh CKA 2026
- Phụ lục C - Tham chiếu mẫu Kustomize
- Phụ lục D - Tham khảo nhanh về lấp đầy khoảng trống CKA

Chương 1 - Kiến trúc Kubernetes: Tổng quan hoàn chỉnh

1.1 Kubernetes là gì?

Kubernetes (thường được viết tắt là K8s - có 8 chữ cái giữa K và s) là một hệ thống điều phối container mã nguồn mở do Google thiết kế ban đầu và có nguồn mở vào tháng 6 năm 2014. Hệ thống này được tặng cho Tổ chức Điện toán Đám mây (CNCF) khi CNCF được thành lập vào năm 2016. Ngày nay, đây là tiêu chuẩn ngành để triển khai, mở rộng quy mô và quản lý các container workloads.

Về cốt lõi, Kubernetes giải quyết ba vấn đề cơ bản phát sinh khi chạy containers trên quy mô lớn:

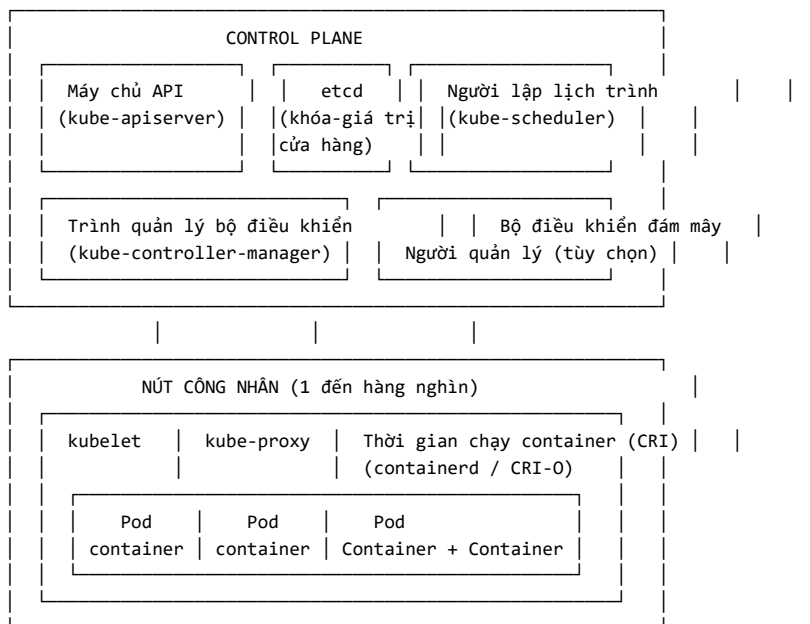
Vị trí - container nên chạy ở đâu? Kubernetes có một bộ lập lịch phức tạp xem xét CPU, bộ nhớ, mối quan hệ node, taints, tolerations và các chính sách tùy chỉnh để quyết định node nào sẽ chạy container nào.

Quản lý vòng đời - Điều gì xảy ra nếu container gặp sự cố? Kubernetes liên tục so sánh *trạng thái mong muốn* mà bạn khai báo với *trạng thái thực tế* của cụm. Nếu container chết, nó sẽ khởi động lại. Nếu node ngoại tuyến, nó sẽ lên lịch lại Pods trên đó.

Kết nối mạng và khám phá - containers tìm thấy nhau bằng cách nào? Kubernetes cung cấp tên DNS ổn định và IP ảo cho các nhóm containers, loại bỏ tính chất tạm thời của các địa chỉ IP container riêng lẻ.

1.2 Mô hình cụm

Cụm Kubernetes bao gồm một hoặc nhiều **nodes** - máy vật lý hoặc máy ảo. Mỗi cụm có hai lớp logic:



1.3 Chi tiết các thành phần control plane

kube-apiserver

Máy chủ API là trung tâm của Kubernetes. Mọi hành động trong cụm đều chảy qua nó - các lệnh `kubectl`, đồng hồ điều khiển, nhịp tim kubelet và các quyết định của bộ lập lịch đều nói chuyện với máy chủ API.

Máy chủ API:

- Xác thực và duy trì các định nghĩa tài nguyên vào etcd
- Hiển thị Kubernetes REST API trên cổng 6443 (HTTPS)
- Xác thực requests (chứng chỉ, mã thông báo tài khoản dịch vụ, OIDC, webhook)
- Cho phép requests thông qua RBAC, ABAC hoặc webhook
- Chạy bộ điều khiển tiếp nhận có thể thay đổi hoặc xác thực tài nguyên trước khi duy trì

Cách kubectl hoạt động: Khi bạn chạy `kubectl apply -f deployment.yaml`, `kubectl` sẽ đọc `~/.kube/config` của bạn, xác thực với máy chủ API bằng chứng chỉ nhúng, tuần tự hóa YAML của bạn thành JSON và gửi PUT hoặc POST đến điểm cuối REST thích hợp (e.g., `/apis/apps/v1/namespaces/default/deployments`).

```
# Xem các cuộc gọi HTTP thực tế mà kubectl thực hiện
kubectl apply -f deployment.yaml -v=8

# Truy vấn trực tiếp máy chủ API REST API
kubectl get --raw /api/v1/namespaces/default/pods | python3 -m json.tool
```

etcd

etcd là kho lưu trữ khóa-giá trị phân tán, là bộ lưu trữ liên tục duy nhất trong Kubernetes. Mọi đối tượng tài nguyên (Pod, Deployment, ConfigMap, Service, Secret, v.v.) được tuần tự hóa vào Bộ đệm giao thức và được lưu trữ trong etcd.

Thuộc tính chính:

- Sử dụng **thuật toán đồng thuận Raft** để đảm bảo tính nhất quán được phân phối
- Lưu trữ dữ liệu tại các đường dẫn như /registry/pods/default/mypod
- Nên chạy với **3 hoặc 5 thành viên** trong quá trình sản xuất để có khả năng chịu lỗi tối thiểu
- Yêu cầu ổ đĩa nhanh - độ trễ etcd ảnh hưởng trực tiếp đến khả năng phản hồi của cụm

```
# Kiểm tra trực tiếp dữ liệu etcd (trên control-plane node)
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/pods/default/ --prefix --keys-only
```

```
# Kiểm tra tình trạng cụm etcd
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health --cluster
```

kube-scheduler

Bộ lập lịch theo dõi Pods mới được tạo chưa có nodeName nào được chỉ định và chọn node tốt nhất để chúng chạy. Quá trình tuyển chọn có hai giai đoạn:

Lọc - Loại bỏ nodes không thể đáp ứng yêu cầu của Pod:

- Không đủ CPU hoặc bộ nhớ
- Nút có taint mà Pod không chấp nhận
- Quy tắc NodeAffinity loại trừ node
- HostPort được yêu cầu của Pod đã được sử dụng

Chấm điểm - Xếp hạng nodes còn lại theo nhiều tiêu chí (lây lan, cân bằng tài nguyên, sở thích về mối quan hệ, v.v.) và chọn node có điểm cao nhất.

Sau đó, bộ lập lịch sẽ ghi nodeName đã chọn trở lại thông số Pod thông qua máy chủ API và kubelet trên node đó sẽ chọn nó.

```
# Xem người lập lịch đã chọn node nào và tại sao
kubectl get pod mypod -o wide
kubectl describe pod mypod | grep -A 5 "Events"

# Ghi đè bộ lập lịch cho một Pod cụ thể (lập lịch thủ công)
kubectl patch pod mypod -p '{"spec":{"nodeName":"worker-2"}}'
```

kube-controller-manager

trình quản lý bộ điều khiển là một tập nhĩ phân duy nhất chạy nhiều vòng điều khiển độc lập ("bộ điều khiển"), mỗi vòng chịu trách nhiệm về một loại tài nguyên:

Bộ điều khiển	Trách nhiệm
Bộ điều khiển Deployment	Đảm bảo tồn tại đúng số lượng bản sửa đổi ReplicaSet

Bộ điều khiển	Trách nhiệm
Bộ điều khiển ReplicaSet	Đảm bảo đúng số lượng Pods đang chạy
Bộ điều khiển nút	Phát hiện lỗi node; đuổi Pods sau khi hết thời gian chờ
Bộ điều khiển Job	Theo dõi các lần hoàn thành Job Pod; tạo Pods mới nếu cần
Bộ điều khiển Endpoints	Điền các đối tượng Endpoint từ bộ chọn Service + Pod
Bộ điều khiển ServiceAccount	Tạo ServiceAccounts mặc định trong namespaces mới
Bộ điều khiển namespace	Xử lý việc xóa và dọn dẹp namespace
Bộ điều khiển CronJob	Kích hoạt Jobs theo lịch trình cron

Mỗi bộ điều khiển chạy một vòng điều chỉnh chặt chẽ:

1. Xem máy chủ API để biết các thay đổi đối với loại tài nguyên của nó
2. So sánh trạng thái thực tế và trạng thái mong muốn
3. Thực hiện các hành động để hội tụ (tài nguyên create/delete/update)

```
# Xem bộ điều khiển đang chạy
kubectl -n kube-system get pod | grep kube-controller-manager

# Xem nhật ký của trình quản lý bộ điều khiển để biết sự đối chiếu cụ thể
kubectl -n kube-system logs kube-controller-manager-controlplane \
  --since=5m | grep -i "deployment controller"
```

1.4 Chi tiết các thành phần worker node

kubelet

kubelet là tác nhân node chính. Nó chạy trên mọi node (bao gồm control-plane node trong các cụm node đơn) và chịu trách nhiệm:

- Đang xem máy chủ API để tìm Pods được gán cho node của nó
- Gọi môi trường chạy container (thông qua CRI) tới create/delete containers
- Volume lắp đặt (ConfigMaps, Secrets, PVCs, EmptyDir)
- Đầu dò Running liveness/readiness/startup
- Báo cáo trạng thái node và Pod trở lại máy chủ API
- Quản lý Pods tĩnh từ `/etc/kubernetes/manifests/`

kubelet là thành phần Kubernetes duy nhất KHÔNG chạy bên trong Kubernetes - nó chạy trực tiếp dưới dạng dịch vụ systemd trên máy chủ.

```
# Trạng thái kubelet và đăng nhập vào node
sudo systemctl status kubelet
sudo journalctl -u kubelet -f
sudo journalctl -u kubelet --since="5 minutes ago" | grep -i error

# Cấu hình kubelet
cat /var/lib/kubelet/config.yaml
cat /etc/kubernetes/kubelet.conf # kubeconfig cho kubelet

# Pods nào mà node của kubelet này quản lý
ls /var/lib/kubelet/pods/
```

kube-proxy

kube-proxy chạy trên mọi node và triển khai tính năng trừu tượng hóa Service bằng cách duy trì các quy tắc mạng (thường là các quy tắc iptables hoặc IPVS) định tuyến lưu lượng truy cập dành cho Service ClusterIP đến một trong các IP Pod dự phòng.

Khi Service được tạo:

1. Bộ điều khiển Endpoints điền vào đối tượng Endpoints với các IP phù hợp với Pods
2. kube-proxy theo dõi đối tượng Endpoints
3. kube-proxy lập trình iptables/IPVS quy tắc cho lưu lượng truy cập DNAT từ ClusterIP:port đến PodIP:port

```
# Xem chế độ và trạng thái kube-proxy
kubectl -n kube-system get pod | grep kube-proxy
kubectl -n kube-system logs kube-proxy-xxxxx

# Xem các quy tắc iptables mà kube-proxy tạo
sudo iptables -t nat -L KUBE-SERVICES -n | grep -i my-service

# Sơ đồ cấu hình kube-proxy
kubectl -n kube-system get configmap kube-proxy -o yaml
```

Môi trường chạy container (CRI)

****Thời gian chạy container** chịu trách nhiệm thực sự kéo images và chạy containers. Kubernetes hiện đại sử dụng **Giao diện môi trường chạy container (CRI)** để tách riêng khỏi mọi thời gian chạy cụ thể. Thời gian chạy phổ biến:

Thời gian chạy	Ghi chú
containerd	Mặc định trong hầu hết các bản phân phối Kubernetes; tối thiểu, nhanh chóng
CRI-O	Thời gian chạy CRI được xây dựng có mục đích dành cho Kubernetes; được sử dụng bởi OpenShift
Động cơ Docker	Được hỗ trợ qua miếng ghép cri-dockerd (Dockershim đã bị loại bỏ trong K8s 1.24)

```
# Tương tác trực tiếp với containerd (thay thế docker CLI)
sudo crictl ps           # Danh sách đang chạy containers
sudo crictl images       # Danh sách kéo images
sudo crictl logs <container-id> # Nhật ký container
sudo crictl inspect <container-id> # Chi tiết container
sudo crictl pods         # Danh sách sandbox pod

# containerd trực tiếp
sudo ctr images list     # Tất cả images trong containerd
sudo ctr containers list # Tất cả containerd containers
```

1.5 Mô hình đối tượng Kubernetes

Mọi thứ trong Kubernetes đều là **đối tượng tài nguyên** có cùng bốn trường cấp cao nhất:

```
apiVersion: apps/v1      # API group/version
kind: Deployment          # Loại tài nguyên
metadata:                 # Danh tính và nhãn
  name: my-app
  namespace: production
  labels:
    app: my-app
    version: "1.0"
  annotations:
    description: "Main application deployment"
spec:                     # Trạng thái mong muốn (bạn viết cái này)
```

```

replicas: 3
...
# trạng thái: # Trạng thái thực tế (Kubernetes viết phần này, chỉ đọc cho người dùng)
# có sẵnBản sao: 3
# điều kiện: [...]

```

Sự phân chia spec/status là cơ bản: bạn khai báo ý định trong `spec`, Kubernetes điều chỉnh theo ý định đó và báo cáo thực tế trong `status`.

Nhóm và phiên bản API

Tài nguyên được tổ chức thành các nhóm API. Cấu trúc đường dẫn là:

```

/api/v1                                # Nhóm cốt lõi (Pods, Services, ConfigMaps, Namespace)
/apis/<group>/<version>                # Nhóm được đặt tên
/apis/apps/v1                          # Deployments, StatefulSets, DaemonSets, ReplicaSets
/apis/batch/v1                         # Jobs, CronJobs
/apis/networking.k8s.io/v1            # Ingress, NetworkPolicy
/apis/storage.k8s.io/v1               # StorageClass, PersistentVolume
/apis/rbac.authorization.k8s.io/v1    # Role, ClusterRole, RoleBinding
/apis/gateway.networking.k8s.io/v1    # Gateway, HTTPRoute (Gateway API)
/apis/apiextensions.k8s.io/v1         # Định nghĩa tài nguyên tùy chỉnh

# Khám phá tất cả các loại tài nguyên trong cụm
kubectl api-resources
kubectl api-resources --namespaced=true # Chỉ các tài nguyên trong phạm vi namespace
kubectl api-resources --namespaced=false # Chỉ các tài nguyên trong phạm vi cụm

# Khám phá tất cả các phiên bản API
kubectl api-versions

# Nhận lược đồ cho bất kỳ tài nguyên nào
kubectl explain pod
kubectl explain pod.spec.containers
kubectl explain deployment.spec.template.spec.containers.resources

```

1.6 Namespace

Namespace cung cấp một phân vùng hợp lý trong một cụm. Các tài nguyên trong các namespace khác nhau được phân tách với nhau theo tên (bạn có thể có hai Deployments đều được đặt tên là `web` trong các namespace khác nhau), và RBAC cũng như ResourceQuotas có thể nằm trong phạm vi mỗi namespace.

```

# namespaces tích hợp
kubectl get namespaces
# NAME STATUS AGE
# mặc định Hoạt động 30d ← Tài nguyên của bạn sẽ đi đâu nếu bạn không chỉ định
# kube-system Active 30d ← Các thành phần hệ thống Kubernetes
# kube-public Active 30d ← ConfigMap có thể đọc công khai (thông tin cụm)
# kube-node-lease Active 30d ← Nút nhịp tim Cho thuê đối tượng

# Tạo namespace
kubectl create namespace staging

# Làm việc trong namespace
kubectl get pods -n staging
kubectl apply -f app.yaml -n staging

# Đặt bối cảnh hiện tại của bạn thành namespace vĩnh viễn
kubectl config set-context --current --namespace=staging

# Liệt kê các tài nguyên trên TẤT CẢ namespaces
kubectl get pods --all-namespaces
kubectl get pods -A # viết tắt

```

Nội dung KHÔNG được đặt tên (tài nguyên trong phạm vi cụm):

- Nút
- PersistentVolumes

- StorageClasses
- ClusterRoles, ClusterRoleBindings
- Bản thân các namespace
- Định nghĩa tài nguyên tùy chỉnh

1.7 Nhãn, bộ chọn và annotation

Nhãn là các cặp khóa-giá trị được gắn vào đối tượng. Chúng là chất keo kết nối các đối tượng Kubernetes với nhau - Services tìm Pods của chúng qua nhãn, Deployments quản lý ReplicaSets của chúng qua nhãn và NetworkPolicies chọn mục tiêu qua nhãn.

```
metadata:
  labels:
    app: frontend          # Nhãn chung theo quy ước
    version: "2.3.1"
    environment: production
    tier: web
```

Bộ chọn tìm các đối tượng khớp với một bộ nhãn:

```
# Bộ chọn dựa trên sự bình đẳng
kubectl get pods -l app=frontend
kubectl get pods -l app=frontend,environment=production

# Bộ chọn dựa trên tập hợp
kubectl get pods -l 'environment in (production, staging)'
kubectl get pods -l 'tier notin (backend)'
kubectl get pods -l '!beta'          # Pods không có nhãn 'beta'

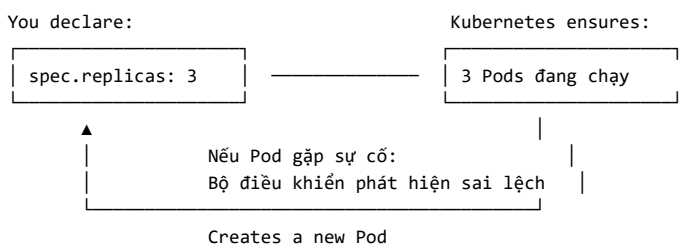
# Hiển thị nhãn trên mỗi pod
kubectl get pods --show-labels
```

Chú thích cũng là cặp khóa-giá trị nhưng chúng dành cho siêu dữ liệu có thể đọc được bằng máy - không được sử dụng để lựa chọn. Sử dụng chú thích để biết thông tin bản dựng, liên kết tài liệu, cấu hình công cụ, v.v.

```
metadata:
  annotations:
    kubernetes.io/change-cause: "Upgraded to nginx 1.25.3 for CVE-2023-xxxx"
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
    deployment.kubernetes.io/revision: "3"
```

1.8 Vòng lặp đối chiếu trạng thái: Trạng thái mong muốn và trạng thái thực tế

Khái niệm quan trọng nhất trong Kubernetes là **vòng điều hòa**. Kubernetes là một hệ thống **khai báo** - bạn nói với nó *điều* bạn muốn chứ không phải *làm thế nào* để thực hiện điều đó.



Điều này về cơ bản khác với các hệ thống mệnh lệnh (như chạy `docker run` ba lần). Nếu trạng thái mong muốn của bạn là `replicas: 3` và một Pod gặp sự cố, Kubernetes sẽ tự động tạo một trạng thái mới. Nếu bạn xóa Pod theo cách thủ công, Kubernetes sẽ tạo một cái mới.


```
# Xem quá trình đối chiếu trạng thái
kubectl run testpod --image=nginx
kubectl delete pod testpod &      # Xóa ngay lập tức
kubectl get pods -w                # Xem Kubernetes tạo lại nó (nếu được quản lý bởi bộ điều khiển)

# "Tại sao" của bất kỳ tài nguyên nào đều nằm trong các sự kiện của nó
kubectl describe deployment my-app | tail -30

# Buộc điều chỉnh bằng cách kích hoạt triển khai
kubectl rollout restart deployment/my-app
```

1.9 Quan hệ giữa các tài nguyên cốt lõi

Hiểu cách các đối tượng Kubernetes liên quan với nhau là rất quan trọng:

```
Deployment
├── manages ─ ReplicaSet
│   └── manages ─ Pod(s)
│       ├── contains ─ Container(s)
│       └── mounts ─ Volume(s)
│           └── backed by ─ PVC ─ PV

Service ─ selects via labels ─ Pod(s)
└── exposes ─ Endpoints object

ConfigMap ─ mounted by ─ Pod(s)
Secret ─ mounted by ─ Pod(s)

ServiceAccount ─ used by ─ Pod(s)
└── bound via ─ RoleBinding ─ Role (RBAC permissions)

NetworkPolicy ─ selects via labels ─ Pod(s) (restricts ingress/egress)
```

1.10 kubectl: CLI chính

kubectl là giao diện của bạn với mọi thứ trong Kubernetes. Nó đọc thông tin xác thực cụm từ ~/.kube/config (tệp kubeconfig).

Các mẫu kubectl cần thiết

```
# Hoạt động CRUD
kubectl get <resource> [name] [-n namespace] [-o yaml|json|wide]
kubectl describe <resource> <name>
kubectl apply -f <file-or-dir>
kubectl delete <resource> <name>

# Tạo mệnh lệnh (tốc độ thi)
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80 --target-port=80 --type=ClusterIP
kubectl run test --image=busybox --restart=Never -- sleep 3600
kubectl create configmap myconfig --from-literal=key=value
kubectl create secret generic mysecret --from-literal=password=s3cret

# Chỉnh sửa tại chỗ
kubectl edit deployment nginx      # Mở $EDITOR (vim theo mặc định trong bài kiểm tra)
kubectl patch deployment nginx -p '{"spec":{"replicas":5}}'

# Tạo tài nguyên bắt buộc (--dry-run cho các mẫu)
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deploy.yaml
kubectl run test --image=busybox --dry-run=client -o yaml > pod.yaml

# Hãy xem và chờ đợi
kubectl get pods -w                # Theo dõi những thay đổi
kubectl wait --for=condition=ready pod -l app=nginx --timeout=60s

# Quản lý triển khai
kubectl rollout status deployment/nginx
kubectl rollout history deployment/nginx
kubectl rollout undo deployment/nginx
kubectl rollout undo deployment/nginx --to-revision=2

# Bối cảnh và cụm
kubectl config get-contexts
kubectl config use-context production
kubectl config set-context --current --namespace=staging
```

Định dạng đầu ra

```
# Đường dẫn JSON cho các trường cụ thể
kubectl get pod mypod -o jsonpath='{.status.podIP}'
kubectl get nodes -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'

# Cột tùy chỉnh
kubectl get pods -o custom-columns='NAME:.metadata.name,IP:.status.podIP,NODE:.spec.nodeName'

# Sắp xếp đầu ra
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pods --sort-by=.status.podIP

# Đầu ra YAML/JSON cho tất cả tài nguyên trong namespace
kubectl get all -n production -o yaml
```

1.11 Tìm hiểu sâu về kubeconfig

Tệp **kubeconfig** (mặc định: ~/.kube/config) lưu trữ thông tin xác thực và thông tin kết nối. Nó có thể chứa nhiều cụm, người dùng và bối cảnh.

```
apiVersion: v1
kind: Config
preferences: {}

clusters:
- name: production-cluster
  cluster:
    server: https://api.production.example.com:6443
    certificate-authority-data: <base64-ca-cert>
```

```

- name: staging-cluster
  cluster:
    server: https://api.staging.example.com:6443
    certificate-authority-data: <base64-ca-cert>

users:
- name: admin-user
  user:
    client-certificate-data: <base64-client-cert>
    client-key-data: <base64-client-key>

- name: readonly-user
  user:
    token: eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

contexts:
- name: prod
  context:
    cluster: production-cluster
    user: admin-user
    namespace: default

- name: staging
  context:
    cluster: staging-cluster
    user: admin-user
    namespace: staging

current-context: prod

# Xem chi tiết bối cảnh hiện tại
kubectl config view
kubectl config view --minify                # Chỉ bối cảnh hiện tại
kubectl config current-context

# Hợp nhất nhiều kubeconfig
KUBECONFIG=~/.kube/config:~/extra-cluster.yaml kubectl config view --flatten \
> ~/.kube/merged-config
export KUBECONFIG=~/.kube/config:~/cluster2.yaml # Sử dụng nhiều tập tin

# Trích xuất một ngữ cảnh vào một tệp mới
kubectl config view --minify --context=staging --kubeconfig=~/.kube/config \
> staging-kubeconfig.yaml

```

1.12 Trạng thái vòng đời Pod

Hiểu trạng thái vòng đời của Pod là điều cần thiết để gỡ lỗi:

Giai đoạn	Mô tả
Pending	Pod đã được cụm chấp nhận nhưng containers vẫn chưa chạy. Có thể đang chờ lập lịch, kéo image hoặc đính kèm volume.
Running	Ít nhất một container đã khởi động và chưa có container nào bị lỗi.
Thành công	Tất cả containers đã kết thúc thành công (mã thoát 0). Trạng thái đầu cuối cho Jobs.
Thất bại	Ít nhất một container được kết thúc bằng mã thoát khác 0.
Không rõ	node chạy Pod không thể truy cập được; không thể xác định được trạng thái.

Trạng thái container trong Pod:

tiểu bang	Mô tả
Đang chờ	Chưa chạy; lý do: ContainerCreating, PodInitializing, ImagePullBackOff, CrashLoopBackOff
Running	Container đang thực thi
Đã chấm dứt	Đã thoát container; kiểm tra mã thoát và lý do

```
# Xem pha Pod
kubectl get pod mypod -o jsonpath='{.status.phase}'

# Xem trạng thái container
kubectl get pod mypod -o jsonpath='{.status.containerStatuses[*].state}'

# Phân tích trạng thái đầy đủ
kubectl describe pod mypod | grep -A 5 "State:"

# Các trạng thái lỗi thường gặp và chẩn đoán
kubectl get pods | grep -v Running | grep -v Completed # pods không tốt cho sức khỏe
kubectl describe pod <pod> | grep -A 20 "Events:"        # Điều gì đã xảy ra
kubectl logs <pod> --previous                             # Nhật ký từ container bị lỗi
```

Giải thích về CrashLoopBackOff

CrashLoopBackOff nghĩa là container liên tục gặp sự cố và Kubernetes đang lùi lại trước khi khởi động lại (10 giây, 20 giây, 40 giây, tối đa 5 phút). Nguyên nhân phổ biến:

- Ứng dụng thoát ngay lập tức (lỗi cấu hình, thiếu file, điểm vào sai)
- Thăm dò Liveness thất bại liên tục
- Tiêu diệt OOM (Hết bộ nhớ)

```
# Chẩn đoán CrashLoopBackOff
kubectl describe pod <pod>                # Kiểm tra "Trạng thái cuối cùng" và mã thoát
kubectl logs <pod> --previous              # Nhật ký từ container bị lỗi trước đó
kubectl logs <pod> -c <container>          # container cụ thể

# OOM kill: mã thoát 137, lý do OOMKilled
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

1.13 Requests và limits của tài nguyên

Mỗi container phải xác định tài nguyên requests và limits. Điều này ảnh hưởng đến cả hành vi lập lịch và thời gian chạy.

```
containers:
- name: app
  image: myapp:1.0
  resources:
    requests:
      memory: "128Mi" # Người lập lịch sử dụng cái này; đảm bảo tối thiểu
      cpu: "250m"     # 250 millicores = lỗi CPU 0.25
    limits:
      memory: "512Mi" # Container bị hủy (OOMKilled) nếu vượt quá mức này
      cpu: "1000m"    # Container bị điều tiết (không bị giết) nếu vượt quá mức này
```

Chất lượng của các lớp Service (QoS) được chỉ định tự động dựa trên requests/limits:

Lớp QoS	tình trạng	Ưu tiên trực xuất
Đảm bảo	requests == limits dành cho CPU và bộ nhớ	Bị đuổi lần cuối
Bù đắp	Ít nhất một bộ request/limit nhưng không bằng	Mức độ ưu tiên trung bình

Lớp QoS	tình trạng	Ưu tiên trực xuất
Nỗ lực tốt nhất	Không có bộ requests hoặc limits	Lần đầu tiên bị đuổi khỏi nhà dưới áp lực

```
# Kiểm tra lớp QoS của Pod
kubectl get pod mypod -o jsonpath='{.status.qosClass}'

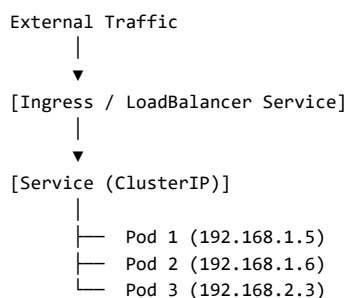
# Xem mức sử dụng tài nguyên so với limits trên nodes
kubectl describe node <node-name> | grep -A 20 "Allocated resources"

# Xem người tiêu dùng tài nguyên hàng đầu
kubectl top pods --all-namespaces --sort-by=memory
kubectl top nodes
```

1.14 Mô hình mạng

Kubernetes áp dụng mô hình mạng phẳng:

1. **Pod-to-Pod** - mọi Pod có thể liên lạc trực tiếp với mọi Pod khác bằng IP mà không cần NAT, thậm chí trên toàn nodes
2. **Pod-to-Service** - Pods tiếp cận Services thông qua cổng ClusterIP:port của họ; kube-proxy thực hiện DNAT thành Pod
3. **Từ bên ngoài đến Service** - Lưu lượng truy cập bên ngoài đi vào qua NodePort, LoadBalancer hoặc Ingress



Pod-to-Pod direct:
192.168.1.5 ————— 192.168.2.3 (no NAT)

Mô hình phẳng này được triển khai bởi các plugin CNI (Calico, Flannel, Weave, Cilium).

1.15 Tổng quan mô hình bảo mật

Xác thực và ủy quyền

Request — Authentication — Authorization — Admission Control — etcd
(who are you?) (can you do it?) (is it valid?)

Phương thức xác thực:

- **Chứng chỉ X.509** - được nhúng trong kubeconfig; được sử dụng bởi người dùng do kubectl và kubeadm tạo
- **Mã thông báo mang** - được sử dụng bởi ServiceAccounts bên trong Pods
- **Mã thông báo OIDC** - tích hợp với các nhà cung cấp danh tính bên ngoài (Google, GitHub, v.v.)
- **Webhook** - dịch vụ xác thực bên ngoài

Ủy quyền (RBAC):

```
# Role: xác định quyền trong namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: staging
```

```

name: pod-reader
rules:
- apiGroups: [""]          # "" = nhóm lõi API
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
# RoleBinding: kết nối chủ thể với Role
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: staging
  name: read-pods-binding
subjects:
- kind: User
  name: developer1
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

# Kiểm tra những gì bạn có thể làm
kubectl auth can-i get pods
kubectl auth can-i get pods --as=developer1 --namespace=staging
kubectl auth can-i '*' '*' --all-namespaces          # Kiểm tra siêu người dùng

# Xem những gì tài khoản dịch vụ có thể làm
kubectl auth can-i list secrets --as=system:serviceaccount:default:my-sa

```

1.16 Tóm tắt: 10 điều cần hiểu về Kubernetes

1. **Máy chủ API là trung tâm** - mọi hoạt động đều thông qua nó; etcd là bộ lưu trữ duy nhất
2. **Mô hình khai báo** - bạn khai báo trạng thái mong muốn; controller liên tục đối chiếu và điều chỉnh trạng thái thực tế
3. **Pods là tạm thời** - không bao giờ dựa vào IP hoặc tên của Pod; sử dụng Services và nhãn
4. **Nhãn là chất kết dính** - chúng kết nối Services với Pods, Deployments với ReplicaSets
5. **Phân vùng namespace** - sử dụng chúng để cách ly environment/team, phạm vi RBAC, hạn ngạch
6. **kubelet là tác nhân node** - đây là thành phần K8 duy nhất không chạy bên trong K8
7. **kube-proxy triển khai Services** - Các quy tắc iptables/IPVS trên mọi node DNAT ClusterIP tới PodIP
8. **CRI/CNI/CSI là các điểm mở rộng** - Thời gian chạy, mạng và bộ lưu trữ của container có thể cắm được
9. **** Yêu cầu dành cho việc lập lịch trình; limits được thực thi trong thời gian chạy**** - Các lớp QoS xuất phát từ đây
10. **RBAC kiểm soát mọi thứ** - mọi yêu cầu máy chủ API đều được xác thực và ủy quyền

1.17 Khắc phục sự cố máy chủ API

Máy chủ API là thành phần quan trọng nhất. Khi không thành công, các lệnh `kubectl` bị treo hoặc trả về lỗi kết nối.

Lỗi thường gặp: Không thể kết nối với máy chủ API

```
$ kubectl get pods
The connection to the server 192.168.1.10:6443 was refused - did you specify the right host or port?

# Các bước chẩn đoán:
# Bước 1: Bạn có thể truy cập vào máy chủ không?
curl -k https://192.168.1.10:6443/healthz

# Bước 2: Máy chủ API pod có đang chạy không?
ssh control-plane-node
sudo crictl ps | grep kube-apiserver
# Nếu trống → vấn đề về manifest pod tĩnh hoặc vấn đề về chứng chỉ

# Bước 3: Kiểm tra nhật ký máy chủ API
sudo crictl logs $(sudo crictl ps -q --name=kube-apiserver)

# Bước 4: Kiểm tra xem manifest pod tĩnh có hợp lệ không
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml
sudo python3 -c "import yaml; yaml.safe_load(open('/etc/kubernetes/manifests/kube-apiserver.yaml'))"
```

Lỗi thường gặp: Không được phép

```
$ kubectl get pods
Error from server (Forbidden): pods is forbidden: User "jane" cannot list resource "pods" in API group "" in the namespace "default"

# Đây là sự cố RBAC, không phải sự cố xác thực
# Kiểm tra những gì Jane có thể làm:
kubectl auth can-i list pods --as=jane
kubectl auth can-i --list --as=jane | grep pods

# Cấp quyền truy cập:
kubectl create rolebinding jane-pod-reader \
  --clusterrole=view \
  --user=jane \
  --namespace=default
```

Lỗi thường gặp: Chứng chỉ x509 được ký bởi cơ quan không xác định

```
$ kubectl get pods
Unable to connect to the server: x509: certificate signed by unknown authority

# Chứng chỉ trong kubeconfig không khớp với cụm CA
# Tái tạo kubeconfig:
sudo kubeadm kubeconfig user --client-name=admin --org=system:masters > ~/.kube/config
# Hoặc sao chép admin.conf từ control plane:
sudo cat /etc/kubernetes/admin.conf > ~/.kube/config
chmod 600 ~/.kube/config
```

Lỗi thường gặp: đã vượt quá thời hạn ngữ cảnh

```
$ kubectl get nodes
Error from server: etcd cluster is unavailable

# etcd không hoạt động hoặc không thể truy cập được
# Kiểm tra etcd pod:
sudo crictl ps | grep etcd
sudo crictl logs $(sudo crictl ps -q --name=etcd) 2>&1 | tail -20

# Kiểm tra dung lượng ổ đĩa thư mục dữ liệu etcd (etcd thường bị lỗi khi đĩa đầy)
df -h /var/lib/etcd
# Khắc phục: giải phóng dung lượng ổ đĩa, sau đó khởi động lại etcd
sudo systemctl restart kubelet
```

1.18 Khắc phục sự cố Trình lập lịch biểu

Triệu chứng: Pods bị kẹt trong Pending mãi mãi

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-app-abc    0/1     Pending   0           15m

# Luôn mô tả đầu tiên
kubectl describe pod my-app-abc

# Các lý do phổ biến của Pending trong phần Sự kiện:

# 1. Không đủ nguồn lực
# Sự kiện: 0/3 nodes có sẵn: 3 Không đủ CPU, 3 Không đủ bộ nhớ
# Khắc phục: giảm requests, thêm nodes hoặc sử dụng node có dung lượng lớn hơn
kubectl get nodes -o custom-columns='NAME:.metadata.name,CPU:.status.allocatable.cpu,MEM:.status.allocatable.memory'

# 2. Không có nút chọn nodes phù hợp
# Sự kiện: 0/3 nodes có sẵn: 3 node không khớp với node affinity/selector của Pod
kubectl get nodes --show-labels | grep disktype
kubectl label node worker-1 disktype=ssd # Thêm nhãn bị thiếu

# 3. Taint không được dung nạp
# Sự kiện: 0/3 nodes có sẵn: 3 node chưa được xử lý taint {node-role.kubernetes.io/control-plane: }
# Khắc phục: Thêm toleration hoặc sử dụng worker nodes

# 4. PVC không bị ràng buộc
# Sự kiện: không tìm thấy Persistvolumeclaim "my-pvc"
# Sự kiện: chờ đợi một ổ đĩa được tạo bởi nhà cung cấp bên ngoài
kubectl get pvc # Kiểm tra trạng thái PVC
kubectl describe pvc my-pvc # Tìm lý do tại sao nó đang chờ xử lý

# 5. Bộ lập lịch không chạy
kubectl -n kube-system get pod | grep scheduler
kubectl -n kube-system logs kube-scheduler-controlplane --tail=30
```

1.19 Tìm hiểu sự kiện và vòng đời của chúng

Sự kiện là công cụ sửa lỗi mạnh mẽ nhất và thường bị bỏ qua. Chúng hết hạn sau 1 giờ theo mặc định.

```
# Hiển thị tất cả các sự kiện Cảnh báo trên toàn bộ cụm (được sắp xếp mới nhất trước)
kubectl get events --all-namespaces --field-selector type=Warning \
  --sort-by='.lastTimestamp'

# Hiển thị cả sự kiện Bình thường
kubectl get events -n production --sort-by='.lastTimestamp' | tail -30

# Xem các sự kiện trong thời gian thực trong quá trình triển khai
kubectl get events -n production -w &
kubectl apply -f deployment.yaml

# Các sự kiện cũng được lưu trữ trong đầu ra mô tả của đối tượng
kubectl describe pod crashing-pod | grep -A 20 "Events:"
# Sự kiện:
#   Nhập lý do Tuổi từ tin nhắn
#   ----
#   Bình thường Lập lịch trình mặc định 2m Đã gắn thành công default/crashing-pod cho worker-1
#   Kéo bình thường 2m kubelet Kéo image "myapp:bad-tag"
#   Cảnh báo không thành công 2m kubelet Không kéo được image: lỗi rpc: ...
#   Cảnh báo BackOff 1m (x5 trên 2m) kubelet Kéo lùi image "myapp:bad-tag"

# Nhận sự kiện cho pod cụ thể
kubectl get events --field-selector involvedObject.name=my-pod,involvedObject.kind=Pod

# Đếm sự kiện theo lý do
kubectl get events -A -o json | python3 -c "
from collections import Counter
import sys, json
data = json.load(sys.stdin)
```



```
reasons = [e['reason'] for e in data['items']]
for r, c in Counter(reasons).most_common(15):
    print(f'{c:4d} {r}')
"
```

1.20 Các lỗi và cách khắc phục kubectl thường gặp

```
# Lỗi: không tìm thấy tài nguyên
$ kubectl get deployments
Error from server (NotFound): deployments.apps "my-app" not found
# Kiểm tra namespace: kubectl get deployment -n staging
# Kiểm tra tên: kubectl get deployment --all-namespaces | grep my

# Lỗi: phiên bản API không hợp lệ
$ kubectl apply -f old-manifest.yaml
error: unable to recognize "old-manifest.yaml": no kind is registered for the version "extensions/v1beta1"
# Khắc phục: cập nhật apiVersion. Di cư phổ biến:
# extensions/v1beta1 Deployment → apps/v1
# extensions/v1beta1 Ingress → networking.k8s.io/v1
# batch/v1beta1 CronJob → batch/v1

# Lỗi: trường không thể thay đổi
$ kubectl apply -f service.yaml
The Service "my-svc" is invalid: spec.clusterIP: Invalid value: "": field is immutable
# Khắc phục: xóa và tạo lại dịch vụ
kubectl delete svc my-svc
kubectl apply -f service.yaml

# Lỗi: phải chỉ định phiên bản tài nguyên khi cập nhật
$ kubectl replace -f pod.yaml
error: Replace failed: Operation cannot be fulfilled on pods "my-pod": the object has been modified
# Cách khắc phục: lấy bản sao mới và thử lại
kubectl get pod my-pod -o yaml > fresh-pod.yaml
# Chỉnh sửa mới-pod.yaml
kubectl replace -f fresh-pod.yaml

# Lỗi: không có kết quả phù hợp cho loại
$ kubectl apply -f crd-resource.yaml
error: no matches for kind "MyCustomResource" in version "mygroup.io/v1"
# CRD chưa được cài đặt
kubectl get crd | grep mygroup
kubectl apply -f crd-definition.yaml # Cài đặt CRD trước
```

Chương 2 - Cài đặt cụm từ đầu với kubeadm

2.1 Tổng quan và điều kiện tiên quyết

kubeadm là công cụ Kubernetes chính thức để khởi động một cụm. Nó xử lý việc tạo chứng chỉ TLS, định cấu hình các thành phần control plane và thiết lập máy chủ Kubernetes API. Hiểu rõ quy trình này từ đầu đến cuối là **bắt buộc** đối với kỳ thi CKA.

Cụm kubeadm tối thiểu yêu cầu:

- **1 control-plane node** (ít nhất 2 CPU, RAM 2 GB)
- **1+ worker nodes** (ít nhất 1 CPU, RAM 1 GB)
- Tất cả nodes trên cùng một mạng, có thể kết nối với nhau
- Tên máy chủ, địa chỉ MAC và UUID sản phẩm duy nhất trên mỗi node
- Các cổng bắt buộc mở (máy chủ API 6443, etcd 2379-2380, kubelet 10250, v.v.)
- Hoán đổi bị vô hiệu hóa trên tất cả nodes

2.2 Bước 1 - Chuẩn bị mọi nút (Control Plane và Worker)

Các bước này phải được chạy trên **tất cả nodes** trước khi chạy kubeadm.

Vô hiệu hóa vĩnh viễn tính năng trao đổi (Kubernetes yêu cầu tắt tính năng trao đổi):

```
# Vô hiệu hóa trao đổi ngay lập tức
sudo swapoff -a

# Vô hiệu hóa trao đổi vĩnh viễn khi khởi động lại
sudo sed -i 's/ swap / s/^(.*)$/#\1/g' /etc/fstab
```

Điều này có tác dụng gì: Quản lý bộ nhớ của Kubernetes và kubelet đều yêu cầu tắt tính năng trao đổi. Lệnh sed nhận xét dòng trao đổi trong /etc/fstab để nó không kích hoạt lại sau khi khởi động lại. Xác minh bằng `free -h` - hàng Hoán đổi sẽ hiển thị 0.

Đặt các mô-đun hạt nhân cần thiết:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

Điều này có tác dụng gì: overlay là trình điều khiển hệ thống tập tin được sử dụng bởi môi trường chạy container. br_netfilter cho phép lọc mạng cầu theo yêu cầu của mạng Kubernetes. Tải chúng bằng modprobe sẽ áp dụng chúng ngay lập tức; tập tin đảm bảo chúng tải mỗi lần khởi động lại.

Đặt các tham số mạng kernel cần thiết (sysctl):

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

# Áp dụng ngay lập tức mà không cần khởi động lại
sudo sysctl --system
```

Điều này có tác dụng gì: Ba tham số này được yêu cầu bởi mạng Kubernetes:

- net.bridge.bridge-nf-call-iptables: Cho phép iptables xem lưu lượng cầu nối (cần thiết cho NetworkPolicy)
- net.ipv4.ip_forward: Cho phép node chuyển tiếp các gói giữa các giao diện mạng (bắt buộc đối với định tuyến Pod)

2.3 Bước 2 - Cài đặt môi trường chạy container (containerd)

Kubernetes sử dụng **Giao diện môi trường chạy container (CRI)** để giao tiếp với môi trường chạy container. containerd là thời gian chạy tiêu chuẩn được sử dụng trong hầu hết các cụm sản xuất.

```
# Cài đặt containerd
sudo apt-get update
sudo apt-get install -y containerd

# Tạo cấu hình mặc định
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# Định cấu hình containerd để sử dụng trình điều khiển nhóm systemd (CRITICS)
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# Khởi động lại và kích hoạt containerd
sudo systemctl restart containerd
sudo systemctl enable containerd

# Xác minh nó đang chạy
```

```
sudo systemctl status containerd
```

Tại sao `SystemdCgroup = false` thực sự: Kubernetes sử dụng trình điều khiển nhóm systemd theo mặc định kể từ v1.22. Nếu containerd sử dụng trình điều khiển cgroup khác (cgroupfs), thì có sự không khớp khiến kubelet bị lỗi và node không ổn định. Luôn đặt `SystemdCgroup = true`.

Xác minh containerd đang hoạt động với crictl:

```
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps
```

2.4 Bước 3 - Cài đặt kubeadm, kubelet và kubectl

```
# Cài đặt phụ thuộc
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

# Thêm kho lưu trữ apt Kubernetes (dành cho Kubernetes 1.35)
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
  https://pkgs.k8s.io/core:/stable:/v1.35/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

# Cài đặt ba thành phần
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl

# Ghim các phiên bản để ngăn chặn việc nâng cấp ngẫu nhiên
sudo apt-mark hold kubelet kubeadm kubectl

# Kích hoạt dịch vụ kubelet (kubeadm sẽ khởi động nó)
sudo systemctl enable kubelet
```

Tác dụng của từng thành phần:

- kubelet - tác nhân chạy trên mọi node và quản lý containers. Nó nhận được hướng dẫn từ máy chủ API và báo cáo lại trạng thái node/pod.
- kubeadm - công cụ khởi động. Chỉ được sử dụng trong quá trình thiết lập và nâng cấp cụm.
- kubectl - công cụ CLI để tương tác với cụm sau khi thiết lập.

2.5 Bước 4 - Khởi tạo nút Control Plane

Chạy chỉ trên control-plane node:

```
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \
  --kubernetes-version=1.35.8 \
  --apiserver-advertise-address=<CONTROL_PLANE_IP>
```

Giải thích về cờ:

Cờ	Mục đích
<code>--pod-network-cidr</code>	Phạm vi CIDR dành cho mạng Pod. Phải khớp những gì plugin CNI của bạn mong đợi (192.168.0.0/16 dành cho Calico, 10.244.0.0/16 dành cho Flannel)
<code>--kubernetes-version</code>	Ghim phiên bản Kubernetes chính xác
<code>--apiserver-advertise-address</code>	Địa chỉ IP mà máy chủ API sẽ quảng cáo - phải có thể truy cập được bởi worker nodes

Sau khi `kubeadm init` hoàn tất thành công, nó sẽ in ba phần đầu ra quan trọng:

```
Your Kubernetes control-plane has initialized successfully!
```

To start using your cluster, run as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You should now deploy a pod network to the cluster...

Then you can join any number of worker nodes by running:

```
kubeadm join 192.168.1.100:6443 --token <token> \
--discovery-token-ca-cert-hash sha256:<hash>
```

Cấu hình truy cập `kubectl` ngay lập tức:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Việc này có tác dụng gì: Sao chép quản trị viên kubeconfig vào thư mục chính của bạn để `kubectl` có thể xác thực với cụm. Nếu không có bước này, `kubectl` sẽ không thành công do lỗi xác thực.

Xác minh control plane đã hoạt động:

```
kubectl get nodes
# Đầu ra: control-plane node ở trạng thái NotReady (bình thường - CNI chưa được cài đặt)

kubectl get pods -n kube-system
# Các Pod control plane cần ở trạng thái Running; CoreDNS chờ CNI
```

2.6 Bước 5 - Cài đặt Plugin CNI (Calico)

Nếu không có plugin CNI, tất cả Pods đều ở trạng thái Pending và CoreDNS sẽ không khởi động.

```
# Cài đặt Calico CNI (khớp với --pod-network-cidr=192.168.0.0/16)
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

```
# Đợi Calico pods xuất hiện
kubectl get pods -n kube-system -w
```

```
# Khi các Pod Calico hoạt động, node chuyển sang Ready
kubectl get nodes
# NAME          STATUS    ROLES          AGE   VERSION
# control-plane Ready     control-plane  3m    v1.35.8
```

Công dụng của plugin CNI: Plugin Giao diện Mạng Container thiết lập mạng Pod. Nó gán địa chỉ IP cho Pods, cho phép giao tiếp Pod-to-Pod trên nodes và thực thi NetworkPolicies. Calico sử dụng định tuyến BGP giữa nodes; Flannel sử dụng đường hầm VXLAN.

2.7 Bước 6 - Tham gia các nút worker

Chạy **trên mỗi worker node** (sử dụng lệnh nói được in bởi `kubeadm init`):

```
sudo kubeadm join 192.168.1.100:6443 \
--token <token> \
--discovery-token-ca-cert-hash sha256:<hash>
```

Nếu mã thông báo tham gia đã hết hạn (mã thông báo hết hạn sau 24 giờ):

```
# Trên control-plane node, tạo mã thông báo mới
kubeadm token create --print-join-command
```

Việc này có tác dụng gì: Tạo mã thông báo tham gia mới và in lệnh `kubeadm join` hoàn chỉnh. Sao chép và chạy nó trên worker node.

Xác minh worker đã tham gia:

```
kubectl get nodes
```

# NAME	STATUS	ROLES	AGE	VERSION
# control-plane	Ready	control-plane	5m	v1.35.8
# worker-1	Ready	<none>	1m	v1.35.8

Gắn nhãn worker node với vai trò của nó:

```
kubectl label node worker-1 node-role.kubernetes.io/worker=worker
```

2.8 Quản lý chứng chỉ bằng kubeadm

Chứng chỉ Kubernetes TLS (được sử dụng bởi tất cả các thành phần control plane) sẽ hết hạn sau **1 năm** theo mặc định.

Kiểm tra ngày hết hạn của chứng chỉ:

```
sudo kubeadm certs check-expiration
```

Đầu ra hiển thị từng chứng chỉ, ngày hết hạn và CA đã ký chứng chỉ đó. Chạy tính năng này thường xuyên trong các cụm sản xuất.

Gia hạn tất cả các chứng chỉ theo cách thủ công:

```
sudo kubeadm certs renew all
```

Gia hạn một chứng chỉ cụ thể:

```
sudo kubeadm certs renew apiserver
sudo kubeadm certs renew etcd-server
```

Sau khi gia hạn chứng chỉ, hãy khởi động lại manifest pod tĩnh bằng cách di chuyển chúng ra ngoài và quay lại:

```
cd /etc/kubernetes/manifests
sudo mv kube-apiserver.yaml /tmp/
sleep 5
sudo mv /tmp/kube-apiserver.yaml .
```

2.9 Đặt lại nút bằng kubeadm

Để xóa sạch Kubernetes khỏi node (để tạo ảnh lại hoặc khắc phục sự cố):

```
# Trên node để thiết lập lại
sudo kubeadm reset

# Sau đó dọn sạch các quy tắc CNI và iptables
sudo rm -rf /etc/cni/net.d
sudo iptables -F && sudo iptables -t nat -F && sudo iptables -t mangle -F && sudo iptables -X
sudo ipvsadm --clear # nếu sử dụng ipv

# Xóa node khỏi cụm (chạy trên control-plane)
kubectl delete node <node-name>
```

2.10 Khắc phục sự cố lỗi init kubeadm

Lỗi: [preflight] đang chạy kiểm tra trước

```
$ sudo kubeadm init
[preflight] Running pre-flight checks
error execution phase preflight: [preflight] Some fatal errors occurred:
    [ERROR NumCPU]: the number of available CPUs 1 is less than the required 2
    [ERROR Mem]: the system RAM (967 MB) is less than the minimum 1700 MB
    [ERROR Swap]: running with swap on is not supported. Please disable swap

# Khắc phục NumCPU/RAM: Sử dụng VM lớn hơn
# Sửa lỗi hoán đổi:
sudo swapoff -a
sudo sed -i 's/ swap / s/^/#/' /etc/fstab

# Buộc init bắt chấp lỗi preflight (chỉ phát triển, không bao giờ sản xuất):
sudo kubeadm init --ignore-preflight-errors=NumCPU,Mem
```

Lỗi: Cổng đã được sử dụng

```
$ sudo kubeadm init
[preflight] Some fatal errors occurred:
    [ERROR Port-6443]: Port 6443 is in use
    [ERROR Port-10259]: Port 10259 is in use

# Lần chạy kubeadm không thành công trước đó khiến các tiến trình đang chạy
# Hoặc một thành phần control-plane trùng lặp đang chạy
sudo netstat -tlnp | grep -E '6443|10259|10257|2379|2380'
sudo fuser -k 6443/tcp 10259/tcp # Tiêu diệt các tiến trình trên các cổng đó
# Hoặc đặt lại hoàn toàn:
sudo kubeadm reset -f
sudo rm -rf /etc/kubernetes /var/lib/etcd /var/lib/kubelet ~/.kube
```

Lỗi: Không tìm thấy node sau khi tham gia

```
$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
control-plane       Ready     control-plane   10m   v1.34.0
# Worker node không xuất hiện

# Trên worker node, hãy kiểm tra kubelet:
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 30

# Nguyên nhân phổ biến: join token hết hạn (token hết hạn sau 24h)
# Khắc phục: tạo mã thông báo mới trên control plane
sudo kubeadm token create --print-join-command

# Nguyên nhân phổ biến: worker không thể truy cập máy chủ API (tường lửa)
# Kiểm tra từ worker:
curl -k https://CONTROL_PLANE_IP:6443/healthz
# Nên trả về "ok"

# Nguyên nhân phổ biến: hàm băm CA sai
# Tạo lại hàm băm chính xác:
openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt | \
openssl rsa -pubin -outform der 2>/dev/null | \
openssl dgst -sha256 -hex | sed 's/^.* //'
```

Lỗi: CrashLoopBackOff dành cho coredns Pods sau khi init

```
$ kubectl get pods -n kube-system
NAME                READY   STATUS             RESTARTS   AGE
coredns-xxx         0/1     CrashLoopBackOff   5           5m
coredns-yyy         0/1     CrashLoopBackOff   5           5m

# Nguyên nhân: Chưa cài đặt plugin CNI
# CoreDNS cần mạng cụm hoạt động
# Trước tiên hãy cài đặt plugin CNI của bạn:
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.31.4/manifests/calico.yaml

# Sau đó đợi CoreDNS tự động khôi phục
kubectl get pods -n kube-system -w
```

Lỗi: kubelet không khởi động sau khi tham gia kubeadm

```
$ sudo systemctl status kubelet
• kubelet.service - kubelet: The Kubernetes Node Agent
  Loaded: loaded (/lib/systemd/system/kubelet.service; enabled)
  Active: activating (auto-restart) (Result: exit-code)

$ sudo journalctl -u kubelet --since "5 minutes ago" | tail -20
"Failed to create kubelet: misconfiguration: kubelet cgroup driver: "cgroupfs" is different from docker cgroup driver:
"systemd""

# Khắc phục: căn chỉnh trình điều khiển cgroup
# containerd sử dụng systemd theo mặc định trong K8 hiện đại
# Kiểm tra trình điều khiển hiện tại:
sudo containerd config dump | grep -A 5 "SystemdCgroup"

# Sửa kubelet để sử dụng systemd:
cat /var/lib/kubelet/config.yaml | grep cgroupDriver
# Chỉnh sửa /var/lib/kubelet/config.yaml:
#   cgroupDriver: systemd
sudo systemctl restart kubelet
```

2.11 Danh sách kiểm tra sau cài đặt

```
# Hoàn tất kiểm tra tình trạng sau khi cài đặt kubeadm:

echo "=== 1. Node Status ==="
kubectl get nodes -o wide

echo "=== 2. System Pods ==="
kubectl get pods -n kube-system

echo "=== 3. Component Status ==="
kubectl get componentstatuses 2>/dev/null || echo "Note: deprecated, use pod checks"

echo "=== 4. API Server Health ==="
kubectl get --raw /healthz
kubectl get --raw /readyz
kubectl get --raw /livez

echo "=== 5. etcd Health ==="
kubectl exec -it etcd-$(hostname) -n kube-system -- \
  etcdctl endpoint health \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

echo "=== 6. DNS Functional Test ==="
kubectl run dns-test --image=busybox:1.35 --restart=Never --rm -it -- \
  nslookup kubernetes.default.svc.cluster.local

echo "=== 7. Pod-to-Pod Connectivity Test ==="
kubectl run pod1 --image=nginx --restart=Never
kubectl run pod2 --image=busybox:1.35 --restart=Never -- sleep 3600
sleep 5
POD1_IP=$(kubectl get pod pod1 -o jsonpath='{.status.podIP}')
kubectl exec pod2 -- wget -qO- http://$POD1_IP
kubectl delete pod pod1 pod2

echo "=== 8. External Connectivity from Pod ==="
kubectl run ext-test --image=busybox:1.35 --restart=Never --rm -it -- \
  wget -qO- http://httpbin.org/ip
```

2.12 Nâng cấp kubeadm từ 1.34 lên 1.35

Không bỏ qua phiên bản minor. Sao lưu etcd, kiểm tra cgroup/CNI/runtime, rồi nâng cấp từng control plane trước các worker. Chọn bản vá **1.35.x** có trong kho; nếu đề bài chỉ định phiên bản thì dùng đúng phiên bản đó. Không coi 1.35.0 là bản vá mới nhất.

Trên từng node cần nâng cấp, đổi kho APT sang 1.35 và xác định gói đích:

```
sudo install -d -m 755 /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key |
  sudo gpg --dearmor --yes -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.35/deb/ /' |
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
apt-cache madison kubeadm

# Ví dụ lấy bản 1.35.x cao nhất hiện có; có thể gán phiên bản đề bài yêu cầu.
K8S_PKG_VERSION=$(apt-cache madison kubeadm | awk ' $3 ~ /^1\.35\./ {print $3}' | sort -V | tail -1)
test -n "$K8S_PKG_VERSION" || exit 1
K8S_TARGET="v${K8S_PKG_VERSION%-*}"
printf 'Gói: %s; Kubernetes: %s\n' "$K8S_PKG_VERSION" "$K8S_TARGET"

sudo apt-mark unhold kubeadm
sudo apt-get install -y "kubeadm=$K8S_PKG_VERSION"
sudo apt-mark hold kubeadm
```

Trên control plane đầu tiên:


```
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply "$K8S_TARGET"
```

Trên mỗi control plane còn lại hoặc worker, sau khi nâng cấp gói kubeadm:

```
sudo kubeadm upgrade node
```

Với **từng node**, dùng máy có kubeconfig quản trị để drain trước khi nâng kubelet. Thay tên bên dưới bằng tên node thật.

--delete-emptydir-data chỉ dùng khi đã chấp nhận mất dữ liệu emptyDir; nếu PDB chặn drain, kiểm tra nguyên nhân trước.

```
# Máy quản trị: mỗi lần chỉ bảo trì một node
NODE_NAME=worker-1
kubectl drain "$NODE_NAME" --ignore-daemonsets

# Trên node đang bảo trì: dùng cùng K8S_PKG_VERSION đã chọn ở trên
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y "kubelet=$K8S_PKG_VERSION" "kubect1=$K8S_PKG_VERSION"
sudo apt-mark hold kubelet kubect1
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# Máy quản trị: xác minh node Ready rồi mở lại lập lịch
kubectl get nodes -o wide
kubectl uncordon "$NODE_NAME"
kubectl get pods -A
```

Nguồn: [quy trình nâng cấp kubeadm 1.35](#) và [chính sách chênh lệch phiên bản](#). Kubelet không được mới hơn kube-apiserver; kubect1 được hỗ trợ trong phạm vi lệch một minor so với kube-apiserver, với giới hạn chặt hơn nếu các API server khác phiên bản.

Chương 3 - Kiến trúc Control Plane có tính sẵn sàng cao

3.1 Tại sao tính sẵn sàng cao lại quan trọng

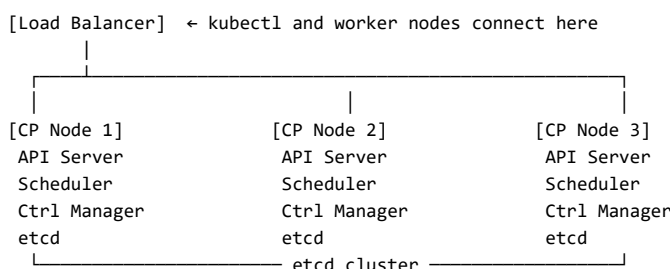
Một node control plane đơn có nghĩa là một điểm lỗi duy nhất. Nếu control-plane node ngừng hoạt động, không thể lên lịch Pods mới, không thể cập nhật workloads và không có hành động quản trị nào hoạt động - ngay cả khi Pods đang chạy vẫn tiếp tục hoạt động. Đối với các cụm sản xuất, control plane có tính sẵn sàng cao (HA) sử dụng **nhiều control-plane nodes** để loại bỏ rủi ro này.

3.2 Các mô hình HA

Kubernetes hỗ trợ hai cấu trúc liên kết HA:

Xếp chồng etcd (phổ biến nhất)

Mỗi control-plane node chạy etcd **cùng với** máy chủ, bộ lập lịch và trình quản lý bộ điều khiển API. etcd được đặt cùng vị trí trên cùng node với các thành phần control-plane khác.



Ưu điểm: Thiết lập đơn giản hơn, cần ít nodes hơn. *Nhược điểm:* etcd và control-plane chia sẻ tài nguyên; lỗi node sẽ mất cả hai.

etcd bên ngoài

etcd chạy trên nodes chuyên dụng riêng biệt. Control plane nodes chỉ chạy máy chủ API, bộ lập lịch và trình quản lý bộ điều khiển.

Ưu điểm: Các miền lỗi được tách biệt; Hiệu suất etcd bị cô lập. *Nhược điểm:* Cần nhiều máy hơn (tối thiểu 6: 3 control-plane + 3 etcd).

3.3 Thiết lập cụm HA với kubeadm

Bước 1: Thiết lập bộ cân bằng tải cho máy chủ API

Bộ cân bằng tải nhận tất cả lưu lượng kubectl và node trên cổng **6443** và phân phối nó trên control-plane nodes. Tùy chọn: HAProxy, AWS NLB, Bộ cân bằng tải GCP TCP, kube-vip.

Cấu hình HAProxy tối thiểu:

```
frontend k8s-api
  bind *:6443
  default_backend k8s-control-planes

backend k8s-control-planes
  balance roundrobin
  server cp1 192.168.1.10:6443 check
  server cp2 192.168.1.11:6443 check
  server cp3 192.168.1.12:6443 check
```

Bước 2: Khởi tạo control-plane node đầu tiên

```
sudo kubeadm init \
  --control-plane-endpoint "LOAD_BALANCER_DNS:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16
```

Các cờ quan trọng đối với HA:

- `--control-plane-endpoint` - Trỏ tới bộ cân bằng tải VIP/DNS, **không** IP của node này. Mọi cấu hình kubeconfig và node sẽ sử dụng địa chỉ này.
- `--upload-certs` - Tải chứng chỉ control-plane lên Kubernetes Secret để họ có thể được tự động phân phối cho control-plane nodes khác trong quá trình tham gia.

Bước 3: Lưu hai lệnh nối từ đầu ra

kubeadm init in hai lệnh nối:

```
# Để có thêm control-plane nodes:
kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>

# Đối với worker nodes:
kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Bước 4: Tham gia thêm control-plane nodes

```
# Trên cp2 và cp3 - sử dụng lệnh nối Control-PLANE (với cờ --control-plane)
sudo kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Xác minh cụm HA:

```
kubect1 get nodes
# NAME      STATUS    ROLES      AGE
# cp1       Ready     control-plane 10m
# cp2       Ready     control-plane 5m
# cp3       Ready     control-plane 2m

# Kiểm tra tình trạng cụm etcd
kubect1 exec -it etcd-cp1 -n kube-system -- sh -c \
  'ETCDCTL_API=3 etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key'
```

3.4 Quorum và khả năng chịu lỗi của etcd

etcd sử dụng **Thuật toán đồng thuận Raft**, yêu cầu phải có quorum (đa số) thành viên để cụm hoạt động.

Tổng số thành viên	Quorum cần thiết	Số thành viên có thể mất
1	1	0
3	2	1 ← HA tối thiểu
5	3	2 ← đề xuất sản xuất
7	4	3

Tại sao lại là số lẻ? Với 4 thành viên, quorum là 3 - chỉ chấp nhận 1 lần thất bại, giống như 3 thành viên nhưng chi phí cao hơn. Luôn sử dụng số lượng thành viên lẻ.

```
# Liệt kê các thành viên etcd và trạng thái của họ
ETCDCTL_API=3 etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table

# Đầu ra:
# +-----+-----+-----+-----+-----+-----+
# | ID | STATUS | NAME | PEER ADDRS | CLIENT ADDRS |
# +-----+-----+-----+-----+-----+
# | 1a6f9aa6b47e62b1 | bắt đầu | cp1 | https://192.168.1.10:2380 | https://192.168.1.10:2379 |
# | 2b7e0bb7c58f73c2 | bắt đầu | cp2 | https://192.168.1.11:2380 | https://192.168.1.11:2379 |
# | 3c8f1cc8d69084d3 | bắt đầu | cp3 | https://192.168.1.12:2380 | https://192.168.1.12:2379 |
# +-----+-----+-----+-----+-----+

# Kiểm tra sức khỏe điểm cuối
ETCDCTL_API=3 etcdctl endpoint health \
  --endpoints=https://192.168.1.10:2379,https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Kiểm tra node nào dẫn đầu
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.10:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
```

```
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
-w table
```

3.5 Sao lưu và khôi phục etcd

Sao lưu etcd là một trong những kỹ năng thi CKA được thử nghiệm nhiều nhất. Bản sao lưu là **ảnh chụp nhanh** của tất cả trạng thái cụm - tất cả các quy tắc Pods, Deployments, Secrets, ConfigMaps, RBAC, v.v.

Chụp nhanh bản sao lưu

```
# Đặt biến môi trường để thuận tiện
export ETCDCTL_API=3

# Chụp ảnh nhanh
etcdctl snapshot save /opt/etcd-backup-$(date +%Y%m%d%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Xác minh ảnh chụp nhanh
etcdctl snapshot status /opt/etcd-backup-20240101120000.db \
  -w table

# Đầu ra:
# +-----+-----+-----+-----+
# | HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
# +-----+-----+-----+-----+
# | 52750ab4 | 1234 | 847 | 3.7 MB |
# +-----+-----+-----+-----+
```

Khôi phục từ Ảnh chụp nhanh

QUAN TRỌNG: Trong kỳ thi CKA, quá trình khôi phục là một nhiệm vụ phổ biến. Thực hiện theo chính xác điều này:

```
# Bước 1: Dừng máy chủ API (nếu sử dụng Pods tĩnh, tạm thời di chuyển manifest)
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
# Đợi máy chủ API dừng (kiểm tra: sudo crictl ps | grep apiserver)

# Bước 2: Khôi phục ảnh chụp nhanh vào thư mục dữ liệu mới
etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored \
  --initial-cluster="cp1=https://192.168.1.10:2380" \
  --initial-advertise-peer-urls="https://192.168.1.10:2380" \
  --name=cp1 \
  --bump-revision=1000000000 \
  --mark-compacted

# Bước 3: Cập nhật manifest Pod tĩnh etcd để sử dụng thư mục dữ liệu mới
# Chỉnh sửa /etc/kubernetes/manifests/etcd.yaml:
# Tìm: --data-dir=/var/lib/etcd
# Đổi thành: --data-dir=/var/lib/etcd-restored
# Đồng thời cập nhật ổ đĩa HostPath cho thư mục dữ liệu

sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-restored|g' /etc/kubernetes/manifests/etcd.yaml

# Bước 4: Khôi phục manifest máy chủ API
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/

# Bước 5: Đợi máy chủ etcd và API khởi động lại
sudo crictl ps | grep etcd
kubectl get nodes # Sẽ hoạt động khi máy chủ API hoạt động trở lại
```

3.6 kube-vip: Giải pháp thay thế cân bằng tải phần mềm

Đối với các cụm tại chỗ hoặc cụm kim loại trần không có bộ cân bằng tải bên ngoài, **kube-vip** cung cấp một IP ảo (VIP) trôi nổi giữa control-plane nodes:

```
# Cài đặt kube-vip dưới dạng Pod tĩnh trên control-plane node đầu tiên
# Đầu tiên, lấy manifest kube-vip
export VIP=192.168.1.100 # IP ảo cho control plane
export INTERFACE=eth0    # Giao diện mạng

docker run --rm ghcr.io/kube-vip/kube-vip:v0.7.0 \
  manifest pod \
  --interface $INTERFACE \
  --address $VIP \
  --controlplane \
  --arp \
  --leaderElection | sudo tee /etc/kubernetes/manifests/kube-vip.yaml
```

Khi kube-vip đang chạy, hãy sử dụng VIP làm --control-plane-endpoint trong kubeadm init.

3.7 Kiểm tra chuyển đổi dự phòng HA

Chỉ thực hiện trong lab có ít nhất ba thành viên etcd và đã sao lưu. Dừng kubelet không dừng các container đang chạy, nên không đủ để mô phỏng mất node. Tắt một máy ảo control plane bằng công cụ quản lý VM, rồi kiểm tra từ node còn lại:

```
# Bước 1: Xác định người lãnh đạo etcd hiện tại
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.10:2379,https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table | grep true # cột LÀ LÃNH ĐẠO

# Bước 2: Tắt VM chứa etcd leader bằng công cụ quản lý máy ảo.
# Chỉ tắt một VM trong lab ba control plane; giữ hai thành viên còn lại hoạt động.

# Bước 3: Xem quá trình bầu chọn người lãnh đạo (từ một control-plane node khác)
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table

# Bước 4: Xác minh cụm vẫn hoạt động
kubectl get pods -A # Vẫn nên hoạt động thông qua cân bằng tải
kubectl create deployment test --image=nginx

# Bước 5: Bật lại VM bằng công cụ quản lý máy ảo và chờ node trở lại Ready.

# node tự động tham gia lại cụm
kubectl get nodes
```

3.8 Quản lý chứng chỉ trong cụm HA

Trong các cụm HA, tất cả control-plane nodes đều có chung **Cơ quan cấp chứng chỉ (CA)**. kubeadm xử lý việc phân phối chứng chỉ khi sử dụng --upload-certs.

```
# Kiểm tra hết hạn chứng chỉ (chạy trên mọi control-plane node)
sudo kubeadm certs check-expiration

# Đầu ra hiển thị hết hạn cho mỗi chứng chỉ:
# CERTIFICATE EXPIRES RESIDUAL TIME ...
# admin.conf Ngày 15 tháng 1 năm 2026 10:00 UTC 364d OK
# apiserver Ngày 15 tháng 1 năm 2026 10:00 UTC 364d OK
# etcd-ca Ngày 12 tháng 1 năm 2034 10:00 UTC 9y OK
```

```
# Gia hạn tất cả các chứng chỉ (làm trước khi chúng hết hạn!)
sudo kubeadm certs renew all

# Sau khi gia hạn, hãy khởi động lại các thành phần Pod tĩnh
sudo crictl rm $(sudo crictl ps -q) # giết chết và khởi động lại tất cả Pods tĩnh

# Cập nhật quản trị viên kubeconfig sau khi gia hạn
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
```

Chương 4 - Giao diện mở rộng: CNI, CSI và CRI

4.1 Kiến trúc mở rộng Kubernetes

Kubernetes được thiết kế có chủ ý để có thể cắm được. Thay vì mã hóa cứng một môi trường chạy container, một giải pháp mạng hoặc một hệ thống lưu trữ, Kubernetes xác định **giao diện tiêu chuẩn** mà mọi triển khai tuân thủ đều có thể đáp ứng. Kiến trúc này là lý do tại sao Kubernetes chạy trên mọi đám mây và hỗ trợ hàng trăm giải pháp lưu trữ và kết nối mạng.

Ba giao diện quan trọng cho kỳ thi CKA là:

Giao diện	Tên đầy đủ	Mục đích
CRI	Giao diện môi trường chạy container	Cách Kubernetes giao tiếp với môi trường chạy container
CNI	Giao diện mạng container	Cách Kubernetes thiết lập mạng Pod
CSI	Giao diện lưu trữ container	Cách Kubernetes cung cấp và gắn kết volume lưu trữ

4.2 CRI - Giao diện môi trường chạy container

CRI là một gRPC API mà kubelet sử dụng để quản lý vòng đời container. Bất kỳ môi trường chạy container nào triển khai CRI đều có thể chạy containers trong Kubernetes.

Thời gian chạy tuân thủ CRI phổ biến:

Thời gian chạy	Mô tả	Trường hợp sử dụng
containerd	Tiêu chuẩn công nghiệp, đi kèm với Docker	Hầu hết các cụm sản xuất
CRI-O	Nhẹ, được thiết kế chuyên dụng cho Kubernetes	OpenShift, triển khai tập trung vào bảo mật
Docker Engine	Thông qua miếng chêm cri-dockerd (không dùng nữa từ k8s 1.24)	Chỉ kế thừa

Kiểm tra thời gian chạy mà node sử dụng:

```
kubectl get node worker-1 -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'
# Đầu ra: containerd://1.7.2
```

Sử dụng crictl để tương tác trực tiếp với môi trường chạy container:

crictl là một công cụ CLI nói trực tiếp giao thức CRI - bỏ qua hoàn toàn kubectl. Nó có sẵn trên mọi Kubernetes node và rất cần thiết để khắc phục sự cố khi máy chủ API ngừng hoạt động.

```
# Liệt kê tất cả containers đang chạy (tương đương docker ps)
sudo crictl ps

# Liệt kê tất cả pods được quản lý bởi thời gian chạy
```

```

sudo crictl pods

# Kéo image
sudo crictl pull nginx:1.25

# Kiểm tra container
sudo crictl inspect <container-id>

# Nhận nhật ký từ container (khi không có nhật ký kubectl)
sudo crictl logs <container-id>

# Xóa container đã dừng
sudo crictl rm <container-id>

```

Định cấu hình crictl để tìm ổ cắm thời gian chạy:

```
sudo crictl config --set runtime-endpoint=unix:///run/containerd/containerd.sock
```

4.3 CNI - Giao diện mạng container

CNI là thông số kỹ thuật và bộ thư viện để định cấu hình giao diện mạng trong Linux containers. Khi Kubernetes tạo Pod, kubelet gọi plugin CNI để:

1. Tạo giao diện mạng (cặp veth) cho Pod
2. Gán địa chỉ IP từ Pod CIDR
3. Thiết lập định tuyến để Pods khác có thể tiếp cận Pod này

Vị trí cấu hình plugin CNI:

```

# Các tệp nhị phân plugin CNI tồn tại ở đây trên mỗi node
ls /opt/cni/bin/
# calico calico-ipam flannel bridge Host-local loopback portmap ...

# Cấu hình mạng CNI tồn tại ở đây
cat /etc/cni/net.d/10-calico.conflist

```

Các plugin CNI phổ biến và đặc điểm của chúng:

Plugin CNI	Mô hình mạng	Hỗ trợ NetworkPolicy	Ghi chú
Calico	BGP hoặc VXLAN	Hỗ trợ đầy đủ	Tiêu chuẩn sản xuất, giàu tính năng nhất
Flannel	VXLAN	Không có hỗ trợ bản địa	Đơn giản, chi phí thấp
Dệt lưới	Lưới	Hỗ trợ đầy đủ	Thiết lập dễ dàng
Cilium	dựa trên eBPF	Hỗ trợ đầy đủ	Hiệu suất cao, hiện đại

Chẩn đoán sự cố CNI:

```

# Kiểm tra xem CNI pods có đang chạy không
kubectl get pods -n kube-system | grep -E "calico|flannel|cilium|weave"

# Kiểm tra nhật ký plugin CNI
kubectl logs -n kube-system -l k8s-app=calico-node --tail=50

# Kiểm tra xem node có được chỉ định PodCIDR hay không
kubectl get node worker-1 -o jsonpath='{.spec.podCIDR}'

# Kiểm tra kết nối pod-to-pod
kubectl run test1 --image=busybox --rm -it --restart=Never -- \
ping -c 3 <pod-ip-of-another-pod>

```

4.4 CSI - Giao diện lưu trữ container

CSI là một tiêu chuẩn để hiển thị các hệ thống lưu trữ tệp và khối tùy ý cho workloads được đóng gói. Trước CSI, trình điều khiển lưu trữ được biên dịch trực tiếp vào cơ sở mã Kubernetes (được gọi là trình điều khiển "trong cây"). CSI cho phép các nhà cung cấp bộ lưu trữ phát triển và cung cấp trình điều khiển của họ một cách độc lập dưới dạng containers, được gọi là **Trình điều khiển CSI**.

Cách CSI hoạt động từ đầu đến cuối:

```
[PVC created] → [StorageClass selects CSI driver] → [CSI driver provisions volume in storage backend]
               → [Volume is bound to PVC] → [CSI driver mounts volume into Pod filesystem]
```

Trình điều khiển CSI phổ biến:

Người lái xe	Phần cuối lưu trữ
ebs.csi.aws.com	Volume AWS EBS
disk.csi.azure.com	Đĩa được quản lý Azure
pd.csi.storage.gke.io	Đĩa liên tục GCE
nfs.csi.k8s.io	Bộ nhớ chia sẻ NFS
rook-ceph.rbd.csi.ceph.com	Lưu trữ khối Ceph

Kiểm tra trình điều khiển CSI đã cài đặt:

```
kubectl get csidriver
# NAME ATTACHREQUIRED PODINFOONMOUNT STORAGECAPACITY
# ebs.csi.aws.com đúng sai sai

kubectl get csistoragecapacities -A
```

Kiểm tra thông tin CSI node:

```
kubectl get csinode
# Hiển thị trình điều khiển CSI nào được đăng ký trên mỗi node
```

Các thành phần trình điều khiển CSI trong cụm:

```
# Trình điều khiển CSI chạy dưới dạng Pods - thường có trong kube-system
kubectl get pods -n kube-system | grep csi
# aws-ebs-csi-controller-... Running
# aws-ebs-csi-node-... Running (DaemonSet - one per node)
```

Thành phần bộ điều khiển xử lý việc cung cấp và đính kèm ổ đĩa ở cấp độ cloud/storage. Thành phần node (DaemonSet) xử lý việc gắn volume vào hệ thống tệp của Pod trên mỗi node.

4.5 Khắc phục sự cố CNI

Triệu chứng: Pods trong quá trình Tạo Container có lỗi mạng

```
$ kubectl describe pod my-pod
Events:
  Warning   Failed    2m   kubelet   Failed to create pod sandbox:
           rpc error: code = Unknown desc = failed to setup network for sandbox:
           plugin type="calico" failed (add): 500 Internal Server Error

# Bước 1: Kiểm tra xem plugin CNI pods có đang chạy không
kubectl get pods -n kube-system | grep -E "calico|flannel|weave|cilium"

# Bước 2: Kiểm tra nhị phân CNI tồn tại trên node
ls /opt/cni/bin/

# Bước 3: Kiểm tra cấu hình CNI
ls /etc/cni/net.d/
cat /etc/cni/net.d/10-calico.conflist

# Bước 4: Kiểm tra nhật ký CNI pod
kubectl logs -n kube-system -l k8s-app=calico-node --tail=50

# Bước 5: Nếu CNI pod CrashLooping, hãy kiểm tra các sự cố cụ thể của node
kubectl describe pod -n kube-system calico-node-xxxxx | grep -A 10 "Events:"
```

Triệu chứng: Pod CIDR chồng chéo với mạng máy chủ

```
# Lỗi trong nhật ký calico-node:
# Không khởi động được Calico: 192.168.0.0/16 trùng lặp với mạng máy chủ

# pod CIDR chồng chéo với mạng hiện tại của bạn
# Khắc phục: chọn CIDR không chồng chéo trong kubeadm init:
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Hoặc cấu hình lại Calico để sử dụng CIDR khác
kubectl -n kube-system edit configmap calico-config
# Thay đổi giá trị CALICO_IPV4POOL_CIDR
```

Xác minh chức năng CNI:

```
# Kiểm tra mạng pod từ đầu đến cuối
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80

# Nhận IP pod
kubectl get pods -l app=nginx -o wide

# Kiểm tra pod-to-pod:
kubectl run test --image=busybox --rm -it --restart=Never -- \
  wget -qO- http://nginx.default.svc.cluster.local

# Kiểm tra giao tiếp xuyên node
kubectl get pods -l app=nginx -o wide | awk '{print $6, $7}' # IP và nút
# Nếu bạn có 3 bản sao trên 2 nodes, tất cả đều có thể truy cập được
```

4.6 Khắc phục sự cố CSI và các vấn đề về lưu trữ

Lỗi thường gặp: PVC bị kẹt trong Pending

```
$ kubectl get pvc
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
data-pvc      Pending                                     fast-ssd       5m

kubectl describe pvc data-pvc
Events:
  Warning ProvisioningFailed 2m rancher.io/local-path
    failed to provision volume:
      node "worker-1" selected by affinity rule "Affinity" not found in NodeList

# Nguyên nhân 1: StorageClass không tồn tại
kubectl get storageclass
kubectl describe storageclass fast-ssd    # Kiểm tra nhà cung cấp

# Nguyên nhân 2: Không có nhà cung cấp bộ nhớ nào
kubectl get pods -n kube-system | grep -i csi
kubectl get pods -n kube-system | grep -i provisioner

# Nguyên nhân 3: Sự cố về mối quan hệ nút với bộ nhớ cục bộ
kubectl describe pvc data-pvc | grep -A 10 "Node Affinity:"
```

Chẩn đoán sự cố trình điều khiển CSI:

```
# Danh sách trình điều khiển CSI đã cài đặt
kubectl get csidrivers

# Kiểm tra CSI node pods (chạy trên mọi node)
kubectl get pods -n kube-system | grep csi-node

# Kiểm tra bộ điều khiển CSI (thường là Deployment hoặc StatefulSet)
kubectl get pods -n kube-system | grep csi-controller

# Kiểm tra nhật ký trình điều khiển CSI
kubectl logs -n kube-system csi-aws-ebs-controller-xxx -c ebs-plugin --tail=30

# Kiểm tra trạng thái đính kèm tập đĩa
kubectl get volumeattachments
kubectl describe volumeattachment <name>
```

4.7 Khắc phục sự cố CRI

Lỗi: không tạo được hộp cát

```
$ kubectl describe pod my-pod
Warning Failed kubelet Failed to create pod sandbox:
  rpc error: code = Unknown desc =
    failed to create containerd task: failed to create shim:
    OCI runtime create failed: container_linux.go:380:
    starting container process caused: process_linux.go:545:
    container init caused: rootfs_linux.go:76:
    mounting "/etc/resolv.conf" to rootfs at "/etc/resolv.conf" caused:
    mount through procd: not a directory: unknown

# Nguyên nhân: /etc/resolv.conf trên node là một liên kết tượng trưng đến một thư mục (phổ biến trong các thiết lập được
# giải quyết systemd)
ls -la /etc/resolv.conf
# Cách khắc phục cho Ubuntu 22+:
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf
sudo systemctl restart containerd kubelet
```

Lỗi: ImagePullBackOff

```
$ kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
my-pod        0/1     ImagePullBackOff   0           5m

kubectl describe pod my-pod
Events:
  Warning   Failed    2m   kubelet   Failed to pull image "myregistry.example.com/myapp:v1.2":
           rpc error: code = Unknown desc = failed to pull and unpack image:
           failed to resolve reference "myregistry.example.com/myapp:v1.2":
           unexpected status code 401 Unauthorized

# Cách 1: Tạo imagePullSecret
kubectl create secret docker-registry registry-credentials \
  --docker-server=myregistry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword \
  --docker-email=admin@example.com

# Tham khảo thông số Pod:
# thông số kỹ thuật:
#   bí mật imagePull:
#   - tên: thông tin đăng ký

# Cách 2: Kiểm tra xem image có thực sự tồn tại không
sudo crictl pull myregistry.example.com/myapp:v1.2

# Cách 3: Thẻ sai - kiểm tra các thẻ có sẵn
curl -u myuser:mypassword \
  https://myregistry.example.com/v2/myapp/tags/list
```

Sử dụng crictl để gỡ lỗi CRI:

```
# List đang chạy containers (như docker ps)
sudo crictl ps

# Liệt kê tất cả containers bao gồm cả đã dừng
sudo crictl ps -a

# Kiểm tra container
sudo crictl inspect <container-id>

# Nhận nhật ký container (bỏ qua kubectl)
sudo crictl logs <container-id>
sudo crictl logs --tail=50 <container-id>

# Danh sách kéo images
sudo crictl images

# Kéo image theo cách thủ công
sudo crictl pull nginx:1.25

# Xóa image
sudo crictl rmi nginx:1.25

# Thông tin hộp cát Pod
sudo crictl pods
sudo crictl inspectp <pod-sandbox-id>

# Chạy lệnh trong container (như docker exec)
sudo crictl exec -it <container-id> sh

# Nhận thông tin thời gian chạy
sudo crictl info
```

Chương 5 - Tìm hiểu Pods và các container

Trước khi đi sâu vào các lệnh, điều quan trọng là phải hiểu mối quan hệ giữa Pods và containers trong Kubernetes, vì điều này trực tiếp thông báo cách cấu trúc các lệnh tương tác.

5.1 Pod là gì?

Pod là đơn vị có thể triển khai nhỏ nhất trong Kubernetes. Hãy coi nó như một lớp bọc mỏng bao quanh một hoặc nhiều containers dùng chung mạng namespace (cùng địa chỉ IP và dung lượng cổng), cùng dung lượng lưu trữ và cùng vòng đời.

Khi bạn chạy `kubectl get pods`, bạn sẽ thấy Pods - không phải trực tiếp containers. Để tương tác với container, trước tiên bạn luôn xử lý Pod, sau đó tùy ý nhắm mục tiêu container cụ thể trong đó.

```
[Kubernetes Node]
└─ [Pod: my-app-pod]
    ├── [Container: app]      ← main application
    ├── [Container: sidecar]  ← helper (e.g., log shipper)
    └─ [Shared Volume: /data]
```

5.2 Pod có một hoặc nhiều container

Mẫu container Pod đơn là mẫu phổ biến nhất. Tuy nhiên, multi-container Pods (sử dụng sidecar, mô hình đại sứ hoặc bộ điều hợp) là phổ biến trong sản xuất. Khi Pod có nhiều containers, hầu hết các lệnh `kubectl` đều yêu cầu bạn chỉ định container nào bạn muốn nhắm mục tiêu bằng cờ `-c` (hoặc `--container`).

5.3 Cấu trúc lệnh kubectl

Tất cả các tương tác Kubernetes CLI đều tuân theo mẫu chung này:

```
kubectl <verb> <resource-type> <resource-name> [flags]
```

Đối với các lệnh dành riêng cho container, cấu trúc chung mở rộng tới:

```
kubectl exec <pod-name> -c <container-name> -n <namespace> -- <command>
```

Dấu gạch ngang kép (--) là quy ước UNIX phân tách các cờ `kubectl` khỏi lệnh được chuyển tới shell của container. Không có nó, việc phân tích đối số có thể không rõ ràng.

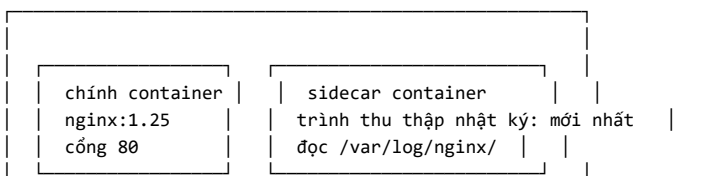
5.4 Cấu trúc chi tiết của Pod

Pod là đơn vị nguyên tử của Kubernetes. Nó **không phải** là container - nó là một môi trường thực thi dùng chung bao bọc một hoặc nhiều containers. Các container trong chia sẻ Pod:

- **Mạng namespace** - chúng chia sẻ cùng một địa chỉ IP và giao diện loopback; giao tiếp qua `localhost`
- **IPC namespace** - có thể sử dụng hàng đợi tin nhắn POSIX và bộ nhớ dùng chung
- **Các tập** - mọi tập đĩa được gắn vào Pod đều có thể truy cập được đối với tất cả containers trong đó

Mỗi Pod có **UTS namespace** (tên máy chủ) riêng và containers có **PID namespaces** riêng theo mặc định (mặc dù chúng có thể được chia sẻ với `shareProcessNamespace: true`).

Pod (shared network: 10.244.3.7, shared volumes)



```
Volume chia sẻ: /var/log/nginx (emptyDir)
```

```
tạm dừng container (cơ sở hạ tầng container)  
(giữ mạng namespace)
```

Container tạm dừng

Mỗi Pod thực sự chứa một **tạm dừng container** ẩn (còn được gọi là cơ sở hạ tầng container hoặc hộp cát container). Nó:

- container đầu tiên có bắt đầu trong Pod không
- Giữ mạng namespace của Pod (địa chỉ IP, quy tắc iptables)
- Hoạt động như quy trình gốc cho tất cả containers' namespaces khác
- Không bao giờ thực hiện bất kỳ công việc thực sự nào - nó chỉ ngủ

```
# Xem phần tạm dừng container trên node  
sudo crictl ps | grep pause  
# Hoặc trực tiếp với containerd  
sudo ctr containers list | grep pause
```

5.5 Init container

Bắt đầu containers chạy và hoàn thành **trước khi** ứng dụng containers khởi động. Chúng hoàn hảo cho:

- Đang chờ một phần phụ thuộc (cơ sở dữ liệu, dịch vụ) sẵn sàng
- Điền trước volume chia sẻ với cấu hình hoặc dữ liệu
- Di chuyển cơ sở dữ liệu Running
- Thực hiện thiết lập bảo mật (e.g., tìm nạp chứng chỉ TLS)

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: app-with-init  
spec:  
  initContainers:  
    - name: wait-for-db  
      image: busybox:1.35  
      command: ['sh', '-c',  
        'until nc -z postgres-service 5432; do echo waiting for database; sleep 2; done']  
      # container này lặp lại cho đến khi cổng 5432 được mở trên dịch vụ postgres  
  
    - name: run-migrations  
      image: myapp:latest  
      command: ['sh', '-c', 'python manage.py migrate']  
      env:  
        - name: DATABASE_URL  
          valueFrom:  
            secretKeyRef:  
              name: db-credentials  
              key: url  
  
  containers:  
    - name: app  
      image: myapp:latest  
      ports:  
        - containerPort: 8080
```

Các hành vi khởi tạo chính của container:

- Chúng chạy theo trình tự (không song song)

- Mỗi lần phải hoàn thành thành công trước khi bắt đầu lần tiếp theo
- Nếu container init không thành công, Kubernetes sẽ khởi động lại Pod (tuân theo restartPolicy)
- `kubectl logs pod-name -c wait-for-db` hiển thị nhật ký init container
- `kubectl describe pod pod-name` hiển thị trạng thái init container trong phần Init Containers

```
# Xem tiến trình init container
kubectl get pod app-with-init -w
# NAME          READY   STATUS    RESTARTS   AGE
# ứng dụng với init 0/1 Ban đầu:0/2 0 5s
# ứng dụng với init 0/1 Ban đầu:1/2 0 12s
# ứng dụng với init 0/1 PodInitializing 0 18s
# app-with-init   1/1     Running    0           20s

# Nhận nhật ký từ một init container cụ thể
kubectl logs app-with-init -c wait-for-db
kubectl logs app-with-init -c run-migrations
```

5.6 Sidecar container

Trong Kubernetes 1.29+, **sidecar containers** có hỗ trợ hạng nhất dưới dạng loại container riêng biệt (trước đây chúng chỉ là containers thông thường theo quy ước). Sidecar chạy cùng với container chính và hỗ trợ ứng dụng:

```
spec:
  initContainers:
    - name: log-collector                # Sidecar được khai báo trong initContainers với restartPolicy
      image: fluentd:v1.16
      restartPolicy: Always              # Điều này làm cho nó trở thành "sidecar gốc" (K8s 1.29+)
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/app

  containers:
    - name: main-app
      image: myapp:latest
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/app

  volumes:
    - name: log-volume
      emptyDir: {}
```

Các mẫu sidecar phổ biến:

- **Trình thu thập nhật ký** - Đọc tệp nhật ký từ ổ đĩa được chia sẻ và gửi tới Elasticsearch/Splunk
- **Proxy lưới Service** - Envoy/Linkerd ủy quyền tất cả lưu lượng truy cập mạng cho mTLS và khả năng quan sát
- **Trình tải lại cấu hình** - Theo dõi các thay đổi của ConfigMap và tải lại ứng dụng
- **Nhà xuất khẩu số liệu** - Hiển thị số liệu ứng dụng ở định dạng Prometheus

5.7 Các mẫu thiết kế Pod có nhiều container

Mẫu đại sứ

Đại sứ container proxy kết nối mạng từ container chính:

```
containers:
  - name: app
    image: myapp
    env:
      - name: DATABASE_HOST
```

```

    value: localhost # Nói chuyện với đại sứ trên localhost
- name: DATABASE_PORT
  value: "6432"

- name: db-ambassador
  image: pgbouncer # Proxy cho cơ sở dữ liệu thực
  env:
    - name: DATABASE_URL
      value: "postgres://prod-db.example.com:5432/mydb"

```

Mẫu bộ điều hợp

Bộ điều hợp chuyển đổi đầu ra từ container chính thành định dạng tiêu chuẩn:

```

containers:
- name: app
  image: legacy-app # Xuất ra định dạng nhật ký tùy chỉnh

- name: log-adapter
  image: log-transformer # Đọc định dạng tùy chỉnh, xuất ra JSON
volumeMounts:
- name: logs
  mountPath: /input

```

5.8 Static Pod

Pods tĩnh được quản lý trực tiếp bởi kubelet trên một node cụ thể, không thông qua máy chủ Kubernetes API. kubelet xem một thư mục (/etc/kubernetes/manifests/ theo mặc định) và tạo Pods từ bất kỳ tệp YAML nào được tìm thấy ở đó.

Thuộc tính chính:

- kubelet tự động khởi động lại chúng nếu chúng gặp sự cố
- Chúng xuất hiện trong `kubectl get pods -n kube-system` dưới dạng **gương Pods** (phản chiếu chỉ đọc)
- Các thành phần control-plane (kube-apiserver, etcd, kube-controller-manager, kube-scheduler) đều là Pods tĩnh trên control-plane node

```

# Xem các biểu hiện Pod tĩnh trên control-plane node
ls /etc/kubernetes/manifests/
# etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml

```

```

# Tạo Pod tĩnh bằng cách đặt một manifest trong thư mục
cat << 'EOF' > /etc/kubernetes/manifests/my-static-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-static-pod
  namespace: kube-system
spec:
  containers:
    - name: nginx
      image: nginx:1.25
EOF

```

```

# kubelet sẽ tạo nó ngay lập tức; xem nó như một tấm gương Pod
kubectl get pods -n kube-system | grep my-static-pod-<node-name>

```

```

# Để xóa: xóa tệp (xóa kubectl sẽ chỉ tạo lại nó)
sudo rm /etc/kubernetes/manifests/my-static-pod.yaml

```

Cấu hình đường dẫn Pod tĩnh:

```

# Tìm nơi kubelet tìm kiếm Pods tĩnh
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# staticPodPath: /etc/kubernetes/manifests

```

5.9 Điều khiển lập lịch Pod

nodeName - Gán trực tiếp vào node

```
spec:
  nodeName: worker-node-2    # Bỏ qua lịch trình; chỉ chạy trên node này
```

nodeSelector - So khớp nhãn đơn giản

```
spec:
  nodeSelector:
    disktype: ssd            # Nút phải có nhãn này
    kubernetes.io/arch: amd64

# Dán nhãn node
kubectl label node worker-1 disktype=ssd

# Xác minh nhãn
kubectl get node worker-1 --show-labels
```

Taint và toleration

Taint hạn chế Pod chạy trên node. **Toleration** tương ứng cho phép scheduler xem xét node đó; Pod vẫn phải đáp ứng các điều kiện lập lịch khác.

```
# Làm hỏng node (ngăn hầu hết Pods chạy ở đây)
kubectl taint node gpu-node-1 gpu=true:NoSchedule

# Liệt kê taints trên nodes
kubectl describe node gpu-node-1 | grep Taints

# Xóa taint
kubectl taint node gpu-node-1 gpu=true:NoSchedule-
```

Hiệu ứng taint:

- NoSchedule - Pods mới sẽ không được lên lịch ở đây (Pods hiện tại không bị ảnh hưởng)
- PreferNoSchedule - Trình lập lịch biểu tránh node này nhưng có thể sử dụng nó nếu cần thiết
- NoExecute - Pods mới sẽ không lên lịch VÀ Pods hiện tại không có toleration sẽ bị trục xuất

```
spec:
  tolerations:
    - key: "gpu"              # Chịu đựng "gpu=true:NoSchedule" taint
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

    - key: "node.kubernetes.io/not-ready"  # taint tích hợp cho NotReady nodes
      operator: "Exists"
      effect: "NoExecute"
      tolerationSeconds: 300    # Chịu đựng 5 phút trước khi trục xuất
```

5.10 Bảo mật Pod và ServiceAccount

Mọi Pod đều chạy với **ServiceAccount** cung cấp thông tin xác thực API:

```
spec:
  serviceAccountName: my-service-account    # Mặc định: SA "mặc định" trong namespace
  automountServiceAccountToken: false       # Tắt nếu Pod không cần truy cập API

  securityContext:                          # Bảo mật cấp độ Pod
    runAsNonRoot: true                     # Đảm bảo container không chạy bằng root
    runAsUser: 1000                         # Run as UID 1000
    runAsGroup: 3000                       # Chạy với GID 3000
    fsGroup: 2000                          # Các tập tin trong tập thuộc sở hữu của GID 2000
```



```

seccompProfile:
  type: RuntimeDefault # Áp dụng hồ sơ seccomp mặc định

containers:
- name: app
  image: myapp:latest
  securityContext:      # Bảo mật cấp container (ghi đè cấp Pod)
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]        # Bỏ tất cả khả năng của Linux
    add: ["NET_BIND_SERVICE"] # Chỉ thêm lại những gì cần thiết

# Tạo tài khoản dịch vụ với quyền hạn chế
kubectl create serviceaccount read-pods-sa -n staging

# Liên kết nó với Role
kubectl create rolebinding read-pods-binding \
  --role=pod-reader \
  --serviceaccount=staging:read-pods-sa \
  -n staging

# Kiểm tra những gì SA có thể làm
kubectl auth can-i list pods --as=system:serviceaccount:staging:read-pods-sa -n staging

# Mã thông báo được gắn bên trong Pod tại:
# /var/run/secrets/kubernetes.io/serviceaccount/token

```

5.11 Quy trình gỡ lỗi Pod

```

Pod Problem → kubectl get pods → identify status
↓
Pending      → kubectl describe pod → check Events
                                   (resources/taints/PVC/selector)
↓
ContainerCreating → kubectl describe pod → Events
                                   (imagePullError/CNI/volume)
↓
CrashLoopBackOff → kubectl logs --previous → see crash reason
                  → kubectl describe pod → see exit code
                                   (exit 1=app error, 137=OOM, 139=segfault)
↓
Running 0/1    → kubectl describe pod → readiness probe failing
                  → kubectl logs → is app healthy?
↓
Terminating stuck → kubectl get pod -o yaml → check finalizers
                  → kubectl patch pod <name> -p '{"metadata":{"finalizers":[]}}' \
                  --type=merge

```

Ví dụ chẩn đoán hoàn chỉnh:

```

# Find all non-Running, non-Completed pods
kubectl get pods --all-namespaces | grep -v -E "Running|Completed|Terminating"

# Với mỗi bài toán pod:
kubectl describe pod <n> -n <ns> 2>&1 | tee pod-diagnosis.txt
kubectl logs <n> -n <ns> --previous 2>&1 >> pod-diagnosis.txt
kubectl logs <n> -n <ns> 2>&1 >> pod-diagnosis.txt

# Kiểm tra mã thoát
kubectl get pod <n> -n <ns> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
# Mã thoát:
# 0 = thành công (thoát sạch)
# 1 = lỗi chung
# 2 = sử dụng sai lệnh shell
# 126 = tìm thấy lệnh nhưng không thực thi được
# 127 = không tìm thấy lệnh
# 128+N = bị tắt bởi tín hiệu N (137=SIGKILL/OOM, 143=SIGTERM)
# 139 = lỗi phân đoạn (SIGSEGV)

```

5.12 Chiến lược cập nhật Deployment

```
# RollingUpdate (mặc định): thay thế dần Pods cũ
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # Pods tối đa mong muốn trong quá trình cập nhật
      maxUnavailable: 0    # Mức tối đa Pods mong muốn dưới mức mong muốn trong quá trình cập nhật
      # maxSurge=1 maxUnavailable=0: Luôn có đủ công suất + 1 pod mới
      # maxSurge=0 maxUnavailable=1: Cập nhật tại chỗ, nhanh chóng -1 pod (tiết kiệm tài nguyên)
      # maxSurge=25% maxUnavailable=25%: Mặc định - cập nhật 25% mỗi lần
  ---
# Tạo lại: tiêu diệt tất cả Pods cũ, sau đó tạo cái mới (thời gian ngừng hoạt động ngắn)
spec:
  strategy:
    type: Recreate

# Xem quá trình cập nhật đang diễn ra
kubectl rollout status deployment/web-app
kubectl rollout history deployment/web-app

# Deployment bị kẹt? Kiểm tra lý do tại sao
kubectl describe deployment web-app | grep -A 20 "Conditions:"
# Tiến độ Hạn chót đã vượt quá: 10 phút trôi qua để chờ triển khai
# Điều này có nghĩa là pods mới không có sẵn - hãy kiểm tra pods!

# Tạm dừng cập nhật luân phiên (để thử nghiệm canary)
kubectl rollout pause deployment/web-app
# ... kiểm tra việc triển khai một phần ...
kubectl rollout resume deployment/web-app

# Nhận chú thích nguyên nhân thay đổi cho quá trình kiểm tra
kubectl annotate deployment/web-app \
  kubernetes.io/change-cause="Upgraded nginx to 1.25.3 for CVE-2023-4966"
kubectl rollout history deployment/web-app
# Hiển thị cột nguyên nhân thay đổi
```

Chương 6 - ConfigMaps và Secrets: Tài liệu tham khảo đầy đủ

6.1 ConfigMaps là gì?

ConfigMap lưu trữ dữ liệu cấu hình không nhạy cảm dưới dạng cặp khóa-giá trị. Họ tách cấu hình khỏi container image, cho phép image tương tự được triển khai trong nhà phát triển, dàn dựng và sản xuất với các cài đặt khác nhau.

ConfigMaps có thể chứa:

- Cặp khóa-giá trị đơn giản (DATABASE_HOST=postgres.production.svc)
- Nội dung tệp cấu hình đầy đủ (nginx.conf, application.properties)
- Dữ liệu JSON hoặc YAML

6.2 Tạo ConfigMaps

Mệnh lệnh từ nghĩa đen:

```
kubectl create configmap app-config \
  --from-literal=DATABASE_HOST=postgres \
  --from-literal=DATABASE_PORT=5432 \
  --from-literal=LOG_LEVEL=info

kubectl get configmap app-config -o yaml
```

Bắt buộc từ một tập tin:

```
# Từ một tệp duy nhất - khóa là tên tệp, giá trị là nội dung tệp
kubectl create configmap nginx-config \
  --from-file=nginx.conf

# Từ một tệp có khóa tùy chỉnh
kubectl create configmap nginx-config \
  --from-file=my-custom-key=./nginx.conf

# Từ một thư mục - tất cả các tệp trong thư mục trở thành cặp khóa-giá trị
kubectl create configmap app-configs \
  --from-file=./config-dir/
```

Khai báo với YAML:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: production
data:
  # Cặp khóa-giá trị đơn giản
  DATABASE_HOST: "postgres.production.svc.cluster.local"
  DATABASE_PORT: "5432"
  LOG_LEVEL: "info"

  # Nội dung tệp đầy đủ dưới dạng giá trị (lưu ý | cho nhiều dòng)
  application.properties: |
    server.port=8080
    spring.datasource.url=jdbc:postgresql://postgres:5432/mydb
    spring.datasource.username=appuser
    logging.level.root=INFO

  nginx.conf: |
    server {
      listen 80;
      server_name _;
      location / {
        proxy_pass http://localhost:8080;
      }
    }
```

6.3 Sử dụng ConfigMaps làm biến môi trường

Chèn các khóa cụ thể làm biến môi trường:

```
spec:
  containers:
  - name: app
    image: my-app:v1
    env:
    - name: DB_HOST                      # Tên của var env trong container
      valueFrom:
        configMapKeyRef:
          name: app-config              # Tên của ConfigMap
          key: DATABASE_HOST           # Chìa khóa trong ConfigMap
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
      optional: true                  # Đừng thất bại nếu ConfigMap hoặc khóa không tồn tại
```

Chèn **TẤT CẢ** các khóa từ ConfigMap dưới dạng biến môi trường (envFrom):

```
spec:
  containers:
  - name: app
    image: my-app:v1
    envFrom:
    - configMapRef:
        name: app-config              # TẤT CẢ các khóa đều trở thành env vars
        prefix: "APP_"              # Tùy chọn: tiền tố tất cả các tên var env
    - configMapRef:
        name: database-config
```

envFrom *làm gì*: Mọi khóa trong app-config đều trở thành biến môi trường có cùng tên trong container. Với prefix: "APP_", khóa DATABASE_HOST trở thành APP_DATABASE_HOST. Điều này thuận tiện nhưng có thể gây ô nhiễm không gian môi trường.

6.4 Sử dụng ConfigMaps làm tệp được gắn

Cách tiếp cận này gắn mỗi khóa ConfigMap dưới dạng một tệp bên trong container tại đường dẫn đã chỉ định. Đây là cách tiếp cận chính xác cho các tệp cấu hình như nginx.conf, application.properties, v.v.

```
spec:
  containers:
  - name: app
    image: my-app:v1
    volumeMounts:
    - name: config-volume
      mountPath: /etc/app/config      # Thư mục nơi tập tin sẽ xuất hiện
      readOnly: true
    - name: nginx-volume
      mountPath: /etc/nginx/conf.d    # Nginx đọc từ đây
      readOnly: true
  volumes:
  - name: config-volume
    configMap:
      name: app-config              # Gắn TẤT CẢ các khóa dưới dạng tệp
  - name: nginx-volume
    configMap:
      name: app-config
      items:
        - key: nginx.conf            # CHỈ gắn các khóa cụ thể dưới dạng tệp
        path: default.conf          # Tệp sẽ có tên default.conf
```

Điều gì xảy ra bên trong container: Mỗi khóa được chỉ định sẽ trở thành một tệp. Nếu app-config có khóa application.properties thì tệp /etc/app/config/application.properties sẽ tồn tại và chứa giá trị của khóa đó. container có thể đọc nó như một tệp thông thường.

6.5 Cập nhật ConfigMaps và Tải lại nóng

Cập nhật ConfigMap:

```
# Chính sửa trực tiếp
kubectl edit configmap app-config

# Vá một khóa cụ thể
kubectl patch configmap app-config \
  --type merge \
  -p '{"data":{"LOG_LEVEL":"debug"}}'

# Thay thế hoàn toàn từ một tập tin
kubectl create configmap app-config \
  --from-file=application.properties \
  --dry-run=client -o yaml | kubectl apply -f -
```

Hành vi tải lại nóng:

Khi ConfigMap được **gắn dưới dạng ổ đĩa**, Kubernetes sẽ tự động truyền các bản cập nhật tới các tệp được gắn trong khoảng 60 giây (được điều khiển bởi `kubelet.syncFrequency`). Các tập tin được thay thế nguyên tử bằng cách sử dụng liên kết tượng trưng.

Tuy nhiên, khi ConfigMap được sử dụng làm **biến môi trường** (`env`: hoặc `envFrom`:), container KHÔNG thấy bản cập nhật. Pod phải được khởi động lại để nhận các giá trị mới.

```
# Buộc Pod khởi động lại để nhận các thay đổi của env var
kubectl rollout restart deployment my-app
```

6.6 ConfigMap bất biến

Không thể sửa đổi ConfigMaps bất biến sau khi được tạo. Điều này ngăn chặn những thay đổi ngẫu nhiên về cấu hình và cải thiện hiệu suất trên quy mô lớn (kubelet không xem các tài nguyên bất biến).

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-v2
immutable: true           # Không thể sửa đổi, chỉ xóa và tạo lại
data:
  DATABASE_HOST: "postgres"
  VERSION: "2.0.0"
```

6.7 Secrets

Secrets tương tự như ConfigMaps nhưng được thiết kế cho dữ liệu nhạy cảm: mật khẩu, khóa API, chứng chỉ TLS, mã thông báo. Secrets được mã hóa base64 trong etcd (không được mã hóa theo mặc định - mã hóa ở trạng thái lưu trữ yêu cầu cấu hình etcd bổ sung) và có thể được mã hóa bằng EncryptionConfiguration.

Quan trọng: Base64 là **mã hóa**, không phải mã hóa. Bất kỳ ai có quyền truy cập đọc vào Đối tượng Secret có thể giải mã nó ngay lập tức. Sử dụng RBAC để hạn chế quyền truy cập Secret.

6.8 Tạo Secrets

Bắt buộc từ chữ (Kubernetes base64-mã hóa tự động):

```
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password='Sup3rS3cr3t!'

# Xác minh (hiển thị các giá trị được mã hóa base64)
kubectl get secret db-credentials -o yaml
```

Từ tập tin:

```
kubect1 create secret generic tls-certs \  
  --from-file=tls.crt=./server.crt \  
  --from-file=tls.key=./server.key  
  
# Tạo bí mật TLS (loại đặc biệt dành cho Ingress/Gateway TLS)  
kubect1 create secret tls my-tls-secret \  
  --cert=./server.crt \  
  --key=./server.key
```

Khai báo (bạn phải tự mình mã hóa các giá trị base64):

```
# Mã hóa giá trị đầu tiên  
echo -n 'admin' | base64          # Đầu ra: YWRtaW4=  
echo -n 'Sup3rS3cr3t!' | base64 # Đầu ra: U3VwM3JTM2NyM3Qh  
  
apiVersion: v1  
kind: Secret  
metadata:  
  name: db-credentials  
  namespace: production  
type: Opaque          # Loại bí mật chung  
data:  
  username: YWRtaW4=    # base64("quản trị viên")  
  password: U3VwM3JTM2NyM3Qh # base64("Sup3rS3cr3t!")
```

Các loại Secret:

Loại	Trường hợp sử dụng
Opaque	Dữ liệu chung tùy ý (mặc định)
kubernetes.io/tls	Cặp khóa và chứng chỉ TLS
kubernetes.io/dockerconfigjson	Thông tin đăng ký Docker
kubernetes.io/service-account-token	Mã thông báo ServiceAccount
kubernetes.io/ssh-auth	Thông tin đăng nhập SSH
kubernetes.io/basic-auth	Xác thực cơ bản HTTP

6.9 Sử dụng Secrets làm biến môi trường

```
spec:  
  containers:  
    - name: app  
      env:  
        - name: DB_USERNAME  
          valueFrom:  
            secretKeyRef:  
              name: db-credentials  
              key: username  
        - name: DB_PASSWORD  
          valueFrom:  
            secretKeyRef:  
              name: db-credentials  
              key: password  
      envFrom:  
        - secretRef:  
            name: db-credentials          # Tất cả các khóa trở thành env vars (được giải mã tự động)
```

6.10 Sử dụng Secrets làm tệp được gắn

```
spec:
  containers:
  - name: app
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets      # Các tệp tin xuất hiện ở đây - được giải mã từ base64
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-credentials
      defaultMode: 0400           # Quyền của tệp: chủ sở hữu chỉ đọc
      items:
      - key: password
        path: db-password.txt     # Có thể truy cập dưới dạng /etc/secrets/db-password.txt
```

6.11 Giải mã Secret để kiểm tra

```
# Xem giá trị base64 thô
kubectl get secret db-credentials -o jsonpath='{.data.password}'
# Đầu ra: U3VwM3JTM2NjM3Qh

# Giải mã nó
kubectl get secret db-credentials \
  -o jsonpath='{.data.password}' | base64 --decode
# Đầu ra: Sup3rS3cr3t!

# Xem tất cả các khóa và giá trị được giải mã của chúng (hữu ích cho việc gỡ lỗi)
kubectl get secret db-credentials -o json | \
  python3 -c "import json,sys,base64; \
  d=json.load(sys.stdin)['data']; \
  [print(f'{k}: {base64.b64decode(v).decode()}') for k,v in d.items()]"
```

6.12 Mã hoá Secret khi lưu trữ

Theo mặc định, Secrets trong etcd chỉ được mã hóa base64. Để bật mã hóa thực sự khi lưu trữ, hãy định cấu hình máy chủ API bằng EncryptionConfiguration:

```
# /etc/kubernetes/enc/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
  providers:
  - aescbc:
    keys:
    - name: key1
      secret: <base64-encoded-32-byte-key> # openssl rand -base64 32
  - identity: {}                          # Dự phòng để đọc các bí mật không được mã hóa hiện có
```

Thêm vào manifest máy chủ API (/etc/kubernetes/manifests/kube-apiserver.yaml):

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --encryption-provider-config=/etc/kubernetes/enc/encryption-config.yaml
    # ...
    volumeMounts:
    - name: enc-config
      mountPath: /etc/kubernetes/enc
      readOnly: true
  volumes:
  - name: enc-config
    hostPath:
      path: /etc/kubernetes/enc
```

Mã hóa các bí mật hiện có sau khi kích hoạt:

```
# Viết lại tất cả các bí mật hiện có thông qua API để mã hóa chúng
kubectl get secrets -A -o json | kubectl replace -f -
```

6.13 Khắc phục sự cố ConfigMaps và Secrets

Lỗi: Không tìm thấy khóa ConfigMap

```
$ kubectl logs my-pod
Error: env variable DATABASE_HOST is empty

# Kiểm tra xem ConfigMap có tồn tại không
kubectl get configmap app-config -n production
# Lỗi: không tìm thấy configmaps "app-config"
# Khắc phục: triển khai ConfigMap trước hoặc kiểm tra namespace

# Xác minh khóa cụ thể tồn tại trong ConfigMap
kubectl get configmap app-config -o jsonpath='{.data.DATABASE_HOST}'
# Nếu trống: tên khóa sai trong thông số Pod hoặc ConfigMap của bạn

# Kiểm tra tên key chính xác trong ConfigMap
kubectl get configmap app-config -o yaml | grep -A 20 "^data:"
```

Lỗi: Secret không gắn kết

```
$ kubectl describe pod my-pod
Events:
  Warning   Failed    1m   kubelet   Error: secret "db-credentials" not found

# Secret sai namespace
kubectl get secret db-credentials # mặc định namespace
kubectl get secret db-credentials -n production # kiểm tra sản xuất namespace
# Secrets nằm trong phạm vi namespace! Phải giống namespace với Pod

# Giải mã và xác minh nội dung bí mật
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d
```

Lỗi: Gắn ổ đĩa hiển thị thư mục trống

```
# Pod gắn ConfigMap dưới dạng ổ đĩa nhưng các tệp trống
# Xác minh tên khóa ConfigMap khớp với mong đợi của Đường dẫn phụ hoặc đường dẫn
kubectl describe pod my-pod | grep -A 20 "Volumes:"
kubectl get configmap nginx-config -o yaml

# Vấn đề: khóa trong ConfigMap là "nginx.conf" nhưng subPath là "nginx.config"
# Chúng phải khớp chính xác

# Gỡ lỗi bằng cách liệt kê những gì trong thư mục được gắn kết
kubectl exec my-pod -- ls -la /etc/nginx/
kubectl exec my-pod -- cat /etc/nginx/nginx.conf
```

Lỗi: Các thay đổi của Secret không được phản ánh trong Pod

```
# Secrets được gắn dưới dạng cập nhật volume tự động (cuối cùng - tối đa 2 phút)
# Nhưng các biến môi trường từ bí mật KHÔNG BAO GIỜ cập nhật (yêu cầu khởi động lại Pod)

# Buộc cập nhật các bí mật env var:
kubectl rollout restart deployment/my-app

# Đối với các bí mật được gắn trên ổ đĩa, hãy kiểm tra xem tệp có thay đổi không:
kubectl exec my-pod -- cat /run/secrets/token | head -1

# Nếu Secret được cập nhật hơn 2 phút trước và volume vẫn hiển thị giá trị cũ:
kubectl describe pod my-pod | grep "Volumes:" -A 20
# Kiểm tra syncPeriod trong cấu hình kubelet:
sudo cat /var/lib/kubelet/config.yaml | grep -i sync
```


Thực tế: Gỡ lỗi thiếu ConfigMap trong Pod đang chạy

```
# Xem pod thực sự có những biến môi trường nào
kubectl exec my-pod -- env | sort | grep -i database

# Xem tập tin nào được gắn kết
kubectl exec my-pod -- find / -path /proc -prune -o -name "*.conf" -print 2>/dev/null

# Kiểm tra ConfigMaps và Secrets nào được tham chiếu
kubectl get pod my-pod -o jsonpath='{.spec.volumes[*].configMap.name}'
kubectl get pod my-pod -o jsonpath='{.spec.containers[0].envFrom[*].configMapRef.name}'

# Đổ đầy tất cả các nguồn env
kubectl get pod my-pod -o json | python3 -c "
import sys, json
pod = json.load(sys.stdin)
for c in pod['spec']['containers']:
    print(f'Container: {c["name"]}')
    for ef in c.get('envFrom', []):
        if 'configMapRef' in ef:
            print(f'  envFrom ConfigMap: {ef["configMapRef"]["name"]}')
        if 'secretRef' in ef:
            print(f'  envFrom Secret: {ef["secretRef"]["name"]}')
    for e in c.get('env', []):
        if 'valueFrom' in e:
            src = e['valueFrom']
            if 'configMapKeyRef' in src:
                print(f'  env {e["name"]} -> ConfigMap {src["configMapKeyRef"]["name"]}:{src["configMapKeyRef"]["key"]}')
            if 'secretKeyRef' in src:
                print(f'  env {e["name"]} -> Secret {src["secretKeyRef"]["name"]}:{src["secretKeyRef"]["key"]}')
"
```

6.14 Thực hành bảo mật Secret

```
# Dữ liệu Secret chỉ được mã hóa base64, KHÔNG được mã hóa theo mặc định
# Kích hoạt mã hóa ở phần còn lại (cấu hình kube-apiserver):
cat << 'EOF' > /etc/kubernetes/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
  providers:
  - aescbc:
    keys:
    - name: key1
      # Tạo: head -c 32 /dev/urandom | cơ sở 64
      secret: $(head -c 32 /dev/urandom | base64)
  - identity: {}
EOF

# Thêm vào manifest pod tính kube-apiserver:
# --encryption-provider-config=/etc/kubernetes/encryption-config.yaml

# Sau khi kích hoạt mã hóa, hãy mã hóa lại tất cả các bí mật hiện có:
kubectl get secrets --all-namespaces -o json | kubectl replace -f -

# Xác minh mã hóa (bí mật sẽ không thể đọc được trong etcd):
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/my-secret | hexdump -C | head
# Nếu được mã hóa: sẽ hiển thị dữ liệu nhị phân, không thể đọc được văn bản
# Nếu bản rõ: bắt đầu bằng "k8s:" theo sau là JSON có thể đọc được
```

6.15 Quản lý Secret bên ngoài (Mẫu sản xuất)

Để sản xuất, hãy sử dụng cửa hàng bí mật bên ngoài:

```
# Sử dụng Operator Secrets bên ngoài với Trình quản lý AWS Secrets
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-password
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials      # Tạo Kubernetes Secret này
    creationPolicy: Owner
  data:
    - secretKey: password    # Nhập K8s Secret
      remoteRef:
        key: prod/database/credentials # Đường dẫn trong Trình quản lý AWS Secrets
        property: password          # Chia khóa bí mật JSON

# Cài đặt toán tử Secrets bên ngoài
helm install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets \
  --create-namespace

# Xác minh đồng bộ hóa externalSecret thành công
kubectl get externalsecret db-password
# NAME          STORE      REFRESH INTERVAL  STATUS  READY
# db-password   aws-secret 1h             Đúng    Đúng
```

Chương 7 - StorageClasses và Cung cấp volume động

7.1 Vòng đời lưu trữ trong Kubernetes

Bộ lưu trữ Kubernetes tuân theo vòng đời rõ ràng:

```
[StorageClass] – defines HOW to provision storage (which CSI driver, what parameters)
↓
[PersistentVolumeClaim (PVC)] – user requests storage of a certain size and access mode
↓
[Dynamic Provisioning] – StorageClass + CSI driver automatically creates the volume
↓
[PersistentVolume (PV)] – the actual storage object representing the provisioned volume
↓
[Pod mounts the PVC] – the volume appears as a filesystem path inside the container
```

Nếu không có StorageClass, PVC phải được liên kết thủ công với các PV được tạo trước (cung cấp tĩnh). Cung cấp động với StorageClasses là tiêu chuẩn sản xuất.

7.2 Tạo StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" # Đặt cái này làm SC mặc định
provisioner: ebs.csi.aws.com # Trình điều khiển CSI để sử dụng
parameters:
  type: gp3 # Loại ổ đĩa AWS EBS
  iops: "3000"
  throughput: "125"
  encrypted: "true"
reclaimPolicy: Delete # Điều gì xảy ra với PV khi PVC bị xóa
volumeBindingMode: WaitForFirstConsumer # Trì hoãn việc cung cấp cho đến khi Pod được lên lịch
allowVolumeExpansion: true # Cho phép thay đổi kích thước PVC sau khi tạo
```

Giải thích về Chính sách đòi lại:

Chính sách	Hành vi khi PVC bị xóa
Delete	PV và dung lượng lưu trữ cơ bản sẽ bị xóa. Mặc định cho việc cung cấp động.
Retain	PV được giữ lại nhưng được đánh dấu Đã phát hành. Yêu cầu dọn dẹp thủ công. Dữ liệu được bảo tồn.
Recycle	Không dùng nữa. Dữ liệu được xóa và PV được cung cấp lại.

Chế độ liên kết volume:

Chế độ	Khi volume được cung cấp
Immediate	Ngay khi PVC được tạo (mặc định)
WaitForFirstConsumer	Phải đến khi Pod sử dụng PVC này mới được lên lịch. Cho phép trình điều khiển CSI cung cấp bộ nhớ trong cùng vùng khả dụng với node của Pod.

```
kubectl apply -f storageclass.yaml
```

```
# Liệt kê tất cả StorageClasses
kubectl get storageclass
# NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE
# fast-ssd (mặc định) ebs.csi.aws.com Xóa WaitForFirstConsumer
# tiêu chuẩn kubernetes.io/no-provisioner Xóa ngay

# Mô tả StorageClass
kubectl describe storageclass fast-ssd
```

7.3 Cung cấp tĩnh - Tạo PV theo cách thủ công

Sử dụng cấp phép tĩnh khi bạn có bộ nhớ hiện có mà bạn muốn hiển thị cho Kubernetes.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv-100gi
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce # Chỉ một node có thể gắn kết để đọc-ghi
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual # Phù hợp với storageClassName của PVC
```

```
hostPath:
  path: /data/myapp          # Chỉ dành cho cụm node đơn hoặc cụm dev!
```

Chế độ truy cập - nodes nào và số lượng có thể gắn ổ đĩa:

Chế độ	Viết tắt	Ý nghĩa
ReadWriteOnce	RWO	Đọc-ghi bằng một node
ReadOnlyMany	ROX	Chỉ đọc đồng thời bởi nhiều nodes
ReadWriteMany	RWX	Đọc-ghi đồng thời bởi nhiều nodes (yêu cầu NFS/Ceph/etc.)
ReadWriteOncePod	RWOP	Đọc-ghi bằng một Pod duy nhất (hạn chế nhất, Kubernetes 1.22+)

```
kubectl apply -f pv.yaml
kubectl get pv
# NAME              CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  STORAGECLASS
# manual-pv-100gi  100Gi    RWO          Retain          Hưởng dẫn sử dụng có sẵn
```

7.4 Tạo PersistentVolumeClaims

PVC là một yêu cầu lưu trữ. Người dùng chỉ định kích thước, chế độ truy cập và tùy chọn StorageClass. Kubernetes liên kết PVC với PV thích hợp.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
  namespace: production
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd    # Bỏ qua hoặc sử dụng "" để sử dụng StorageClass mặc định
  resources:
    requests:
      storage: 20Gi
```

```
kubectl apply -f pvc.yaml
```

```
# Xem trạng thái PVC - nó chuyển sang Pending → Giới hạn
kubectl get pvc -w
# NAME              STATUS    VOLUME              CAPACITY  ACCESS MODES
# app-data-pvc      Pending   ← WaitForFirstConsumer
# app-data-pvc      Ràng buộc PVC-a1b2c3d4-... 20Gi RWO ← sau Pod được lên lịch
```

7.5 Gắn PVC vào Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-storage
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: data-volume
          mountPath: /usr/share/nginx/html    # Nơi volume xuất hiện bên trong container
          readOnly: false
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: app-data-pvc              # Tham chiếu PVC theo tên
```

7.6 Thay đổi kích thước PVC (Mở rộng volume)

Nếu StorageClass có `allowVolumeExpansion: true`, bạn có thể mở rộng PVC trực tuyến:

```
# Chỉnh sửa PVC để yêu cầu thêm dung lượng
kubectl patch pvc app-data-pvc \
  -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'

# Hoặc chỉnh sửa trực tiếp
kubectl edit pvc app-data-pvc
# Thay đổi dung lượng: 20Gi → dung lượng lưu trữ: 50Gi

# Theo dõi trạng thái mở rộng
kubectl describe pvc app-data-pvc | grep -A 5 Conditions
# Loại: FileSystemResizePending - ổ đĩa đang được thay đổi kích thước
# Loại: Thay đổi kích thước → cuối cùng biến mất khi hoàn tất
```

Quan trọng: Bạn chỉ có thể **tăng** kích thước PVC. Việc thu nhỏ PVC không được hỗ trợ và sẽ bị máy chủ API từ chối.

7.7 Khắc phục sự cố lưu trữ

PVC bị kẹt ở trạng thái Pending:

```
kubectl describe pvc app-data-pvc
# Xem phần Sự kiện:
# "không có volume liên tục nào có sẵn cho xác nhận quyền sở hữu này và không có lớp lưu trữ nào được đặt"
#   → StorageClass không tồn tại hoặc sai chính tả
# "đang chờ một tập đĩa được tạo bởi nhà cung cấp bên ngoài..."
#   → WaitForFirstConsumer - bình thường cho đến khi Pod được lên lịch
# "Không tìm thấy storageclass.storage.k8s.io"
#   → StorageClass được chỉ định không tồn tại

# Liệt kê các lớp lưu trữ
kubectl get sc

# Kiểm tra xem trình điều khiển CSI có đang chạy không
kubectl get pods -n kube-system | grep csi
```

PV và PVC trong Released/Available nhưng không đóng bìa:

```
# Một PV ở trạng thái "Đã phát hành" có yêu cầuRef từ PVC trước đó
# Xóa yêu cầuRef để làm cho nó có sẵn trở lại
kubectl patch pv <pv-name> \
  -p '{"spec":{"claimRef": null}}'
```

7.8 Quy trình xử lý sự cố PV/PVC

Tìm hiểu quy trình ràng buộc

```
PVC Created → PV found matching storage class/capacity/access modes
              → PV.claimRef points to this PVC
              → Both enter Bound state
```

Triệu chứng: PVC trong Pending

```
kubectl describe pvc my-pvc
# Xem phần Sự kiện

# Trường hợp 1: Không tồn tại PV phù hợp (cung cấp tĩnh)
# Lỗi: không có tập liên tục nào cho yêu cầu này
kubectl get pv # Liệt kê các PV hiện có
kubectl describe pv my-pv | grep -E "Access|Capacity|StorageClass"
# Phải khớp: AccessModes, dung lượng (PV >= PVC request), storageClassName

# Trường hợp 2: Cung cấp động không hoạt động
# Lỗi: đang chờ một tập đĩa được tạo bởi nhà cung cấp bên ngoài
# "kubernetes.io/no-provisioner" hoặc được tạo thủ công
# Nhà cung cấp StorageClass là "không có nhà cung cấp" - tạo PV theo cách thủ công:
cat << 'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: local-storage
  hostPath:
    path: /data/volumes/pv1
EOF

# Trường hợp 3: StorageClass không tồn tại
kubectl get storageclass # Kiểm tra các lớp có sẵn
# Lỗi: Không tìm thấy storageclass.storage.k8s.io "fast-ssd"
kubectl get pvc -o yaml | grep storageClassName # Kiểm tra PVC requests là gì
```

Triệu chứng: Bị ràng buộc bởi PVC nhưng Pod không thể gắn kết

```
$ kubectl describe pod my-pod
Events:
  Warning FailedMount 2m kubelet
    MountVolume.SetUp failed for volume "data" :
    mount /var/lib/kubelet/pods/.../volumes/kubernetes.io~csi/... failed:
    rpc error: code = Internal desc = NodePublishVolume failed:
    error NodePublishVolume for volume failed: error mounting "":
    exit status 32

# Nguyên nhân phổ biến:
# 1. PVC có namespace khác với Pod (PVC có phạm vi namespace)
kubectl get pvc -A | grep my-pvc # Kiểm tra namespace nào

# 2. Nút không có trình điều khiển CSI
kubectl get pods -n kube-system | grep csi-node
kubectl describe node $(kubectl get pod my-pod -o jsonpath='{.spec.nodeName}') | grep csi

# 3. PV/PVC ReadWriteOnce đã được liên kết với một node khác
kubectl get pv data-pv -o jsonpath='{.spec.claimRef.name}'
kubectl describe pv data-pv | grep -E "Access|Node"
# RWO (ReadWriteOnce) chỉ có thể được gắn trên MỘT node tại một thời điểm

# Khắc phục xung đột RWO: giảm quy mô triển khai cũ, đợi tách volume, sau đó tăng quy mô triển khai mới
kubectl scale deployment old-app --replicas=0
kubectl get volumeattachments # Đợi cho đến khi tệp đính kèm biến mất
kubectl scale deployment new-app --replicas=1
```

Triệu chứng: Mất dữ liệu sau khi khởi động lại Pod

```
# Pod đang sử dụng EmptyDir (mất khi khởi động lại) thay vì PVC (liên tục)
kubectl get pod my-pod -o jsonpath='{.spec.volumes[*].emptyDir}'

# Kiểm tra loại volume của mỗi tập:
kubectl get pod my-pod -o json | python3 -c "
import sys, json
pod = json.load(sys.stdin)
for v in pod['spec']['volumes']:
    vtype = list(v.keys() - {'name'})[0] if len(v) > 1 else 'unknown'
    print(f'{v["name"]}: {vtype}')
"

# Dữ liệu trốngDir bị mất khi Pod bị xóa hoặc lên lịch lại
# Dữ liệu PVC vẫn tồn tại ngay cả khi Pod bị xóa (miễn là ReclaimPolicy được giữ lại)
```

Chính sách đòi lại PersistentVolume

```
# Kiểm tra chính sách thu hồi
kubectl get pv -o custom-columns='NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy,STATUS:.status.phase'

# ReclaimPolicy = Retain: Sau khi PVC bị xóa, PV chuyển sang trạng thái "Đã phát hành"
# Dữ liệu vẫn tồn tại nhưng PV không thể tự động phục hồi sang PVC mới
# Để sử dụng lại PV đã phát hành:
kubectl patch pv my-pv -p '{"spec":{"claimRef": null}}' # Xóa khiếu nạiTham khảo
kubectl get pv my-pv # Bây giờ hiển thị Có sẵn
# Tạo PVC mới tham chiếu PV này bằng storageClass/volumeName

# ReclaimPolicy = Xóa: PV và bộ lưu trữ cơ bản bị xóa khi PVC bị xóa
# Hãy hết sức cẩn thận với điều này trong sản xuất!

# Thay đổi chính sách thu hồi:
kubectl patch pv my-pv -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'
```

7.9 Các loại volume và ví dụ

```
# trốngDir: tạm thời, được chia sẻ giữa containers, đã xóa với Pod
volumes:
- name: cache
  emptyDir: {}
# Hoặc với giới hạn kích thước:
# trốngDir:
#   giới hạn kích thước: 500Mi
#   trung bình: Bộ nhớ # tmpfs được hỗ trợ bởi RAM

# HostPath: gắn kết đường dẫn node (sử dụng cẩn thận - liên kết Pod với node cụ thể)
- name: host-logs
  hostPath:
    path: /var/log/app
    type: DirectoryOrCreate # Tạo thư mục nếu chưa tồn tại
    # Các loại khác: Thư mục, Tập, FileOrCreate, Ổ cứng, BlockDevice

# volume bản đồ cấu hình
- name: app-config
  configMap:
    name: my-config
    defaultMode: 0644 # Quyền tập tin
    items:             # Tùy chọn: chọn các phím cụ thể
    - key: nginx.conf
      path: nginx.conf
      mode: 0600       # Ghi đè chế độ trên mỗi tệp

# volume bí mật
- name: tls-certs
  secret:
    secretName: tls-secret
    defaultMode: 0400 # Secrets phải ở chế độ chỉ đọc

# dự kiến: kết hợp nhiều nguồn vào một tập
- name: combined
  projected:
    sources:
    - configMap:
        name: app-config
    - secret:
        name: app-secret
    - serviceAccountToken:
        path: token
        expirationSeconds: 3600
    - downwardAPI:
        items:
        - path: "pod-name"
          fieldRef:
            fieldPath: metadata.name

# nfs: hệ thống tệp mạng (không cần CSI)
- name: nfs-data
  nfs:
    server: 10.0.0.100
    path: /exports/data
    readOnly: false
```

7.10 Lưu trữ cho StatefulSet

StatefulSets sử dụng volumeClaimTemplates - mỗi Pod có PVC riêng:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
```



```

matchLabels:
  app: postgres
template:
  spec:
    containers:
      - name: postgres
        image: postgres:15
        env:
          - name: PGDATA
            value: /var/lib/postgresql/data/pgdata
        volumeMounts:
          - name: postgres-data
            mountPath: /var/lib/postgresql/data
    volumeClaimTemplates:
      # Mỗi pod có PVC riêng
    - metadata:
        name: postgres-data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 10Gi

# Quy ước đặt tên PVC StatefulSet: <template-name>-<statefulset-name>-<ordinal>
kubectl get pvc -l app=postgres
# NAME                                STATUS    VOLUME
# postgres-data-postgres-0            Giới hạn PVC-abc123
# postgres-data-postgres-1            Giới hạn PVC-def456
# postgres-data-postgres-2            Giới hạn PVC-ghi789

# PVC từ StatefulSets KHÔNG bị xóa khi StatefulSet bị xóa!
# Điều này bảo vệ dữ liệu của bạn. Xóa thủ công khi chắc chắn:
kubectl delete pvc postgres-data-postgres-0

```

Chương 8 - Tự động thay đổi quy mô workload với HPA và VPA

8.1 Tự động tăng giảm số Pod bằng HPA

Bộ co giãn tự động Pod ngang tự động co giãn số lượng bản sao Pod trong Deployment, StatefulSet hoặc ReplicaSet dựa trên các số liệu được quan sát (CPU, bộ nhớ hoặc số liệu tùy chỉnh). Đây là một trong những chủ đề chính trong miền Workload & Lập lịch.

HPA yêu cầu **Máy chủ số liệu** được cài đặt trong cụm để đọc dữ liệu CPU/memory.

Cài đặt Máy chủ số liệu (nếu không có):

```

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Xác minh nó đang chạy
kubectl get deployment metrics-server -n kube-system
kubectl top nodes    # Sẽ hoạt động khi máy chủ số liệu đã sẵn sàng

```

8.2 Tạo HPA bằng lệnh

Cách nhanh nhất để thi CKA:

```

# Tự động co giãn Deployment từ 2 đến 10 bản sao, nhằm mục tiêu sử dụng CPU 50%
kubectl autoscale deployment my-app \
  --min=2 \
  --max=10 \
  --cpu-percent=50

```

Điều này có tác dụng gì: Tạo tài nguyên HPA để theo dõi my-app Deployment. Nếu CPU trung bình trên tất cả Pods vượt quá 50% CPU requests của họ, thì HPA sẽ bổ sung thêm nhiều bản sao hơn (tối đa 10). Nếu CPU giảm xuống dưới 50%, HPA sẽ loại bỏ các bản sao (xuống còn 2). Theo mặc định, việc giảm tỷ lệ có cửa sổ ổn định 5 phút để tránh hiện tượng rung.

Kiểm tra trạng thái HPA:

```
kubectl get hpa
# NAME          REFERENCE          TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
# ứng dụng của tôi Deployment/my-app 23%/50% 2 10 3 5m

kubectl describe hpa my-app
# Hiển thị các sự kiện như: Thay đổi tỷ lệ thành công từ 3 đến 5 bản sao
```

8.3 Tạo HPA với YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50          # Mục tiêu: 50% CPU requests
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 256Mi           # Mục tiêu: Bộ nhớ trung bình 256Mi trên mỗi Pod
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # Đợi 5 phút trước khi thu nhỏ lại
      policies:
      - type: Percent
        value: 10                      # Loại bỏ tối đa 10% số bản sao mỗi kỳ
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0    # Tăng quy mô ngay lập tức
      policies:
      - type: Percent
        value: 100                    # Nhân đôi bản sao mỗi kỳ nếu cần
        periodSeconds: 30
```

Các trường chính được giải thích:

- scaleTargetRef - Deployment/StatefulSet có thể mở rộng quy mô
- metrics - những gì cần đo. Utilization là phần trăm của requests; AverageValue là tuyệt đối
- behavior - kiểm soát tốc độ tăng và giảm tỷ lệ, ngăn chặn sự tăng vọt và va đập đột ngột

Điều kiện tiên quyết quan trọng - Deployment PHẢI có tài nguyên requests được xác định:

```
resources:
  requests:
    cpu: "200m"      # HPA tính tỷ lệ phần trăm so với giá trị này
    memory: "256Mi"
```

Nếu không có `resources.requests.cpu`, HPA không thể tính tỷ lệ phần trăm sử dụng và sẽ báo cáo các mục tiêu <unknown>, ngăn chặn việc mở rộng quy mô.

8.4 Mô phỏng tải để kiểm tra HPA

```
# Trong một thiết bị đầu cuối: xem trạng thái HPA
kubectl get hpa my-app -w

# Trong một thiết bị đầu cuối khác: tạo tải CPU
kubectl run load-gen \
  --image=busybox \
  --rm -it \
  --restart=Never \
  -- sh -c "while true; do wget -q -O- http://my-app-service/; done"
```

Điều này có tác dụng gì: Tạo vòng lặp vô hạn HTTP requests, tăng mức sử dụng CPU trong Pods mục tiêu. Xem thiết bị đầu cuối HPA - bạn sẽ thấy MỤC TIÊU tăng trên 50% và RPLICAS tăng. Nhấn Ctrl+C để dừng trình tạo tải. Sau ~5 phút, HPA giảm quy mô trở lại.

8.5 Xóa HPA

```
# Xóa HPA (Deployment giữ số lượng bản sao hiện tại)
kubectl delete hpa my-app-hpa

# Kiểm tra xem Deployment vẫn có bản sao được mở rộng
kubectl get deployment my-app

# Thu nhỏ lại theo cách thủ công nếu cần
kubectl scale deployment my-app --replicas=2
```

8.6 Cấu hình HPA nâng cao

HPA dựa trên tài nguyên

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70      # Mục tiêu sử dụng CPU trung bình 70%
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 400Mi        # Nhắm mục tiêu bộ nhớ trung bình 400MiB trên mỗi Pod
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60      # Đợi 60 giây trước khi quyết định mở rộng quy mô
      policies:
      - type: Percent
        value: 100                        # Có thể nhân đôi số bản sao mỗi kỳ
        periodSeconds: 60
      - type: Pods
        value: 5                          # Hoặc thêm 5 pods mỗi kỳ (tùy theo mức nào cao hơn)
        periodSeconds: 60
      selectPolicy: Max                    # Sử dụng chính sách bổ sung nhiều Pods nhất
    scaleDown:
      stabilizationWindowSeconds: 300     # Đợi 5 phút trước khi thu nhỏ quy mô
      policies:
      - type: Percent
        value: 10                         # Loại bỏ tối đa 10% số bản sao mỗi kỳ
        periodSeconds: 60
```

Số liệu bên ngoài và tùy chỉnh HPA

HPA cũng có thể mở rộng quy mô theo số liệu tùy chỉnh từ hệ thống giám sát:

```
spec:
  metrics:
  # Số liệu bên ngoài: độ sâu hàng đợi từ hệ thống bên ngoài (e.g., RabbitMQ, SQS)
  - type: External
    external:
      metric:
        name: rabbitmq_queue_messages
        selector:
          matchLabels:
            queue: orders
      target:
        type: AverageValue
        averageValue: 100      # Co giãn khi > 100 tin nhắn trên mỗi Pod

  # Số liệu tùy chỉnh: requests mỗi giây từ Prometheus thông qua số liệu tùy chỉnh API
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
```

```

    type: AverageValue
    averageValue: 1000          # Co giãn khi > 1000 req/s trên mỗi Pod

# Số liệu đối tượng: được gán với một đối tượng Kubernetes khác (như Ingress)
- type: Object
  object:
    metric:
      name: requests-per-second
    describedObject:
      apiVersion: networking.k8s.io/v1
      kind: Ingress
      name: main-ingress
    target:
      type: Value
      value: 10000              # Co giãn khi tổng lượng xâm nhập req/s > 10000

```

Giám sát trạng thái HPA

```

# Tạo HPA bắt buộc (phương pháp thi nhanh)
kubectl autoscale deployment web-app --cpu-percent=70 --min=2 --max=20

# Liệt kê HPA
kubectl get hpa -n production

# Trạng thái HPA chi tiết
kubectl describe hpa web-app -n production
# Đầu ra cho thấy:
#   Số liệu: mức sử dụng tài nguyên CPU (phần trăm yêu cầu): 65% / 70%
#   Bản sao tối thiểu: 2, Bản sao tối đa: 20
#   Bản sao hiện tại: 4, Bản sao mong muốn: 4

# Xem các quyết định mở rộng quy mô HPA trong thời gian thực
kubectl get hpa web-app -n production -w

# Đồng hồ pods cân up/down
kubectl get pods -l app=web-app -n production -w

# Sự kiện hiển thị điều gì đã kích hoạt việc mở rộng quy mô
kubectl describe hpa web-app | grep -A 10 Events

```

8.7 Tự động điều chỉnh tài nguyên Pod bằng VPA

Trong khi HPA co giãn số lượng bản sao theo chiều ngang, **VPA** tự động điều chỉnh CPU và bộ nhớ requests/limits của từng containers riêng lẻ.

```

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  updatePolicy:
    updateMode: "Auto"          # Tùy chọn: Tắt, Ban đầu, Tạo lại, Tự động
  resourcePolicy:
    containerPolicies:
      - containerName: web
        minAllowed:
          cpu: 100m
          memory: 128Mi
        maxAllowed:
          cpu: 4000m
          memory: 4Gi
        controlledResources: ["cpu", "memory"]
        controlledValues: RequestsAndLimits    # hoặc Chỉ yêu cầu

```

Chế độ cập nhật VPA:

Chế độ	Hành vi
Off	VPA chỉ tính toán các đề xuất, không bao giờ áp dụng chúng (chỉ xem)
Initial	Chỉ áp dụng các đề xuất cho Pods mới, không áp dụng các đề xuất đang chạy
Recreate	Loại bỏ và tạo lại Pods để áp dụng các giá trị tài nguyên mới
Auto	Hiện tại giống như Tái tạo; tương lai có thể cho phép cập nhật tại chỗ

```
# Cài đặt VPA (không được cài đặt theo mặc định)
git clone https://github.com/kubernetes/autoscaler.git
./autoscaler/vertical-pod-autoscaler/hack/vpa-up.sh

# Xem đề xuất VPA
kubectl get vpa web-app-vpa -o yaml | grep -A 20 "recommendation:"
# Hiển thị lowBound, target, UpperBound cho CPU và bộ nhớ

# Ở chế độ Tắt, chỉ cần xem VPA sẽ đề xuất những gì
kubectl describe vpa web-app-vpa | grep -A 10 "Recommendation:"
```

Cảnh báo xung đột HPA + VPA: Bạn không thể sử dụng cả HPA (trên CPU/memory) và VPA (trên CPU/memory) cho cùng một Deployment - chúng sẽ xung đột với nhau. Sử dụng một trong hai:

- HPA trên CPU + VPA chỉ trên bộ nhớ (với `controlledResources: ["memory"]`)
- HPA trên số liệu tùy chỉnh + VPA trên CPU/memory (VPA quản lý kích thước, HPA quản lý số lượng)

8.8 KEDA: Kubernetes Tự động điều chỉnh quy mô theo sự kiện

Đối với các kịch bản mở rộng nâng cao hơn ngoài CPU/memory, **KEDA** là tiêu chuẩn ngành:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: rabbitmq-scaledobject
spec:
  scaleTargetRef:
    name: order-processor
  pollingInterval: 15           # Kiểm tra số liệu cứ sau 15 giây
  cooldownPeriod: 300          # Đợi 5 phút trước khi thu nhỏ về 0
  minReplicaCount: 0           # Tỷ lệ về 0 khi hàng đợi trống
  maxReplicaCount: 50
  triggers:
  - type: rabbitmq
    metadata:
      protocol: amqp
      queueName: orders
      mode: QueueLength
      value: "10"               # 1 Pod trên 10 tin nhắn trong hàng đợi
```

KEDA hỗ trợ hơn 60 loại trình kích hoạt: Kafka, RabbitMQ, AWS SQS, Azure Service Bus, Prometheus, lịch trình Cron, v.v.

8.9 Khắc phục sự cố HPA

Lỗi: HPA không thể tính số lượng bản sao

```
$ kubectl describe hpa web-app
Events:
  Warning   FailedGetScale   1m   horizontal-pod-autoscaler
          failed to get cpu utilization: unable to get metrics for resource cpu:
          no metrics returned from resource metrics API

# Nguyên nhân: Máy chủ Metric chưa được cài đặt
kubectl get apiservice v1beta1.metrics.k8s.io
# NAME                               AVAILABLE   AGE
# v1beta1.metrics.k8s.io Sai 5m ← Không có sẵn!

# Cài đặt máy chủ số liệu:
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Xác minh:
kubectl top pods # Sẽ hoạt động trong vòng 2 phút
```

Lỗi: Mục tiêu HPA hiển thị <unknown>/70%

```
$ kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS
web-app       Deployment/web-app  <unknown>/70%    2         20        2

# Nguyên nhân: Triển khai mục tiêu không xác định CPU requests
kubectl get deployment web-app -o jsonpath='{.spec.template.spec.containers[0].resources}'
# {} ← trống! HPA không thể tính toán mức sử dụng nếu không có đường cơ sở yêu cầu

# Khắc phục: Thêm CPU requests vào quá trình triển khai
kubectl set resources deployment web-app \
  --containers=app --requests=cpu=100m

# Đợi ~60 giây để HPA tính toán lại
kubectl get hpa web-app -w
```

HPA không mở rộng quy mô như mong đợi

```
# HPA sẽ không mở rộng quy mô do cửa sổ ổn định (mặc định là 0 để mở rộng quy mô)
# Kiểm tra trạng thái hiện tại:
kubectl describe hpa web-app | grep -A 20 "Conditions:"
# Điều kiện:
#   Nhập trạng thái Lý do Thông báo
#   Kích thước được đề xuất của AbleToScale True ReadyForNewScale phù hợp với kích thước hiện tại
#   ScalingActive True ValidMetricTim thấy HPA đã có thể tính toán thành công...
#   ScalingLimited True TooManyReplicas số lượng bản sao mong muốn nhỏ hơn mức tối thiểu

# AbleToScale=Sai với lý do BackoffBoth:
# HPA đang trong giai đoạn lùi sau sự kiện quy mô gần đây
# Thang đo mặc địnhDown stableWindowSeconds: 300 (5 phút)

# Kiểm tra xem maxReplicas có quá thấp không:
kubectl get hpa web-app -o jsonpath='{.spec.maxReplicas}'
kubectl get hpa web-app -o jsonpath='{.status.currentReplicas}'
# Nếu currentReplicas == maxReplicas → ScalingLimited:TooManyReplicas
# Khắc phục: tăng maxReplicas

# Buộc cân theo cách thủ công để xác minh HPA tiếp quản:
kubectl scale deployment web-app --replicas=1
# HPA sẽ mở rộng quy mô trở lại trong vòng 30 giây
```

Chương 9 - Cơ chế tự phục hồi: probe và PodDisruptionBudget

9.1 Probe của container

Kubernetes cung cấp ba loại **đầu dò** - cơ chế kiểm tra tình trạng mà kubelet sử dụng để xác định trạng thái của container. Thăm dò là một trong những chủ đề quan trọng nhất của kỳ thi CKA trong phần Workload & Lập lịch.

Loại đầu dò	Nó làm gì	Hành động khi thất bại
Sống động	container có phải là healthy/alive không?	Giết và khởi động lại container
Sẵn sàng	container đã sẵn sàng nhận lưu lượng truy cập chưa?	Xóa khỏi điểm cuối Service (không khởi động lại)
Khởi động	container đã khởi động xong chưa?	Tắt và khởi động lại nếu chưa sẵn sàng trong ngưỡng

9.2 Các cơ chế probe

Tất cả ba loại đầu dò đều hỗ trợ bốn cơ chế:

HTTP GET - phổ biến nhất cho các dịch vụ web:

```
livenessProbe:
  httpGet:
    path: /healthz           # Điểm cuối HTTP để gọi
    port: 8080               # Cổng container để sử dụng
    httpHeaders:
      - name: Custom-Header
        value: Awesome
    initialDelaySeconds: 10   # Đợi 10 giây sau khi container khởi động trước khi thăm dò lần đầu
    periodSeconds: 15        # Kiểm tra cứ sau 15 giây
    failureThreshold: 3      # Thất bại 3 lần liên tiếp trước khi hành động
    successThreshold: 1      # 1 thành công được coi là khỏe mạnh (sự sống động chỉ cho phép 1)
    timeoutSeconds: 5        # Mỗi đầu dò phải phản hồi trong vòng 5 giây
```

**** Ô cắm TCP - dành cho các dịch vụ không phải HTTP (cơ sở dữ liệu, hàng đợi):****

```
livenessProbe:
  tcpSocket:
    port: 5432               # Cố gắng kết nối TCP với cổng này
    initialDelaySeconds: 15
    periodSeconds: 20
```

Lệnh Exec - chạy lệnh bên trong container:

```
livenessProbe:
  exec:
    command:
      - /bin/sh
      - -c
      - "pg_isready -U postgres" # Trả về 0 nếu khỏe mạnh, khác 0 nếu không
    initialDelaySeconds: 30
    periodSeconds: 10
```

gRPC - dành cho các dịch vụ gRPC (Kubernetes 1.24+):

```
livenessProbe:
  grpc:
    port: 9090
    service: "my.grpc.HealthService"
```


9.3 Thông số Pod hoàn chỉnh với cả ba đầu dò

```
apiVersion: v1
kind: Pod
metadata:
  name: robust-app
spec:
  containers:
  - name: app
    image: my-app:v2
    ports:
    - containerPort: 8080
    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"

# Thăm dò khởi động - giúp các ứng dụng khởi động chậm có thời gian khởi chạy
# Đã kiểm tra đầu tiên. Đầu dò Liveness/readiness bị TẮT cho đến khi khởi động thành công
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30          # 30 lần thất bại x khoảng thời gian 10 giây = thời gian khởi động tối đa 300 giây (5
                                # phút)
  periodSeconds: 10

# Đầu dò sự sống - nếu điều này không thành công, container sẽ bị tắt và khởi động lại
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 0        # Thăm dò khởi động đã xử lý độ trễ
  periodSeconds: 15
  failureThreshold: 3           # 3 lần thất bại = khởi động lại (45 giây không phản hồi)
  timeoutSeconds: 5

# Thăm dò mức độ sẵn sàng - nếu điều này không thành công, Pod sẽ bị xóa khỏi điểm cuối Service
# container tiếp tục chạy; nó chỉ ngừng nhận lưu lượng truy cập
readinessProbe:
  httpGet:
    path: /ready                # Điểm cuối khác nhau - kiểm tra kết nối DB, bộ đệm, v.v.
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
  successThreshold: 1
```

Thông tin chi tiết về thiết kế: Điểm cuối /healthz và /ready phục vụ các mục đích khác nhau. /healthz là một biện pháp kiểm tra độ sống tối thiểu - chỉ là "quá trình đang chạy". /ready phong phú hơn - nó có thể kiểm tra kết nối cơ sở dữ liệu, tính khả dụng của dịch vụ hạ nguồn và độ ấm của bộ nhớ đệm. Việc sử dụng cùng một điểm cuối cho cả mức độ hoạt động và mức độ sẵn sàng là một lỗi phổ biến: nếu DB của bạn tạm thời không khả dụng, bạn muốn xóa Pod khỏi vòng quay Service (lỗi sẵn sàng) nhưng KHÔNG được khởi động lại (trạng thái hoạt động vẫn vượt qua).

9.4 Gỡ lỗi probe

```
# Xem các sự kiện lỗi thăm dò
kubectl describe pod robust-app | tail -30
# Sự kiện:
#   Cảnh báo Đầu dò độ sống kubelet 5m không tốt cho sức khỏe: Đầu dò HTTP không thành công
#   với mã trạng thái: 503
#   Cảnh báo Giết ứng dụng Container kubelet 5m không thành công trong việc thăm dò độ sống,
#   sẽ được khởi động lại

# Kiểm tra số lần khởi động lại (tăng dần với mỗi lần khởi động lại được kích hoạt bằng hoạt động)
kubectl get pod robust-app -o jsonpath='{.status.containerStatuses[0].restartCount}'

# Xem nhật ký của container trước đó (điều gì đã xảy ra trước sự cố)
kubectl logs robust-app --previous

# Kiểm tra thủ công điểm cuối tình trạng từ bên trong cụm
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl -v http://<pod-ip>:8080/healthz
```

9.5 PodDisruptionBudget (PDB)

A **PodDisruptionBudget** limits có bao nhiêu Pods trong một quá trình triển khai có thể không khả dụng đồng thời trong thời gian gián đoạn tự nguyện - tiêu hao node, nâng cấp cụm và trực xuất thủ công.

Nếu không có PDB, việc rút node với tất cả Pods của Deployment có thể khiến tất cả các bản sao ngoại tuyến cùng một lúc. PDB ngăn chặn điều này.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
  namespace: production
spec:
  # Tùy chọn 1: Mức tối thiểu khả dụng (ít nhất N pods phải đang chạy)
  minAvailable: 2

  # Tùy chọn 2: Không khả dụng tối đa (tối đa N pods có thể ngừng hoạt động cùng một lúc)
  # maxKhông có sẵn: 1

  selector:
    matchLabels:
      app: my-app          # Nhắm mục tiêu pods với nhãn này

kubectl apply -f pdb.yaml

kubectl get pdb
# NAME          MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS  AGE
# my-app-pdb 2 N/A 1 1m

# SỰ TUYỆT VỜI ĐƯỢC PHÉP = bản sao hiện tại - minAvailable
# Nếu bản sao=3 và minAvailable=2 thì được phép thực hiện 1 lần gián đoạn tại một thời điểm
```

Kiểm tra PDB trong quá trình xả node:

```
# Cố gắng tiêu hao node - PDB sẽ tăng tốc độ trực xuất
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# Nếu chỉ tồn tại 2 bản sao và minAvailable=2, hãy rút BLOCKS bằng:
# "Không thể trực xuất pod vì nó sẽ vi phạm ngân sách gián đoạn của pod"
```

9.6 Khắc phục sự cố lỗi đầu dò

Triệu chứng: Pod trong CrashLoopBackOff do thăm dò độ sống

```
kubectl describe pod my-pod
Events:
  Warning Unhealthy 5m kubelet
    Liveness probe failed: Get "http://10.244.1.5:8080/healthz":
    dial tcp 10.244.1.5:8080: connect: connection refused

# Nguyên nhân 1: Sai cổng
# Xác minh container thực sự đang lắng nghe trên cổng đầu dò:
kubectl exec my-pod -- ss -tlnp
# or
kubectl exec my-pod -- netstat -tlnp

# Nguyên nhân 2: Ứng dụng chưa sẵn sàng - initDelaySeconds quá ngắn
# Kiểm tra xem ứng dụng mất bao lâu để bắt đầu:
kubectl logs my-pod | head -20 # Tìm dấu thời gian của thông báo "máy chủ đã bắt đầu"
# Khắc phục: tănginitDelaySeconds hoặc thêm startupProbe

# Nguyên nhân 3: Điểm cuối tình trạng trả về mã trạng thái sai
kubectl exec my-pod -- wget -qO- http://localhost:8080/healthz
# Nếu nó trả về không phải 2xx, thăm dò không thành công
# Kiểm tra: /healthz có trả về 200 không? Một số trả về 204 (cũng được) nhưng những cái sai trả về 500

# Nguyên nhân 4: Thời gian chờ thăm dò quá ngắn
# Nếu ứng dụng đang bận, đầu dò có thể hết thời gian chờ trước khi phản hồi
# Khắc phục: tăng thời gian chờGiây từ 1 giây mặc định
kubectl describe pod my-pod | grep -A 10 "Liveness:"
```

Triệu chứng: Pod là Running nhưng không nhận được lưu lượng truy cập

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-pod        0/1     Running   0           2m # 0/1 có nghĩa là chưa sẵn sàng!

kubectl describe pod my-pod
Readiness:  http-get http://:8080/ready delay=0s timeout=1s period=10s #success=1 #failure=3

# Thăm dò mức độ sẵn sàng không thành công - pod bị loại trừ khỏi điểm cuối Service
kubectl get endpoints my-service
# NAME          ENDPOINTS          AGE
# dịch vụ của tôi <none> 2m ← trống! Pod chưa sẵn sàng

# Gỡ lỗi điểm cuối sẵn sàng:
kubectl exec my-pod -- wget -qO- -S http://localhost:8080/ready
# HTTP/1.1 503 Service Không khả dụng ← Ứng dụng cho biết chưa sẵn sàng

# Kiểm tra lý do ứng dụng chưa sẵn sàng (e.g. vẫn đang kết nối với cơ sở dữ liệu)
kubectl logs my-pod | grep -i "ready\|database\|connect"

# Nếu ứng dụng không có điểm cuối /ready, hãy sử dụng đầu dò exec:
# sẵn sàngProbe:
# thực thi:
# lệnh: ["/bin/sh", "-c", "test -f /tmp/ready"]
```

Triệu chứng: Pod liên tục khởi động lại với OOMKilled

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-pod        0/1     OOMKilled  3          10m

kubectl describe pod my-pod | grep -A 5 "Last State:"
# Trạng thái cuối cùng: Đã chấm dứt
# Lý do: OOMKilled ← Bị giết bởi kẻ giết người hết bộ nhớ kernel
# Mã thoát: 137

# Kiểm tra mức sử dụng bộ nhớ so với limits
kubectl top pod my-pod
kubectl get pod my-pod -o jsonpath='{.spec.containers[0].resources}'

# Sửa: tăng giới hạn bộ nhớ
kubectl set resources deployment my-app --limits=memory=512Mi

# Hoặc tìm rõ ràng bộ nhớ: lấy số liệu theo thời gian
kubectl top pod my-pod --no-headers | awk '{print $3}' # Cột bộ nhớ

# Kiểm tra xem có xu hướng bộ nhớ trong các sự kiện không:
kubectl describe pod my-pod | grep -i "oom\|memory\|kill"
```

Sự sống động, Sự sẵn sàng và Khởi nghiệp - Hướng dẫn Quyết định

```
Use LIVENESS when:
├─ App can deadlock (all threads stuck, but process running)
├─ App cache grows unbounded and needs periodic restart
└─ App enters error loop and needs restart to recover

Use READINESS when:
├─ App needs time to load data/cache before serving requests
├─ App depends on external service being available
└─ App periodically becomes temporarily unavailable (maintenance mode)

Use STARTUP when:
├─ Container needs >30s to start (would be killed by liveness)
├─ Initialization time is variable (could be 10s or 10min)
└─ You want different probe behavior during startup vs steady state

Example: Use all three together for a Java application:
startupProbe:          # Thời gian chờ rộng rãi để khởi động JVM
  httpGet: {path: /health, port: 8080}
  failureThreshold: 30  # 30 lần thử x 10 giây = thời gian khởi động tối đa 5 phút
  periodSeconds: 10

livenessProbe:          # Nghiêm ngặt - khởi động lại nếu bế tắc
  httpGet: {path: /health, port: 8080}
  initialDelaySeconds: 0 # startupProbe đã xử lý độ trễ
  periodSeconds: 10
  failureThreshold: 3    # 3 lần thất bại = 30 giây trước khi khởi động lại

readinessProbe:         # Xóa khỏi điểm cuối nếu tạm thời bận
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  failureThreshold: 2    # Đã xóa khỏi Service sau 10 giây
  successThreshold: 1    # Trở lại Service sau thành công đầu tiên
```

9.7 Xử lý sự cố PodDisruptionBudget

Triệu chứng: Cổng kubectl bị chặn bởi PDB

```
$ kubectl drain worker-1 --ignore-daemonsets
evicting pod production/api-7d8f9-abc12
error when evicting pods/"api-7d8f9-abc12" -n "production"
  (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.

# Kiểm tra PDB:
kubectl get pdb -n production
kubectl describe pdb api-pdb -n production
# Được phép gián đoạn: 0 ← Đây chính là vấn đề!
# Hiện tại: 2/2 pods khỏe mạnh, PDB cho biết minCó sẵn: cho phép gián đoạn 2 → 0

# Sửa lỗi Tùy chọn 1: Xóa PDB tạm thời (nếu cửa sổ bảo trì)
kubectl delete pdb api-pdb -n production
kubectl drain worker-1 --ignore-daemonsets
# Sau đó tạo lại PDB sau khi bảo trì

# Sửa lỗi Tùy chọn 2: Tạm thời mở rộng quy mô triển khai
kubectl scale deployment api --replicas=3 -n production
# Hiện tại 3/2 > minAvailable(2), do đó cho phép 1 lần gián đoạn
kubectl drain worker-1 --ignore-daemonsets

# Khắc phục Tùy chọn 3: Buộc thoát (đuổi pods bỏ qua PDB - nguy cơ mất dữ liệu!)
kubectl drain worker-1 --ignore-daemonsets --disable-eviction --force
# CẢNH BÁO: Chỉ dành cho các tình huống emergency/already-down
```

Chương 10 - Định nghĩa và toán tử tài nguyên tùy chỉnh

10.1 Mở rộng Kubernetes API với CRDs

Kubernetes cung cấp một bộ loại tài nguyên cố định: Pod, Deployment, Service, ConfigMap, v.v. **Định nghĩa tài nguyên tùy chỉnh (CRDs)** cho phép bạn mở rộng Kubernetes API bằng các loại tài nguyên của riêng bạn, tạo ra chúng công dân hạng nhất - họ có điểm cuối API của riêng mình, hoạt động với kubectl, hỗ trợ labels/annotations và có thể được theo dõi bởi bộ điều khiển.

CRD xác định lược đồ của loại tài nguyên mới. Sau khi cài đặt, người dùng có thể tạo các phiên bản của tài nguyên đó (được gọi là **Tài nguyên tùy chỉnh** hoặc **CRs**) giống như tạo Pods.

Ví dụ: CRD cho tài nguyên Database

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.mycompany.com      # Phải là <plural>.<group>
spec:
  group: mycompany.com              # Nhóm API cho tài nguyên này
  names:
    kind: Database                  # Loại tài nguyên (số ít, PascalCase)
    listKind: DatabaseList
    plural: databases               # Đường dẫn URL: /apis/mycompany.com/v1/databases
    singular: database
    shortNames:
      - db                          # kubectl giúp db hoạt động!
  scope: Namespaced                 # Hoặc Cụm cho các tài nguyên trong phạm vi cụm
  versions:
    - name: v1
      served: true                   # Phiên bản này đang hoạt động
      storage: true                  # Phiên bản này được lưu trữ trong etcd
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                  enum: ["postgres", "mysql", "redis"]
                version:
                  type: string
                replicas:
                  type: integer
                  minimum: 1
                  maximum: 5
                storageGB:
                  type: integer
```

Cài đặt CRD:

```
kubectl apply -f database-crd.yaml

# Xác minh nó đã được đăng ký
kubectl get crd databases.mycompany.com

# Kiểm tra xem nó có sẵn trong API không
kubectl api-resources | grep databases
# cơ sở dữ liệu db mycompany.com/v1 cơ sở dữ liệu đúng
```

Tạo Tài nguyên tùy chỉnh (phiên bản của CRD):

```
# của tôi-database.yaml
apiVersion: mycompany.com/v1
kind: Database
metadata:
  name: production-postgres
  namespace: default
spec:
  engine: postgres
  version: "15.4"
  replicas: 3
  storageGB: 100

kubectl apply -f my-database.yaml

# Tương tác với nó giống hệt như tài nguyên tích hợp
kubectl get databases
kubectl get db                  # tên ngắn hoạt động
kubectl describe database production-postgres
kubectl delete database production-postgres
```

Liệt kê tất cả CRDs trong cụm:

```
kubectl get crd
kubectl get crd -o wide
```

10.2 Operator: controller cho tài nguyên tùy chỉnh

Riêng CRD chỉ là một cấu trúc dữ liệu được lưu trữ trong etcd. Nó không tự làm gì cả. **Operator** là bộ điều khiển theo dõi Tài nguyên tùy chỉnh và thực hiện hành động để làm cho trạng thái trong thế giới thực khớp với trạng thái mong muốn được xác định trong CR.

Mẫu toán tử = CRD (lược đồ trạng thái mong muốn) + Bộ điều khiển (vòng điều hòa).

Ví dụ: Operator cơ sở dữ liệu làm gì

Khi bạn tạo CR cơ sở dữ liệu `production-postgres`, Operator cơ sở dữ liệu:

1. Xem CR mới (thông qua Đồng hồ trên Kubernetes API)
2. Cung cấp StatefulSet với 3 PostgreSQL Pods
3. Tạo PVC cho mỗi bản sao bằng cách sử dụng GB lưu trữ được chỉ định
4. Tạo Service để truy cập cơ sở dữ liệu
5. Liên tục theo dõi CR và điều chỉnh nếu có gì sai lệch

Cài đặt Operator (ví dụ: Operator Prometheus):

```
# Hầu hết các toán tử được phân phối dưới dạng biểu đồ Helm hoặc gói YAML
# Cách 1: Qua Helm
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace

# Cách 2: Qua YAML thô (phổ biến cho các tình huống thi CKA)
kubectl apply -f https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/main/bundle.yaml
```

Xác minh Operator đang chạy:

```
# Kiểm tra toán tử Deployment
kubectl get deployment -n monitoring prometheus-operator

# Kiểm tra CRDs nhà điều hành đã đăng ký
kubectl get crd | grep monitoring.coreos.com
# alertmanagerconfigs.monitoring.coreos.com
# prometheuses.monitoring.coreos.com
# prometheusrules.monitoring.coreos.com
# servicemonitors.monitoring.coreos.com

# Tạo ServiceMonitor (phiên bản CRD)
kubectl get servicemonitors -A
```

Các lệnh CRD chính cho kỳ thi CKA:

```
# Liệt kê tất cả CRDs
kubectl get crd

# Nhận định nghĩa CRD
kubectl get crd databases.mycompany.com -o yaml

# Mô tả CRD (hiển thị lược đồ, phiên bản được chấp nhận)
kubectl describe crd databases.mycompany.com

# Xóa CRD (cũng xóa TẤT CẢ các phiên bản của tài nguyên đó)
kubectl delete crd databases.mycompany.com

# Liệt kê tất cả các phiên bản tài nguyên tùy chỉnh trên tất cả namespaces
kubectl get databases -A
```

Cảnh báo: Xóa CRD là có tính hủy diệt. Tất cả Tài nguyên tùy chỉnh thuộc loại đó là vĩnh viễn đã bị xóa khỏi etcd ngay lập tức. Không có giai đoạn xóa mềm hoặc gia hạn.

10.5 Xử lý sự cố CRD và Operator

Lỗi: Không tìm thấy tài nguyên CRD sau khi đăng ký

```
$ kubectl apply -f my-database.yaml
error: no matches for kind "Database" in version "databases.example.com/v1"

# CRD chưa được cài đặt
kubectl get crd | grep databases.example.com
# Trống → CRD chưa được cài đặt

# Trước tiên hãy cài đặt CRD:
kubectl apply -f database-crd.yaml

# Xác minh CRD đã sẵn sàng:
kubectl get crd databases.example.com
# NAME                      CREATED AT
# databases.example.com     2024-01-15T10:00:00Z
kubectl wait --for=condition=Established crd/databases.example.com --timeout=30s
```

Lỗi: Lỗi xác thực CRD

```
$ kubectl apply -f my-database.yaml
The Database "mydb" is invalid:
  spec.version: Unsupported value: "5.6": supported values: "5.7", "8.0"

# CRD có xác thực (thông qua lược đồ OpenAPI) và giá trị của bạn không được phép
# Kiểm tra giá trị nào hợp lệ:
kubectl explain database.spec.version
# Hoặc kiểm tra lược đồ CRD:
kubectl get crd databases.example.com -o jsonpath='{.spec.versions[0].schema}' | python3 -m json.tool | grep -A 5 "version"
```

Operator không đổi chiếu tài nguyên

```
# Kiểm tra xem toán tử có đang chạy không
kubectl get pods -n operator-system
# NAME                      READY   STATUS    RESTARTS
# cơ sở dữ liệu-toán tử-abc12 0/1 Pending 0 ← không chạy!

kubectl describe pod database-operator-abc12 -n operator-system
# Sự kiện: 0/3 nodes có sẵn: Không đủ bộ nhớ

# Nhà điều hành đang chờ xử lý - tăng tài nguyên hoặc giải phóng nodes
# Kiểm tra yêu cầu bộ nhớ của Operator:
kubectl get deployment database-operator -n operator-system \
  -o jsonpath='{.spec.template.spec.containers[0].resources}'

# Sự cố thường gặp của nhà điều hành: Thiếu quyền RBAC
kubectl logs database-operator-abc12 -n operator-system | grep -i "forbidden\|permission\|access"
# Lỗi: không lấy được tài nguyên cơ sở dữ liệu:
# databases.example.com "mydb" bị cấm:
# Người dùng "system:serviceaccount:operator-system:database-operator"
# không thể lấy "cơ sở dữ liệu" tài nguyên trong nhóm API "databases.example.com"
# Khắc phục: Kiểm tra và cập nhật ClusterRole của nhà điều hành
kubectl get clusterrole database-operator-role -o yaml
```


Gỡ lỗi tài nguyên do CRD quản lý

```
# Xem tất cả các phiên bản của CRD
kubectl get databases --all-namespaces

# Nhận trạng thái của phiên bản CRD (toán tử ghi vào .status)
kubectl describe database mydb

# Kiểm tra các điều kiện (Operator nên điền những điều kiện này)
kubectl get database mydb -o jsonpath='{.status.conditions}' | python3 -m json.tool

# Xem các sự kiện tài nguyên CRD
kubectl get events --field-selector involvedObject.kind=Database,involvedObject.name=mydb

# Kiểm tra nhật ký của nhà điều hành để biết tài nguyên cụ thể này
kubectl logs -n operator-system -l control-plane=controller-manager \
  --tail=50 | grep "mydb\|reconcil"
```

Chương 11 - Gateway API: Quản lý lưu lượng Ingress hiện đại

11.1 Tại sao Gateway API tồn tại

Tài nguyên Kubernetes Ingress truyền thống có những hạn chế đáng kể - nó chỉ hỗ trợ HTTP/S, có khả năng định tuyến hạn chế và các nhà cung cấp phải sử dụng các chú thích không chuẩn để thêm các tính năng. **Gateway API** là sản phẩm thay thế hệ tiếp theo, mang lại khả năng biểu đạt phong phú hơn, định tuyến dựa trên vai trò và hỗ trợ cho việc truyền qua TCP, UDP, gRPC và TLS - tất cả đều có tài nguyên Kubernetes gốc.

Gateway API hiện là một phần của **giáo trình giảng dạy chính thức của CKA 2025/2026** thuộc Services và miền Mạng.

11.2 Các tài nguyên cốt lõi của Gateway API

Gateway API giới thiệu ba loại tài nguyên chính:

```
[GatewayClass] → defines the controller/implementation (e.g., nginx, istio, cilium)
    ↓
[Gateway]      → defines a load balancer instance with listeners (ports, protocols)
    ↓
[HTTPRoute]    → defines routing rules (which paths/hosts go to which Services)
```

Sự tách biệt này cung cấp **sự ủy quyền dựa trên vai trò**:

- Người vận hành cụm tạo GatewayClass Và Gateway
- Các nhóm ứng dụng tạo HTTPRoute (không cần quyền truy cập cụm)

11.3 Cài đặt Gateway API CRDs

Gateway API không được cài đặt theo mặc định. Trước tiên hãy cài đặt CRDs của nó:

```
# Cài đặt Gateway API CRDs (kênh chuẩn)
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-install.yaml

# Xác minh CRDs đã được cài đặt
kubectl get crd | grep gateway
# gatewayclasses.gateway.networking.k8s.io
# gateways.gateway.networking.k8s.io
# httproutes.gateway.networking.k8s.io
# grpcroutes.gateway.networking.k8s.io
# referencegrants.gateway.networking.k8s.io
```

11.4 GatewayClass - Xác định việc triển khai

GatewayClass nằm trong phạm vi cụm và xác định bộ điều khiển nào sẽ triển khai Cổng. Nó tương tự như IngressClass của Ingress API cũ.

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx-gateway
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller

kubectl apply -f gatewayclass.yaml
kubectl get gatewayclass
# NAME          CONTROLLER          ACCEPTED
# nginx-gateway gateway.nginx.org/nginx-gateway-controller Đúng
```

11.5 Gateway - Xác định cân bằng tải

Gateway có phạm vi namespace và xác định trình nghe thực tế (cổng, giao thức, cấu hình TLS).

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
  namespace: production
spec:
  gatewayClassName: nginx-gateway      # Tham khảo GatewayClass
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same                      # Chỉ cho phép các tuyến từ namespace này
  - name: https
    port: 443
    protocol: HTTPS
    tls:
      mode: Terminate                  # Chấm dứt TLS tại cổng
      certificateRefs:
        - name: my-tls-secret          # Kubernetes TLS Secret
    allowedRoutes:
      namespaces:
        from: All                      # Cho phép các tuyến đường từ bất kỳ namespace nào

kubectl apply -f gateway.yaml

kubectl get gateway -n production
# NAME          CLASS          ADDRESS          PROGRAMMED    AGE
# my-gateway    nginx-gateway  203.0.113.5      Đúng         2m
```

11.6 HTTPRoute - Xác định quy tắc định tuyến

HTTPRoute xác định cách lưu lượng truy cập HTTP từ Gateway được định tuyến đến Services.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-route
  namespace: production
spec:
  parentRefs:
  - name: my-gateway                  # Gắn vào Gateway ở trên
    namespace: production
  hostnames:
  - "myapp.example.com"              # Khớp tên máy chủ này
  rules:
  - matches:
```

```

- path:
  type: PathPrefix
  value: /api # Khớp các đường dẫn/api/*
backendRefs:
- name: api-service
  port: 8080
  weight: 100
- matches:
  - path:
    type: PathPrefix
    value: / # Mặc định: khớp với tất cả các đường dẫn khác
  backendRefs:
  - name: frontend-service
    port: 3000

kubectl apply -f httproute.yaml

kubectl get httproute -n production
kubectl describe httproute my-app-route -n production

```

11.7 Chia lưu lượng bằng HTTPRoute (canary deployment)

```

rules:
- matches:
  - path:
    type: PathPrefix
    value: /
  backendRefs:
  - name: app-stable # 90% lưu lượng truy cập
    port: 8080
    weight: 90
  - name: app-canary # 10% lưu lượng truy cập vào phiên bản mới
    port: 8080
    weight: 10

```

Điều này có tác dụng gì: Phân phối lưu lượng truy cập dựa trên trọng số. Với trọng lượng 90/10, khoảng 90% requests chuyển đến app-stable và 10% chuyển đến app-canary. Điều này cho phép triển khai liên tục mà không cần thay đổi cấu hình DNS hoặc Service.

11.8 So sánh Ingress và Gateway API

tính năng	Ingress	Gateway API
Giao thức	Chỉ HTTP/S	HTTP, HTTPS, TCP, UDP, gRPC, TLS
Logic định tuyến	Bị giới hạn (đường dẫn, máy chủ)	Đường dẫn, máy chủ, tiêu đề, phương thức, thông số truy vấn
Phân chia lưu lượng truy cập	Thông qua chú thích (không chuẩn)	Phân chia dựa trên trọng lượng bản địa
Tách Role	Không có - tài nguyên duy nhất	GatewayClass (cụm), Gateway (hoạt động), HTTPRoute (nhà phát triển)
Báo cáo trạng thái	Tối thiểu	Điều kiện trạng thái có cấu trúc phong phú
Tính ổn định	Ingress v1 ổn định từ Kubernetes 1.19	GatewayClass, Gateway và HTTPRoute v1 ổn định từ Gateway API 1.0; phiên bản độc lập với Kubernetes

Nguồn: [Ingress và Gateway API 1.0](#).

11.9 Khắc phục sự cố Gateway API

Lỗi: Gateway API CRDs chưa được cài đặt

```
$ kubectl apply -f gateway.yaml
error: resource mapping not found for name: "my-gateway"
      namespace: "" from "gateway.yaml":
      no matches for kind "Gateway" in version "gateway.networking.k8s.io/v1"

# Cài đặt Gateway API CRDs:
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/standard-install.yaml

# Xác minh:
kubectl get crd | grep gateway.networking.k8s.io
# gatewayclasses.gateway.networking.k8s.io
# gateways.gateway.networking.k8s.io
# httproutes.gateway.networking.k8s.io
```

Lỗi: GatewayClass không được chấp nhận

```
$ kubectl get gatewayclass
NAME          CONTROLLER          ACCEPTED  AGE
nginx         k8s.nginx.org/nginx-class  Unknown  2m

kubectl describe gatewayclass nginx
# Điều kiện:
#   Kiểu: Được chấp nhận
#   Tình trạng: Không rõ
#   Lý do: Pending
#   Thông báo: Đang chờ bộ điều khiển điều chỉnh

# Bộ điều khiển GatewayClass (bộ điều khiển xâm nhập) không chạy
kubectl get pods -n nginx-gateway
# Nếu trống: cài đặt bộ điều khiển cổng nginx
helm install nginx-gateway oci://ghcr.io/nginx/charts/nginx-gateway-fabric \
  --namespace nginx-gateway --create-namespace
```

HTTPRoute không định tuyến lưu lượng

```
$ curl http://my-app.example.com/api
curl: (7) Failed to connect to my-app.example.com port 80: Connection refused

# Bước 1: Kiểm tra Gateway đã sẵn sàng
kubectl get gateway my-gateway
# NAME          CLASS    ADDRESS          PROGRAMMED
# my-gateway    nginx   10.0.0.100    Đúng ← Đã gán địa chỉ, Đã lập trình=True

# Bước 2: Kiểm tra HTTPRoute đã được đính kèm
kubectl get httproute my-route
kubectl describe httproute my-route | grep -A 10 "Status:"
# Cha mẹ:
#   Phụ huynh: Kind=Gateway, Name=my-gateway
#   Điều kiện:
#     Đã chấp nhận: Đúng
#     Đã giải quyếtRefs: Đúng ← Sai có nghĩa là không tìm thấy Service phụ trợ

# Bước 3: Nếu ResolvedRefs=False - sự cố dịch vụ phụ trợ
kubectl describe httproute my-route | grep -A 10 "ResolvedRefs"
# Thông báo: BackendNotFound: Không tìm thấy "dịch vụ api" Service trong "sản xuất" namespace

# Bước 4: Xác minh dịch vụ phụ trợ tồn tại và có điểm cuối
kubectl get svc api-service -n production
kubectl get endpoints api-service -n production

# Bước 5: Kiểm tra phần phụ trợ trực tiếp bỏ qua Gateway
kubectl port-forward svc/api-service 8080:80 -n production &
curl http://localhost:8080/api
```

Chương 12 - Kubernetes Services: ClusterIP, NodePort, LoadBalancer và hơn thế nữa

12.1 Service là gì và tại sao nó tồn tại?

Pods chỉ tồn tại trong thời gian ngắn - chúng nhận được địa chỉ IP mới mỗi khi khởi động lại hoặc lên lịch lại. Nếu giao diện Pod của bạn mã hóa cứng IP của phụ trợ Pod, nó sẽ bị hỏng mỗi khi Pod phụ trợ khởi động lại. **Services** giải quyết vấn đề này bằng cách cung cấp **IP ảo ổn định (ClusterIP)** được hỗ trợ bởi bộ Pods động, được phát hiện thông qua bộ chọn nhãn.

```
Client Pod (10.244.1.5)
|
▼ curl http://10.96.45.23:80 (ClusterIP - stable)
[Service: backend-svc]
|
├── Pod-A (10.244.1.10:8080) - healthy
├── Pod-B (10.244.2.11:8080) - healthy
└── Pod-C (10.244.1.12:8080) - dead (not in Endpoints)
```

kube-proxy trên mọi node duy trì iptables/IPVS quy định rằng DNAT lưu lượng truy cập từ ClusterIP:port đến một trong các IP Pod lành mạnh. Bộ điều khiển Endpoints giúp cập nhật danh sách IP Pod.

12.2 Các loại Service

Kubernetes có bốn loại Service, mỗi loại có thêm các khả năng:

```
ClusterIP (default) → accessible only inside cluster
↑
NodePort             → exposes on every node's IP:port (30000-32767)
↑
LoadBalancer         → provisions a cloud load balancer in front of NodePort
↑
ExternalName         → CNAME alias to an external DNS name (no proxying)
```

12.3 ClusterIP Service - Lưu lượng truy cập nội bộ

Loại mặc định. Tạo một IP ảo chỉ có thể truy cập được trong cụm.

```
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: production
spec:
  type: ClusterIP          # mặc định nếu không được chỉ định
  selector:
    app: backend           # Các tuyến tới Pods có nhãn này
    tier: api
  ports:
    - name: http
      protocol: TCP
      port: 80             # Cổng Service nghe trên
      targetPort: 8080     # Cổng trên Pod (có thể là tên hoặc số)
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
  sessionAffinity: None    # hoặc ClientIP (phiên cố định theo IP của khách hàng)

# Tạo một cách bắt buộc
kubectl create service clusterip backend-svc --tcp=80:8080
```

```
# Hoặc lộ Deployment
kubectl expose deployment backend --port=80 --target-port=8080 --name=backend-svc

# Xác minh
kubectl get svc backend-svc
# NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
# backend-svc   ClusterIP     10.96.45.23   <none>         80/TCP     1m

# Kiểm tra từ bên trong cụm
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://backend-svc.production.svc.cluster.local
```

12.4 NodePort Service - Truy cập bên ngoài thông qua IP nút

Hiển thị Service trên mỗi IP của node tại một cổng cụ thể (30000-32767).

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - port: 80          # ClusterIP port (internal)
      targetPort: 3000  # Cổng Pod
      nodePort: 31080   # Cổng nút (tùy chọn: bỏ qua để tự động gán trong 30000-32767)
      protocol: TCP

# Tạo một cách bắt buộc
kubectl create service nodeport frontend-svc --tcp=80:3000 --node-port=31080

# Tìm nodePort được gán tự động
kubectl get svc frontend-svc -o jsonpath='{.spec.ports[0].nodePort}'

# Truy cập dịch vụ từ cụm bên ngoài
curl http://<any-node-ip>:31080

# Nhận tất cả IP node
kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="ExternalIP")].address}'
# hoặc InternalIP nếu không có bên ngoài:
kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}'
```

12.5 LoadBalancer Service - Cân bằng tải được cung cấp trên nền tảng đám mây

Được xây dựng trên NodePort và yêu cầu thêm nhà cung cấp đám mây cung cấp bộ cân bằng tải thực sự.

```
apiVersion: v1
kind: Service
metadata:
  name: web-lb
  annotations:
    # Chú thích dành riêng cho đám mây:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"          # AWS: Mạng LB
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
    service.beta.kubernetes.io/azure-load-balancer-internal: "false" # Azure: bên ngoài
    cloud.google.com/load-balancer-type: "External"                 # GKE
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
  loadBalancerSourceRanges:      # Tùy chọn: hạn chế IP được phép tiếp cận LB
```

- 10.0.0.0/8
- 203.0.113.0/24

Theo dõi cho đến khi IP bên ngoài được chỉ định (cung cấp đám mây mất ~ 1-2 phút)

kubectl get svc web-lb -w

```
# NAME      TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
# web-lb    LoadBalancer 10.96.100.1      <pending>         80:31234/TCP     10s
# web-lb    LoadBalancer 10.96.100.1      34.120.45.67     80:31234/TCP     75s
```

Đối với các cụm tại chỗ (không có đám mây), hãy sử dụng MetalLB để nhận hỗ trợ LoadBalancer:

kubectl apply -f <https://raw.githubusercontent.com/metallb/metallb/v0.14.3/config/manifests/metallb-native.yaml>

12.6 ExternalName Service: bí danh DNS CNAME

Được sử dụng để tích hợp các dịch vụ trong cụm với các hệ thống bên ngoài. Tạo CNAME, không cần ủy quyền.

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
  namespace: production
spec:
  type: ExternalName
  externalName: mydb.database.windows.net # Tên DNS thành bí danh
  # Không Cần bộ chọn

# Pods hiện có thể tiếp cận DB bên ngoài bằng DNS bên trong:
# kubectl thực thi my-pod -- nslookup external-db.production.svc.cluster.local
# → CNAME → mydb.database.windows.net

# Hữu ích cho: di chuyển từ dịch vụ bên ngoài sang dịch vụ nội bộ (chỉ cần thay đổi externalName thành bộ chọn)
# hoặc để đặt tên nội bộ ổn định cho dịch vụ SaaS bên ngoài
```

12.7 Headless Service: khám phá Pod trực tiếp

Đôi khi bạn không muốn cân bằng tải - bạn muốn khám phá trực tiếp tất cả các IP Pod. StatefulSets sử dụng các dịch vụ không đầu cho các tên Pod DNS ổn định.

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None # ← Làm cho nó không có đầu (không có VIP, không có cân bằng tải)
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432

# Với dịch vụ không đầu, DNS trả về TẤT CẢ IP Pod:
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup postgres-headless.default.svc.cluster.local
# Máy chủ: 10.96.0.10
# Địa chỉ 1: 10.244.1.10 postgres-0.postgres-headless.default.svc.cluster.local
# Địa chỉ 2: 10.244.2.15 postgres-1.postgres-headless.default.svc.cluster.local
# Địa chỉ 3: 10.244.3.20 postgres-2.postgres-headless.default.svc.cluster.local

# Đối với StatefulSets, mỗi Pod có tên DNS ổn định:
# <pod-name>.<service-name>.<namespace>.svc.cluster.local
# postgres-0.postgres-headless.default.svc.cluster.local
```

12.8 Endpoints và EndpointSlices

Services không theo dõi trực tiếp Pods - họ sử dụng Endpoints (và trong K8 hiện đại, EndpointSlices):

```
# Endpoints là auto-created/updated bởi bộ điều khiển điểm cuối
kubectl get endpoints backend-svc
# NAME          ENDPOINTS          AGE
# phụ trợ-svc 10.244.1.10:8080,10.244.2.11:8080 5m

# Nếu Endpoints hiển thị <none> thì bộ chọn không khớp với Pods:
kubectl get pods -l app=backend # Nên trả về pods
kubectl describe svc backend-svc | grep Selector # Kiểm tra bộ chọn

# EndpointSlices (K8s 1.21+ mặc định, có khả năng mở rộng cao hơn Endpoints)
kubectl get endpointslices -l kubernetes.io/service-name=backend-svc

# Tạo thủ công Endpoint để trỏ Service tới IP bên ngoài:
apiVersion: v1
kind: Endpoints
metadata:
  name: external-legacy-db # Phải khớp với tên Service
subsets:
- addresses:
  - ip: 192.168.100.50 # IP máy chủ bên ngoài
  ports:
  - port: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: external-legacy-db
spec:
  ports:
  - port: 5432
# Không có bộ chọn - sử dụng Endpoints thủ công ở trên
```

12.9 Xử lý sự cố Service

Vấn đề: Service không có điểm cuối (ĐIỂM CUỐI trống)

```
kubectl get endpoints my-svc
# NAME          ENDPOINTS          AGE
# my-svc <none> 5m

# Bước 1: Kiểm tra bộ chọn Service
kubectl describe svc my-svc | grep Selector
# Bộ chọn: ứng dụng=backend,version=v2

# Bước 2: Kiểm tra xem có Pods nào khớp với bộ chọn đó không
kubectl get pods -l app=backend,version=v2
# Không tìm thấy tài nguyên ← Nhân Pod không bao gồm version=v2

# Bước 3: Kiểm tra nhãn Pod
kubectl get pods -l app=backend --show-labels
# NAME          READY  STATUS   LABELS
# backend-0     1/1    Running  app=backend,version=v1 ← version=v1, not v2!

# Tùy chọn sửa lỗi:
# Trả lời: Cập nhật bộ chọn Service để khớp với nhãn Pod
kubectl patch svc my-svc -p '{"spec":{"selector":{"app":"backend"}}}'
# B: Thêm nhãn phiên bản vào Pod (nếu cố ý)
kubectl label pod backend-0 version=v2
```


Vấn đề: Service có thể truy cập được bằng ClusterIP nhưng không thể truy cập bằng tên DNS

```
# Test by ClusterIP
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://10.96.45.23:80 # Tác phẩm

# Kiểm tra bởi DNS
kubectl run test --image=busybox:1.35 --rm -it --restart=Never -- \
  wget -qO- http://my-svc.default.svc.cluster.local # Thất bại

# Sự cố CoreDNS - kiểm tra CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns | grep -i error

# Kiểm tra cấu hình DNS của Pod
kubectl exec some-pod -- cat /etc/resolv.conf
# tìm kiếm default.svc.cluster.local svc.cluster.local cluster.local
# nameserver 10.96.0.10 ← should be CoreDNS ClusterIP
kubectl get svc kube-dns -n kube-system # Verify ClusterIP matches
```

Sự cố: NodePort không thể truy cập được từ bên ngoài

```
# Check if the NodePort is listening
ssh <node-ip>
sudo ss -tlnp | grep 31080

# Kiểm tra quy tắc tường lửa
sudo iptables -t nat -L | grep 31080
# Sẽ hiển thị quy tắc DNAT

# Tường lửa đám mây có thể đang chặn cổng
# AWS: Kiểm tra quy tắc gửi đến của Nhóm bảo mật cho cổng 31080
# GCP: Kiểm tra quy tắc tường lửa: danh sách quy tắc tường lửa tính toán gcloud
# Azure: Kiểm tra quy tắc gửi đến NSG

# Kiểm tra từ bên trong cụm trước
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://<node-ip>:31080
```

Sự cố: LoadBalancer bị kẹt trong <pending>

```
kubectl get svc my-lb
# NAME      TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
# my-lb    LoadBalancer 10.96.1.1     <pending>      80:30000/TCP

# Không có nhà cung cấp đám mây nào cung cấp IP bên ngoài
# Giải pháp:
# 1. Sử dụng MetalLB (tại chỗ)
# 2. Use NodePort instead
# 3. Sử dụng cổng chuyển tiếp để thử nghiệm

kubectl describe svc my-lb | grep -A 5 "Events:"
# Không có sự kiện nào nếu không có bộ điều khiển đám mây nào đang chạy
# Kiểm tra trình quản lý bộ điều khiển đám mây:
kubectl get pods -n kube-system | grep cloud-controller
```

12.10 Tra cứu DNS của Service

Tất cả Services đều nhận được các mục DNS ở định dạng:

```
<service-name>.<namespace>.svc.<cluster-domain>
```

Examples:

```
backend.production.svc.cluster.local
postgres.database.svc.cluster.local
kubernetes.default.svc.cluster.local ← The API server itself
```

Short forms (from within same namespace):

```
backend           → works from 'production' namespace
backend.production → works from any namespace
backend.production.svc → works from any namespace
```

```
backend.production.svc.cluster.local → fully qualified
```

SRV records for named ports:

```
_http._tcp.backend.production.svc.cluster.local  
→ returns port number and Pod IPs
```

Từ bên trong pod, hãy kiểm tra tất cả các dạng:

```
kubectl exec my-pod -n production -- sh -c "
```

```
nslookup backend # cùng loại namespace ngắn
```

```
nslookup backend.production # chéo namespace ngắn
```

```
nslookup backend.production.svc.cluster.local # FQDN
```

```
nslookup _http._tcp.backend.production.svc.cluster.local # SRV
```

```
"
```

Chương 13 - Ingress: Định tuyến HTTP và chấm dứt TLS

13.1 Ingress là gì?

Ingress là tài nguyên Kubernetes hiển thị các tuyến HTTP và HTTPS từ bên ngoài cụm đến Services bên trong cụm. Nó là một đối tượng API xác định các quy tắc định tuyến; việc định tuyến thực tế được thực hiện bởi **Bộ điều khiển Ingress** - Pod đang chạy trong cụm.

```
Internet  
|  
▼ HTTPS (443)  
[Ingress Controller Pod] ← reads Ingress rules  
|  
├─ /api/* → api-service:8080  
├─ /web/* → web-service:80  
└─ /static/* → cdn-service:3000
```

Không có Ingress: mỗi Service cần có LoadBalancer riêng (đặt tiền trên đám mây). Với Ingress: một LoadBalancer định tuyến tất cả lưu lượng truy cập HTTP dựa trên quy tắc host/path.

13.2 Cài đặt bộ điều khiển Ingress

Kubernetes không đi kèm bộ điều khiển xâm nhập. Bạn phải cài đặt một:

```
# Bộ điều khiển NGINX Ingress (phổ biến nhất)  
# Ví dụ lịch sử, không triển khai controller đã ngừng bảo trì lên cụm mới.  
# kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.5/deploy/static/provider/cloud/deploy.yaml
```

```
# Xác minh nó đang chạy
```

```
kubectl get pods -n ingress-nginx
```

# NAME	READY	STATUS	RESTARTS	AGE
# ingress-nginx-controller-6dd4c6d8f-xk2p9	1/1	Running	0	2m

```
# Nhận IP bên ngoài của Service của bộ điều khiển xâm nhập
```

```
kubectl get svc -n ingress-nginx ingress-nginx-controller
```

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
# bộ điều khiển ingress-nginx LoadBalancer	LoadBalancer	10.96.50.1	34.120.55.99	80:32080/TCP,443:32443/TCP

```
# Đối với minikube:
```

```
minikube addons enable ingress
```

```
# Đối với bare-metal/on-Premise (không có LB trên nền tảng đám mây):
```

```
# Ví dụ lịch sử, không triển khai controller đã ngừng bảo trì lên cụm mới.
```

```
# kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.5/deploy/static/provider/bare-metal/deploy.yaml
```

```
# Then use NodePort instead of LoadBalancer external IP
```

13.3 Ingress cơ bản cho một Service

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: / # Tách tiền tố đường dẫn trước khi chuyển tiếp
spec:
  ingressClassName: nginx # Bộ điều khiển nào xử lý việc này (bắt buộc trong K8s 1.22+)
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix # Tiền tố, Chính xác hoặc Triển khai cụ thể
            backend:
              service:
                name: frontend-svc
                port:
                  number: 80

kubectl apply -f ingress.yaml
kubectl get ingress simple-ingress
# NAME          CLASS    HOSTS          ADDRESS          PORTS    AGE
# nginx xâm nhập đơn giản myapp.example.com 34.120.55.99 80 1m

# Điểm DNS (hoặc /etc/hosts để kiểm tra):
echo "34.120.55.99 myapp.example.com" | sudo tee -a /etc/hosts
curl http://myapp.example.com
```

13.4 Định tuyến theo đường dẫn

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-based-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
  ingressClassName: nginx
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /api/v1
        pathType: Prefix
        backend:
          service:
            name: api-v1-svc
            port:
              number: 8080
      - path: /api/v2
        pathType: Prefix
        backend:
          service:
            name: api-v2-svc
            port:
              number: 8080
      - path: /static
        pathType: Prefix
        backend:
          service:
            name: static-assets-svc
            port:
              number: 80
      - path: /
        # Tổng hợp mặc định
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
```

13.5 Định tuyến theo hostname

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-host-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: www.example.com      # Giao diện sản xuất
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: prod-frontend
                port:
                  number: 80
    - host: api.example.com      # Dịch vụ API
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 8080
    - host: staging.example.com  # Môi trường dàn dựng
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: staging-frontend
                port:
                  number: 80
```

13.6 Chấm dứt TLS

```
# Bước 1: Tạo TLS Secret
kubectl create secret tls tls-secret \
  --cert=tls.crt \
  --key=tls.key \
  --namespace=production

# Hoặc tạo chứng chỉ tự ký để thử nghiệm:
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=myapp.example.com/O=myapp"
kubectl create secret tls tls-secret --cert=tls.crt --key=tls.key

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true" # Chuyển hướng HTTP → HTTPS
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - myapp.example.com
        - api.example.com
      secretName: tls-secret # Chứa tls.crt và tls.key
  rules:
    - host: myapp.example.com
      http:
```

```

    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: frontend-svc
          port:
            number: 80
- host: api.example.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: api-svc
          port:
            number: 8080

```

trình quản lý chứng chỉ: TLS tự động với Let's Encrypt

```

# Cài đặt trình quản lý chứng chỉ
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.13.0/cert-manager.yaml

```

```

# Tạo ClusterIssuer cho Let's Encrypt
cat << 'YAML' | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
    - http01:
        ingress:
          class: nginx

```

YAML

```

# Ingress với TLS tự động thông qua trình quản lý chứng chỉ:
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: auto-tls-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod # Tự động cung cấp TLS
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - myapp.example.com
    secretName: myapp-tls # người quản lý chứng chỉ sẽ tạo Secret này
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80

```

13.7 Chú thích Ingress phổ biến (NGINX)

```
metadata:
  annotations:
    # Viết lại/thao tác URL
    nginx.ingress.kubernetes.io/rewrite-target: /$2          # Tiền tố đường dẫn dài
    nginx.ingress.kubernetes.io/use-regex: "true"           # Kích hoạt biểu thức chính quy trong đường dẫn

    # Giới hạn tỷ lệ
    nginx.ingress.kubernetes.io/limit-rps: "10"              # 10 req/s mỗi IP
    nginx.ingress.kubernetes.io/limit-connections: "5"       # 5 kết nối đồng thời

    # Hết giờ
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "5"   # hết thời gian kết nối 5s
    nginx.ingress.kubernetes.io/proxy-send-timeout: "60"     # Hết thời gian gửi 60 giây
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"     # Hết thời gian đọc 60 giây

    # CORS
    nginx.ingress.kubernetes.io/enable-cors: "true"
    nginx.ingress.kubernetes.io/cors-allow-origin: "https://myapp.example.com"

    # Xác thực
    nginx.ingress.kubernetes.io/auth-type: basic
    nginx.ingress.kubernetes.io/auth-secret: basic-auth-secret
    nginx.ingress.kubernetes.io/auth-realm: "Restricted Area"

    # Tiêu đề tùy chỉnh
    nginx.ingress.kubernetes.io/configuration-snippet: |
      more_set_headers "X-Frame-Options: SAMEORIGIN";
      more_set_headers "X-Content-Type-Options: nosniff";

    # Giao thức phụ trợ
    nginx.ingress.kubernetes.io/backend-protocol: "GRPC"     # Dành cho chương trình phụ trợ gRPC
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"    # Đối với chương trình phụ trợ mTLS

    # Mối quan hệ phiên
    nginx.ingress.kubernetes.io/affinity: "cookie"
    nginx.ingress.kubernetes.io/session-cookie-name: "SERVERID"
```

13.8 Khắc phục sự cố Ingress

Sự cố: Không tìm thấy 404 từ Ingress

```
# Bước 1: Kiểm tra Ingress đã được tạo và có địa chỉ chưa
kubectl get ingress my-ingress
# NAME          CLASS    HOSTS          ADDRESS          PORTS    AGE
# my-ingress    nginx    myapp.example.com <none> 80 5m
# ĐỊA CHỈ trống → bộ điều khiển xâm nhập không nhận được

# Bước 2: Kiểm tra nhật ký bộ điều khiển xâm nhập
kubectl logs -n ingress-nginx \
  $(kubectl get pods -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx -o jsonpath='{.items[0].metadata.name}') \
  --tail=50

# Bước 3: Kiểm tra ingressClassName
kubectl get ingress my-ingress -o jsonpath='{.spec.ingressClassName}'
# Nếu trống và bộ điều khiển yêu cầu:
kubectl patch ingress my-ingress -p '{"spec":{"ingressClassName":"nginx"}}'

# Bước 4: Xác minh dịch vụ phụ trợ tồn tại và có điểm cuối
kubectl get svc frontend-svc
kubectl get endpoints frontend-svc

** Vấn đề: 502 Gateway lỗi **

# NGINX có thể tiếp cận phần phụ trợ nhưng phần phụ trợ đang trả về lỗi
# Bước 1: Kiểm tra trực tiếp phần phụ trợ
kubectl port-forward svc/frontend-svc 8080:80
curl http://localhost:8080
```

```
# Bước 2: Kiểm tra xem targetPort có khớp với nội dung ứng dụng nghe hay không
kubectl get svc frontend-svc -o jsonpath='{.spec.ports[0].targetPort}'
kubectl exec frontend-pod -- ss -tlnp # Ứng dụng đang ở cổng nào?
```

```
# Bước 3: Kiểm tra cấu hình bộ điều khiển xâm nhập
kubectl get ingress my-ingress -o yaml
# Đảm bảo đường dẫn và phụ trợ phù hợp
```

Vấn đề: Lỗi chứng chỉ SSL

```
# Kiểm tra chứng chỉ có hợp lệ
kubectl get secret tls-secret -o jsonpath='{.data.tls.crt}' | base64 -d | \
openssl x509 -text -noout | grep -E "Subject:|DNS:|Not After"

# Kiểm tra người quản lý chứng chỉ đã cấp chứng chỉ
kubectl get certificates -n production
kubectl describe certificate myapp-tls -n production

# thử thách quản lý chứng chỉ bị kẹt?
kubectl get challenges -A
kubectl describe challenge <challenge-name> -n production
# Sự cố thường gặp: Thử thách HTTP01 không thành công nếu Ingress không thể truy cập công khai
# Sử dụng thử thách DNS01 thay thế cho các cụm riêng tư
```

Chương 14 - NetworkPolicy: Kiểm soát lưu lượng và phân đoạn vi mô

14.1 Mặc định: Không cách ly mạng

Theo mặc định, Kubernetes cho phép tất cả lưu lượng truy cập Pod-to-Pod. Bất kỳ Pod nào cũng có thể tiếp cận bất kỳ Pod nào khác trong bất kỳ namespace nào. NetworkPolicies cho phép bạn hạn chế điều này - nhưng chỉ khi plugin CNI của bạn hỗ trợ chúng (Calico, Cilium, Weave làm; Flannel KHÔNG).

```
# Kiểm tra xem CNI của bạn có hỗ trợ NetworkPolicy không
# Kiểm tra bằng cách tạo từ chối tất cả và xem liệu lưu lượng truy cập có bị chặn không
# Nếu lưu lượng truy cập vẫn chảy sau khi từ chối tất cả, CNI của bạn không hỗ trợ nó
kubectl get pods -n kube-system | grep -E "calico|cilium|weave|canal"
```

NetworkPolicy là **phụ gia** - Pods không áp dụng NetworkPolicy có lưu lượng truy cập không hạn chế. Khi bất kỳ NetworkPolicy nào chọn Pod, tất cả lưu lượng truy cập không được cho phép rõ ràng sẽ bị từ chối.

14.2 Từ chối tất cả: Đường cơ sở bảo mật

Luôn bắt đầu bằng mặc định từ chối tất cả, sau đó thêm quy tắc cho phép rõ ràng:

```
# Từ chối tất cả sự xâm nhập vào namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: production
spec:
  podSelector: {} # {} = chọn TẤT CẢ pods trong namespace
  policyTypes:
    - Ingress # Hạn chế lưu lượng truy cập vào
  # Không có quy tắc xâm nhập = từ chối tất cả xâm nhập
---
# Từ chối tất cả các lối ra từ namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
```



```

    namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Egress
  # Không có quy tắc đi ra = từ chối tất cả đi ra
  ---
  # Từ chối tất cả theo cả hai hướng (hạn chế nhất)
  apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: default-deny-all
    namespace: production
  spec:
    podSelector: {}
    policyTypes:
    - Ingress
    - Egress

```

14.3 Các mẫu cho phép cụ thể

Cho phép lưu lượng truy cập từ Pods cụ thể

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend          # Chính sách này áp dụng cho phụ trợ Pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend      # Chỉ frontend Pods mới có thể gửi xâm nhập vào backend
    ports:
    - protocol: TCP
      port: 8080

```

Cho phép lưu lượng truy cập từ Namespace cụ thể

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: production
spec:
  podSelector: {}          # Tất cả pods đang được sản xuất
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: monitoring      # Chỉ từ giám sát namespace
    ports:
    - port: 9090          # Cổng tạo Prometheus
    - port: 9091

```

Cho phép lưu lượng truy cập từ Namespace VÀ Pod cụ thể (VÀ logic)

```
# QUAN TRỌNG: from[] với cả hai bộ chọn trong CÙNG danh sách item = AND (cả hai phải khớp)
# from[] với các bộ chọn trong danh sách RIÊNG các mục = OR (có thể khớp)

# VÀ: phải từ giám sát namespace VÀ có nhãn role=prometheus
ingress:
- from:
  - namespaceSelector:      # Một mục có CẢ HAI bộ chọn = AND
    matchLabels:
      name: monitoring
  podSelector:
    matchLabels:
      role: prometheus

# HOẶC: từ giám sát namespace HOẶC bất kỳ pod nào có nhãn role=prometheus
ingress:
- from:
  - namespaceSelector:      # Các mục riêng biệt = HOẶC
    matchLabels:
      name: monitoring
  - podSelector:
    matchLabels:
      role: prometheus
```

Cho phép lưu lượng truy cập từ IP bên ngoài

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-external-cidr
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: public-api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - ipBlock:
        cidr: 0.0.0.0/0      # Tất cả các IP bên ngoài
        except:
        - 10.0.0.0/8         # Ngoại trừ mạng nội bộ
        - 172.16.0.0/12
        - 192.168.0.0/16
    ports:
    - port: 443
```

14.4 Chính sách lưu lượng đi ra (egress)

```
# Chỉ cho phép pods tiếp cận DNS + các dịch vụ cụ thể
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-egress-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Egress
  egress:
    # Cho phép DNS (bắt buộc để phân giải tên)
    - to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kube-system
        podSelector:
          matchLabels:
            k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # Cho phép tiếp cận cơ sở dữ liệu
    - to:
      - podSelector:
          matchLabels:
            app: postgres
      ports:
        - port: 5432
    # Cho phép tiếp cận API bên ngoài
    - to:
      - ipBlock:
          cidr: 142.250.0.0/15    # Dải IP API của Google
      ports:
        - port: 443
```

14.5 Mẫu NetworkPolicy trong thực tế

Ứng dụng ba tầng (Giao diện người dùng → Phần phụ trợ → Cơ sở dữ liệu)

Cấp 1: Frontend có thể nhận từ bất cứ đâu, chỉ có thể gửi đến backend

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: frontend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - {} # Cho phép tất cả các hoạt động xâm nhập (đối mặt với công chúng)
  egress:
  - to:
    - podSelector:
        matchLabels:
          tier: backend
      ports:
      - port: 8080
  - to: # Luôn cho phép DNS
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
      ports:
      - port: 53
        protocol: UDP
---
```

Cấp 2: Chỉ phụ trợ từ giao diện người dùng, chỉ cho cơ sở dữ liệu

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: frontend
      ports:
      - port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          tier: database
      ports:
      - port: 5432
  - to: # DNS
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
      ports:
      - port: 53
        protocol: UDP
---
```

Cấp 3: Cơ sở dữ liệu chỉ từ phụ trợ

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: database-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              tier: backend
      ports:
        - port: 5432
```

14.6 Kiểm tra NetworkPolicy

```
# Thiết lập môi trường thử nghiệm
kubectl create namespace np-test
kubectl label namespace np-test kubernetes.io/metadata.name=np-test

# Triển khai thử nghiệm pods
kubectl run server --image=nginx --labels="app=server" -n np-test
kubectl run client --image=curlimages/curl --labels="app=client" -n np-test \
  --command -- sleep 3600

# Nhận IP máy chủ
SERVER_IP=$(kubectl get pod server -n np-test -o jsonpath='{.status.podIP}')

# Kiểm tra TRƯỚC bất kỳ chính sách nào (nên hoạt động)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# Nền trả về nginx HTML

# Áp dụng từ chối tất cả
kubectl apply -f - << 'YAML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: np-test
spec:
  podSelector: {}
  policyTypes:
    - Ingress
YAML

# Kiểm tra SAU KHI từ chối tất cả (nên thất bại)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# Curl: (28) Đã hết thời gian kết nối ← bị chặn!

# Áp dụng quy tắc cho phép
kubectl apply -f - << 'YAML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-client-to-server
  namespace: np-test
spec:
  podSelector:
    matchLabels:
      app: server
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: client
      ports:
        - port: 80
YAML

# Kiểm tra sau quy tắc cho phép (sẽ hoạt động trở lại)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# Trả về nginx HTML ← được phép!

# Dọn dẹp
kubectl delete namespace np-test
```

14.7 Khắc phục sự cố NetworkPolicies

Vấn đề: Lưu lượng truy cập bị chặn khi đáng lẽ phải cho phép

```
# Liệt kê tất cả NetworkPolicies trong namespace
kubectl get networkpolicies -n production -o yaml

# Kiểm tra xem chính sách có đang chọn Pod mà bạn nghĩ không
kubectl get networkpolicies -n production -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for pol in data['items']:
    sel = pol['spec'].get('podSelector', {}).get('matchLabels', {})
    print(f"Policy: {pol['metadata']['name']}, selector: {sel}")
"

# Kiểm tra bộ chọn chính sách khớp nhãn Pod
kubectl get pod my-pod -n production --show-labels

# Xác minh nhãn namespace (đối với namespaceSelector)
kubectl get namespace production --show-labels
# Tìm kiếm: kubernetes.io/metadata.name=sản xuất (tự động cài đặt trong K8s 1.21+)
# Nếu thiếu (K8 cũ hơn):
kubectl label namespace production kubernetes.io/metadata.name=production

# Lỗi thường gặp: AND vs OR nhầm lẫn
# Đây KHÔNG giống nhau:
# từ: # HOẶC: bộ chọn phù hợp
# - podSelector: {app: frontend}
# - namespaceSelector: {name: giám sát}
#
# từ: # VÀ: cả hai phải khớp
# - podSelector: {app: frontend}
#   namespaceSelector: {name: giám sát}
```

Sự cố: DNS ngừng hoạt động sau khi áp dụng chính sách đầu ra

```
# Đây là lỗi NetworkPolicy phổ biến số 1!
# Nếu bạn hạn chế đi ra, bạn PHẢI cho phép rõ ràng DNS trên cổng UDP/TCP 53

# Triệu chứng: Pod có thể tiếp cận IP trực tiếp nhưng DNS không thành công
kubectl exec my-pod -- nslookup google.com      # Không thành công (DNS bị chặn)
kubectl exec my-pod -- curl http://8.8.8.8      # Có thể hoạt động nếu cho phép đầu ra IP

# Khắc phục: thêm quy tắc đầu ra DNS vào mọi chính sách đầu ra
egress:
- to:
  - namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
```

Chương 15 - RBAC: Kiểm soát truy cập dựa trên Role

15.1 Mẫu RBAC

Kubernetes RBAC trả lời một câu hỏi: "**Chủ thể X có thể thực hiện hành động Y trên tài nguyên Z không?**"

Subject (who)	Verb (what)	Resource (which object)
User	get	Pods
Group	list	deployments
ServiceAccount	create	secrets
	update	configmaps
	patch	services
	delete	namespaces
	watch	nodes
	*	* (wildcard)

Bốn đối tượng RBAC:

Role → permissions in ONE namespace
ClusterRole → permissions cluster-wide (or reusable in any namespace)
RoleBinding → grants a Role or ClusterRole to subjects IN ONE namespace
ClusterRoleBinding → grants a ClusterRole to subjects CLUSTER-WIDE

15.2 Roles và ClusterRoles

```
# Role: Quyền trong phạm vi namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-manager
  namespace: staging
rules:
- apiGroups: [""] # "" = nhóm API lõi (pods, dịch vụ, sơ đồ cấu hình, v.v.)
  resources: ["pods", "pods/log", "pods/exec"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: ["apps"] # nhóm ứng dụng (triển khai, bản sao, v.v.)
  resources: ["deployments"]
  verbs: ["get", "list", "update", "patch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
  resourceNames: ["app-config", "feature-flags"] # Chỉ những chiếc ConfigMaps cụ thể này
---
# ClusterRole: quyền trên toàn cụm (được sử dụng cho thông tin node, PV, CRDs, v.v.)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["get", "list"]
- apiGroups: ["storage.k8s.io"]
  resources: ["storageclasses"]
  verbs: ["get", "list"]

# Tạo một cách bắt buộc
kubectl create role pod-manager \
  --verb=get,list,watch,create,delete \
  --resource=pods \
  --namespace=staging

kubectl create clusterrole node-reader \
  --verb=get,list,watch \
  --resource=nodes,persistentvolumes

# Liệt kê tất cả roles/clusterroles
```



```
kubectl get roles -n staging
kubectl get clusterroles | grep -v system: # Loại trừ hệ thống ClusterRoles

# Mô tả một vai trò (hiển thị các quy tắc ở định dạng có thể đọc được)
kubectl describe role pod-manager -n staging
```

15.3 RoleBindings và ClusterRoleBindings

```
# RoleBinding: cấp quyền Role cho các đối tượng trong MỘT namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-pod-manager
  namespace: staging
subjects:
- kind: User
  name: alice # Người dùng theo tên (từ chứng chỉ CN)
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: dev-team # Nhóm theo tên
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci-runner # ServiceAccount
  namespace: staging # Cần có SA namespace
roleRef:
  kind: Role
  name: pod-manager
  apiGroup: rbac.authorization.k8s.io
---
```

```
# RoleBinding cũng có thể tham chiếu ClusterRole (phạm vi limits đến namespace)
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-view-staging
  namespace: staging
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole # ClusterRole, nhưng nằm trong phạm vi 'dàn dựng' namespace
  name: view # ClusterRole tích hợp
  apiGroup: rbac.authorization.k8s.io
---
```

```
# ClusterRoleBinding: cấp quyền cho ClusterRole trên toàn cụm
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: ops-team-cluster-admin
subjects:
- kind: Group
  name: ops-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin # Truy cập siêu người dùng đầy đủ
  apiGroup: rbac.authorization.k8s.io

# Tạo một cách bắt buộc
kubectl create rolebinding alice-pod-manager \
  --role=pod-manager \
  --user=alice \
  --namespace=staging

kubectl create clusterrolebinding ops-admin \
  --clusterrole=cluster-admin \
  --group=ops-team

# Liên kết với ServiceAccount
kubectl create rolebinding ci-runner-binding \
  --role=pod-manager \
```

```
--serviceaccount=staging:ci-runner \
--namespace=staging
```

15.4 ClusterRole có sẵn

Kubernetes được trang bị ClusterRoles tích hợp hữu ích:

ClusterRole	Những gì nó cho phép
cluster-admin	Siêu người dùng đầy đủ - mọi thứ
admin	Toàn quyền truy cập trong namespace (không thể sửa đổi namespace hoặc hạn ngạch)
edit	Read/write truy cập vào hầu hết các tài nguyên trong namespace
view	Quyền truy cập chỉ đọc vào hầu hết các tài nguyên trong namespace
system:node	Được sử dụng bởi kubelets (đọc pods/services, ghi trạng thái node/pod)
system:kube-scheduler	Được sử dụng bởi bộ lập lịch
system:controller:*	Được sử dụng bởi nhiều bộ điều khiển khác nhau

```
# Gán vai trò tích hợp nhanh chóng
kubectl create rolebinding alice-edit \
  --clusterrole=edit \
  --user=alice \
  --namespace=production

kubectl create rolebinding bob-view \
  --clusterrole=view \
  --user=bob \
  --namespace=production

# Xem các quyền của vai trò tích hợp
kubectl describe clusterrole edit | head -40
```

15.5 ServiceAccount RBAC

ServiceAccounts là danh tính để Pods truy cập Kubernetes API:

```
# Tạo ServiceAccount chuyên dụng cho một ứng dụng
kubectl create serviceaccount app-sa -n production

# Tạo Role với các quyền cần thiết
kubectl create role app-role \
  --verb=get,list \
  --resource=configmaps,secrets \
  --namespace=production

# Liên kết SA với Role
kubectl create rolebinding app-sa-binding \
  --role=app-role \
  --serviceaccount=production:app-sa \
  --namespace=production

# Tham chiếu SA trong Pod/Deployment
spec:
  serviceAccountName: app-sa          # Sử dụng SA tùy chỉnh của chúng tôi
  automountServiceAccountToken: true  # Gắn mã thông báo (mặc định: true)
  containers:
  - name: app
    image: myapp:1.0
```

```
# Bên trong Pod, mã thông báo nằm ở:
# /var/run/secrets/kubernetes.io/serviceaccount/token
# Chứng chỉ CA có tại:
# /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

# Ứng dụng có thể gọi API bằng mã thông báo này:
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -H "Authorization: Bearer $TOKEN" \
  --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
  https://kubernetes.default.svc/api/v1/namespaces/production/configmaps
```

15.6 Kiểm tra và kiểm toán RBAC

```
# Kiểm tra xem người dùng hiện tại có thể làm gì
kubectl auth can-i create pods
kubectl auth can-i delete secrets --namespace=production
kubectl auth can-i '*' '*' # Bạn có phải là siêu người dùng?

# Mạo danh người dùng để kiểm tra quyền của họ
kubectl auth can-i list pods --as=alice
kubectl auth can-i list pods --as=alice --namespace=staging
kubectl auth can-i delete nodes --as=alice # Không nên

# Mạo danh ServiceAccount
kubectl auth can-i list configmaps \
  --as=system:serviceaccount:production:app-sa \
  --namespace=production

# Liệt kê TẤT CẢ các quyền cho người dùng (kiểm tra toàn diện)
kubectl auth can-i --list --as=alice --namespace=staging

# Kiểm tra ClusterRoles và Roles nào liên kết với người dùng
kubectl get rolebindings,clusterrolebindings --all-namespaces \
  -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for item in data['items']:
    for subj in item.get('subjects', []):
        if subj.get('name') == 'alice':
            ns = item['metadata'].get('namespace', 'cluster-wide')
            role = item['roleRef']['name']
            print(f'{ns}: {item["metadata"]["name"]} -> {role}')
"
```

```
# Tìm tất cả các ràng buộc cho một ServiceAccount cụ thể
kubectl get rolebindings,clusterrolebindings --all-namespaces \
  -o custom-columns='NAME:.metadata.name,NAMESPACE:.metadata.namespace,SUBJECTS:.subjects[*].name' \
  | grep app-sa
```

15.7 RBAC Khắc phục sự cố

Lỗi: Bị cấm - Quyền truy cập RBAC bị từ chối

```
# Lỗi đầy đủ:
Error from server (Forbidden): deployments.apps is forbidden:
  User "alice" cannot list resource "deployments" in API group "apps"
  in the namespace "production"

# Chẩn đoán:
# Bước 1: Xác định những gì cần thiết
#   tài nguyên: "triển khai", apiGroup: "ứng dụng", động từ: "danh sách", namespace: "sản xuất"

# Bước 2: Kiểm tra xem Alice có ràng buộc không
kubectl get rolebindings,clusterrolebindings -A \
  -o wide | grep alice

# Bước 3: Kiểm tra xem vai trò của ràng buộc có được phép không
kubectl describe rolebinding alice-edit -n production
# Tìm kiếm: triển khai trong nhóm ứng dụng với động từ danh sách

# Bước 4: Thêm quyền nếu thiếu
kubectl create role deployment-reader \
  --verb=list,get,watch \
  --resource=deployments \
  --namespace=production

kubectl create rolebinding alice-deployment-reader \
  --role=deployment-reader \
  --user=alice \
  --namespace=production

# Bước 5: Kiểm tra ngay
kubectl auth can-i list deployments --as=alice -n production
```

Lỗi: ServiceAccount không thể truy cập ConfigMap

```
# Nhật ký Pod hiển thị:
# Lỗi: configmaps "app-config" bị cấm:
#   Người dùng "system:serviceaccount:production:default" không thể lấy tài nguyên "configmaps"

# pod đang sử dụng tài khoản dịch vụ DEFAULT (không phải tài khoản tùy chỉnh có quyền)
# Khắc phục: tạo SA với quyền và tham chiếu nó trong Pod

# Sửa nhanh để thử nghiệm (không được khuyến nghị cho sản xuất):
kubectl create rolebinding default-sa-configmap-reader \
  --clusterrole=view \
  --serviceaccount=production:default \
  --namespace=production

# Tốt hơn: tạo SA chuyên dụng
kubectl create serviceaccount app-sa -n production
kubectl create rolebinding app-sa-configmap \
  --role=configmap-reader \
  --serviceaccount=production:app-sa \
  --namespace=production
# Sau đó đặt serviceAccountName: app-sa trong thông số Pod
```

Kiểm tra RBAC: Tìm ServiceAccounts có đặc quyền cao hơn

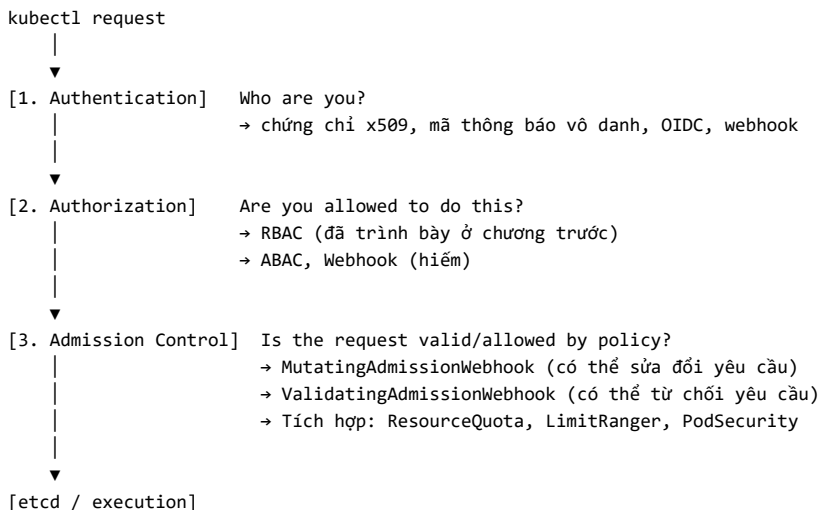
```
# Tìm tất cả các SA có liên kết quản trị viên cụm hoặc quản trị viên (nên ở mức tối thiểu)
kubectl get clusterrolebindings -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for b in data['items']:
    role = b['roleRef']['name']
    if role in ('cluster-admin', 'admin'):
        for subj in b.get('subjects', []):
            if subj['kind'] == 'ServiceAccount':
                print(f'SA {subj["namespace"]}/{subj["name"]} has {role}')
"
```

```
# Tìm tất cả các SA có động từ ký tự đại diện (nguy hiểm!)
kubectl get roles,clusterroles -A -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for item in data['items']:
    for rule in item.get('rules', []):
        if '*' in rule.get('verbs', []):
            ns = item['metadata'].get('namespace', 'cluster-wide')
            print(f'{ns}/{item["metadata"]["name"]}: wildcard verbs on {rule.get("resources")}')
" | grep -v system:
```

Chương 16 - Bảo mật Kubernetes: Xác thực, Bảo mật Pod và TLS

16.1 Xác thực, phân quyền và admission

Yêu cầu tới máy chủ API đi qua ba cổng:



16.2 Xác thực: người dùng và chứng chỉ

Kubernetes không có cơ sở dữ liệu người dùng tích hợp. Người dùng được xác thực thông qua:

```
# Cách 1: Chứng chỉ khách hàng x509 (phổ biến nhất cho kỳ thi CKA)
# CN (Common Name) trở thành tên người dùng, O (Tổ chức) trở thành nhóm

# Tạo khóa riêng cho người dùng 'alice'
openssl genrsa -out alice.key 2048

# Tạo CSR
```

```

openssl req -new -key alice.key \
  -subj "/CN=alice/O=dev-team" \
  -out alice.csr

# Yêu cầu CA cùm ký tên
sudo openssl x509 -req \
  -in alice.csr \
  -CA /etc/kubernetes/pki/ca.crt \
  -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial \
  -out alice.crt \
  -days 365

# Hoặc sử dụng Giấy chứng nhậnSigningRequest API (phương pháp hiện đại):
cat << 'CSR' | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: $(cat alice.csr | base64 | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400 # 24 giờ
  usages:
    - client auth
CSR

kubectl certificate approve alice
kubectl get csr alice -o jsonpath='{.status.certificate}' | base64 -d > alice.crt

# Xây dựng kubeconfig cho Alice
kubectl config set-credentials alice \
  --client-certificate=alice.crt \
  --client-key=alice.key \
  --embed-certs=true \
  --kubeconfig=alice.kubeconfig

kubectl config set-context alice-context \
  --cluster=kubernetes \
  --user=alice \
  --namespace=staging \
  --kubeconfig=alice.kubeconfig

```

16.3 Pod Security Standards (PSS)

Đã thay thế PodSecurityPolicy không được dùng nữa trong K8s 1.25+. Ba cấp độ:

```

privileged → unrestricted (no policy)
baseline   → minimal restrictions, prevents known escalations
restricted → hardened, follows best practices

```

Thực thi thông qua nhãn namespace:

```

# Áp dụng chính sách 'hạn chế' cho namespace (chế độ kiểm tra trước)
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=v1.35 \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# Chế độ:
# thực thi → từ chối Pods vi phạm chính sách
# kiểm tra → ghi lại các vi phạm nhưng cho phép
# cảnh báo → cảnh báo hướng tới người dùng nhưng cho phép

```

Pod Security - Yêu cầu chính sách bị hạn chế:

```
spec:
  securityContext:
    runAsNonRoot: true          # Bắt buộc bởi bị hạn chế
    seccompProfile:
      type: RuntimeDefault      # Bắt buộc bởi bị hạn chế
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false  # Bắt buộc
      capabilities:
        drop: ["ALL"]                # Bắt buộc
        add: ["NET_BIND_SERVICE"]     # Tùy chọn: chỉ thêm lại những gì cần thiết
      runAsNonRoot: true              # Bắt buộc
      runAsUser: 1000                 # Thực hành tốt
      readOnlyRootFilesystem: true    # Thực hành tốt
```

16.4 Secrets: Mã hóa và xử lý an toàn

```
# Xác minh bí mật KHÔNG được mã hóa khi lưu trữ (trạng thái mặc định):
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/my-secret | strings | head -10
# Nếu bạn có thể đọc dữ liệu dưới dạng văn bản → không được mã hóa!

# Kích hoạt tính năng mã hóa ở trạng thái lưu trữ (được trình bày chi tiết trong Chương 6)
# Thủ tục xoay khóa:
# 1. Thêm khóa mới trước trong cấu hình mã hóa (với tư cách là nhà cung cấp đầu tiên)
# 2. Khởi động lại api-server để nhận cấu hình mới
# 3. Mã hóa lại tất cả bí mật: kubectl get secrets -A -o json | kubectl replace -f -
# 4. Xóa khóa cũ khỏi cấu hình
# 5. Khởi động lại máy chủ api một lần nữa
```

16.5 Ghi log kiểm toán (audit log)

Theo dõi mọi yêu cầu máy chủ API để kiểm tra bảo mật:

```
# /etc/kubernetes/audit-policy.yaml
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
- RequestReceived          # Không đăng nhập khi bắt đầu mọi yêu cầu
rules:
# Ghi lại tất cả hoạt động trong namespace nhạy cảm ở cấp độ Yêu cầu phản hồi
- level: RequestResponse
  namespaces: ["production", "finance"]
  resources:
  - group: ""
    resources: ["secrets", "configmaps"]

# Ghi lại pod exec/attach ở cấp Siêu dữ liệu (ghi lại hành động chứ không phải dữ liệu)
- level: Metadata
  resources:
  - group: ""
    resources: ["pods/exec", "pods/attach", "pods/portforward"]

# Ghi log lỗi xác thực
- level: Request
  users: ["system:anonymous"]

# Ghi nhật ký tối thiểu cho requests khác (giảm tiếng ồn)
- level: Metadata
  resources:
  - group: ""
```

```

resources: ["pods", "services", "deployments"]

# Không ghi lại các hoạt động của hệ thống chỉ đọc
- level: None
  users: ["system:kube-proxy", "system:node"]
  verbs: ["get", "list", "watch"]

# Kích hoạt trong kube-apiserver pod tĩnh:
# --audit-policy-file=/etc/kubernetes/audit-policy.yaml
# --audit-log-path=/var/log/kubernetes/audit.log
# --audit-log-maxage=30
# --audit-log-maxbackup=3
# --audit-log-maxsize=100

# Truy vấn nhật ký kiểm tra
sudo cat /var/log/kubernetes/audit.log | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        e = json.loads(line)
        if e.get('level') != 'None':
            user = e.get('user', {}).get('username', 'unknown')
            verb = e.get('verb', '')
            resource = e.get('objectRef', {}).get('resource', '')
            ns = e.get('objectRef', {}).get('namespace', '')
            name = e.get('objectRef', {}).get('name', '')
            code = e.get('responseStatus', {}).get('code', '')
            print(f'{user} {verb} {resource}/{name} in {ns} → {code}')
    except:
        pass
" | head -30

```

16.6 Chứng chỉ TLS trong Kubernetes

Kubernetes PKI có một số hệ thống phân cấp chứng chỉ:

```

Cluster CA (/etc/kubernetes/pki/ca.crt)
├─ API Server cert (kube-apiserver.crt)
├─ API Server kubelet client cert
├─ Controller Manager cert
├─ Scheduler cert
└─ Admin (user) certs

etcd CA (/etc/kubernetes/pki/etcd/ca.crt)
├─ etcd server cert (server.crt)
├─ etcd peer cert (peer.crt)
└─ API server etcd client cert (apiserver-etcd-client.crt)

Front Proxy CA (/etc/kubernetes/pki/front-proxy-ca.crt)
└─ front-proxy-client.crt (for aggregated APIs)

# Kiểm tra tất cả ngày hết hạn của chứng chỉ
sudo kubeadm certs check-expiration
# CERTIFICATE          EXPIRES          RESIDUAL TIME    CERTIFICATE AUTHORITY
# admin.conf Ngày 15 tháng 1 năm 2025 10:00 UTC 29 ngày
# apiserver Ngày 15 tháng 1 năm 2025 10:00 UTC 29 ngày ca
# etcd-peer ngày 15 tháng 1 năm 2025 10:00 UTC 29 ngày etcd-ca
# ...

# Gia hạn TẤT CẢ các chứng chỉ (làm điều này trước khi chúng hết hạn!)
sudo kubeadm certs renew all

# Hoặc gia hạn chứng chỉ cụ thể:
sudo kubeadm certs renew apiserver
sudo kubeadm certs renew scheduler.conf
sudo kubeadm certs renew controller-manager.conf

# Sau khi gia hạn, khởi động lại các thành phần control plane:
sudo crictl ps | grep -E "api|scheduler|controller"
# Giết từng cái để kích hoạt khởi động lại:
sudo kill $(sudo crictl ps | grep kube-apiserver | awk '{print $1}')

```



```
# Xác minh thời hạn mới
sudo kubeadm certs check-expiration | grep admin
```

```
# Chạy kube-bench để kiểm tra việc tuân thủ Điểm chuẩn CIS Kubernetes
kubectl run kube-bench --image=aquasec/kube-bench:latest \
  --restart=Never \
  --overrides='{ "spec": { "hostPID": true, "hostNetwork": true, "tolerations": [{ "operator": "Exists" }, { "volumes": [{ "name": "var-lib-etcd", "hostPath": { "path": "/var/lib/etcd" } }, { "name": "etc-kubernetes", "hostPath": { "path": "/etc/kubernetes" } } ] }, { "containers": [{ "name": "kube-bench", "image": "aquasec/kube-bench:latest", "command": [ "kube-bench" ], "volumeMounts": [{ "name": "var-lib-etcd", "mountPath": "/var/lib/etcd" }, { "name": "etc-kubernetes", "mountPath": "/etc/kubernetes" } ] } ] } } }' \
  -- node

kubectl logs kube-bench | head -60
# [PASS] 4.1.1 Đảm bảo rằng quyền của tập dịch vụ kubelet được đặt thành 600
# [FAIL] 4.1.2 Đảm bảo rằng quyền sở hữu tập dịch vụ kubelet được đặt thành root:root
# [CẢNH BÁO] 4.1.3 Nếu tệp kubeconfig proxy tồn tại, hãy đảm bảo quyền được đặt thành 600
```

Thao tác này sẽ đọc tệp trạng thái quy trình riêng của kernel cho PID 1 (quy trình init của container).

Ví dụ: Đọc nội dung của tệp cấu hình

```
kubectl exec my-app-pod -- cat /etc/nginx/nginx.conf
```

Điều này làm gì: In nội dung của tệp cấu hình trực tiếp vào thiết bị đầu cuối của bạn mà không cần sao chép nó ra khỏi container.

12.3 Mở phiên Shell tương tác

Khi bạn cần khám phá container một cách tương tác, điều hướng hệ thống tệp, chạy nhiều lệnh hoặc khắc phục sự cố động, bạn mở shell tương tác.

Giải thích về cờ `-it`:

Cờ	Dạng dài	Mục đích
<code>-i</code>	<code>--stdin</code>	Giữ STDIN mở để đầu vào bàn phím của bạn đạt tới container
<code>-t</code>	<code>--tty</code>	Phân bổ một TTY giả (thiết bị đầu cuối), cho phép các tính năng tương tác như lời nhắc và màu sắc

Cả hai cờ cùng nhau (`-it`) cung cấp cho bạn phiên thiết bị đầu cuối tương tác đầy đủ bên trong container.

Ví dụ: Mở bash shell

```
kubectl exec -it my-app-pod -- /bin/bash
```

Việc này có tác dụng gì: Mở một bash shell bên trong container. Bạn sẽ thấy lời nhắc như `root@my-app-pod:/#` và có thể chạy các lệnh một cách tương tác. Gõ `exit` hoặc nhấn `Ctrl+D` để thoát.

Ví dụ: Mở sh shell (dành cho minimal/Alpine-based images)

Nhiều images sản xuất được xây dựng trên Alpine Linux hoặc images không phân phối không bao gồm bash. Trong những trường hợp này, hãy sử dụng sh:

```
kubectl exec -it my-app-pod -- /bin/sh
```

Ví dụ: Hãy thử bash trước, quay lại sh

```
kubectl exec -it my-app-pod -- bash 2>/dev/null || kubectl exec -it my-app-pod -- sh
```

Việc này có tác dụng gì: Cố gắng bắt đầu bash; nếu thất bại (mã thoát khác 0), quay trở lại sh.

12.4 Nhắm mục tiêu một container cụ thể trong nhiều container Pod

Khi Pod có nhiều container, bạn phải chỉ định container nào sẽ nhắm mục tiêu bằng `-c`.

Ví dụ: Liệt kê containers trong Pod

```
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[*].name}'
```

Ví dụ đầu ra: app sidecar

Ví dụ: Thực thi cụ thể vào sidecar container

```
kubectl exec -it my-app-pod -c sidecar -- /bin/sh
```

Điều này có tác dụng gì: Chỉ mở shell bên trong sidecar container. Nếu không có `-c`, kubectl sẽ mặc định là container đầu tiên trong thông số Pod, đây có thể không phải là điều bạn muốn.

Ví dụ: Chạy lệnh trong container cụ thể không tương tác

```
kubectl exec my-app-pod -c app -- cat /var/log/app.log
```

12.5 Truyền đối số với các ký tự đặc biệt

Khi lệnh của bạn chứa các đường dẫn, chuyển hướng hoặc toán tử shell, bạn cần gọi shell một cách rõ ràng bên trong container để diễn giải chúng, thay vì để kubectl cố gắng phân tích cú pháp chúng.

Ví dụ: Sử dụng grep bên trong container (SAU cách)

```
# Điều này sẽ KHÔNG hoạt động - kubectl không giải thích các toán tử shell
kubectl exec my-app-pod -- cat /var/log/app.log | grep "ERROR"
```

Ổng | ở trên được diễn giải bởi shell cục bộ của BẠN, không phải shell của container. Grep chạy cục bộ dựa trên đầu ra, nhưng mục đích là lọc bên trong container.

Ví dụ: Sử dụng grep bên trong container (cách ĐÚNG)

```
# Gói lệnh trong sh -c để shell của container diễn giải lệnh đó
kubectl exec my-app-pod -- sh -c 'cat /var/log/app.log | grep "ERROR"'
```

Việc này có tác dụng gì: Chuyển toàn bộ chuỗi tới sh -c bên trong container. Sau đó, shell của container sẽ diễn giải đường ống. Luôn sử dụng dấu ngoặc đơn để ngăn shell cục bộ của bạn mở rộng các biến hoặc toán tử.

Ví dụ: Kiểm tra mức sử dụng đĩa bên trong container

```
kubectl exec my-app-pod -- sh -c 'df -h && du -sh /app/*'
```

Ví dụ: Đặt một biến và sử dụng nó trong container

```
kubectl exec my-app-pod -- sh -c 'TARGET=/app; ls -la $TARGET'
```

17.8 Khắc phục sự cố Lỗi thực thi kubectl

Lỗi: Thực thi thời gian chạy OCI không thành công

```
$ kubectl exec -it my-pod -- /bin/bash
OCI runtime exec failed: exec failed: container_linux.go:380:
starting container process caused:
exec: "/bin/bash": stat /bin/bash: no such file or directory

# container image không có bash (phổ biến trong Alpine/distroless images)
# Thay vào đó hãy thử sh:
kubectl exec -it my-pod -- /bin/sh

# Hoặc sử dụng busybox sh cho images ở mức tối thiểu:
kubectl exec -it my-pod -- /busybox/sh

# Đối với containers không phân phối (hoàn toàn không có shell), hãy sử dụng bản gỡ lỗi tạm thời container:
kubectl debug -it my-pod --image=busybox:1.35 --target=my-container
```

Lỗi: không thể nâng cấp kết nối: Không tìm thấy container

```
$ kubectl exec -it my-pod -c wrong-container -- sh
error: unable to upgrade connection: container not found ("wrong-container")

# Liệt kê containers trong pod:
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
# Hoặc chi tiết hơn:
kubectl describe pod my-pod | grep "Container ID:" -B 3

# Sử dụng tên container chính xác:
kubectl exec -it my-pod -c correct-container-name -- sh
```

Lỗi: lệnh bị chấm dứt với mã thoát 126/127

```
$ kubectl exec my-pod -- mycommand
command terminated with exit code 127 # lệnh không tìm thấy
command terminated with exit code 126 # lệnh không thực thi được (quyền)

# 127: lệnh không có trong PATH
# Kiểm tra xem nó có tồn tại không:
kubectl exec my-pod -- which mycommand
kubectl exec my-pod -- find / -name "mycommand" 2>/dev/null

# 126: file tồn tại nhưng không thể thực thi được
kubectl exec my-pod -- ls -la /usr/local/bin/mycommand
kubectl exec my-pod -- chmod +x /usr/local/bin/mycommand

# Sử dụng đường dẫn đầy đủ một cách rõ ràng:
kubectl exec my-pod -- /usr/local/bin/mycommand
```

Lỗi: exec: lỗi đọc từ kết nối

```
# Thường do gián đoạn mạng hoặc ngắt kết nối VPN
# Thường an toàn khi chỉ chạy lại lệnh exec
# Đối với các phiên tương tác, hãy cân nhắc sử dụng tmux bên trong container:
kubectl exec -it my-pod -- tmux new-session -d -s main
kubectl exec -it my-pod -- tmux attach -t main
```

17.9 Chạy lệnh một lần để gỡ lỗi

```
# Kết xuất các biến môi trường
kubectl exec my-pod -- env

# Kiểm tra quá trình nào đang lắng nghe trên một cổng
kubectl exec my-pod -- ss -tlnp
kubectl exec my-pod -- netstat -tlnp # nếu cài đặt công cụ mạng

# Kiểm tra mức sử dụng đĩa bên trong container
kubectl exec my-pod -- df -h
kubectl exec my-pod -- du -sh /var/log/*

# Kiểm tra mức sử dụng bộ nhớ bên trong container
kubectl exec my-pod -- cat /proc/meminfo
kubectl exec my-pod -- free -h # nếu có

# Kiểm tra các tiến trình đang chạy
kubectl exec my-pod -- ps aux

# Kiểm tra múi giờ (vấn đề cấu hình phổ biến)
kubectl exec my-pod -- date
kubectl exec my-pod -- cat /etc/timezone

# Kiểm tra /etc/hosts (ghi đề DNS)
kubectl exec my-pod -- cat /etc/hosts

# Kiểm tra mô tả tập tin đang mở
kubectl exec my-pod -- ls -la /proc/1/fd | wc -l

# Chạy tập lệnh Python nhanh bên trong container
kubectl exec my-pod -- python3 -c "
import socket
print('Hostname:', socket.gethostname())
print('IP:', socket.gethostbyname(socket.gethostname()))
"

# Đuôi một tệp nhật ký bên trong container
kubectl exec -it my-pod -- tail -f /var/log/app/application.log

# Chạy một loạt lệnh gỡ lỗi và lưu kết quả
kubectl exec my-pod -- sh -c "
echo '=== Environment ==='; env | sort;
echo '=== Processes ==='; ps aux;
echo '=== Network ==='; ss -tlnp;
echo '=== Disk ==='; df -h;
" > pod-debug-$(date +%Y%m%d).txt
```

17.10 Thực thi lệnh trong Pod có nhiều container

```
# Khi pod có nhiều containers (e.g., app + sidecar + init)
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
# Đầu ra: trình xuất số liệu thu thập nhật ký ứng dụng

# Chạy lệnh trong mỗi container
for container in app log-collector metrics-exporter; do
    echo "=== $container ==="
    kubectl exec my-pod -c $container -- ps aux 2>/dev/null || echo "ps not available"
done

# Truyền dữ liệu giữa containers qua ổ đĩa chung
kubectl exec my-pod -c app -- sh -c "echo 'test data' > /shared/data.txt"
kubectl exec my-pod -c log-collector -- cat /shared/data.txt
```

Chương 18 - Xem và truyền phát nhật ký container

18.1 Lệnh `kubectl logs`

Kubernetes ghi lại các luồng đầu ra tiêu chuẩn (stdout) và lỗi tiêu chuẩn (stderr) của mọi container và lưu trữ chúng dưới dạng nhật ký. `kubectl logs` truy xuất các nhật ký này. Điều này khác với các tệp nhật ký cấp ứng dụng được ghi vào đĩa bên trong container.

Truy xuất nhật ký cơ bản 13.2

Ví dụ: Xem tất cả nhật ký từ container mặc định của Pod

```
kubectl logs my-app-pod
```

Việc này có tác dụng gì: Lưu toàn bộ lịch sử nhật ký của container chính vào thiết bị đầu cuối của bạn. Đối với các ứng dụng bận rộn, đây có thể là hàng nghìn dòng.

Ví dụ: Xem nhật ký từ một container cụ thể trong nhiều container Pod

```
kubectl logs my-app-pod -c sidecar
```

13.3 Truyền trực tuyến nhật ký thời gian thực

Ví dụ: Theo dõi nhật ký (đuôi) trong thời gian thực

```
kubectl logs -f my-app-pod
```

Điều này có tác dụng gì: Cờ `-f` (theo dõi) giữ cho kết nối luôn mở và truyền phát các dòng nhật ký mới khi chúng được tạo. Tương đương với `tail -f` trên tệp nhật ký truyền thống. Nhấn `Ctrl+C` để dừng phát trực tuyến.

Ví dụ: Theo dõi nhật ký của một container cụ thể

```
kubectl logs -f my-app-pod -c app
```

13.4 Giới hạn volume đầu ra

Ví dụ: Chỉ xem 50 dòng nhật ký cuối cùng

```
kubectl logs my-app-pod --tail=50
```

Việc này có tác dụng gì: Chỉ hiển thị 50 dòng nhật ký gần đây nhất. Cờ `--tail` chấp nhận bất kỳ số nguyên dương nào. Sử dụng `--tail=-1` để hiển thị tất cả các dòng (giống như không có cờ).

Ví dụ: Kết hợp theo dõi và theo dõi để xem trực tiếp các nhật ký gần đây

```
kubectl logs -f my-app-pod --tail=100
```

Điều này có tác dụng gì: Hiển thị 100 dòng cuối cùng ngay lập tức, sau đó tiếp tục truyền phát các dòng mới. Đây là mẫu phổ biến nhất để gỡ lỗi tích cực.

Lọc nhật ký dựa trên thời gian 13.5

Ví dụ: Xem nhật ký trong 1 giờ qua

```
kubectl logs my-app-pod --since=1h
```

Điều này làm gì: Chỉ trả về các dòng nhật ký được tạo trong giờ qua. Đơn vị hợp lệ: `s` (giây), `m` (phút), `h` (giờ).

Ví dụ: Xem nhật ký kể từ dấu thời gian cụ thể (định dạng RFC3339)

```
kubectl logs my-app-pod --since-time="2024-06-01T09:00:00Z"
```

Điều này làm gì: Trả về tất cả các dòng nhật ký được tạo sau dấu thời gian UTC đã chỉ định. Hữu ích khi gỡ lỗi một sự cố cụ thể mà bạn biết thời gian bắt đầu.

13.6 Xem nhật ký container trước đó (Khôi phục sự cố)

Khi container gặp sự cố và khởi động lại, Kubernetes sẽ giữ ngắn gọn nhật ký của phiên bản trước đó.

Ví dụ: Xem nhật ký từ phiên bản crashed/previous container

```
kubectl logs my-app-pod --previous
```

Điều này có tác dụng gì: Cờ `--previous` yêu cầu Kubernetes trả về nhật ký của phiên bản container đã chấm dứt gần đây nhất. Đây thường là điều đầu tiên bạn làm khi container gặp sự cố, vì nó cho thấy điều gì đã xảy ra ngay trước khi xảy ra sự cố.

Ví dụ: Kết hợp previous với tail cho những khoảnh khắc cuối cùng trước khi xảy ra sự cố

```
kubectl logs my-app-pod --previous --tail=200
```

13.7 Xem nhật ký từ tất cả Pods trong Deployment

Ví dụ: Nhận nhật ký từ tất cả Pods khớp với bộ chọn nhãn

```
kubectl logs -l app=my-app --all-containers=true
```

Việc này có tác dụng gì: Chọn tất cả Pods có nhãn `app=my-app` và truy xuất nhật ký từ mọi container trong mỗi Pods đó. Cờ `--all-containers=true` đảm bảo containers với bất kỳ tên nào trong nhiều container Pods đều được bao gồm.

18.2 Ghi và phân tích log có cấu trúc

Các ứng dụng hiện đại phát ra nhật ký có cấu trúc JSON. `kubectl logs` có thể được kết hợp với `jq` (hoặc `python -m json.tool`) để phân tích chúng:

```
# Ứng dụng phát ra nhật ký JSON
kubectl logs api-pod-abc123 | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        obj = json.loads(line)
        print(f'{{obj.get(\"timestamp\", \"\")}} [{{obj.get(\"level\", \"\")}}] {obj.get(\"message\", \"\")}')
    except: print(line, end='')
"

# Chỉ lọc nhật ký mức LỖI JSON
kubectl logs api-pod | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        obj = json.loads(line)
        if obj.get('level') == 'ERROR':
            print(json.dumps(obj, indent=2))
    except: pass
"
```

18.3 Tổng hợp log từ nhiều Pod

```
# Nhật ký từ TẤT CẢ pods khớp với bộ chọn nhãn
kubectl logs -l app=frontend --all-containers --since=1h

# Xem log của Pod mới nhất trong Deployment
kubectl logs deployment/frontend --since=30m

# Theo dõi nhật ký từ tất cả pods trong thời gian thực với tiền tố tên pod
kubectl logs -l app=frontend -f --prefix

# Đầu ra bao gồm tiền tố tên pod:
# [pod/frontend-6d8b4d9-xk2p9] 2024-01-15T10:30:00Z NHẬN /api/users 200 45 mili giây
# [pod/frontend-6d8b4d9-m3n7r] 2024-01-15T10:30:01Z NHẬN /api/health 200 2ms
```

18.4 Lưu giữ log và xem log của container trước đó

```
# Nhận nhật ký từ phiên bản container TRƯỚC (trước khi gặp sự cố)
kubectl logs my-pod --previous
kubectl logs my-pod -c main-container --previous

# Điều này rất quan trọng để chẩn đoán CrashLoopBackOff - container hiện tại
# mới bắt đầu nên chưa có nhật ký; --previous cho thấy lý do tại sao nó bị hỏng

# Chỉ lấy N dòng cuối cùng
kubectl logs my-pod --tail=50
kubectl logs my-pod --tail=100 --previous

# Nhận nhật ký kể từ dấu thời gian
kubectl logs my-pod --since-time="2024-01-15T10:00:00Z"

# Nhận nhật ký phát ra trong 30 phút qua
kubectl logs my-pod --since=30m

# Kết hợp: 100 dòng cuối cùng trong 1 giờ qua
kubectl logs my-pod --tail=100 --since=1h
```

18.5 Kiến trúc thu thập log

Để sản xuất, truyền nhật ký đến hệ thống trung tâm. Ba mẫu phổ biến:

Mẫu 1: Trình thu thập nhật ký DaemonSet (Tác nhân nút)

```
# Fluent Bit hoặc Fluentd chạy trên mọi node dưới dạng DaemonSet
# và đọc tệp nhật ký container trực tiếp từ /var/log/pods/
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels:
      name: fluent-bit
  template:
    spec:
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:2.1
          volumeMounts:
            - name: varlog
              mountPath: /var/log
            - name: varlibdockercontainers
              mountPath: /var/lib/docker/containers
              readOnly: true
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
        - name: varlibdockercontainers
          hostPath:
            path: /var/lib/docker/containers
```

Mẫu 2: Chuyển tiếp nhật ký Sidecar Mỗi Pod có một sidecar đọc nhật ký ứng dụng từ ổ đĩa chung và chuyển tiếp đến dịch vụ trung tâm.

Mẫu 3: Ghi nhật ký ứng dụng trực tiếp Ứng dụng ghi trực tiếp vào stdout/stderr; Đại lý node thu thập và vận chuyển. Đây là phương pháp tiếp cận ứng dụng 12 yếu tố được đề xuất - containers không nên quản lý tệp nhật ký của riêng mình.

```
# Nơi Kubernetes lưu trữ các tệp nhật ký container trên node
ls /var/log/pods/<namespace>_<podname>_<uid>/<container-name>/
# 0.log ← nhật ký hiện tại
# 0.log.gz ← nhật ký đã xoay
```

Chương 19 - Sao chép tập tin đến và từ container

19.1 Lệnh `kubectl cp`

`kubectl cp` cho phép bạn chuyển các tệp và thư mục giữa hệ thống tệp cục bộ của bạn và hệ thống tệp của container. Về cơ bản, nó sử dụng `tar` bên trong container, vì vậy mã nhị phân `tar` phải có trong container image.

Cú pháp cơ bản

```
kubectl cp <source> <destination>
```

Đối với đường dẫn container, hãy sử dụng định dạng: `<pod-name>:<path>` hoặc `<pod-name>/<container-name>:<path>`

14.2 Sao chép từ container sang cục bộ

Ví dụ: Sao chép tệp nhật ký từ container vào thư mục hiện tại của bạn

```
kubectl cp my-app-pod:/var/log/app/error.log ./error.log
```

Việc này có tác dụng gì: Tải xuống `/var/log/app/error.log` từ bên trong container và lưu nó dưới dạng `error.log` trong thư mục làm việc hiện tại của bạn.

Ví dụ: Sao chép toàn bộ thư mục từ container

```
kubectl cp my-app-pod:/etc/nginx ./nginx-config-backup/
```

Việc này có tác dụng gì: Sao chép đệ quy toàn bộ thư mục `/etc/nginx` từ container vào một thư mục cục bộ có tên `nginx-config-backup`.

Ví dụ: Sao chép từ một container cụ thể trong nhiều container Pod

```
kubectl cp my-app-pod:/var/log/app.log -c sidecar ./app.log
```

14.3 Sao chép từ cục bộ sang container

Ví dụ: Tải tệp cấu hình lên container đang chạy

```
kubectl cp ./updated-config.yaml my-app-pod:/etc/app/config.yaml
```

Việc này có tác dụng gì: Sao chép tệp cục bộ `updated-config.yaml` vào container theo đường dẫn đã chỉ định. Điều này hữu ích cho các cấu hình vá nóng trong quá trình gỡ lỗi mà không cần xây dựng lại image.

Cảnh báo quan trọng: Những thay đổi được thực hiện theo cách này **không tồn tại lâu dài**. Khi Pod khởi động lại, container trở lại trạng thái image ban đầu. Để có những thay đổi liên tục, hãy cập nhật ConfigMaps, Secrets hoặc chính container image.

Ví dụ: Tải tập lệnh lên cho mục đích thử nghiệm

```
kubectl cp ./test-script.sh my-app-pod:/tmp/test-script.sh
kubectl exec my-app-pod -- chmod +x /tmp/test-script.sh
kubectl exec my-app-pod -- /tmp/test-script.sh
```

Việc này có tác dụng gì: Tải lên tập lệnh shell, làm cho nó có thể thực thi được và chạy nó bên trong container. Đây là kỹ thuật gỡ lỗi phổ biến khi bạn cần chạy thử nghiệm nhiều bước phức tạp.

Ví dụ: Copy toàn bộ thư mục vào container

```
kubectl cp ./local-test-data/ my-app-pod:/tmp/test-data/
```

19.3 Khắc phục sự cố kubectl cp

Lỗi: tar: xóa phần đầu '/'

```
$ kubectl cp my-pod:/etc/nginx/nginx.conf ./nginx.conf
tar: removing leading '/' from member names

# Đây chỉ là CẢNH BÁO, không phải lỗi - sao chép thành công
# / Hàng đầu bị loại bỏ để đảm bảo an toàn (đường dẫn tương đối trong tar)
# Tệp sẽ ở ./nginx.conf (không có dấu / ở đầu)
```

Lỗi: Không tìm thấy container

```
$ kubectl cp ./myfile.txt my-pod:/tmp/myfile.txt
error: container not found for pod my-pod

# Pod có nhiều containers - chỉ định cái nào:
kubectl cp ./myfile.txt my-pod:/tmp/myfile.txt -c main-container

# Tìm tên container:
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
```

Lỗi: không có tập tin hoặc thư mục (nguồn) như vậy

```
$ kubectl cp my-pod:/nonexistent/path ./output
tar: /nonexistent/path: Cannot stat: No such file or directory
error: exit status 1
```

```
# Xác minh đường dẫn tồn tại trước:
kubectl exec my-pod -- ls -la /nonexistent/
kubectl exec my-pod -- find / -name "filename" 2>/dev/null
```

Sao chép các tập tin lớn một cách hiệu quả

```
# kubectl cp không lý tưởng cho các tập tin lớn (truyền phát qua máy chủ API)
# Để truyền tập tin lớn, hãy sử dụng chuyển tiếp cổng + kết nối trực tiếp:

# Phương pháp 1: Chuyển tiếp cổng tới máy chủ chuyển tiếp
kubectl run transfer-pod --image=python:3.11 --restart=Never --command -- \
  python3 -m http.server 8080
kubectl port-forward pod/transfer-pod 8080:8080 &
# Tải lên qua curl
curl -X PUT http://localhost:8080/ --data-binary @largefile.tar.gz

# Cách 2: Sử dụng tar + exec để lưu trữ thư mục
kubectl exec my-pod -- tar czf - /var/data | tar xzf - -C ./local-backup/

# Cách 3: Sao chép qua ổ đĩa chung (đối với cùng namespace pods)
# Gắn cùng một PVC vào pod gỡ lỗi và sao chép dữ liệu vào đó
```

19.4 Thao tác tập phổ biến thông qua kubectl exec + cp

```
# Mẫu: sửa đổi tệp cấu hình trong container đang chạy
# Bước 1: Sao chép cấu hình hiện tại
kubectl cp my-pod:/etc/app/config.json ./config.json

# Bước 2: Chỉnh sửa cục bộ
vim config.json

# Bước 3: Sao chép lại
kubectl cp ./config.json my-pod:/etc/app/config.json

# Bước 4: Tải lại ứng dụng (phương pháp tùy thuộc vào ứng dụng)
kubectl exec my-pod -- kill -HUP 1
# or
kubectl rollout restart deployment/my-app

# Sao lưu toàn bộ thư mục từ pod
kubectl exec my-pod -- tar czf - /var/lib/app/data/ > app-data-backup.tar.gz
ls -lh app-data-backup.tar.gz

# Khôi phục bản sao lưu vào pod
cat app-data-backup.tar.gz | kubectl exec -i my-pod -- tar xzf - -C /

# Sao chép từ một pods sang cái khác (thông qua localhost)
kubectl cp source-pod:/data/file.db ./temp-file.db
kubectl cp ./temp-file.db target-pod:/data/file.db
```

Chương 20 - Kiểm tra và mô tả container

20.1 Nhận thông tin chi tiết về Pod và container

```
kubectl describe pod
```

Lệnh `describe` cung cấp bản tóm tắt mà con người có thể đọc được về tài nguyên Kubernetes. Đối với Pods, nó bao gồm container images, tài nguyên requests và limits, các biến môi trường, gắn ổ đĩa, sự kiện và trạng thái container hiện tại.

Ví dụ: Mô tả Pod

```
kubectl describe pod my-app-pod
```

Các phần chính cần xem ở đầu ra:

- **Phần container:** Hiển thị tên image, ID container, cổng, biến môi trường, ổ đĩa gắn kết, tài nguyên limits và trạng thái hiện tại (Running, Đang chờ, Đã chấm dứt) cùng với lý do và mã thoát.
- **Phần sự kiện:** Hiển thị các sự kiện Kubernetes gần đây cho Pod - quyết định lập lịch, kéo image, container khởi động, OOMKills và lỗi. Đây thường là nơi đầu tiên để tìm kiếm manh mối.

Ví dụ: Mô tả tất cả Pods trong namespace

```
kubectl describe pods -n my-namespace
```

15.2 Nhận đầu ra YAML/JSON thô

Ví dụ: Nhận thông số Pod đầy đủ dưới dạng YAML

```
kubectl get pod my-app-pod -o yaml
```

Điều này làm gì: Xuất ra định nghĩa đối tượng Kubernetes đầy đủ dưới dạng YAML. Điều này rất hữu ích để xem mọi trường, bao gồm cả những trường không được `describe` hiển thị, chẳng hạn như `podIP`, `hostIP`, `nodeName`, tất cả các chú thích và thông số container đầy đủ.

Ví dụ: Nhận một trường cụ thể bằng JSONPath

```
# Lấy tên container image
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[0].image}'

# Nhận tất cả tên container trong Pod
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[*].name}'

# Lấy địa chỉ IP của Pod
kubectl get pod my-app-pod -o jsonpath='{.status.podIP}'

# Nhận node Pod được lên lịch vào
kubectl get pod my-app-pod -o jsonpath='{.spec.nodeName}'
```

JSONPath làm gì: Cho phép bạn trích xuất các trường cụ thể từ bản trình bày JSON của một tài nguyên. Đường dẫn `.spec.containers[*].name` trả về tên của mọi container trong Pod.

15.3 Kiểm tra trạng thái container và mã thoát

Ví dụ: Kiểm tra lý do tại sao container lại có trong CrashLoopBackOff

```
kubectl get pod my-app-pod -o jsonpath='{.status.containerStatuses[0]}'
```

Điều này làm gì: Trả về đối tượng `containerStatus` đầy đủ bao gồm: `state` (hiện tại), `lastState` (chấm dứt trước đó), `exitCode`, `reason`, `message` và `restartCount`.

Ví dụ: Lấy mã thoát của container đã kết thúc gần đây nhất

```
kubectl get pod my-app-pod \
-o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
```

Ý nghĩa mã thoát:

Mã thoát	Ý nghĩa
0	thành công
1	Lỗi chung
137	OOMKilled (Hết bộ nhớ, tín hiệu 9)
143	Chấm dứt duyên dáng (tín hiệu 15)

Mã thoát	Ý nghĩa
255	Thoát khỏi shell mà không cần mã

15.4 liệt kê tất cả các container trên tất cả Pods

Ví dụ: Liệt kê mọi Pod và container images của nó trong cụm

```
kubectl get pods -A -o=custom-columns=\
'NAMESPACE:.metadata.namespace,POD:.metadata.name,CONTAINERS:.spec.containers[*].name,IMAGES:.spec.containers[*].image'
```

Điều này làm gì: Sử dụng định dạng đầu ra cột tùy chỉnh để hiển thị tên namespace, pod, tất cả tên container và tất cả container images ở định dạng bảng trên tất cả namespaces.

20.2 So sánh trạng thái tài nguyên bằng diff

```
# So sánh tài nguyên trực tiếp với tệp cục bộ
kubectl diff -f deployment.yaml

# Điều này cho thấy những gì SẼ thay đổi nếu bạn áp dụng tệp
# Xanh (+) = sẽ được thêm, Đỏ (-) = sẽ bị xóa

# So sánh hai nguồn tài nguyên khác nhau
kubectl get deployment frontend -o yaml > frontend-live.yaml
kubectl get deployment backend -o yaml > backend-live.yaml
diff frontend-live.yaml backend-live.yaml
```

20.3 Kiểm tra tình trạng tài nguyên với các điều kiện

Mọi đối tượng Kubernetes đều có **Điều kiện** cung cấp thông tin sức khỏe chi tiết:

```
# Điều kiện Pod
kubectl get pod mypod -o jsonpath='{.status.conditions}' | python3 -m json.tool

# Các loại:
#   PodScheduled: Đúng → Pod đã được gán cho node
#   ContainersReady: True → Tất cả containers đã sẵn sàng vượt qua
#   Đã khởi tạo: Đúng → Tất cả containers init đã hoàn thành
#   Ready: True → Pod is ready to serve traffic

# Điều kiện Deployment
kubectl describe deployment myapp | grep -A 30 "Conditions:"
# Có sẵn: Đúng → bản sao tối thiểu có sẵn đã sẵn sàng
# Đang tiến hành: Đúng → đang tiến hành cập nhật hoặc hoàn thành
# Lỗi bản sao: Sai → không có lỗi tạo bản sao

# Điều kiện nút
kubectl describe node worker-1 | grep -A 20 "Conditions:"
# Áp suất bộ nhớ: Sai → node có đủ bộ nhớ
# DiskPressure: Sai → node có đủ đĩa
# PIDPressure: Sai → node có đủ PID
# Ready: True → node is healthy
```

20.4 Sử dụng jsonpath và jq để kiểm tra nâng cao

```
# Nhận tất cả IP Pod trong namespace
kubectl get pods -n production -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}}{end}'

# Nhận tất cả container images trên tất cả pods
kubectl get pods --all-namespaces \
  -o jsonpath='{range .items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\t"}{range .spec.containers[*]}{.image}{"\n"}}{end}{end}'

# Tìm pods không có tài nguyên requests được xác định
kubectl get pods --all-namespaces -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for pod in data['items']:
    for c in pod['spec'].get('containers', []):
        if not c.get('resources', {}).get('requests'):
            print(f'{pod["metadata"]["namespace"]}/{pod["metadata"]["name"]}: {c["name"]} has no requests')
"

# Nhận tất cả các dịch vụ với bộ chọn và cổng của chúng
kubectl get svc --all-namespaces \
  -o custom-columns='NS:.metadata.namespace,NAME:.metadata.name,SELECTOR:.spec.selector,PORTS:.spec.ports[*].port'

# Tìm Deployments không có bản sao mong muốn
kubectl get deployment --all-namespaces \
  -o custom-columns='NS:.metadata.namespace,NAME:.metadata.name,DESIRED:.spec.replicas,READY:.status.readyReplicas' \
  | awk '$3 != $4 {print}'
```

20.5 Sự kiện: công cụ chẩn đoán

```
# Sự kiện là nơi đầu tiên để tìm kiếm khi có sự cố xảy ra
kubectl get events -n production --sort-by='.lastTimestamp'

# Sự kiện cho một tài nguyên cụ thể
kubectl get events -n production --field-selector involvedObject.name=my-pod

# Xem các sự kiện trong thời gian thực (xuất sắc trong quá trình triển khai)
kubectl get events -n production -w

# Sự kiện có bộ lọc lý do
kubectl get events --field-selector reason=Failed -A
kubectl get events --field-selector reason=BackOff -A
kubectl get events --field-selector type=Warning -A

# Các sự kiện không tồn tại node khởi động lại theo mặc định (TTL: 1 giờ)
# Đối với các sự kiện liên tục, hãy cài đặt trình xuất sự kiện vào Elasticsearch hoặc Prometheus
```

Chương 21 - Chuyển tiếp cảng tới container

21.1 Lệnh `kubectl port-forward`

`kubectl port-forward` tạo một đường hầm an toàn giữa một cổng trên máy cục bộ của bạn và một cổng trên container bên trong cụm. Lưu lượng truy cập được ủy quyền thông qua máy chủ Kubernetes API và không yêu cầu cấu hình Service hoặc Ingress.

Điều này là vô giá đối với:

- Gỡ lỗi các dịch vụ không được hiển thị bên ngoài
- Thử nghiệm các dịch vụ mới trước khi giới thiệu chúng
- Truy cập trực tiếp vào giao diện người dùng, cơ sở dữ liệu hoặc điểm cuối số liệu của quản trị viên

Cú pháp cơ bản

```
kubectl port-forward <pod-name> <local-port>:<container-port>
```

16.2 Chuyển tiếp một cổng

Ví dụ: Truy cập nginx container chạy trên cổng 80

```
kubectl port-forward my-nginx-pod 8080:80
```

Điều này có tác dụng gì: Chuyển tiếp cổng cục bộ 8080 của bạn sang cổng 80 bên trong container. Sau đó, bạn có thể mở `http://localhost:8080` trong trình duyệt của mình hoặc chạy `curl http://localhost:8080` và yêu cầu được chuyển vào container.

Nhà ga vẫn bị chiếm giữ bởi đường hầm. Nhấn `Ctrl+C` để đóng nó.

Ví dụ: Chuyển tiếp tới cơ sở dữ liệu container trên cổng 5432

```
kubectl port-forward my-postgres-pod 5432:5432
```

Điều này có tác dụng gì: Cho phép bạn kết nối với PostgreSQL đang chạy trong cụm bằng ứng dụng khách `psql` cục bộ của bạn: `psql -h localhost -p 5432 -U myuser`

16.3 Chuyển tiếp nhiều cổng

Ví dụ: Chuyển tiếp hai cổng cùng lúc

```
kubectl port-forward my-app-pod 8080:80 9090:9090
```

Điều này có tác dụng gì: Chuyển tiếp cổng cục bộ 8080 sang cổng container 80 (HTTP) và cổng cục bộ 9090 đến cổng container 9090 (e.g., điểm cuối chỉ số Prometheus).

Chuyển tiếp cổng 16.4 tới Service hoặc Deployment

Bạn cũng có thể chuyển tiếp tới Service hoặc Deployment và Kubernetes chọn một trong các Pod sao lưu.

Ví dụ: Chuyển tiếp tới Service

```
kubectl port-forward service/my-app-service 8080:80
```

Ví dụ: Chuyển tiếp tới Deployment

```
kubectl port-forward deployment/my-app-deployment 8080:80
```

16.5 Chạy chuyển tiếp cổng trong nền

```
kubectl port-forward my-app-pod 8080:80 &  
PF_PID=$!  
# ... làm công việc của bạn ...  
kill $PF_PID
```

Việc này có tác dụng gì: Chạy chuyển tiếp cổng ở chế độ nền (&), ghi lại PID của nó và cho phép bạn tắt nó khi hoàn tất.

21.2 Chuyển tiếp cổng để truy cập cơ sở dữ liệu

Chuyển tiếp cổng đặc biệt hữu ích cho việc truy cập cơ sở dữ liệu trong quá trình gỡ lỗi:

```
# Chuyển tiếp tới PostgreSQL pod  
kubectl port-forward pod/postgres-0 5432:5432 -n database  
  
# Chuyển tiếp qua Service (tốt hơn - vẫn tồn tại khi khởi động lại pod)  
kubectl port-forward svc/postgres 5432:5432 -n database &  
  
# Bây giờ hãy kết nối từ localhost với bất kỳ ứng dụng khách PostgreSQL nào:  
psql -h localhost -U postgres -d mydb  
pg_dump -h localhost -U postgres mydb > backup.sql
```

```
# Chuyển tiếp tới Redis
kubectl port-forward svc/redis 6379:6379 -n cache &
redis-cli -h localhost ping      # PONG

# Chuyển tiếp tới Bảng điều khiển Kubernetes
kubectl port-forward svc/kubernetes-dashboard -n kubernetes-dashboard 8443:443 &
# Hiện đang mở: https://localhost:8443
```

21.3 Chuyển tiếp nhiều cổng đồng thời

```
# Chuyển tiếp nhiều cổng của cùng một pod
kubectl port-forward pod/myapp 8080:8080 9090:9090 6060:6060 &
# 8080 → cổng ứng dụng
# 9090 → Số liệu Prometheus
# 6060 → gỡ lỗi pprof

# Chuyển tiếp cổng ở chế độ nền với tính năng theo dõi PID
kubectl port-forward svc/elasticsearch 9200:9200 &
PF_PID=$!
echo "Port forward running with PID $PF_PID"

# Hãy làm công việc của bạn...
curl localhost:9200/_cluster/health

# Dọn dẹp khi hoàn tất
kill $PF_PID
```

21.4 Hạn chế của chuyển tiếp cổng

Chuyển tiếp cổng là **công cụ gỡ lỗi**, không phải là cơ chế định tuyến lưu lượng truy cập sản xuất:

- Mỗi lần chỉ có một khách hàng có thể sử dụng đường hầm (đó là một kết nối TCP duy nhất)
- Dừng khi quá trình kubectl thoát
- Không có TLS cho phía cục bộ (kết nối đơn giản là TCP cục bộ)
- Không thể chuyển tiếp UDP
- Không thể chuyển tiếp tới Services không đầu (không có ClusterIP)

Để truy cập bên ngoài sản xuất, hãy sử dụng:

- `Service.type=NodePort` - hiển thị trên mọi IP của node
 - `Service.type=LoadBalancer` - tạo bộ cân bằng tải đám mây
 - `Ingress / Gateway API` - Định tuyến HTTP với quy tắc hostname/path
-

Chương 22 - Giám sát tài nguyên bên trong container

22.1 Sử dụng `kubectl top`

`kubectl top` hiển thị mức tiêu thụ CPU và bộ nhớ theo thời gian thực của Pods và containers của chúng. Nó yêu cầu **Máy chủ số liệu** được cài đặt trong cụm của bạn.

Ví dụ: Xem mức sử dụng tài nguyên của tất cả Pods trong namespace hiện tại

```
kubectl top pods
```


Ví dụ: Xem mức sử dụng tài nguyên được chia nhỏ theo container

```
kubectl top pod my-app-pod --containers
```

Ví dụ đầu ra:

POD	NAME	CPU(cores)	MEMORY(bytes)
my-app-pod	app	15m	128Mi
my-app-pod	sidecar	2m	32Mi

Điều này cho bạn biết điều gì: app container đang sử dụng 15 millicores CPU và 128 MiB bộ nhớ. Điều này hữu ích cho việc định cỡ đúng tài nguyên requests và limits.

Ví dụ: Sắp xếp theo mức sử dụng bộ nhớ

```
kubectl top pods --sort-by=memory
```

Ví dụ: Sắp xếp theo CPU trên tất cả namespaces

```
kubectl top pods -A --sort-by=cpu
```

22.2 Kiểm tra các yêu cầu và giới hạn tài nguyên

Ví dụ: Xem tài nguyên requests và limits qua mô tả

```
kubectl describe pod my-app-pod | grep -A 6 "Limits\|Requests"
```

Ví dụ: Nhận tài nguyên requests qua JSONPath

```
kubectl get pod my-app-pod \
-o jsonpath='{.spec.containers[0].resources}'
```

22.3 Kiến trúc Metrics Server

Máy chủ số liệu là giải pháp trong cây dành cho các số liệu cốt lõi (CPU/memory). Đó là điều kiện tiên quyết cho `kubectl top` và HPA.

```
# Cài đặt máy chủ số liệu
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Đối với môi trường phòng thí nghiệm nơi nodes không có chứng chỉ TLS hợp lệ:
kubectl patch deployment metrics-server -n kube-system \
--type=json \
-p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
```

```
# Xác minh Máy chủ số liệu đang chạy và hoạt động tốt
kubectl get deployment metrics-server -n kube-system
kubectl top nodes
kubectl top pods --all-namespaces
```

22.4 ResourceQuota và LimitRange

ResourceQuota - Giới hạn tổng mức tiêu thụ namespace

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: production-quota
  namespace: production
spec:
  hard:
    # Tài nguyên tính toán
    requests.cpu: "20"           # Tổng CPU requests trong namespace
    requests.memory: 40Gi        # Tổng bộ nhớ requests
    limits.cpu: "40"             # Tổng CPU limits
    limits.memory: 80Gi          # Tổng bộ nhớ limits

    # Số lượng đối tượng limits
    pods: "100"                  # Tối đa Pods
    services: "20"               # Tối đa Services
    secrets: "50"                # Tối đa Secrets
    configmaps: "50"             # Tối đa ConfigMaps
    persistentvolumeclaims: "20" # Max PVCs

    # Hạn chế loại Service
    services.loadbalancers: "2"  # Chỉ 2 LoadBalancer Services
    services.nodeports: "0"      # No NodePort Services allowed

# Tạo ResourceQuota
kubectl apply -f quota.yaml

# Kiểm tra việc sử dụng hạn ngạch
kubectl describe resourcequota production-quota -n production
# Đầu ra:
# Tài nguyên được sử dụng nhiều
# -----
# limits.cpu           8      40
# limits.memory 16Gi 80Gi
# pods                 23     100
# requests.cpu         4      20
# requests.memory 8Gi 40Gi

# Khi đạt đến hạn ngạch, Pods sẽ không tạo được:
# Lỗi: pods "new-pod" bị cấm: vượt quá hạn ngạch: hạn ngạch sản xuất,
#      đã yêu cầu: pods=1, đã sử dụng: pods=100, bị giới hạn: pods=100
```

LimitRange - Đặt giới hạn và yêu cầu mặc định

Nếu không có LimitRange, người dùng có thể tạo Pods mà không cần tài nguyên requests, điều này khiến việc lập lịch không thể đoán trước được. LimitRange tự động đặt mặc định:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: production
spec:
  limits:
    - type: Container
      default:
        # Áp dụng làm giới hạn nếu không được chỉ định
        cpu: 500m
        memory: 512Mi
      defaultRequest:
        # Áp dụng theo yêu cầu nếu không được chỉ định
        cpu: 250m
        memory: 256Mi
      max:
        # Container không thể vượt quá những
        cpu: "4"
        memory: 4Gi
      min:
        # Container ít nhất phải yêu cầu điều này
        cpu: 100m
        memory: 128Mi
```

```
- type: Pod
  max:          # Tổng Pod limits không thể vượt quá
    cpu: "8"
    memory: 8Gi

- type: PersistentVolumeClaim
  max:
    storage: 50Gi    # PVC không thể yêu cầu nhiều hơn 50Gi
  min:
    storage: 1Gi     # PVC phải yêu cầu ít nhất 1Gi

# Xem phạm vi giới hạn trong namespace
kubectl describe limitrange default-limits -n production

# Pod mới không có tài nguyên sẽ nhận mặc định từ LimitRange:
kubectl run test --image=nginx -n production
kubectl describe pod test -n production | grep -A 5 "Limits:"
# Giới hạn:
#   cpu: 500m ← Được đặt theo mặc định LimitRange
#   bộ nhớ: 512Mi
# Yêu cầu:
#   cpu: 250m ← Được đặt bởi LimitRange defaultRequest
#   bộ nhớ: 256Mi
```

22.5 Lập kế hoạch năng lực cụm và node

```
# Tổng tài nguyên có thể phân bổ trên mỗi node
kubectl describe nodes | grep -A 5 "Allocatable:"

# Tổng tài nguyên được yêu cầu so với dung lượng trên tất cả nodes
kubectl describe nodes | grep -A 10 "Allocated resources:"

# Tìm nodes dưới áp lực bộ nhớ
kubectl get nodes -o custom-columns=\
'NAME:.metadata.name,CONDITIONS:.status.conditions[?(@.type=="MemoryPressure")].status'

# Kiểm tra các sự kiện áp lực tài nguyên trên nodes
kubectl describe node <node-name> | grep -A 20 "Events:"

# Liệt kê tất cả Pods được sắp xếp theo yêu cầu CPU
kubectl get pods -A -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
pods = []
for pod in data['items']:
    for c in pod['spec'].get('containers', []):
        cpu = c.get('resources', {}).get('requests', {}).get('cpu', '0')
        pods.append((pod['metadata']['namespace'], pod['metadata']['name'], c['name'], cpu))
for p in sorted(pods, key=lambda x: x[3], reverse=True)[:20]:
    print(f'{p[0]}/{p[1]} container={p[2]} cpu={p[3]}')
"
```

Chương 23 - Loại bỏ và sửa đổi các container trong Pods

23.1 Các trường bất biến của Pod đang chạy

Một trong những khái niệm quan trọng nhất cần hiểu trước khi cố gắng xóa hoặc sửa đổi containers trong Pod là **Kubernetes Pods phần lớn không thể thay đổi sau khi được tạo**. Danh sách container cốt lõi của Pod đang chạy - mảng `spec.containers[]` - không thể chỉnh sửa tại chỗ theo cách bạn có thể chỉnh sửa tệp văn bản. Đây là một quyết định thiết kế có chủ ý: thông số Pod xác định trạng thái mong muốn mà Kubernetes thực thi tại thời điểm tạo và việc sửa đổi nó một cách tùy ý sẽ khiến trạng thái thực tế không thể đoán trước được.

Tuy nhiên, điều này không có nghĩa là bạn bị mắc kẹt. Kubernetes cung cấp cho bạn một số cơ chế để đạt được kết quả thực tế trong việc loại bỏ hoặc thay đổi containers, tùy thuộc vào loại tài nguyên quản lý Pod.

Có bốn kịch bản chính mà chương này đề cập đến:

1. Xóa container khỏi **Pod trần (không được quản lý)** - yêu cầu xóa và tạo lại.
2. Xóa container khỏi Pod do **Deployment, StatefulSet hoặc DaemonSet quản lý** - cập nhật spec và Kubernetes triển khai thay đổi.
3. **Loại bỏ tiến trình của container** bên trong Pod để buộc khởi động lại mà không tạo lại Pod.
4. Sử dụng **bản vá hợp nhất chiến lược** và **bản vá JSON** để sửa đổi kỹ lưỡng các thông số kỹ thuật của Pod.

23.2 Vì sao không thể xóa container khỏi Pod đang chạy

Kubernetes 1.35: tài nguyên CPU/bộ nhớ của từng container có thể thay đổi qua `/resize`; danh sách container và lệnh khởi chạy vẫn có ràng buộc bất biến. Xem bài [resize Pod](#).

Cập nhật 1.35: CPU/bộ nhớ của từng container có thể thay đổi qua `/resize`. Danh sách hoặc tên container và lệnh khởi chạy vẫn không được sửa tùy ý trên Pod đang tồn tại. Xem bài [resize Pod](#).

Kubernetes thực thi các trường sau là bất biến khi chạy Pods:

- `spec.containers` (danh sách đầy đủ - bạn không thể thêm, xóa hoặc đổi tên containers)
- `spec.initContainers`
- `spec.volumes` (hầu hết là bất biến, một số trường có thể thay đổi)
- `spec.nodeName`
- `spec.serviceAccountName`

Nếu bạn cố gắng vá một Pod đang chạy để xóa mục nhập container, máy chủ API sẽ từ chối nó:

```
# Điều này sẽ bị máy chủ API từ chối
kubectl patch pod my-app-pod --type=json \
  -p='[{"op": "remove", "path": "/spec/containers/1"}]'
```

Lỗi dự kiến:

```
The Pod "my-app-pod" is invalid:
spec.containers: Forbidden: pod updates may not add or remove containers
```

Đây không phải là lỗi - đó là sự bảo vệ. Môi trường chạy container trên node quản lý vòng đời của container được kết hợp chặt chẽ với thông số Pod được sử dụng khi tạo. Việc thay đổi nó giữa chuyến bay sẽ khiến thời gian chạy ở trạng thái không nhất quán.

23.3 Xóa container khỏi Pod độc lập

Nếu Pod của bạn không được quản lý bởi bộ điều khiển cấp cao hơn (không có Deployment, không có StatefulSet), cách duy nhất để xóa container khỏi nó là xóa Pod và tạo lại nó bằng thông số kỹ thuật đã cập nhật.

Bước 1: Xuất thông số Pod hiện tại sang tệp YAML

```
kubectl get pod my-app-pod -o yaml > my-app-pod.yaml
```

Việc này có tác dụng gì: Xuất thông số kỹ thuật Pod trực tiếp đầy đủ sang tệp YAML. Điều này bao gồm tất cả các trường mà Kubernetes đã thêm (trạng thái, phiên bản tài nguyên, v.v.) cũng như thông số ban đầu của bạn.

Bước 2: Dọn dẹp YAML đã xuất

YAML đã xuất chứa các trường chỉ chạy trong thời gian chạy sẽ gây ra lỗi nếu được gửi lại cho API. Xóa các trường này:

```
# Xóa các trường được quản lý thời gian chạy bằng kubectl để thực hiện áp dụng chạy thử
# Hoặc xóa thủ công các phần này khỏi tệp YAML:
```

Các trường cần xóa khỏi YAML đã xuất trước khi gửi lại:

- metadata.uid
- metadata.resourceVersion
- metadata.creationTimestamp
- metadata.selfLink (nếu có)
- status (toàn bộ khối)
- spec.nodeName (bộc phải lên lịch lại)

Bước 3: Chỉnh sửa YAML để xóa container

Mở file YAML và tìm phần spec.containers:

```
# TRƯỚC - hai containers
spec:
  containers:
    - name: app
      image: my-app:v1
      ports:
        - containerPort: 8080
    - name: sidecar          # ← Xóa toàn bộ khối này
      image: log-shipper:latest
      volumeMounts:
        - name: logs
          mountPath: /var/log

# SAU - sidecar đã bị xóa
spec:
  containers:
    - name: app
      image: my-app:v1
      ports:
        - containerPort: 8080
```

Bước 4: Xóa Pod cũ và tạo lại với thông số kỹ thuật mới

```
# Xóa Pod đang chạy (lưu ý: đối với pods trần, điều này gây ra thời gian ngừng hoạt động)
kubectl delete pod my-app-pod
```

```
# Tạo lại với thông số kỹ thuật được cập nhật
kubectl apply -f my-app-pod.yaml
```

Tắt cả trong một với kubectl thay thế --force (nhanh hơn nhưng có tính hủy diệt)

```
# Chỉnh sửa tệp tin trước, sau đó:
kubectl replace --force -f my-app-pod.yaml
```

--force *làm gì*: Xóa tài nguyên hiện có và tạo lại ngay tài nguyên đó khỏi tệp. Điều này tương đương với delete + create trong một lệnh. **Cảnh báo**: Có một khoảng thời gian ngừng hoạt động ngắn giữa quá trình xóa và giải trí.

23.4 Xóa container khỏi Pod do Deployment quản lý

Đây là phương pháp sản xuất đúng đắn. Khi Pods được quản lý bởi Deployment, bạn cập nhật thông số mẫu Pod của Deployment - không phải trực tiếp Pod. Kubernetes sau đó thực hiện cập nhật luân phiên, thay thế Pods cũ (bằng sidecar) bằng Pods mới (không có sidecar).

Phương pháp 1: Chỉnh sửa kubectl (tương tác)

```
kubectl edit deployment my-app-deployment
```

Việc này có tác dụng gì: Mở thông số Deployment trong trình chỉnh sửa mặc định của bạn (được đặt bởi \$EDITOR hoặc \$KUBE_EDITOR). Điều hướng đến spec.template.spec.containers, tìm container bạn muốn xóa và xóa toàn bộ khối của nó. Lưu và đóng tập tin. Kubernetes ngay lập tức bắt đầu cập nhật liên tục.

Phương pháp 2: Bản vá kubectl với Bản vá JSON (chính xác, có thể viết được tập lệnh)

JSON Hoạt động của bản vá cho phép bạn sửa đổi các phần tử mảng cụ thể một cách khéo léo. Để xóa container thứ hai (chỉ mục 1) khỏi danh sách containers:

```
kubectl patch deployment my-app-deployment --type=json \
-p='[{"op": "remove", "path": "/spec/template/spec/containers/1"}]'
```

Giải thích về hoạt động của Bản vá JSON:

hoạt động	Giá trị op	Hiệu ứng
Xóa phần tử	"remove"	Xóa phần tử tại đường dẫn
Thay thế giá trị	"replace"	Thay thế giá trị tại đường dẫn
Thêm phần tử	"add"	Thêm vào đường dẫn
Sao chép phần tử	"copy"	Sao chép từ đường dẫn này sang đường dẫn khác

Phương pháp 3: Bản vá kubectl với Bản vá hợp nhất chiến lược (theo tên)

Bản vá hợp nhất chiến lược là phần mở rộng dành riêng cho Kubernetes của bản vá hợp nhất JSON. Đối với containers, bạn có thể hướng dẫn Kubernetes xóa container theo tên bằng cách sử dụng lệnh \$patch: delete:

```
kubectl patch deployment my-app-deployment --type=strategic \
-p='{"spec":{"template":{"spec":{"containers":[{"name":"sidecar","$patch":"delete"}]}}}}'
```

Điều này có tác dụng gì: Kubernetes sử dụng container name làm khóa hợp nhất. Việc đặt \$patch: delete trên container có tên sẽ yêu cầu hợp nhất chiến lược loại bỏ container đó khỏi mảng, thay vì cố gắng hợp nhất nó. Đây là dựa trên tên, không dựa trên chỉ mục, làm cho nó an toàn hơn Bản vá JSON.

Phương pháp 4: Chỉnh sửa và áp dụng từ một tệp

```
# Nhận thông số triển khai
kubectl get deployment my-app-deployment -o yaml > deployment.yaml

# Chỉnh sửa deployment.yaml - xóa container khỏi spec.template.spec.containers

# Áp dụng thay đổi
kubectl apply -f deployment.yaml
```

23.5 Theo dõi quá trình triển khai sau khi loại bỏ container

Sau khi chỉnh sửa Deployment để xóa container, Kubernetes thực hiện cập nhật luân phiên. Bạn có thể theo dõi điều này trong thời gian thực.

Ví dụ: Xem trạng thái triển khai

```
kubectl rollout status deployment/my-app-deployment
```

Đầu ra trong khi lần:

```
Waiting for deployment "my-app-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "my-app-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "my-app-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "my-app-deployment" successfully rolled out
```

Ví dụ: Xem Pods đang được thay thế

```
kubectl get pods -w -l app=my-app
```

Cờ `-w` (xem) truyền phát các cập nhật trực tiếp về các thay đổi trạng thái Pod cho thiết bị đầu cuối của bạn.

Ví dụ: Khôi phục nếu có sự cố

```
kubectl rollout undo deployment/my-app-deployment
```

Việc này thực hiện: Hoàn nguyên Deployment về bản sửa đổi trước đó, khôi phục container đã bị xóa. Kubernetes lưu giữ lịch sử sửa đổi có thể định cấu hình (mặc định: 10).

Ví dụ: Xem lịch sử triển khai

```
kubectl rollout history deployment/my-app-deployment
```

```
# Xem những gì đã thay đổi trong một bản sửa đổi cụ thể
kubectl rollout history deployment/my-app-deployment --revision=3
```

23.6 Xóa các container khỏi StatefulSets và DaemonSets

Cách tiếp cận `kubectl patch` và `kubectl edit` tương tự áp dụng cho StatefulSets và DaemonSets, với một điểm khác biệt quan trọng: StatefulSets thực hiện cập nhật từng đợt một Pod (theo thứ tự) và DaemonSets cập nhật một node mỗi lần.

Ví dụ: Xóa container khỏi StatefulSet

```
kubectl patch statefulset my-statefulset --type=json \
-p='[{"op": "remove", "path": "/spec/template/spec/containers/1"}]'
```

Ví dụ: Xóa container khỏi DaemonSet

```
kubectl patch daemonset my-daemonset --type=strategic \
-p='{ "spec": { "template": { "spec": { "containers": [ { "name": "log-agent", "$patch": "delete" } ] } } } }'
```

23.7 Dừng tiến trình container để buộc khởi động lại

Đôi khi bạn không muốn xóa container vĩnh viễn - bạn chỉ muốn buộc nó khởi động lại (e.g., để chọn cấu hình mới hoặc để khôi phục sau lỗi). Bạn có thể tắt quy trình chính của container từ bên trong Pod. Kubernetes sẽ phát hiện quá trình thoát và khởi động lại container theo `restartPolicy` của Pod.

Ví dụ: Gửi SIGTERM (tắt máy nhẹ nhàng) tới PID 1 trong container

```
kubectl exec my-app-pod -c sidecar -- kill -15 1
```

Điều này có tác dụng gì: Gửi tín hiệu 15 (SIGTERM) đến PID 1 bên trong `sidecar` container. Một ứng dụng hoạt động tốt sẽ tắt một cách duyên dáng. Kubernetes thấy container thoát ra và khởi động lại nó.

Ví dụ: Gửi SIGKILL (tiêu diệt ngay lập tức, mạnh mẽ) tới PID 1

```
kubectl exec my-app-pod -c sidecar -- kill -9 1
```

Điều này có tác dụng gì: Tín hiệu 9 (SIGKILL) không thể bị bắt hoặc bỏ qua bởi quy trình - kernel sẽ tắt nó ngay lập tức. Sử dụng tính năng này khi container bị đóng băng và không phản hồi với SIGTERM.

Ví dụ: Tìm và hủy một quy trình cụ thể (không phải PID 1)

```
# Bước 1: Tìm ID tiến trình
kubectl exec my-app-pod -- ps aux | grep "stuck-process"
```

```
# Bước 2: Tiêu diệt nó bằng PID của nó
kubectl exec my-app-pod -- kill -9 <PID>
```

Ví dụ: Khởi động lại tất cả containers trong Pod bằng cách xóa Pod (dành cho Deployment do Pods quản lý)

```
kubectl delete pod my-app-pod-xyz123
```

Điều này có tác dụng gì: Kubernetes (thông qua bộ điều khiển ReplicaSet của Deployment) ngay lập tức tạo một Pod thay thế. Đây là cách rõ ràng nhất để buộc Pod khởi động lại hoàn toàn khi Pod được quản lý bởi Deployment. Pod cũ kết thúc một cách duyên dáng và một cái mới được lên lịch.

23.8 Loại bỏ init container

Khởi tạo containers chạy đến khi hoàn thành trước khi containers chính bắt đầu. Chúng cũng có thể được loại bỏ bằng cách sử dụng cách tiếp cận bản vá hợp nhất chiến lược tương tự:

Ví dụ: Xóa init container theo tên

```
kubectl patch deployment my-app-deployment --type=json \
  -p='[{"op": "remove", "path": "/spec/template/spec/initContainers/0"}]'
```

Lưu ý: containers ban đầu có tại /spec/template/spec/initContainers, không phải /spec/template/spec/containers.

23.9 Thu nhỏ xuống 0 (Tạm thời loại bỏ tất cả các container)

Nếu bạn cần "dừng" tất cả containers do Deployment quản lý một cách hiệu quả mà không xóa chính Deployment:

Ví dụ: Co giãn Deployment thành 0 bản sao

```
kubectl scale deployment my-app-deployment --replicas=0
```

Việc này có tác dụng gì: Chấm dứt tất cả Pods đang chạy của Deployment. Không chạy containers. Bản thân đối tượng Deployment, cấu hình của nó, ConfigMaps và Secrets đều vẫn còn nguyên.

Ví dụ: Khôi phục bằng cách sao lưu quy mô

```
kubectl scale deployment my-app-deployment --replicas=3
```

Mẫu này hữu ích cho:

- Các cửa sổ bảo trì mà bạn cần tắt tất cả containers đã tạm thời dừng lại.
- Gỡ lỗi Deployment mà không có lưu lượng truy cập vào nó.
- Tiết kiệm chi phí trong môi trường phi sản xuất.

Ví dụ: Giảm quy mô tất cả Deployments trong namespace xuống 0 (chế độ bảo trì)

```
kubectl get deployments -n my-namespace -o name | \
  xargs -I {} kubectl scale {} --replicas=0 -n my-namespace
```

23.10 Khắc phục sự cố lỗi sửa đổi Pod

Lỗi: không thể cập nhật trường thông số pod

```
$ kubectl edit pod my-pod
# Sau khi lưu:
pods "my-pod" was not valid:
* spec: Forbidden: pod updates may not change fields other than
  `spec.containers[*].image`, `spec.initContainers[*].image`,
  `spec.activeDeadlineSeconds`, `spec.tolerations` (only additions),
  `spec.terminationGracePeriodSeconds`

# Bạn đã cố gắng thay đổi một trường bất biến (lệnh, cổng, ổ đĩa, v.v.)
# Giải pháp: xuất, chỉnh sửa, xóa, tạo lại
kubectl get pod my-pod -o yaml > pod.yaml
# Chỉnh sửa pod.yaml - thực hiện các thay đổi của bạn
kubectl delete pod my-pod
kubectl apply -f pod.yaml
```


Cập nhật container image của Deployment (không ngừng hoạt động)

Khuyến nghị: Đặt lệnh image (kích hoạt cập nhật luân phiên)

```
kubectl set image deployment/web app=nginx:1.25.3
```

Xem bản cập nhật cuộn

```
kubectl rollout status deployment/web
```

Đang chờ quá trình triển khai "web" triển khai kết thúc:

1 trong 3 bản sao mới đã được cập nhật...

2 trong số 3 bản sao mới đã được cập nhật...

3 trong số 3 bản sao mới đã được cập nhật...

triển khai "web" được triển khai thành công

Xem pods trong khi cập nhật

```
kubectl get pods -l app=web -w
```

# NAME	READY	STATUS	RESTARTS	AGE
# web-abc123	1/1	Running	0	2m
# web-def456	1/1	Running	0	2m

web-ghi789 0/1 ContainerTạo 0 5s ← mới

web-abc123 1/1 Chấm dứt 0 2m ← cũ

web-jkl012 0/1 ContainerTạo 0 5s ← mới

Hoàn tác nếu có sự cố xảy ra

```
kubectl rollout undo deployment/web
```

Hoàn tác bản sửa đổi cụ thể

```
kubectl rollout history deployment/web
```

```
kubectl rollout undo deployment/web --to-revision=3
```

Buộc khởi động lại Pod mà không thay đổi thông số kỹ thuật

Cách 1: Xóa Pod (Deployment tạo lại)

```
kubectl delete pod -l app=my-app # Xóa tất cả pods phù hợp (được tạo lại bằng cách triển khai)
```

Phương pháp 2: Thêm chú thích để kích hoạt khởi động lại

```
kubectl rollout restart deployment/my-app
```

Điều này thêm chú thích restartAt vào mẫu pod, kích hoạt cập nhật luân phiên

Phương pháp 3: Co giãn về 0 và sao lưu (thời gian ngừng hoạt động ngắn)

```
kubectl scale deployment my-app --replicas=0
```

```
kubectl scale deployment my-app --replicas=3
```

Cách 4: Đối với StatefulSet

```
kubectl rollout restart statefulset/my-db
```

23.11 Vá Deployment đang hoạt động

```
# Thêm biến môi trường vào triển khai đang chạy
kubectl set env deployment/my-app NEW_VAR=new-value

# Xóa một biến môi trường
kubectl set env deployment/my-app NEW_VAR-

# Thêm giới hạn tài nguyên
kubectl set resources deployment/my-app \
  --containers=app \
  --limits=cpu=500m,memory=256Mi \
  --requests=cpu=250m,memory=128Mi

# Cập nhật với bản vá JSON (thay đổi phễu thuật)
kubectl patch deployment my-app \
  --type=json \
  -p='[{"op":"replace","path":"/spec/template/spec/containers/0/image","value":"nginx:1.25"}]'
```

```
# Bản vá hợp nhất chiến lược (bản vá hợp nhất, cú pháp dễ dàng hơn)
kubectl patch deployment my-app --type=strategic -p='
spec:
  template:
    spec:
      containers:
      - name: app
        env:
        - name: LOG_LEVEL
          value: debug
      ,

# Thêm ổ đĩa vào quá trình triển khai hiện có
kubectl patch deployment my-app --type=strategic -p='
spec:
  template:
    spec:
      volumes:
      - name: tmp-cache
        emptyDir: {}
      containers:
      - name: app
        volumeMounts:
        - name: tmp-cache
          mountPath: /tmp/cache
      ,
```

Chương 24 - Tạo và xóa container thử nghiệm tại chỗ

24.1 Khái niệm về container thử nghiệm tạm thời

Một trong những mô hình mạnh mẽ nhất trong quá trình gỡ lỗi Kubernetes là tạo một container tạm thời, dùng một lần bên trong cụm để thực hiện kiểm tra - gửi HTTP requests, dịch vụ ping, phân giải tên DNS, theo dõi các tuyến đường - và sau đó hủy nó khi hoàn tất.

Cách tiếp cận này tốt hơn so với việc chạy thử nghiệm từ máy cục bộ của bạn vì:

- **Cùng mạng namespace:** container chạy bên trong cụm và có quyền truy cập vào Services, ClusterIP và DNS nội bộ không thể truy cập được từ bên ngoài.
- **Các chính sách tương tự của RBAC/network:** Bạn kiểm tra các chính sách mạng thực tế đang có hiệu lực.
- **Môi trường sạch sẽ:** container được cách ly và dùng một lần.

24.2 Mẫu `kubectl run --rm -it`

Mẫu phổ biến nhất sử dụng `kubectl run` với cờ `--rm` (xóa khi thoát) và `-it` (thiết bị đầu cuối tương tác).

Cú pháp cơ bản

```
kubectl run <pod-name> \  
  --image=<image> \  
  --rm \  
  -it \  
  --restart=Never \  
  -- <command>
```

Giải thích về cờ:

Cờ	Mục đích
<code>--rm</code>	Tự động xóa Pod khi container thoát
<code>-it</code>	Thiết bị đầu cuối tương tác (stdin + TTY)
<code>--restart=Never</code>	Không khởi động lại container (chạy một lần và thoát)
<code>--</code>	Dấu phân cách: mọi thứ sau đây là lệnh chạy trong container

24.3 Tạo thử nghiệm Busybox Pod cho các tiện ích mạng

`busybox` là một image cực kỳ nhỏ (khoảng 1-5 MB) đóng gói hàng chục tiện ích UNIX vào một tệp nhị phân duy nhất. Nó bao gồm `ping`, `nslookup`, `wget`, `telnet`, `nc` (`netcat`), v.v.

Ví dụ: Tạo một busybox tương tác shell

```
kubectl run test-shell \  
  --image=busybox \  
  --rm \  
  -it \  
  --restart=Never \  
  -- /bin/sh
```

Điều này làm gì: Tạo một Pod có tên `test-shell` đang chạy `busybox`, mở shell bên trong nó. Khi bạn nhập `exit`, Kubernetes sẽ tự động xóa Pod. Bây giờ bạn có shell bên trong cụm của mình mà từ đó bạn có thể chạy bất kỳ thử nghiệm mạng nào.

Ví dụ: Chạy ping một lần và hủy

```
kubectl run ping-test \  
  --image=busybox \  
  --rm \  
  -it \  
  --restart=Never \  
  -- ping -c 4 google.com
```

Việc này làm gì: Tạo một Pod, ping `google.com` 4 lần, in kết quả, sau đó Pod sẽ tự động bị xóa. Cờ `-c 4` limits ping tới 4 gói.

Ví dụ: Ping Kubernetes Service nội bộ theo tên DNS của nó

```
kubectl run ping-test \  
  --image=busybox \  
  --rm \  
  -it \  
  --restart=Never \  
  -- ping -c 4 my-service.my-namespace.svc.cluster.local
```

Điều này có tác dụng gì: Kiểm tra khả năng kết nối với Kubernetes Service bằng tên miền đủ điều kiện (FQDN). Điều này cũng gián tiếp kiểm tra xem CoreDNS có đang phân giải chính xác tên dịch vụ hay không.

24.4 Tạo container thử nghiệm có curl

wget của busybox bị giới hạn so với curl. Để kiểm tra HTTP đầy đủ, hãy sử dụng curlimages/curl hoặc nicolaka/netshoot.

Ví dụ: Chạy một yêu cầu curl và hủy Pod

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -s http://my-service.my-namespace.svc.cluster.local
```

Điều này làm gì: Tạo Pod bằng cách sử dụng curlimages/curl image nhẹ, chạy curl dựa trên điểm cuối Service nội bộ, in phản hồi HTTP tới thiết bị đầu cuối của bạn, sau đó xóa Pod.

Ví dụ: Kiểm tra một dịch vụ nội bộ với kết quả đầu ra dài dòng

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -v http://my-service:8080/health
```

Công dụng của cờ -v: Cho phép xuất chi tiết, hiển thị đầy đủ tiêu đề yêu cầu, tiêu đề phản hồi, chi tiết bắt tay TLS (nếu HTTPS) và nội dung phản hồi. Cần thiết để gỡ lỗi các sự cố HTTP.

Ví dụ: Kiểm tra với tiêu đề tùy chỉnh và nội dung POST

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -X POST \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer mytoken" \
    -d '{"key":"value"}' \
    http://my-api-service:8080/api/endpoint
```

Điều này có tác dụng gì: Gửi yêu cầu POST có nội dung JSON và tiêu đề xác thực tới dịch vụ API nội bộ. Đây chính xác là cách bạn sẽ kiểm tra một microservice mà không để lộ nó ra bên ngoài.

Ví dụ: Đo thời gian phản hồi HTTP

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -o /dev/null -s -w \
    "Time to connect: %{time_connect}s\nTime total: %{time_total}s\nHTTP Status: %{http_code}\n" \
    http://my-service:8080
```

Điều này có tác dụng gì: Ngăn chặn đầu ra nội dung (-o /dev/null), chạy im lặng (-s) và in bản tóm tắt được định dạng về số liệu thời gian. Hữu ích cho việc đo độ trễ cơ bản.

24.5 Sử dụng netshoot: image chứa bộ công cụ mạng

nicolaka/netshoot là container image được thiết kế nhằm mục đích khắc phục sự cố mạng. Nó bao gồm: curl, wget, ping, traceroute, nmap, netstat, ss, dig, nslookup, iperf3, tcpdump, nftables, iptables, mtr, socat, jq, và nhiều hơn nữa.

Ví dụ: Khởi chạy phiên tương tác netshoot

```
kubectl run netshoot \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- bash
```

Sau khi vào trong, bạn có bộ công cụ đầy đủ:

```
# Bên trong netshoot container:

# Ping một dịch vụ nội bộ
ping -c 3 my-service.default.svc.cluster.local

# Tra cứu DNS
dig my-service.default.svc.cluster.local

# Theo dõi lộ trình mạng
traceroute my-service.default.svc.cluster.local

# Quét các cổng đang mở trên một dịch vụ
nmap -p 80,443,8080 my-service.default.svc.cluster.local

# Kiểm tra xem cổng TCP có mở không (netcat)
nc -zv my-service 8080

# Yêu cầu HTTP với đầy đủ chi tiết
curl -v http://my-service:8080/health

# Chụp các gói trên giao diện mạng
tcpdump -i eth0 -n host my-service
```

Ví dụ: Chạy traceroute trực tiếp (không tương tác)

```
kubectl run trace-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- traceroute my-service.default.svc.cluster.local
```

****Ví dụ: Kiểm tra xem một cổng TCP cụ thể có mở không ****

```
kubectl run nc-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- nc -zv my-postgres-service 5432
```

Điều này làm gì: Sử dụng netcat (nc) ở chế độ zero-I/O (-z) với đầu ra dài dòng (-v) để kiểm tra xem cổng 5432 (PostgreSQL) có chấp nhận kết nối TCP trên dịch vụ đích hay không. Đây là một thử nghiệm kết nối thuần túy không truyền dữ liệu.

24.6 Chạy Pod thử nghiệm trong namespace cụ thể

Theo mặc định, `kubectl run` tạo Pod trong namespace hiện tại. Để kiểm tra khả năng kết nối giữa các dịch vụ, bạn thường cần chạy thử nghiệm Pod trong một namespace cụ thể.

Ví dụ: Tạo Pod thử nghiệm trong namespace cụ thể

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -n production \
  -- curl -s http://api-service.production.svc.cluster.local:8080/health
```

Điều này có tác dụng gì: Tạo Pod thử nghiệm trong production namespace, cho phép bạn thử nghiệm từ phối cảnh mạng của namespace đó, tôn trọng NetworkPolicies áp dụng cho namespace đó.

24.7 Kết nối vào Pod thử nghiệm đang chạy

Đôi khi bạn muốn tạo Pod mà không cần lệnh và gắn riêng vào nó.

Ví dụ: Tạo thử nghiệm chạy dài Pod

```
kubectl run debug-pod \
  --image=nicolaka/netshoot \
  --restart=Never \
  -- sleep 3600
```

Ví dụ: Gắn shell vào Pod đang chạy

```
kubectl exec -it debug-pod -- bash
```

Ví dụ: Xóa Pod theo cách thủ công khi hoàn tất

```
kubectl delete pod debug-pod
```

24.8 Tạo thử nghiệm Pod với các biến môi trường

Ví dụ: Chèn các biến môi trường để thử nghiệm

```
kubectl run test-pod \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  --env="API_URL=http://my-service:8080" \
  --env="API_TOKEN=mytoken" \
  -- sh -c 'curl -H "Authorization: Bearer $API_TOKEN" $API_URL/endpoint'
```

Điều này làm gì: Đưa các biến môi trường vào container thử nghiệm và sử dụng chúng trong lệnh. Điều này mô phỏng cách ứng dụng containers của bạn sử dụng các biến này.

24.9 Xác minh xóa Pod sau --rm

Sau khi thoát thử nghiệm Pod, hãy xác nhận rằng nó đã được dọn sạch:

```
kubectl get pods | grep test
```

Nếu vì lý do nào đó --rm không thành công (kết nối e.g., kubectl bị rớt), hãy dọn dẹp thủ công:

```
# Liệt kê và xóa bài kiểm tra bị kẹt pods
kubectl delete pod curl-test ping-test netshoot debug-pod --ignore-not-found=true
```

Cờ --ignore-not-found=true ngăn lỗi nếu Pod đã bị xóa.

Chương 25 - Kiểm tra mạng từ các container tạm thời

25.1 Kiểm tra kết nối HTTP

Phần này cung cấp các ví dụ được nhắm mục tiêu, sẵn sàng sản xuất về các thử nghiệm mạng.

Ví dụ: Kiểm tra HTTP và chỉ chụp mã phản hồi

```
kubectl run http-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -o /dev/null -s -w "%{http_code}" http://my-service:8080/
```

Đầu ra dự kiến: 200 - hoặc bất kỳ mã trạng thái HTTP nào mà dịch vụ trả về.

Ví dụ: Kiểm tra HTTPS khi xác minh chứng chỉ bị tắt (đối với chứng chỉ tự ký nội bộ)

```
kubectl run https-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -k -s -o /dev/null -w "%{http_code}" https://my-secure-service:8443/
```

-k làm gì: Vô hiệu hóa xác minh chứng chỉ SSL. Hữu ích khi thử nghiệm các dịch vụ có chứng chỉ tự ký trong các cụm nội bộ.

Không bao giờ sử dụng -k trong sản xuất requests.

Ví dụ: Theo dõi chuyển hướng HTTP

```
kubectl run redirect-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -L -s -o /dev/null -w "%{http_code} final URL: %{url_effective}\n" \
    http://my-service:8080/old-path
```

-L làm gì: Đi theo các chuyển hướng HTTP. Biến `%{url_effective}` hiển thị URL cuối cùng sau tất cả các chuyển hướng.

25.2 Kiểm tra kết nối TCP

Ví dụ: Kiểm tra cổng TCP bằng netcat (nc)

```
kubectl run tcp-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'nc -zv my-db-service 5432 && echo "Port OPEN" || echo "Port CLOSED"'
```

Việc này có tác dụng gì: Kiểm tra xem cổng TCP 5432 có chấp nhận kết nối hay không. Logic `&&` và `||` in kết quả mà con người có thể đọc được.

Ví dụ: Kiểm tra nhiều cổng trong một lần chạy

```
kubectl run port-scanner \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'for port in 80 443 8080 9090; do
    nc -zv my-service $port 2>&1 && echo " $port: OPEN" || echo " $port: CLOSED"
done'
```

Việc này có tác dụng gì: Duyệt qua danh sách các cổng và kiểm tra từng cổng. Điều này rất hữu ích để nhanh chóng kiểm tra xem dịch vụ nào đang thực sự lắng nghe trên cổng nào.

25.3 Kiểm tra độ trễ và băng thông

Ví dụ: Đo độ trễ khứ hồi bằng ping

```
kubectl run latency-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- ping -c 10 my-service.default.svc.cluster.local
```

Nội dung cần tìm ở đầu ra: Dòng min/avg/max/stddev ở cuối. Giá trị stddev cao gợi ý hiện tượng giật mạng. Tỷ lệ mất gói cho thấy vấn đề kết nối.

Ví dụ: Kiểm tra băng thông với iperf3 (yêu cầu máy chủ iperf3 trên mục tiêu)

```
# Đầu tiên: Khởi động máy chủ iperf3 trong mục tiêu pod
kubectl exec my-target-pod -- iperf3 -s -D
```

```
# Sau đó: Chạy thử nghiệm máy khách iperf3
kubectl run iperf-client \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- iperf3 -c my-target-service -p 5201 -t 10
```

25.4 Ma trận xử lý sự cố mạng

Symptom	→ Check
Pod can't reach Service	→ Service selector matches Pod labels? kubectl get endpoints <svc> (should not be empty) kube-proxy running on node?
Pod can't reach external URL	→ DNS resolving? (nslookup from pod) NetworkPolicy blocking egress? Node has internet access? (curl from node)
Service accessible from pod but not from outside cluster	→ Ingress rules correct? NodePort/LoadBalancer Service type? Cloud firewall rules? Ingress controller running?
Pod DNS not resolving	→ CoreDNS running? (kubectl get pods -n kube-system) Pod's dnsPolicy correct? /etc/resolv.conf in pod correct?
Intermittent connection failures	→ Pod anti-affinity spreading? Resource limits causing restarts? HPA scaling creating new pods?

25.5 Lệnh kiểm tra mạng nâng cao

```
# Kiểm tra kết nối hoàn chỉnh từ bản gỡ lỗi pod
kubectl run nettest --image=nicolaka/netshoot --rm -it --restart=Never -- bash

# Netshoot bên trong:
# Kiểm tra DNS
nslookup kubernetes.default.svc.cluster.local
dig +short my-service.production.svc.cluster.local

# Kiểm tra khả năng tiếp cận cổng TCP
nc -zv my-service.production.svc.cluster.local 8080
# Kết nối với cổng my-service.production.svc.cluster.local (10.96.45.23) 8080 [tcp/*] đã thành công!

# Traceroute (đường dẫn qua mạng)
traceroute my-service.production.svc.cluster.local

# Kiểm tra sâu HTTP
curl -v http://my-service.production.svc.cluster.local/api/health
# Hiển thị đầy đủ các tiêu đề request/response, bắt tay TLS, thời gian

# Kiểm tra kích thước MTU/packet (phát hiện các vấn đề phân mảnh)
ping -M do -s 1450 my-service.production.svc.cluster.local

# Kiểm tra kết nối node-to-node (chạy từ pod được lên lịch trên mỗi node)
for node_ip in 10.0.0.1 10.0.0.2 10.0.0.3; do
  echo -n "Node $node_ip: "
  nc -zv -w2 $node_ip 10250 2>&1 | grep -E "open|refused|timeout"
done

# Kiểm tra băng thông giữa pods
kubectl run iperf-server --image=networkstatic/iperf3 --restart=Never -- -s
kubectl run iperf-client --image=networkstatic/iperf3 --restart=Never -- \
  -c iperf-server.default.svc.cluster.local -t 10
```

25.6 Chẩn đoán chặn NetworkPolicy

```
# Kiểm tra xem có NetworkPolicies nào tồn tại trong namespace không
kubectl get networkpolicies -n my-namespace

# Mô phỏng kết nối đang bị chặn
kubectl run test-client --image=curlimages/curl --rm -it --restart=Never \
  -n source-namespace -- curl -v http://my-service.target-namespace.svc.cluster.local

# Nếu bị chặn, hãy kiểm tra NetworkPolicies trong mục tiêu namespace
kubectl get networkpolicies -n target-namespace -o yaml

# Kiểm tra xem chính sách "từ chối tất cả" có hiệu lực không
kubectl get networkpolicies -n target-namespace | grep deny-all

# Theo dõi chính sách phù hợp (chính sách nào phù hợp với kết nối này?)
# Không có công cụ tích hợp sẵn, nhưng Cilium cung cấp công cụ này:
# kubectl exec -n kube-system cilium-xxx -- giám sát cilium --type thả
# Đối với Calico:
# kubectl exec -n kube-system calico-node-xxx -- khớp chính sách calicoctl

# Tạm thời vô hiệu hóa NetworkPolicy để thử nghiệm (không bao giờ được sản xuất!)
kubectl delete networkpolicy deny-all -n target-namespace
curl -v http://my-service.target-namespace.svc.cluster.local
kubectl apply -f deny-all-policy.yaml # Khôi phục
```

Chương 26 - CoreDNS: Kiểm tra, chẩn đoán và xác minh phân giải DNS

26.1 CoreDNS là gì?

CoreDNS là máy chủ DNS mặc định trong cụm Kubernetes (thay thế kube-dns kể từ Kubernetes 1.11). Nó chạy dưới dạng Deployment trong kube-system namespace và mọi Pod trong cụm đều sử dụng nó làm trình phân giải DNS.

CoreDNS chịu trách nhiệm:

- Giải quyết tên Kubernetes Service thành địa chỉ ClusterIP
- Giải quyết tên Pod DNS
- Chuyển tiếp các truy vấn DNS bên ngoài (e.g., google.com) tới các trình phân giải ngược dòng
- Hỗ trợ cấu hình DNS tùy chỉnh thông qua Corefile của nó

Nếu CoreDNS không hoạt động bình thường thì Pods không thể giao tiếp với nhau bằng tên dịch vụ, điều này hầu như phá vỡ mọi kiến trúc vi dịch vụ.

26.2 Quy ước đặt tên DNS trong Kubernetes

Hiểu các mẫu đặt tên là điều cần thiết để thử nghiệm DNS.

Tên Service DNS:

mẫu	Giải quyết để	Ví dụ
<service>	ClusterIP (tương tự namespace)	my-service
<service>.<namespace>	ClusterIP	my-service.production
<service>.<namespace>.svc	ClusterIP	my-service.production.svc
<service>.<namespace>.svc.cluster.local	ClusterIP (FQDN)	my-service.production.svc.cluster.local

Tên Pod DNS (khi hostname được đặt):

```
<pod-ip-dashes>.<namespace>.pod.cluster.local  
# e.g.: 10-0-1-5.default.pod.cluster.local
```

26.3 Kiểm tra trạng thái CoreDNS

Ví dụ: Xem CoreDNS Deployment

```
kubectl get deployment coredns -n kube-system
```

Những gì cần tìm: Cột READY sẽ hiển thị tất cả các bản sao đã sẵn sàng (e.g., 2/2). Nếu nó hiển thị 0/2 hoặc 1/2 thì CoreDNS có vấn đề và service discovery sẽ bị suy giảm.

Ví dụ: Xem CoreDNS Pods

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Ví dụ: Kiểm tra lỗi trong nhật ký CoreDNS

```
kubectl logs -n kube-system -l k8s-app=kube-dns --all-containers=true
```

Nội dung cần tìm: Các dòng chứa SERVFAIL, REFUSED, error hoặc các truy vấn lặp lại không có phản hồi. Những điều này cho thấy CoreDNS đang nhận được truy vấn nhưng không giải quyết được chúng.

Ví dụ: Theo dõi nhật ký CoreDNS trong thời gian thực trong khi kiểm tra DNS

```
kubectl logs -f -n kube-system -l k8s-app=kube-dns
```

Mẹo chuyên nghiệp: Mở phần này trong một thiết bị đầu cuối riêng biệt, sau đó chạy thử nghiệm DNS của bạn. Bạn có thể thấy CoreDNS nhận và xử lý từng truy vấn trong thời gian thực.

26.4 Xem cấu hình CoreDNS (Corefile)

CoreDNS được định cấu hình thông qua ConfigMap có tên coredns trong kube-system namespace.

Ví dụ: Xem CoreDNS Corefile

```
kubectl get configmap coredns -n kube-system -o yaml
```

Những gì cần tìm ở đầu ra:

```
data:
  Corefile: |
    .:53 {
      errors
      health {
        lameduck 5s
      }
      ready
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
      }
      prometheus :9153
      forward . /etc/resolv.conf {
        max_concurrent 1000
      }
      cache 30
      loop
      reload
      loadbalance
    }
```

Các khối cấu hình chính:

- kubernetes cluster.local: Xử lý độ phân giải cho tên .cluster.local (tất cả các dịch vụ Kubernetes)
- forward . /etc/resolv.conf: Chuyển tiếp các truy vấn DNS bên ngoài lên thượng nguồn (trình phân giải của node)
- cache 30: Lưu phản hồi vào bộ nhớ đệm trong 30 giây
- loop: Phát hiện và ngăn chặn các vòng truy vấn DNS

26.5 Kiểm tra phân giải DNS bằng nslookup

nslookup được bao gồm trong busybox và là công cụ nhanh nhất để thử nghiệm DNS.

Ví dụ: Kiểm tra độ phân giải của Kubernetes Service

```
kubectl run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup kubernetes.default.svc.cluster.local
```

Việc này có tác dụng gì: Giải quyết kubernetes Service trong default namespace (máy chủ Kubernetes API Service). Đây là Service tích hợp sẵn luôn tồn tại và là công cụ kiểm tra tình trạng tốt nhất cho CoreDNS.

Sản lượng dự kiến:

```
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   kubernetes.default.svc.cluster.local
Address: 10.96.0.1
```

Dòng Server hiển thị máy chủ DNS đang được sử dụng (CoreDNS ClusterIP). Dòng Address hiển thị ClusterIP của kubernetes Service. Nếu điều này không thành công, CoreDNS bị hỏng.

Ví dụ: Kiểm tra độ phân giải của dịch vụ của bạn

```
kubect1 run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup my-service.my-namespace.svc.cluster.local
```

Ví dụ: Kiểm tra độ phân giải tên ngắn (dựa trên miền tìm kiếm)

```
kubect1 run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -n my-namespace \
  -- nslookup my-service
```

Điều này có tác dụng gì: Kiểm tra xem my-service có giải quyết được bằng cách sử dụng các miền tìm kiếm mặc định trong namespace hay không. Kubernetes định cấu hình /etc/resolv.conf của mỗi Pod với:

```
search <namespace>.svc.cluster.local svc.cluster.local cluster.local
```

Điều này có nghĩa là nslookup my-service sẽ thử my-service.my-namespace.svc.cluster.local trước tiên.

26.6 Kiểm tra phân giải DNS bằng dig

dig (Công cụ thu thập thông tin tên miền) cung cấp thông tin truy vấn DNS chi tiết hơn nhiều so với nslookup. Nó có sẵn trong nicolaka/netshoot image.

Ví dụ: Truy vấn một dịch vụ bằng dig

```
kubect1 run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig my-service.default.svc.cluster.local
```

Kết quả hiển thị:

- QUESTION SECTION: Nội dung được truy vấn
- ANSWER SECTION: (Các) địa chỉ IP và TTL đã được giải quyết
- Query time: CoreDNS mất bao lâu để phản hồi (mili giây)
- SERVER: Máy chủ DNS nào đã được sử dụng

Ví dụ: Truy vấn trực tiếp CoreDNS (bỏ qua bộ đệm)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig @10.96.0.10 my-service.default.svc.cluster.local
```

Điều này làm gì: @10.96.0.10 chỉ định truy vấn trực tiếp CoreDNS ClusterIP. Thay thế 10.96.0.10 bằng CoreDNS ClusterIP của bạn (được tìm thấy với `kubectl get svc kube-dns -n kube-system`). Điều này bỏ qua bất kỳ trình phân giải cục bộ nào và kiểm tra trực tiếp CoreDNS.

Ví dụ: Thực hiện tra cứu ngược DNS (IP thành tên máy chủ)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig -x 10.96.0.1
```

Điều này có tác dụng gì: Truy vấn bản ghi PTR của ClusterIP, phân giải nó trở lại tên máy chủ của nó. Điều này kiểm tra phân giải DNS đảo ngược, được sử dụng bởi một số công cụ giám sát và ghi nhật ký.

Ví dụ: Chỉ truy vấn phần câu trả lời (đầu ra sạch)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig +short my-service.default.svc.cluster.local
```

Điều này có tác dụng gì: Cờ +short chặn tất cả đầu ra ngoại trừ câu trả lời - chỉ địa chỉ IP. Hữu ích cho việc kiểm tra DNS theo kịch bản.

26.7 Kiểm tra cấu hình DNS của Pod

Mọi Pod đều có `/etc/resolv.conf` được cấu hình bởi Kubernetes. Việc kiểm tra nó sẽ cho bạn biết chính xác phân giải DNS sẽ hoạt động như thế nào đối với Pod đó.

Ví dụ: Xem resolv.conf bên trong Pod đang chạy

```
kubectl exec my-app-pod -- cat /etc/resolv.conf
```

Đầu ra điển hình:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Ý nghĩa của từng dòng:

- `nameserver 10.96.0.10` - CoreDNS ClusterIP. Tất cả các truy vấn DNS đều có ở đây.
- `search default.svc.cluster.local svc.cluster.local cluster.local` - Tìm kiếm tên miền được nối thêm thành tên viết tắt. Khi bạn truy vấn `my-service`, trình phân giải sẽ thử từng miền tìm kiếm theo thứ tự.
- `options ndots:5` - Nếu tên có ít hơn 5 dấu chấm, miền tìm kiếm sẽ được thử trước khi xử lý nó dưới dạng FQDN. Điều này ảnh hưởng đến hiệu suất vì các tên bên ngoài như `api.external.com` trước tiên sẽ được thử với các miền tìm kiếm được nối thêm (tạo ra các tra cứu không thành công) trước khi được giải quyết nguyên trạng.

Ví dụ: Xem resolv.conf trong bản thử nghiệm container

```
kubectl run dns-inspect \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- cat /etc/resolv.conf
```

26.8 Kiểm tra phân giải DNS bên ngoài

Ví dụ: Xác minh DNS bên ngoài hoạt động từ bên trong cụm

```
kubectl run dns-external-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup google.com
```

Việc này có tác dụng gì: Kiểm tra xem CoreDNS có thể chuyển tiếp các truy vấn bên ngoài ngược dòng hay không. Nếu DNS bên trong hoạt động nhưng DNS bên ngoài không thành công thì sự cố nằm ở khối `forward` của CoreDNS Corefile hoặc không thể truy cập được trình phân giải ngược dòng của node.

Ví dụ: So sánh độ phân giải bên trong và bên ngoài

```
kubectl run dns-compare \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- sh -c '
    echo "=== Internal Service ==="
    dig +short kubernetes.default.svc.cluster.local

    echo "=== External Domain ==="
    dig +short google.com

    echo "=== CoreDNS Info ==="
    cat /etc/resolv.conf
  '
```

Điều này làm gì: Chạy ba lần kiểm tra theo trình tự trong một Pod, so sánh độ phân giải bên trong và bên ngoài cạnh nhau.

26.9 Chẩn đoán CoreDNS bằng thử nghiệm thời gian chờ 5 giây

Ví dụ: Kiểm tra DNS với thời gian chờ rõ ràng

```
kubectl run dns-timeout-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'nslookup -timeout=5 my-service.default.svc.cluster.local && echo "DNS OK" || echo "DNS FAILED"'
```

Điều này có tác dụng gì: Nếu CoreDNS không phản hồi trong vòng 5 giây, lệnh sẽ không thành công với "DNS FAILED". Trong điều kiện bình thường, phân giải DNS bên trong cụm sẽ mất dưới 5 mili giây.

26.10 Kiểm tra CoreDNS với dnsutils / dnsperf

Ví dụ: Khởi chạy dnsutils container để có bộ công cụ DNS đầy đủ

```
kubectl run dnsutils \
  --image=gcr.io/kubernetes-e2e-test-images/dnsutils:1.3 \
  --restart=Never \
  -- sleep 3600
```

```
kubectl exec -it dnsutils -- bash
```

Bên trong dnsutils, bạn có quyền truy cập vào:

```
# Đào đầy đủ với tất cả các chi tiết
dig kubernetes.default.svc.cluster.local

# nslookup với máy chủ DNS cụ thể
nslookup my-service 10.96.0.10

# Kiểm tra mở rộng miền tìm kiếm
host my-service

# Kiểm tra TCP DNS (thay vì UDP)
dig +tcp my-service.default.svc.cluster.local
```

Ví dụ: Dọn dẹp dnsutils pod khi hoàn tất

```
kubectl delete pod dnsutils
```

26.11 Metrics và endpoint kiểm tra sức khỏe CoreDNS

Ví dụ: Chuyển tiếp tới điểm cuối tình trạng CoreDNS

```
# Nhận tên CoreDNS pod
COREDNS_POD=$(kubectl get pods -n kube-system -l k8s-app=kube-dns -o jsonpath='{.items[0].metadata.name}')

# Chuyển tiếp cổng sức khỏe
kubectl port-forward -n kube-system pod/$COREDNS_POD 8080:8080
```

Sau đó, trong một thiết bị đầu cuối khác:

```
curl http://localhost:8080/health
# Dự kiến: Được

curl http://localhost:8080/ready
# Dự kiến: Được
```

Ví dụ: Truy cập số liệu Prometheus của CoreDNS

```
kubectl port-forward -n kube-system pod/$COREDNS_POD 9153:9153
curl http://localhost:9153/metrics | grep coredns_dns_requests_total
```

Điều này hiển thị: Tổng số truy vấn DNS mà CoreDNS đã nhận và xử lý. Hãy tìm số lượng SERVFAIL cao cho biết độ phân giải không thành công.

26.12 Chẩn đoán lỗi CoreDNS

Triệu chứng: Phân giải DNS thỉnh thoảng không thành công

```
# Kiểm tra CoreDNS pods để khởi động lại
kubectl get pods -n kube-system -l k8s-app=kube-dns
# NAME          READY   STATUS    RESTARTS   AGE
# coredns-abc123 1/1     Running   12          2d    ← 12 restarts is suspicious!

kubectl logs coredns-abc123 -n kube-system | tail -30 | grep -i "error\|fail\|panic"

# CoreDNS OOMKilled (bộ nhớ đã cạn kiệt bởi bộ đệm DNS)
kubectl describe pod coredns-abc123 -n kube-system | grep -A 5 "Last State"
# Trạng thái cuối cùng: Đã chấm dứt
# Lý do: OOMKilled
# Khắc phục: tăng giới hạn bộ nhớ CoreDNS:
kubectl -n kube-system get deployment coredns -o yaml | grep -A 5 resources
kubectl -n kube-system set resources deployment coredns \
  --containers=coredns --limits=memory=256Mi --requests=memory=128Mi
```

Triệu chứng: NXDOMAIN dành cho các dịch vụ trong cụm

```
# Kiểm tra DNS từ pod
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup my-service.production.svc.cluster.local

# Máy chủ: 10.96.0.10
# Địa chỉ 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
# nslookup: không thể giải quyết 'my-service.production.svc.cluster.local'

# Bước 1: Xác minh dịch vụ thực sự tồn tại
kubectl get svc my-service -n production
# Lỗi: không tìm thấy dịch vụ "my-service" ← Đó là vấn đề!

# Bước 2: Kiểm tra xem DNS có hoạt động với các dịch vụ khác không
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup kubernetes.default.svc.cluster.local
# Điều này sẽ luôn luôn hoạt động. Nếu không, CoreDNS đã bị hỏng.

# Bước 3: Kiểm tra miền cụm
kubectl get configmap coredns -n kube-system -o yaml | grep "cluster.local"
# Đảm bảo /etc/resolv.conf của pod của bạn sử dụng cùng một miền:
kubectl exec my-pod -- cat /etc/resolv.conf
# tìm kiếm default.svc.cluster.local svc.cluster.local cluster.local
```

Triệu chứng: DNS bên ngoài không phân giải được từ pods

```
# DNS bên ngoài (e.g., google.com) không thành công nhưng bên trong hoạt động
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- sh
> nslookup kubernetes.default # Tác phẩm
> nslookup google.com        # Thất bại

# Kiểm tra cấu hình chuyển tiếp CoreDNS
kubectl get configmap coredns -n kube-system -o yaml
# Corefile nên có:
# .:53 {
#     ...
#     chuyển tiếp . /etc/resolv.conf ← điều này chuyển tiếp lên DNS ngược dòng
# }

# Nếu CoreDNS không thể truy cập ngược dòng DNS, hãy kiểm tra resolv.conf của node:
cat /etc/resolv.conf # Trên node
# Nên có một máy chủ tên hợp lệ

# Kiểm tra trực tiếp từ CoreDNS pod
kubectl exec -n kube-system coredns-abc123 -- nslookup google.com 8.8.8.8
```


Tùy chỉnh CoreDNS cho các mẫu phổ biến

```
# Thêm mục nhập DNS tùy chỉnh cho một máy chủ cụ thể
# Chỉnh sửa sơ đồ cấu hình CoreDNS:
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health { lameduck 5s }
      ready
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
      }
      hosts {                                # Mục máy chủ tùy chỉnh
        10.1.2.3   mylegacyserver
        10.1.2.4   another-host.local
        fallthrough
      }
      rewrite name exact \
        old-service.example.com \
        new-service.production.svc.cluster.local # Viết lại DNS
      prometheus :9153
      forward . /etc/resolv.conf {
        max_concurrent 1000
      }
      cache 30
      loop
      reload
      loadbalance
    }

# Sau khi chỉnh sửa CoreDNS ConfigMap, nó sẽ tự động tải lại trong vòng 30 giây
# Buộc tải lại ngay lập tức:
kubectl rollout restart deployment/coredns -n kube-system

# Kiểm tra mục DNS tùy chỉnh của bạn
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup mylegacyserver
```

Chương 27 - Gỡ lỗi nâng cao với các container tạm thời

27.1 Container tạm thời là gì?

Ephemeral container ổn định từ Kubernetes **1.25**. Không giống như containers thông thường trong thông số Pod, containers tạm thời chỉ là tạm thời - chúng dùng chung namespaces của Pod (mạng, PID, v.v.) nhưng có thể được thêm vào Pod đang chạy mà không cần khởi động lại nó.

Điều này giải quyết một vấn đề nghiêm trọng: images không phân phối và tối thiểu không có shell cũng như không có công cụ gỡ lỗi. Một container tạm thời có thể được đưa vào cùng một Pod trong thời gian chạy để gỡ lỗi.

22.2 Thêm một container tạm thời vào Running Pod

Ví dụ: Đưa container gỡ lỗi vào Pod đang chạy

```
kubectl debug -it my-app-pod \
  --image=nicolaka/netshoot \
  --target=app \
  -- bash
```

Điều này có tác dụng gì: Thêm netshoot container dưới dạng container tạm thời vào my-app-pod đang chạy. Cờ --target=app nhắm mục tiêu quy trình namespace của app container (bạn có thể xem các quy trình của nó bằng cách sử dụng ps aux từ bên trong bản gỡ lỗi container). Khi bạn thoát, Pod vẫn tiếp tục chạy nhưng container tạm thời sẽ chấm dứt.

Ví dụ: Gỡ lỗi container không phân phối (không có shell)

```
kubectl debug -it my-distroleless-pod \
  --image=busybox \
  --target=app \
  -- sh
```

Điều này có tác dụng gì: Vì images không có distro không có shell nên thao tác này sẽ đưa container busybox vào cùng Pod namespace, mang lại cho bạn một shell chia sẻ ngăn xếp mạng với container không có distro.

22.3 Tạo bản sao gỡ lỗi của Pod

Nếu bạn cần sửa đổi thông số pod để gỡ lỗi (e.g., thay đổi điểm vào, thêm khả năng), bạn có thể tạo bản sao của Pod có sửa đổi.

Ví dụ: Tạo bản sao gỡ lỗi bằng shell thay thế điểm vào

```
kubectl debug my-app-pod \
  -it \
  --copy-to=my-app-debug \
  --image=busybox \
  -- sh
```

Điều này làm gì: Tạo một Pod mới có tên my-app-debug, đây là bản sao của my-app-pod nhưng với container image được thay thế bằng busybox. Hữu ích khi bạn muốn khám phá cùng một ổ đĩa gắn kết và các biến môi trường với một image khác.

Ví dụ: Dọn dẹp bản gỡ lỗi Pod

```
kubectl delete pod my-app-debug
```

22.4 Running một nút gỡ lỗi Pod (Đặc quyền)

Để gỡ lỗi các sự cố hệ thống tệp hoặc mạng cấp node:

Ví dụ: Tạo Pod đặc quyền trên node cụ thể

```
kubectl debug node/my-node-name \
  -it \
  --image=nicolaka/netshoot
```

Điều này có tác dụng gì: Tạo một Pod đặc quyền trên node được chỉ định, cho phép bạn truy cập vào hệ thống tệp máy chủ của node được gắn tại /host, mạng máy chủ và máy chủ PID namespace. Sử dụng cẩn thận.

27.2 Xử lý sự cố hệ thống trên node

Khi node hiển thị NotReady, hãy sử dụng phương pháp có hệ thống này:

```
# Bước 1: Kiểm tra trạng thái và điều kiện của node
kubectl get nodes
kubectl describe node <problem-node> | grep -A 20 "Conditions:"

# Các điều kiện NotReady phổ biến và ý nghĩa của chúng:
# Condition: Ready=False
```

```
# Lý do: KubeletNotReady, KubeletHasInfullMemory, KubeletHasDiskPressure

# Bước 2: Kiểm tra trạng thái kubelet trên node (yêu cầu quyền truy cập SSH hoặc node)
ssh worker-1
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 50 --no-pager

# Bước 3: Nguyên nhân lỗi kubelet thường gặp
# - Chứng chỉ đã hết hạn: "không tải được chứng chỉ" trong nhật ký
# - Môi trường chạy container không chạy: "Lỗi lấy thông tin node... quay số unix /run/containerd/containerd.sock"
# - Đĩa đầy: "dung lượng đĩa dưới ngưỡng"

# Bước 4: Kiểm tra môi trường chạy container
sudo systemctl status containerd
sudo crictl ps

# Bước 5: Kiểm tra tài nguyên hệ thống
free -h          # Bộ nhớ
df -h            # Đĩa
top              # CPU
```

27.3 Khắc phục sự cố các thành phần Control Plane

```
# Kiểm tra tất cả trạng thái thành phần control plane
kubectl get pods -n kube-system
kubectl get componentstatuses # cũ nhưng hữu ích: hiển thị etcd, trình quản lý bộ điều khiển, tình trạng bộ lập lịch

# Máy chủ API không hoạt động - kết nối trực tiếp với control-plane node
ssh control-plane-node
sudo crictl logs $(sudo crictl ps -q --name=kube-apiserver)
cat /etc/kubernetes/manifests/kube-apiserver.yaml | grep -- '--'

# Sự cố kube-controller-manager
kubectl -n kube-system logs kube-controller-manager-<node> --tail=50 | grep -i error

# kube-scheduler không lập lịch Pods (Pods bị kẹt trong Pending)
kubectl -n kube-system logs kube-scheduler-<node> --tail=50
kubectl describe pod <pending-pod> | grep -A 10 "Events:"
# "0/3 nodes có sẵn: 3 CPU không đủ" → cần thêm tài nguyên
# "0/3 nodes có sẵn: 3 node chưa dung nạp taint" → thêm toleration
```

27.4 Khắc phục sự cố Services và Mạng

```
# Pod không thể đạt Service bằng DNS?
# Bước 1: Kiểm tra phân giải DNS
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup my-service.staging.svc.cluster.local

# Step 2: Test Service directly by ClusterIP
kubectl get svc my-service -n staging -o jsonpath='{.spec.clusterIP}'
kubectl run curltest --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://10.96.100.5:80

# Bước 3: Kiểm tra Endpoints - nếu trống, bộ chọn nhãn sai
kubectl get endpoints my-service -n staging
# Nếu "my-service <none>" → không có Pods nào khớp với bộ chọn Service

# Bước 4: Xác minh nhãn Pod khớp với bộ chọn Service
kubectl get svc my-service -n staging -o jsonpath='{.spec.selector}'
# {"app://frontend"}
kubectl get pods -n staging -l app=frontend # Nên trả về Pods

# Bước 5: Kiểm tra xem kube-proxy có đang chạy không
kubectl -n kube-system get pods -l k8s-app=kube-proxy
kubectl -n kube-system logs kube-proxy-<name> --tail=20

# Bước 6: Xác minh quy tắc iptables tồn tại (trên node)
sudo iptables -t nat -L KUBE-SERVICES | grep my-service
```

27.5 Danh sách kiểm tra sự cố cấp cụm

Danh sách kiểm tra có hệ thống cho các tình huống khắc phục sự cố trong kỳ thi CKA:

Pod stuck in Pending:

```
Describe pod → check Events section
Check if node has enough CPU/memory: kubectl describe nodes | grep "Allocated"
Check for taints: kubectl describe node | grep Taints
Check PVC is bound (if using storage): kubectl get pvc
Check resource quotas: kubectl describe resourcequota
```

Pod in CrashLoopBackOff:

```
kubectl logs <pod> --previous
kubectl describe pod → check Last State exit code
Check if command/args are correct
Check if readiness probe is misconfigured
Check if env vars / ConfigMaps / Secrets are correct
```

Pod stuck in ContainerCreating:

```
kubectl describe pod → Events: "failed to pull image"
Check imagePullSecret if private registry
Check PVC is available
Check CNI plugin is running
```

Service not accessible:

```
kubectl get endpoints <service> → should not be <none>
kubectl get pods -l <service-selector> → pods must be Running
Test from within cluster with curl/wget
Check NetworkPolicy: kubectl get networkpolicy -n <namespace>
```

Chương 28 - Các lệnh nhận biết namespace và quản lý bối cảnh

28.1 Làm việc với namespace

Tất cả các lệnh `kubectl` đều hoạt động trong namespace. Nếu bạn không chỉ định một, `kubectl` sử dụng namespace được định cấu hình trong ngữ cảnh hiện tại của bạn (thường là `default`).

Ví dụ: Chạy lệnh trong namespace cụ thể

```
kubectl exec -it my-pod -n production -- bash
kubectl logs my-pod -n staging
kubectl run test -n development --image=busybox --rm -it --restart=Never -- sh
```

Ví dụ: Liệt kê tất cả namespaces

```
kubectl get namespaces
```

Ví dụ: Xem Pods trong tất cả namespaces

```
kubectl get pods -A
# -A là viết tắt của --all-namespaces
```

28.2 Chuyển đổi bối cảnh và namespace

Ví dụ: Xem bối cảnh hiện tại

```
kubectl config current-context
```

Ví dụ: Đặt namespace mặc định cho ngữ cảnh hiện tại

```
kubectl config set-context --current --namespace=production
```

Điều này làm gì: Đặt namespace mặc định cho tất cả các lệnh tiếp theo mà không cần `-n` mỗi lần. Đây là cài đặt liên tục được lưu trữ trong kubeconfig của bạn.

Ví dụ: Xem tất cả ngữ cảnh

```
kubectl config get-contexts
```

Ví dụ: Chuyển sang ngữ cảnh cụm khác

```
kubectl config use-context my-production-cluster
```

28.3 Thực thi trên tất cả Pods trong Deployment

Ví dụ: Thực thi lệnh trong mọi Pod của Deployment

```
for pod in $(kubectl get pods -l app=my-app -o jsonpath='{.items[*].metadata.name}'); do
  echo "=== $pod ==="
  kubectl exec $pod -- cat /etc/hostname
done
```

Điều này có tác dụng gì: Lặp lại tất cả Pods phù hợp với nhãn `app=my-app` và chạy một lệnh trong mỗi cái. Hữu ích để kiểm tra tính nhất quán trên tất cả các bản sao.

28.4 RBAC nằm trong phạm vi namespace

RBAC cho phép bạn cấp cho các nhóm khác nhau các quyền khác nhau trên mỗi namespace:

```
# Tạo namespace cho mỗi nhóm
kubectl create namespace team-alpha
kubectl create namespace team-beta

# Tạo Role trong team-alpha cho phép quản lý Pods và Deployments
kubectl create role developer-role \
  --verb=get,list,watch,create,update,patch,delete \
  --resource=pods,deployments,services,configmaps \
  -n team-alpha

# Liên kết người dùng với vai trò trong namespace đó
kubectl create rolebinding alice-binding \
  --role=developer-role \
  --user=alice \
  -n team-alpha

# Alice có thể làm việc trong team-alpha nhưng không có quyền truy cập vào team-beta
kubectl auth can-i create pods --as=alice -n team-alpha # vâng
kubectl auth can-i create pods --as=alice -n team-beta  # no
kubectl auth can-i get pods --as=alice -n kube-system   # no
```

28.5 NetworkPolicy để cách ly namespace

Theo mặc định, tất cả Pods có thể giao tiếp với tất cả Pods khác trong cụm (không cách ly mạng). NetworkPolicies hạn chế điều này:

```
# Theo mặc định, từ chối tất cả sự xâm nhập vào dàn namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: staging
spec:
  podSelector: {} # Áp dụng cho TẤT CẢ pods trong namespace
  policyTypes:
    - Ingress # Hạn chế lưu lượng truy cập vào
    # Không có quy tắc xâm nhập = từ chối tất cả
  ---
# Cho phép dàn dựng chỉ nhận lưu lượng truy cập từ sản xuất namespace
```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-production
  namespace: staging
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: production
---
# Cho phép liên lạc từ pod đến pod cụ thể trên namespaces
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-from-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          team: frontend
      podSelector:
        matchLabels:
          app: frontend
  ports:
  - port: 8080

# Gắn nhãn namespace (bắt buộc để khớp với namespaceSelector)
kubectl label namespace production kubernetes.io/metadata.name=production

# Kiểm tra chính sách mạng
kubectl run test --image=busybox -n staging --rm -it --restart=Never -- \
  wget -qO- http://api-service.backend.svc.cluster.local:8080

# Liệt kê tất cả các chính sách mạng
kubectl get networkpolicies --all-namespaces

```

28.6 Truy cập Service ở namespace khác

Services có thể định địa chỉ DNS từ namespaces khác:

```

# Định dạng DNS: <service-name>.<namespace>.svc.cluster.local
# Từ bất kỳ Pod nào trong bất kỳ namespace nào:
curl http://my-service.staging.svc.cluster.local:8080

# Trong cùng namespace (dạng ngắn):
curl http://my-service:8080

# FQDN đầy đủ (từ cụm bên ngoài hoặc gỡ lỗi):
# của tôi-service.staging.svc.cluster.local

```

Bạn cũng có thể tạo Service trỏ đến một dịch vụ trong namespace khác bằng ExternalName:

```

# Trong namespace "sản xuất", hãy tạo một dịch vụ ủy quyền cho việc dàn dựng
apiVersion: v1
kind: Service
metadata:
  name: staging-api
  namespace: production
spec:

```

```
type: ExternalName
externalName: api.staging.svc.cluster.local
# Bây giờ sản xuất Pods có thể sử dụng: Curl http://staging-api:8080
# và nó phân giải thành api.staging.svc.cluster.local
```

28.7 Vòng đời và dọn dẹp namespace

```
# Xóa namespace (xóa TẤT CẢ tài nguyên trong đó)
kubectl delete namespace old-staging

# Đây là ASYNC - namespace chuyển sang trạng thái Chấm dứt
kubectl get namespace old-staging
# NAME          STATUS      AGE
# dàn cũ Chấm dứt 2m

# Nếu bị kẹt trong Chấm dứt, nó có thể có phần hoàn thiện
kubectl get namespace old-staging -o json | grep finalizers

# Buộc tháo namespace bị kẹt (sử dụng cẩn thận)
kubectl proxy &
curl -k -H "Content-Type: application/json" -X PUT \
  --data-binary '{"kind":"Namespace","apiVersion":"v1","metadata":{"name":"old-staging"},"spec":{"finalizers":[]}}' \
  http://127.0.0.1:8001/api/v1/namespaces/old-staging/finalize

# Liệt kê tất cả tài nguyên trong namespace trước khi xóa
kubectl get all -n old-staging
kubectl api-resources --verbs=list --namespaced -o name | \
  xargs -I {} kubectl get {} -n old-staging --ignore-not-found
```

Chương 29 - kubectl Kustomize: Quản lý cấu hình ở quy mô

29.1 Kustomize là gì?

Kustomize là công cụ quản lý cấu hình gốc Kubernetes cho phép bạn tùy chỉnh các manifest YAML thô mà không cần mẫu, không sửa đổi tệp gốc và không có công cụ kết xuất riêng. Nó được tích hợp trực tiếp vào kubectl bắt đầu từ phiên bản 1.14, nghĩa là bạn không cần cài đặt thêm công cụ nào - kubectl kustomize có sẵn trên mọi bản cài đặt Kubernetes hiện đại.

Triết lý của Kustomize về cơ bản khác với Helm:

Khía cạnh	Kustomize	Helm
Cách tiếp cận	Vá lớp phủ khai báo	Công cụ tạo khuôn mẫu với các mẫu Go
Đường cong học tập	Thấp hơn (YAML thuần túy)	Cao hơn (cú pháp mẫu)
Tập tin gốc	Còn lại chưa sửa đổi	Phải được viết dưới dạng mẫu
Nhiều môi trường	Lớp phủ trên nền	Tập giá trị (values.yaml)
Kubernetes-bản địa	Có (được tích hợp trong kubectl)	Yêu cầu nhị phân riêng biệt

Kustomize hoạt động bằng cách:

- Bạn xác định **cơ sở** - YAML chuẩn cho tài nguyên của bạn.
- Bạn tạo **lớp phủ** cho từng môi trường (nhà phát triển, dàn dựng, sản xuất).
- Lớp phủ sử dụng **bản vá**, **bộ biến đổi** và **trình tạo** để áp dụng các thay đổi được nhắm mục tiêu lên trên của cơ sở mà không cần chạm vào các tệp tin cơ sở.

4. `kubectl kustomize` hiển thị đầu ra được hợp nhất cuối cùng mà bạn chuyển thành `kubectl apply`.

29.2 Tập `kustomization.yaml`

Mỗi thư mục Kustomize phải chứa một tệp có tên chính xác là `kustomization.yaml`. Tệp này là manifest của cấu hình Kustomize - nó liệt kê những tài nguyên cần đưa vào, bản vá nào cần áp dụng, những chuyển đổi nào cần thực hiện và trình tạo nào sẽ chạy.

`kustomization.yaml` tối thiểu:

```
# kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- service.yaml
```

Điều này làm gì: Tuyên bố rằng tùy chỉnh này quản lý hai tài nguyên: `deployment.yaml` và `service.yaml`. Running `kubectl kustomize` . trong thư mục này xuất ra cả hai tệp được hợp nhất và hiển thị.

29.3 Cấu trúc thư mục: base và overlay

Bố cục dự án Kustomize tiêu chuẩn tách biệt **cơ sở** (cấu hình dùng chung) khỏi **lớp phủ** (các tùy chỉnh dành riêng cho môi trường):

```
my-app/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
└── overlays/
    ├── development/
    │   ├── kustomization.yaml
    │   └── bản vá-replicas.yaml
    ├── staging/
    │   ├── kustomization.yaml
    │   └── bản vá-replicas.yaml
    └── production/
        ├── kustomization.yaml
        ├── patch-replicas.yaml
        └── patch-resources.yaml
```

Cấu trúc này đạt được những gì: Thư mục `base/` chứa các manifest Kubernetes hoàn chỉnh, không phụ thuộc vào môi trường. Mỗi thư mục lớp phủ chứa `kustomization.yaml` tham chiếu cơ sở và áp dụng các bản vá dành riêng cho môi trường. Các tập tin cơ sở không bao giờ được sửa đổi.

29.4 Xây dựng và áp dụng tùy chỉnh

Hai lệnh cốt lõi

Lệnh 1: Tùy chỉnh `kubectl` - kết xuất mà không áp dụng

```
kubectl kustomize <directory>
```

Điều này làm gì: Đọc tất cả các tài nguyên và bản vá trong thư mục tùy chỉnh, xử lý chúng và xuất YAML được hợp nhất cuối cùng thành stdout. Không có gì được gửi đến cụm. Điều này rất cần thiết để xem xét những gì sẽ được áp dụng trước khi thực sự áp dụng nó.

```
# Kết xuất cơ sở
kubectl kustomize ./base
```



```
# Hiển thị lớp phủ cụ thể
kubect1 kustomize ./overlays/production

# Lưu đầu ra được kết xuất vào một tệp
kubect1 kustomize ./overlays/production > rendered-production.yaml

# Xem lại đầu ra (đường dẫn đến ít hơn cho đầu ra lớn)
kubect1 kustomize ./overlays/production | less
```

Lệnh 2: Áp dụng kubect1 -k - kết xuất và áp dụng trong một bước

```
kubect1 apply -k <directory>
```

Điều này làm gì: Cờ -k yêu cầu kubect1 apply coi thư mục này là thư mục Kustomize thay vì tệp YAML đơn giản. Nó hiển thị tùy chỉnh và ngay lập tức áp dụng nó cho cụm. Tương đương với kubect1 kustomize <dir> | kubect1 apply -f -.

```
# Áp dụng cơ sở trực tiếp
kubect1 apply -k ./base

# Áp dụng lớp phủ sản xuất
kubect1 apply -k ./overlays/production

# Chạy thử để xem điều gì sẽ thay đổi nếu không áp dụng
kubect1 apply -k ./overlays/production --dry-run=client

# Chạy thử trên máy chủ (xác thực dựa trên lược đồ API)
kubect1 apply -k ./overlays/production --dry-run=server
```

Lệnh 3: kubect1 xóa -k - xóa tất cả tài nguyên theo tùy chỉnh

```
kubect1 delete -k ./overlays/production
```

Việc này có tác dụng gì: Xóa tất cả tài nguyên mà quá trình tùy chỉnh quản lý. Hữu ích cho việc phá bỏ toàn bộ môi trường một cách sạch sẽ.

29.5 Ví dụ cấu hình base

Chúng ta hãy xây dựng một ví dụ hoạt động hoàn chỉnh từ đầu.

base/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: my-app:latest
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
          env:
            - name: LOG_LEVEL
              value: "info"
```

base/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml

# Xác minh cơ sở hiển thị rõ ràng
kubectl kustomize ./base
```

29.6 Overlay: tùy chỉnh cho từng môi trường

Lớp phủ phát triển

overlays/development/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Tham khảo cơ sở
resources:
- ../../base

# Thêm tiền tố namespace vào tất cả tài nguyên
namespace: development

# Thêm nhãn vào tất cả tài nguyên
commonLabels:
  environment: development

# Ghi đè thẻ image để phát triển
images:
- name: my-app
  newTag: dev-latest

# Áp dụng các bản vá
patches:
- path: patch-replicas.yaml
```

overlays/development/patch-replicas.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1          # Phát triển chỉ cần 1 bản sao

# Hiển thị lớp phủ phát triển và kiểm tra
kubectl kustomize ./overlays/development

# Áp dụng cho cụm
kubectl apply -k ./overlays/development
```

Lớp phủ sản xuất

overlays/production/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

namespace: production

commonLabels:
  environment: production

images:
- name: my-app
  newName: registry.company.com/my-app  # Đăng ký khác nhau trong sản phẩm
  newTag: v1.5.2                        # Thẻ phiên bản được ghim

patches:
- path: patch-replicas.yaml
- path: patch-resources.yaml
- path: patch-env.yaml
```

overlays/production/patch-replicas.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 5          # Sản xuất cần 5 bản sao cho HA
```

overlays/production/patch-resources.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
      - name: app
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "1000m"
            memory: "1Gi"
```

overlays/production/patch-env.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
      - name: app
        env:
          - name: LOG_LEVEL
            value: "warn"          # Giảm mức chi tiết của log trong môi trường production
          - name: DATABASE_URL
            valueFrom:
              secretKeyRef:
                name: db-credentials
                key: url
```

29.7 Bản vá Kustomize: strategic merge và JSON 6902

Kustomize hỗ trợ hai loại bản vá, mỗi loại có điểm mạnh riêng.

Bản vá hợp nhất chiến lược

Bản vá hợp nhất chiến lược là một tài liệu YAML một phần được hợp nhất với tài nguyên đích. Kubernetes sử dụng lược đồ của tài nguyên để hiểu cách hợp nhất các mảng (e.g., containers hợp nhất bằng khóa name, không phải theo vị trí chỉ mục).

```
# kustomization.yaml
patches:
- path: my-patch.yaml
  target:
    kind: Deployment
    name: my-app

# my-patch.yaml - thêm biến môi trường mới mà không thay thế biến môi trường hiện có
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
```

```
template:
  spec:
    containers:
      - name: app
        env:
          - name: NEW_FEATURE_FLAG
            value: "enabled"
```

Điều này làm gì: Vì Kubernetes biết mảng env hợp nhất bằng khóa name, nên bản vá này THÊM biến mới mà không xóa biến hiện có.

Bản vá JSON 6902 (Phẫu thuật chính xác)

Các bản vá JSON 6902 cho phép bạn thực hiện các thao tác chính xác: thêm, xóa, thay thế, sao chép và di chuyển theo các đường dẫn chính xác trong tài nguyên. Chúng dài dòng hơn nhưng rõ ràng hơn.

```
# kustomization.yaml - bản vá JSON 6902 nội tuyến
patches:
  - target:
      kind: Deployment
      name: my-app
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 3
      - op: add
        path: /spec/template/spec/containers/0/env/-
        value:
          name: FEATURE_FLAG
          value: "true"
      - op: remove
        path: /spec/template/spec/containers/0/env/0
```

Đường dẫn /spec/template/spec/containers/0/env/-: ở cuối là quy ước Con trở JSON có nghĩa là "nối vào cuối mảng". /env/0 có nghĩa là phần tử đầu tiên của mảng env.

29.8 Transformer: thay đổi tài nguyên trên toàn cấu hình

Bộ biến đổi áp dụng các thay đổi cho TẤT CẢ các tài nguyên theo cách tùy chỉnh. Chúng được khai báo trực tiếp trong kustomization.yaml và cực kỳ mạnh mẽ để áp dụng siêu dữ liệu nhất quán trên toàn bộ tập hợp tài nguyên.

namePrefix và nameSuffix

```
# kustomization.yaml
namePrefix: prod-      # Tất cả các tên tài nguyên đều được thêm vào trước "prod-"
nameSuffix: -v2        # Tất cả tên tài nguyên đều được thêm "-v2"
```

Kết quả: Deployment có tên my-app trở thành prod-my-app-v2. Điều này thường được sử dụng để phân biệt các môi trường có chung một cụm.

nhãn chung

```
# kustomization.yaml
commonLabels:
  app.kubernetes.io/managed-by: kustomize
  app.kubernetes.io/environment: production
  team: platform-engineering
```

Điều này có tác dụng gì: Thêm TẤT CẢ các nhãn được liệt kê vào mọi tài nguyên VÀ vào bộ chọn mẫu Pod. Điều này có nghĩa là các nhãn này xuất hiện trên Deployments, Services, ConfigMaps và tất cả Pods được quản lý của chúng.

Cảnh báo: commonLabels cũng sửa đổi spec.selector trên Services và Deployments. Nếu những bộ chọn này thay đổi trên các tài nguyên hiện có, Kubernetes có thể từ chối bản cập nhật. sử dụng commonAnnotations dành cho siêu dữ liệu thông tin thuần túy để tránh xung đột bộ chọn.

chú thích chung

```
# kustomization.yaml
commonAnnotations:
  team-contact: "platform@company.com"
  docs-url: "https://wiki.internal/my-app"
  last-deployed-by: "ci-pipeline"
```

Điều này làm gì: Thêm chú thích vào mọi tài nguyên mà không ảnh hưởng đến bộ chọn. Chú thích an toàn để thêm vào tài nguyên hiện có.

namespace

```
# kustomization.yaml
namespace: production
```

Việc này có tác dụng gì: Đặt metadata.namespace trên TẤT CẢ các tài nguyên trong quá trình tùy chỉnh. Đây là cách sử dụng bộ biến đổi phổ biến nhất - đảm bảo tất cả các nguồn tài nguyên đều nằm trong namespace chính xác.

29.9 Generator: tạo ConfigMap và Secret từ tệp

Kustomize có thể tự động tạo ConfigMaps và Secrets từ các tệp và chữ, với hậu tố tên dựa trên hàm băm tự động. Đây là một trong những tính năng mạnh mẽ nhất của Kustomize để quản lý cấu hình.

Trình tạo ConfigMap

```
# kustomization.yaml
configMapGenerator:
- name: app-config
  files:
    - configs/app.properties      # Từ một tập tin (key = tên tập tin)
    - application.conf=configs/app.conf # Tên khóa tùy chỉnh
  literals:
    - LOG_LEVEL=info
    - MAX_CONNECTIONS=100
  options:
    disableNameSuffixHash: false # Mặc định: đúng. Đặt sai để tắt hậu tố băm
```

Hậu tố băm làm gì: Theo mặc định, Kustomize gắn thêm hàm băm 8 ký tự của nội dung ConfigMap vào tên của nó (e.g., app-config-5gh29kd7). Khi nội dung thay đổi, hàm băm thay đổi, tên thay đổi và Kubernetes buộc phải cập nhật liên tục tất cả Pods tham chiếu đến nó. Điều này đảm bảo Pods luôn sử dụng cấu hình mới nhất.

configs/app.properties (tệp cấu hình văn bản thuần túy):

```
database.host=prod-db.internal
database.port=5432
database.name=myapp_production
cache.ttl=3600
```

Trình tạo Secret

```
# kustomization.yaml
secretGenerator:
- name: db-credentials
  type: Opaque
  files:
    - secrets/db-password.txt # Nội dung trở thành giá trị bí mật
  literals:
    - username=myapp_user
  envs:
    - secrets/.env.production # Tải từ định dạng tệp .env
  options:
    disableNameSuffixHash: true # Tên ổn định cho các bí mật (được tham chiếu bởi pods)
```

secrets/.env.production:

```
DB_PASSWORD=super-secret-password-123
API_KEY=abcdef123456
```

Lưu ý bảo mật: Không bao giờ chuyển các tệp bí mật sang kiểm soát phiên bản. Sử dụng quản lý bí mật hệ thống (Vault, AWS Secrets Manager, Sealed Secrets) và chỉ tham chiếu chúng trong kustomization.yaml.

29.10 Thay đổi image

Trường `images` trong `kustomization.yaml` là một biến áp chuyên dụng để cập nhật tên và thẻ container image trên TẤT CẢ các tài nguyên trong tùy chỉnh. Đây là cơ chế chính để tạo phiên bản image dành riêng cho môi trường.

```
# kustomization.yaml
images:
- name: my-app                                # Khớp tên image này trong bất kỳ container nào
  newName: registry.company.com/my-app        # Thay thế bằng tên này
  newTag: v1.5.2                               # Thay thế thẻ

- name: sidecar-image
  newTag: stable                               # Chỉ đổi tag, giữ nguyên tên

- name: old-image-name
  newName: new-image-name                     # Đổi tên mà không thay đổi thẻ

- name: my-app
  digest: sha256:abc123def456...              # Ghim vào digest chính xác (không thay đổi)
```

Tại sao nên sử dụng bộ biến đổi images thay vì chỉnh sửa trực tiếp deployment.yaml?

Bởi vì `deployment.yaml` cơ sở sử dụng tên logic `my-app:latest` và mỗi lớp phủ có thể đặt độc lập phiên bản image chính xác phù hợp với môi trường đó. Tệp cơ sở không bao giờ cần phải sửa đổi.

```
# Xác minh ghi đề image được áp dụng chính xác
kubect1 kustomize ./overlays/production | grep "image:"
```

29.11 Hợp nhất nhiều tùy chỉnh với các thành phần

Kustomize 4.1+ đã giới thiệu **Thành phần** - các phần cấu hình có thể tái sử dụng và có thể được đưa vào nhiều lớp phủ. Điều này giải quyết vấn đề về các tính năng được chia sẻ (e.g., giám sát, TLS) áp dụng cho một số chứ không phải tất cả các lớp phủ.

components/monitoring/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1alpha1
kind: Component

patches:
- path: add-prometheus-annotations.yaml
```

components/monitoring/add-prometheus-annotations.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
        prometheus.io/path: "/metrics"
```

overlays/production/kustomization.yaml (có thành phần)

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

components:
- ../../components/monitoring # Bao gồm thành phần giám sát
- ../../components/tls        # Bao gồm thành phần TLS
```

29.12 Các lệnh quy trình công việc Kustomize thực tế

So sánh trước khi áp dụng

Trước khi áp dụng tùy chỉnh, cách tốt nhất là xem điều gì sẽ thay đổi. Sự kết hợp giữa `kubectl kustomize` và `kubectl diff` mang đến cho bạn sự khác biệt chính xác so với những gì hiện đang chạy:

```
# Xem điều gì sẽ thay đổi trong cụm nếu bạn áp dụng lớp phủ sản xuất
kubectl diff -k ./overlays/production
```

Điều này có tác dụng gì: Hiển thị sự khác biệt thống nhất giữa trạng thái tài nguyên hiện tại trong cụm và những gì tùy chỉnh sẽ tạo ra. Các dòng bắt đầu bằng `-` là những dòng sẽ bị xóa, những dòng bắt đầu bằng `+` là những dòng sẽ được thêm vào.

Áp dụng và xác minh

```
# Áp dụng và kiểm tra ngay trạng thái triển khai
kubectl apply -k ./overlays/production && \
  kubectl rollout status deployment/my-app -n production
```

Xem tất cả tài nguyên được quản lý

```
# Xem tất cả các tài nguyên mà tùy chỉnh quản lý
kubectl kustomize ./overlays/production | grep "^kind:|^ name:"
```

Xóa tài nguyên bằng cách tùy chỉnh

```
# Phá bỏ tất cả các tài nguyên từ việc tùy chỉnh
kubectl delete -k ./overlays/development
```

Gỡ lỗi đầu ra Kustomize

```
# Kiểm tra lỗi cú pháp YAML trong quá trình tùy chỉnh của bạn
kubectl kustomize ./overlays/production 2>&1 | head -50

# Xác thực dựa trên lược đồ Kubernetes API mà không áp dụng
kubectl apply -k ./overlays/production --dry-run=server

# Đầu ra dưới dạng JSON thay vì YAML
kubectl kustomize ./overlays/production -o json | jq '.metadata.name'
```

29.13 Sử dụng base từ xa với Kustomize

Kustomize có thể tham chiếu các cơ sở được lưu trữ trong kho Git từ xa, rất mạnh mẽ để chia sẻ cấu hình chung giữa các nhóm và dự án.

```
# kustomization.yaml - tham chiếu cơ sở từ xa từ GitHub
resources:
- github.com/company/k8s-base//apps/my-app/base?ref=v1.2.0
  # Định dạng: github.com/<org>/<repo>//<path-in-repo>?ref=<branch-or-tag>

# Tham chiếu cơ sở từ thẻ Git cụ thể
resources:
- https://github.com/company/k8s-base//base?ref=v2.0.0
```


Lưu ý sản xuất: Luôn ghim các cơ sở từ xa vào một thẻ cụ thể hoặc cam kết hàm băm, không bao giờ ghim vào một nhánh có thể thay đổi như main. Điều này đảm bảo việc triển khai của bạn có thể lặp lại và không bị hỏng bởi những thay đổi ngược dòng.

29.14 Kustomize trong đường ống CI/CD

Trong đường dẫn CI/CD, mẫu điển hình là:

```
# 1. Kết xuất lớp phủ cho môi trường đích
kubect1 kustomize ./overlays/${ENVIRONMENT} > rendered.yaml

# 2. Xác thực đầu ra được hiển thị (tùy chọn nhưng được khuyến nghị)
kubect1 apply -f rendered.yaml --dry-run=server

# 3. Khác biệt với trạng thái cụm hiện tại
kubect1 diff -f rendered.yaml

# 4. Áp dụng
kubect1 apply -f rendered.yaml

# 5. Chờ triển khai
kubect1 rollout status deployment/my-app -n ${ENVIRONMENT} --timeout=300s
```

Tại sao phải kết xuất thành tệp trước? Việc lưu trữ manifest được kết xuất dưới dạng cấu phần phần mềm CI/CD sẽ cung cấp cho bạn bản ghi chính xác về những gì đã được triển khai - hữu ích cho việc kiểm tra, gỡ lỗi và khắc phục thảm họa.

29.15 Khắc phục sự cố Kustomize

Lỗi: đang tích lũy tài nguyên

```
$ kubect1 kustomize overlays/production
Error: accumulating resources:
  accumulation err='merging resources from '../base':
  no such file or directory'

# Đường dẫn cơ sở sai trong kustomization.yaml
# Kiểm tra đường dẫn tương đối từ lớp phủ đến cơ sở:
ls overlays/production/../../base/
# Sửa kustomization.yaml:
# tài nguyên:
# - ../../base ← phải khớp với đường dẫn tương đối thực tế
```

Lỗi: Xung đột bản vá hợp nhất chiến lược

```
$ kubect1 apply -k overlays/production
Error: strategic merge patch: ...
  "spec.template.spec.containers[name="app"]":
  unable to find api field in struct Container for the key "unknown-field"

# Bản vá tham chiếu đến một trường không tồn tại
# Kiểm tra tệp vá - tên trường phải chính xác:
kubect1 explain deployment.spec.template.spec.containers | grep -i "field\|resource"
# Những lỗi thường gặp:
# "tài nguyên" thay vì "tài nguyên"
# "bản sao" ở cấp độ sai (đó là thông số kỹ thuật, không phải spec.template.spec)
```

Xác thực đầu ra tùy chỉnh trước khi áp dụng

```
# Xem trước những gì Kustomize sẽ tạo ra (không áp dụng)
kubectl kustomize overlays/production

# Xác thực cú pháp YAML
kubectl kustomize overlays/production | kubectl apply --dry-run=client -f -

# Khác biệt với những gì đang chạy trong cụm
kubectl diff -k overlays/production

# Kiểm tra xem Kustomize có thể giải quyết tất cả tài nguyên không
kubectl kustomize overlays/production 2>&1 | grep -i error
```

Các vấn đề phổ biến về Kustomize

```
# Vấn đề 1: Các bản vá không được áp dụng - namePrefix thay đổi tên
# Nếu cơ sở có tên: ứng dụng và lớp phủ sẽ thêm namePrefix: prod-
# thì bản vá phải tham chiếu: prod-app (không phải ứng dụng)
# Sử dụng tên được tạo cuối cùng trong các bản vá

# Gotcha 2: Trình tạo ConfigMap không cập nhật khi nội dung thay đổi
# Thêm cờ --load-restrictor nếu các tệp nằm ngoài root tùy chỉnh:
kubectl kustomize --load-restrictor LoadRestrictionsNone ./overlays/prod

# Gotcha 3: Biến áp hình ảnh chỉ áp dụng cho containers, không áp dụng cho initContainers
# Sử dụng bản vá riêng cho initContainers:
# bản vá lỗi:
# - mục tiêu:
#   loại: Deployment
#   bản vá: |-
#     - op: thay thế
#       đường dẫn: /spec/template/spec/initContainers/0/image
#       giá trị: myimage:v2.0

# Gotcha 4: Tên ConfigMap được băm theo mặc định
# Điều này tốt (ngăn chặn cấu hình cũ) nhưng có thể phá vỡ tài liệu tham khảo
# Vô hiệu hóa hàm băm cho một trình tạo cụ thể:
# configMapGenerator:
# - tên: cấu hình ứng dụng
#   tùy chọn:
#     vô hiệu hóaNameSuffixHash: true ← Chỉ sử dụng khi bạn cần tên ổn định
```

Chương 30 - Chứng nhận CKA 2026: Nghiên cứu đầy đủ và tham khảo lệnh

30.1 Giới thiệu về kỳ thi CKA

Kỳ thi **Quản trị viên Kubernetes được chứng nhận (CKA)** được tổ chức bởi Tổ chức điện toán đám mây gốc (CNCF) và Tổ chức Linux. Đây là bài kiểm tra thực hành dựa trên hiệu suất - bạn được cấp một cụm Kubernetes trực tiếp và phải hoàn thành các nhiệm vụ trong thời gian giới hạn chỉ bằng cách sử dụng thiết bị đầu cuối. Không có câu hỏi trắc nghiệm.

Chi tiết kỳ thi năm 2026

Thuộc tính	Chi tiết
Thời lượng	2 giờ
Câu hỏi	~17 nhiệm vụ dựa trên hiệu suất
Điểm đậu	66%
Chế độ thi	Giám sát trực tuyến (thiết bị đầu cuối dựa trên trình duyệt)
Phiên bản Kubernetes	Kubernetes 1.35 (môi trường thi CKA; kiểm tra 17/09/2026)
Chính sách thi lại	Bao gồm 1 lần thi lại miễn phí
hiệu lực	3 năm kể từ ngày qua
Môi trường	Trình duyệt bảo mật PSI Bridge + được cung cấp kubeconfig

Tài nguyên được phép trong kỳ thi

Kỳ thi CKA cho phép bạn mở **một tab trình duyệt bổ sung** trở tới tài liệu Kubernetes chính thức tại kubernetes.io/docs. Bạn KHÔNG được sử dụng bất kỳ tài nguyên bên ngoài nào khác. Bạn nên đánh dấu các trang tài liệu quan trọng trước kỳ thi.

30.2 Các lĩnh vực và tỷ trọng kỳ thi CKA

Chương trình giảng dạy CKA được chia thành năm lĩnh vực. Tỷ lệ phần trăm cho biết mỗi miền đạt được bao nhiêu điểm trong bài kiểm tra.

Miền 1: Kiến trúc, cài đặt và cấu hình cụm (25%)

Đây là miền nặng nhất. Nó bao gồm việc thiết lập và quản lý Kubernetes control plane, RBAC, kubeadm và etcd.

Chủ đề chính:

- Cài đặt cụm kubeadm, tham gia nodes và nâng cấp cụm
- Kiểm soát truy cập dựa trên Role (RBAC): Roles, ClusterRoles, RoleBindings, ClusterRoleBindings
- ServiceAccounts và các quyền của họ
- Sao lưu và khôi phục etcd
- Quản lý tệp Kubeconfig và bối cảnh nhiều cụm
- Cờ và cấu hình máy chủ Kubernetes API
- Khái niệm control plane có tính sẵn sàng cao

Bài kiểm tra ví dụ: *"Tạo ClusterRole có tên pod-reader cho phép liệt kê và nhận pods, sau đó liên kết nó với người dùng jane trong development namespace."*

```
# Tạo ClusterRole
kubectl create clusterrole pod-reader \
  --verb=get,list,watch \
  --resource=pods

# Tạo RoleBinding trong quá trình phát triển namespace
# (RoleBinding liên kết ClusterRole trong phạm vi namespace)
kubectl create rolebinding jane-pod-reader \
  --clusterrole=pod-reader \
  --user=jane \
  --namespace=development
```

```
# Xác minh các công việc ràng buộc
kubectl auth can-i list pods \
  --namespace=development \
  --as=jane
# Sản lượng dự kiến: có
```

Miền 2: Workload & Lập lịch (15%)

Miền này bao gồm việc triển khai và quản lý ứng dụng trên Kubernetes.

Chủ đề chính:

- Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, CronJobs
- Cập nhật và khôi phục
- Tài nguyên requests và limits, LimitRanges, ResourceQuotas
- ConfigMaps và Secrets - tạo, cập nhật, gắn kết
- Bộ chọn nút, ái lực node, taints và tolerations
- Ưu tiên và ưu tiên Pod
- Bộ co giãn tự động Pod ngang (HPA)
- Các mẫu Multi-container pod (ban đầu containers, sidecar)

Nhiệm vụ kiểm tra ví dụ: *"Co giãn Deployment web-app trong namespace production thành 5 bản sao, sau đó cập nhật image lên nginx:1.25 và xác minh quá trình triển khai hoàn tất thành công."*

```
# quy mô
kubectl scale deployment web-app -n production --replicas=5

# Cập nhật image
kubectl set image deployment/web-app nginx=nginx:1.25 -n production

# Xem buổi giới thiệu
kubectl rollout status deployment/web-app -n production

# Xác minh
kubectl get pods -n production -l app=web-app
```

Bài kiểm tra ví dụ: *"Tạo Pod sử dụng ConfigMap để đặt biến môi trường."*

```
# Tạo ConfigMap
kubectl create configmap app-config \
  --from-literal=DB_HOST=postgres.default.svc.cluster.local \
  --from-literal=DB_PORT=5432

# Tạo pod bằng configmap
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "env | grep DB && sleep 3600"]
    envFrom:
    - configMapRef:
        name: app-config
EOF
```

Miền 3: Services & Mạng (20%)

Miền này bao gồm cách các ứng dụng giao tiếp trong và ngoài cụm.

Chủ đề chính:

- Các loại Service: ClusterIP, NodePort, LoadBalancer, ExternalName
- Tài nguyên Ingress và bộ điều khiển Ingress
- NetworkPolicies - hạn chế giao tiếp Pod-to-Pod
- DNS và CoreDNS (được trình bày đầy đủ trong Chương 10 của hướng dẫn này)
- Nguyên tắc cơ bản về định tuyến kube-proxy và Service
- Gateway API (bổ sung mới hơn cho chương trình giảng dạy 2025/2026)

Nhiệm vụ kiểm tra ví dụ: *"Tạo NetworkPolicy trong namespace api chỉ cho phép Pods có nhãn role=frontend giao tiếp với Pods có nhãn role=backend trên cổng 8080."*

```
# mạng-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-to-backend
  namespace: api
spec:
  podSelector:
    matchLabels:
      role: backend          # Chính sách này áp dụng cho phần phụ trợ pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend     # Chỉ cho phép TỪ giao diện pods
  ports:
  - protocol: TCP
    port: 8080
```

```
kubectl apply -f network-policy.yaml
```

Kiểm tra: tạo giao diện người dùng pod và thử curl phần phụ trợ

```
kubectl run frontend-test \
  --image=curlimages/curl \
  --labels="role=frontend" \
  --rm -it --restart=Never \
  -n api \
  -- curl -s http://backend-service:8080/health
```

Kiểm tra: tạo pod không có nhãn (nên BLOCKED)

```
kubectl run blocked-test \
  --image=curlimages/curl \
  --rm -it --restart=Never \
  -n api \
  -- curl -s --max-time 5 http://backend-service:8080/health
```

Điều này sẽ là timeout/fail – bị chặn bởi NetworkPolicy

Bài kiểm tra ví dụ: *"Hiển thị Deployment my-app dưới dạng dịch vụ NodePort trên cổng 30080."*

```
kubectl expose deployment my-app \
  --type=NodePort \
  --port=80 \
  --target-port=8080 \
  --name=my-app-nodeport
```

Patch to set the specific NodePort

```
kubectl patch service my-app-nodeport \
  --type='json' \
  -p='[{"op": "replace", "path": "/spec/ports/0/nodePort", "value": 30080}]'
```

Nhiệm vụ thi mẫu: *"Tạo một Ingress định tuyến /api tới dịch vụ api-service:8080 và /web tới web-service:80."*

```
apiVersion: networking.k8s.io/v1
```

```

kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80

kubectl apply -f ingress.yaml
kubectl get ingress my-ingress
kubectl describe ingress my-ingress

```

Miền 4: Lưu trữ (10%)

Miền này bao gồm việc lưu trữ dữ liệu liên tục trong Kubernetes.

Chủ đề chính:

- PersistentVolumes (PV) và PersistentVolumeClaims (PVC)
- StorageClasses và cung cấp động
- Chế độ truy cập volume: ReadWriteOnce, ReadOnlyMany, ReadWriteMany
- Các loại ổ đĩa: HostPath, EmptyDir, nfs, configMap, secret
- Gắn volume trong containers

Bài kiểm tra ví dụ: *"Tạo PersistentVolume gồm 1Gi bằng HostPath /data/myapp, sau đó tạo PVC xác nhận quyền sở hữu đó, sau đó gắn nó vào Pod tại /app/data."*

```

# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: myapp-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  hostPath:
    path: /data/myapp
  storageClassName: manual

# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myapp-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:

```

```

    requests:
      storage: 1Gi
    storageClassName: manual

# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: app-storage
          mountPath: /app/data
  volumes:
    - name: app-storage
      persistentVolumeClaim:
        claimName: myapp-pvc

kubectl apply -f pv.yaml -f pvc.yaml -f pod.yaml

# Xác minh PVC bị ràng buộc
kubectl get pvc myapp-pvc
# TRẠNG THÁI phải là "Bị ràng buộc"

# Xác minh gắn kết
kubectl exec myapp-pod -- df -h /app/data

```

Miền 5: Khắc phục sự cố (30%)

Khắc phục sự cố là **lĩnh vực lớn nhất** trong CKA và ảnh xạ trực tiếp tới các kỹ năng được đề cập trong toàn bộ hướng dẫn này. Nó yêu cầu chẩn đoán và sửa chữa Pods, nodes bị hỏng, các thành phần cụm và kết nối mạng.

Chủ đề chính:

- Chẩn đoán lỗi Pod: OOMKilled, CrashLoopBackOff, ImagePullBackOff, Pending
- Xử lý sự cố nút: Lỗi NotReady nodes, kubelet
- Lỗi thành phần cụm: máy chủ API, bộ lập lịch, trình quản lý bộ điều khiển, etcd
- Sự cố mạng ứng dụng
- Phân tích nhật ký và kiểm tra sự kiện

Bài thi mẫu: *"Pod broken-app trong namespace qa nằm trong CrashLoopBackOff. Hãy tìm nguyên nhân và khắc phục."*

```

# Bước 1: Mô tả Pod
kubectl describe pod broken-app -n qa

# Nhìn vào:
# - Phần "Sự kiện": Lỗi kéo image, OOMKill, lỗi thăm dò hoạt động
# - "Trạng thái" của container: lý do, mã thoát, tín hiệu
# - "Trạng thái cuối cùng": chuyện gì đã xảy ra trước vụ tai nạn hiện tại

# Bước 2: Kiểm tra nhật ký hiện tại và trước đó
kubectl logs broken-app -n qa
kubectl logs broken-app -n qa --previous

# Bước 3: Dựa vào mã thoát:
# Thoát 137 (OOMKill) → tăng bộ nhớ limits
# Thoát 1 (lỗi ứng dụng) → kiểm tra nhật ký ứng dụng xem có lỗi khởi động không
# CrashLoop không có nhật ký → Lỗi entrypoint/command

# Bước 4: Khắc phục – e.g., sửa biến môi trường không chính xác
kubectl set env deployment/broken-app -n qa DB_HOST=correct-hostname

# Hoặc chỉnh sửa triển khai
kubectl edit deployment broken-app -n qa

```

Bài kiểm tra ví dụ: "node ở trạng thái NotReady. Hãy điều tra và khôi phục nó."

```
# Bước 1: Xác định NotReady node
kubectl get nodes

# Bước 2: Mô tả node cho các sự kiện
kubectl describe node <node-name>

# Bước 3: SSH vào node (nếu đề thi cho phép)
# Kiểm tra trạng thái kubelet
systemctl status kubelet

# Kiểm tra nhật ký kubelet
journalctl -u kubelet -n 50 --no-pager

# Các cách sửa lỗi phổ biến:
# kubelet bị dừng → khởi động systemctl kubelet
# Lỗi cấu hình kubelet → kiểm tra /var/lib/kubelet/config.yaml
# Sự cố môi trường chạy container → trạng thái systemctl containerd

# Khởi động lại kubelet
systemctl restart kubelet

# Bước 4: Xác minh node phục hồi
kubectl get nodes -w
```

30.3 Sao lưu và khôi phục etcd

Sao lưu và khôi phục etcd là một kỹ năng CKA quan trọng xuất hiện trong miền Kiến trúc cụm. Tiện ích `etcdctl` được sử dụng cho thao tác này.

Ví dụ: Sao lưu etcd vào một tệp

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

Giải thích về cờ:

- `ETCDCTL_API=3` - Biến dùng trong ví dụ `etcdctl` cũ; các phiên bản `etcdctl` hiện đại mặc định dùng API v3
- `--endpoints` - Điểm cuối etcd (thường là 127.0.0.1:2379 trên control plane node)
- `--cacert`, `--cert`, `--key` - Chứng chỉ TLS để truy cập xác thực vào etcd

Ví dụ: Xác minh bản sao lưu

```
etcdctl snapshot status /tmp/etcd-backup.db \
--write-out=table
```

Đầu ra hiển thị: Giá trị băm, bản sửa đổi, tổng số khóa và kích thước. Luôn xác minh sau khi sao lưu.

Ví dụ: Khôi phục etcd từ bản sao lưu

```
etcdctl snapshot restore /tmp/etcd-backup.db \
--data-dir=/var/lib/etcd-restored \
--bump-revision=1000000000 \
--mark-compacted
```

Sau đó cập nhật manifest Pod tĩnh etcd để sử dụng thư mục dữ liệu mới:

```
# Chỉnh sửa /etc/kubernetes/manifests/etcd.yaml
# Thay đổi: --data-dir=/var/lib/etcd
# Tới: --data-dir=/var/lib/etcd-restored
# Và cập nhật volume HostPath cho phù hợp
```


30.4 Nâng cấp cụm bằng kubeadm

Thực hành theo quy trình đầy đủ tại mục 2.12: đổi kho APT, chọn bản vá 1.35.x, nâng kubeadm, nâng control plane đầu tiên bằng `kubeadm upgrade apply`, rồi chạy `kubeadm upgrade node` trên từng control plane còn lại và worker. Drain từng node trước khi nâng kubelet và uncordon sau khi xác minh Ready.

```
# Máy có kubeconfig quản trị: kiểm tra trước và sau mỗi node
kubectl get nodes -o wide
kubectl get pods -A
kubectl get --raw='/readyz?verbose'
```

Không bỏ qua minor, không nâng kubelet vượt phiên bản API server, không xử lý đồng thời nhiều thành viên etcd. Chọn đúng bản vá đề bài yêu cầu; không dùng tên gói kiểu 1.30.0-00 của kho cũ.

30.5 RBAC - Roles, ClusterRoles, RoleBinding và ClusterRoleBinding

RBAC là một trong những khu vực được thử nghiệm thường xuyên nhất trong CKA. Nó kiểm soát ai có thể làm gì với tài nguyên nào trong đó namespaces.

Mô hình RBAC có bốn loại tài nguyên:

Tài nguyên	Phạm vi	Mục đích
Role	Namespace	Xác định quyền trong một namespace
ClusterRole	Toàn cụm	Xác định quyền trên tất cả namespaces hoặc cho các tài nguyên trong phạm vi cụm
RoleBinding	Namespace	Cấp Role hoặc ClusterRole cho user/group/SA trong namespace
ClusterRoleBinding	Toàn cụm	Cấp ClusterRole cho user/group/SA trên toàn bộ cụm

Ví dụ: Tạo Role chỉ cho phép liệt kê pod trong namespace

```
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  --namespace=development
```

Ví dụ: Liên kết Role với người dùng

```
kubectl create rolebinding dev-pod-reader \
  --role=pod-reader \
  --user=developer1 \
  --namespace=development
```

Ví dụ: Tạo ServiceAccount với các quyền cụ thể

```
# Tạo ServiceAccount
kubectl create serviceaccount monitoring-sa -n monitoring

# Tạo ClusterRole để đọc số liệu cụm
kubectl create clusterrole metrics-reader \
  --verb=get,list \
  --resource=nodes,pods,services,endpoints

# Liên kết với ServiceAccount
kubectl create clusterrolebinding monitoring-metrics-reader \
  --clusterrole=metrics-reader \
  --serviceaccount=monitoring:monitoring-sa
```

Ví dụ: Kiểm tra quyền bằng kubectl auth can-i

```
# Kiểm tra với tư cách người dùng cụ thể
kubectl auth can-i create deployments --namespace=development --as=developer1

# Kiểm tra dưới dạng ServiceAccount
kubectl auth can-i get pods \
  --namespace=monitoring \
  --as=system:serviceaccount:monitoring:monitoring-sa

# Liệt kê tất cả các quyền cho người dùng hiện tại
kubectl auth can-i --list

# Liệt kê tất cả các quyền cho người dùng trong namespace cụ thể
kubectl auth can-i --list --namespace=development --as=developer1
```

30.6 Jobs và CronJobs

Ví dụ: Tạo Job chạy tính toán

```
kubectl create job compute-pi \
  --image=perl \
  -- perl -Mbignum=bpi -wle 'print bpi(2000)'

# Xem Job chạy
kubectl get jobs -w

# Xem đầu ra
kubectl logs -l job-name=compute-pi
```

Ví dụ: Tạo CronJob chạy 5 phút một lần

```
kubectl create cronjob health-check \
  --image=curlimages/curl \
  --schedule="*/5 * * * *" \
  -- curl -s http://my-service/health

# Danh sách CronJobs
kubectl get cronjobs

# Kích hoạt thủ công CronJob
kubectl create job --from=cronjob/health-check manual-run-001

# Xem vài lần chạy gần đây nhất
kubectl get jobs -l job-name=health-check --sort-by=.metadata.creationTimestamp
```

30.7 Quản lý node: cordon, drain và taint

Ví dụ: Kết nối node (ngăn Pods mới lên lịch)

```
kubectl cordon <node-name>
```

Việc này có tác dụng gì: Đánh dấu node là `SchedulingDisabled`. Pods hiện tại tiếp tục chạy nhưng sẽ không có Pods mới nào được lên lịch trên đó. Sử dụng trước khi bảo trì.

Ví dụ: Xả node (đuổi tất cả Pods một cách duyên dáng)

```
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --grace-period=60
```

Việc này có tác dụng gì: Loại bỏ tất cả Pods khỏi node (ngoại trừ DaemonSet Pods và Pods phản chiếu). Cờ `--ignore-daemonsets` là bắt buộc vì DaemonSet Pods không thể bị loại bỏ. `--delete-emptydir-data` là bắt buộc nếu bất kỳ Pods nào sử dụng ổ `emptyDir` (dữ liệu của chúng sẽ bị mất).

Ví dụ: Rút dây node sau khi bảo trì

```
kubectl uncordon <node-name>
```

Ví dụ: Thêm taint vào node

```
# NoSchedule: pods mới không có toleration sẽ không được lên lịch tại đây
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule

# PreferNoSchedule: bộ lập lịch cố gắng tránh node này nhưng sẽ sử dụng nó nếu cần
kubectl taint nodes <node-name> dedicated=gpu:PreferNoSchedule

# NoExecute: loại bỏ pods hiện có không chấp nhận taint
kubectl taint nodes <node-name> maintenance=true:NoExecute
```

Ví dụ: Xóa taint khỏi node

```
# Thêm dấu gạch ngang (-) vào taint để xóa nó
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule-
```

30.8 Lập lịch Pod: affinity, nodeSelector và toleration

Ví dụ: Lên lịch Pod trên node cụ thể bằng cách sử dụng nodeSelector

```
# Đầu tiên, gắn nhãn mục tiêu node
kubectl label node <node-name> disktype=ssd

# Sau đó tạo Pod bằng nodeSelector
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-app
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: app
    image: nginx
EOF
```

Ví dụ: Lên lịch Pod với mối quan hệ node (ưu tiên)

```
apiVersion: v1
kind: Pod
metadata:
  name: affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: region
            operator: In
            values:
            - us-east-1
  containers:
  - name: app
    image: nginx
```

Ví dụ: Thêm toleration để lên lịch trên node có taint

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"
  containers:
  - name: app
    image: nvidia/cuda:11.0-base
```

30.9 ResourceQuotas và LimitRanges

Ví dụ: Tạo ResourceQuota cho namespace

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: development
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
    pods: "20"
    services: "10"
    persistentvolumeclaims: "5"
```

```
kubectl apply -f quota.yaml
```

```
# Xem mức sử dụng hạn ngạch
kubectl describe resourcequota dev-quota -n development
```

Ví dụ: Tạo LimitRange để đặt tài nguyên mặc định requests

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: development
spec:
  limits:
  - default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
    max:
      cpu: "2"
      memory: "2Gi"
    min:
      cpu: "50m"
      memory: "64Mi"
    type: Container
  # limits mặc định (được sử dụng nếu container không chỉ định)
  # requests mặc định (được sử dụng nếu container không chỉ định)
  # Giá trị tối đa được phép
  # Giá trị tối thiểu được phép
```

```
kubectl apply -f limitrange.yaml
```

```
# Xác minh
kubectl describe limitrange default-limits -n development
```

30.10 Chiến lược thi và quản lý thời gian của CKA

Hiểu nội dung kỹ thuật là cần thiết nhưng chưa đủ để vượt qua CKA. Kỳ thi yêu cầu hoàn thành khoảng 17 nhiệm vụ trong 2 giờ - khoảng 7 phút cho mỗi nhiệm vụ. Tốc độ và hiệu quả cũng quan trọng như kiến thức.

25.10.1 Thiết lập môi trường Shell bài kiểm tra của bạn

Điều đầu tiên cần làm khi kỳ thi bắt đầu là định cấu hình shell của bạn để đạt hiệu quả. Chỉ điều này thôi cũng có thể tiết kiệm được 5-10 phút cho toàn bộ bài kiểm tra.

```
# Bật tính năng tự động hoàn thành tab kubectl (hoạt động trong môi trường thi)
source <(kubectl completion bash)
echo "source <(kubectl completion bash)" >> ~/.bashrc

# Tạo bí danh cho các lệnh được sử dụng nhiều nhất
alias k=kubectl
alias kgp='kubectl get pods'
alias kgs='kubectl get services'
alias kgd='kubectl get deployments'
alias kgn='kubectl get nodes'
alias kdp='kubectl describe pod'
alias kns='kubectl config set-context --current --namespace'

# Đặt bí danh 'k' cũng sử dụng tính năng tự động hoàn thành
complete -F __start_kubectl k

# Đặt vim làm trình chỉnh sửa mặc định cho chỉnh sửa kubectl
export KUBE_EDITOR=vim

# Áp dụng tất cả các bí danh ngay lập tức
source ~/.bashrc
```

Lệnh bắt buộc 25.10.2 so với tệp YAML

CKA khen thưởng tốc độ. **Sử dụng các lệnh mệnh lệnh bất cứ khi nào có thể** thay vì viết YAML từ đầu. Mỗi giây tiết kiệm được khi tạo tệp là một giây dành cho các tác vụ khó hơn.

So sánh tốc độ:

```
# CHẬM: Viết Deployment YAML từ đầu
vim deployment.yaml
# (phút đánh máy...)

# NHANH CHÓNG: Sáng tạo bắt buộc
kubectl create deployment my-app --image=nginx --replicas=3

# NHANH CHÓNG: Tạo YAML và chỉnh sửa
kubectl create deployment my-app --image=nginx --dry-run=client -o yaml > my-app.yaml
# (thực hiện các chỉnh sửa tối thiểu đối với tệp được tạo)
kubectl apply -f my-app.yaml
```

Chạy thử + đầu ra: Kỹ thuật thi hiệu quả nhất

```
# Tạo Pod YAML mà không cần tạo nó
kubectl run my-pod --image=nginx --dry-run=client -o yaml > pod.yaml

# Tạo Deployment
kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml > dep.yaml

# Tạo Service
kubectl expose deployment my-dep --port=80 --dry-run=client -o yaml > svc.yaml

# Tạo ConfigMap
kubectl create configmap my-cm --from-literal=key=value --dry-run=client -o yaml > cm.yaml

# Tạo Secret
kubectl create secret generic my-secret --from-literal=password=secret --dry-run=client -o yaml > secret.yaml

# Tạo ServiceAccount
kubectl create serviceaccount my-sa --dry-run=client -o yaml > sa.yaml

# Tạo Role
kubectl create role my-role --verb=get,list --resource=pods --dry-run=client -o yaml > role.yaml

# Tạo ClusterRole
kubectl create clusterrole my-cr --verb=get,list --resource=pods --dry-run=client -o yaml > cr.yaml

# Tạo RoleBinding
kubectl create rolebinding my-rb --role=my-role --user=jane --dry-run=client -o yaml > rb.yaml
```

Chuyển đổi bối cảnh 25.10.3 giữa các cụm

Kỳ thi CKA thường trình bày nhiều cụm. Mỗi tác vụ chỉ định bối cảnh cụm nào sẽ sử dụng. **Luôn chuyển ngữ cảnh là bước ĐẦU TIÊN của mỗi tác vụ.** Làm việc trong sai cụm là một lỗi rất phổ biến và tốn kém.

```
# Liệt kê các bối cảnh có sẵn
kubectl config get-contexts

# Chuyển sang một cụm cụ thể
kubectl config use-context k8s-cluster-1

# Xác minh bạn đang ở đúng bối cảnh
kubectl config current-context

# Chuyển namespace trong ngữ cảnh hiện tại (tránh -n trên mọi lệnh)
kubectl config set-context --current --namespace=kube-system
```

25.10.4 Sử dụng kubectl giải thích để tham khảo nhanh

Trong kỳ thi, `kubectl explain` là tài liệu nội bộ của bạn. Nó hiển thị các trường của bất kỳ tài nguyên Kubernetes nào, loại và mô tả của chúng.

```
# Giải thích tất cả các trường Pod
kubectl explain pod

# Giải thích một trường lồng nhau cụ thể
kubectl explain pod.spec.containers

# Giải thích tolerations
kubectl explain pod.spec.tolerations

# Giải thích các quy tắc quan hệ
kubectl explain pod.spec.affinity.nodeAffinity

# Giải thích thông số PersistentVolumeClaim
kubectl explain pvc.spec

# Giải thích đệ quy (hiển thị cây đầy đủ)
kubectl explain deployment --recursive | head -50
```

Khi nào nên sử dụng: Khi bạn biết tài nguyên nhưng không thể nhớ chính xác tên trường hoặc cấu trúc. Ví dụ: nếu bạn quên đó là `imagePullPolicy` hay `image-pull-policy`, `kubectl explain pod.spec.containers` sẽ hiển thị cho bạn ngay lập tức.

Mẹo Vim 25.10.5 cho kỳ thi CKA

Hầu hết các ứng viên CKA đều sử dụng vim làm biên tập viên của họ. Biết các phím tắt này giúp giảm đáng kể thời gian chỉnh sửa.

```
# Điều hướng
gg          - Go to first line
G          - Go to last line
:<number>   - Go to specific line number (e.g., :42)
/text      - Search forward for "text"
n          - Next search result

# Chỉnh sửa
dd          - Delete current line
yy          - Copy (yank) current line
p          - Paste after current line
u          - Undo
Ctrl+r     - Redo

# Thụt lề (quan trọng đối với YAML)
>>         - Indent current line
<<         - Dedent current line
Visual mode + > or < - Indent/dedent selected lines

# Cài đặt vim dành riêng cho YAML (thêm vào ~/.vimrc trong bài kiểm tra)

# Chạy phần này để định cấu hình vim để chỉnh sửa YAML:
cat >> ~/.vimrc << 'EOF'
set expandtab      " Use spaces instead of tabs
set tabstop=2     " Tab width = 2 spaces
set shiftwidth=2  " Indent width = 2 spaces
set autoindent    " Auto-indent new lines
EOF
```

25.10.6 Kiểm tra công việc của bạn sau mỗi nhiệm vụ

Sau khi hoàn thành một nhiệm vụ, hãy luôn xác minh nó:

```
# Sau khi tạo Pod
kubectl get pod <pod-name>
kubectl describe pod <pod-name> | tail -20  # Kiểm tra sự kiện

# Sau khi tạo Deployment
kubectl get deployment <name>
kubectl rollout status deployment/<name>

# Sau khi tạo Service
kubectl get svc <name>
kubectl describe svc <name>

# Sau khi tạo RBAC
kubectl auth can-i <verb> <resource> --as=<user> -n <namespace>

# Sau hoạt động node
kubectl get nodes

# Sau khi sao lưu etcd
ls -la /tmp/etcd-backup.db
etcdctl snapshot status /tmp/etcd-backup.db
```

30.11 Static Pod trong CKA

Pods tĩnh là Pods được quản lý trực tiếp bởi kubelet trên một node cụ thể, không phải bởi máy chủ Kubernetes API hay bất kỳ bộ điều khiển nào. Chúng được định nghĩa là các tệp YAML được đặt trong một thư mục trên hệ thống tệp của node, điển hình là `/etc/kubernetes/manifests/`. Bản thân các thành phần control plane (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) chạy dưới dạng Pods tĩnh.

Ví dụ: Xác định vị trí thư mục manifest Pod tĩnh

```
# Trên control plane node
ls /etc/kubernetes/manifests/
# Đầu ra: etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml
```

Ví dụ: Tạo Pod tĩnh trên node

```
# SSH vào mục tiêu node
# Tạo một manifest trong thư mục pods tĩnh
cat << EOF > /etc/kubernetes/manifests/static-nginx.yaml
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
  namespace: default
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
EOF
```

Điều gì xảy ra: kubelet phát hiện tệp mới trong vài giây và tự động khởi động Pod. Không cần `kubectl apply`. Pod xuất hiện trong `kubectl get pods` với tên `static-nginx-<node-name>`.

Ví dụ: Xóa Pod tĩnh

```
# Điều này KHÔNG hoạt động - kubelet ngay lập tức tạo lại nó
kubectl delete pod static-nginx-<node-name>

# Cách đúng: xóa file kê khai trên node
rm /etc/kubernetes/manifests/static-nginx.yaml
```

kubelet phát hiện việc xóa tệp và chấm dứt Pod.

Ví dụ: Tìm đường dẫn Pod tĩnh từ cấu hình kubelet

```
# Nếu đường dẫn mặc định không hoạt động, hãy kiểm tra cấu hình kubelet
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# or
ps aux | grep kubelet | grep staticPodPath
```

30.12 Helm trong bối cảnh của CKA

Mặc dù Helm tập trung hơn vào chứng chỉ CKAD (Nhà phát triển ứng dụng Kubernetes được chứng nhận), nhưng chương trình giảng dạy CKA 2025/2026 lại bao gồm cách sử dụng Helm cơ bản. Hiểu mối quan hệ giữa Helm và các lệnh `kubectl/Kustomize` thì rất quan trọng.

Ví dụ: Thêm kho lưu trữ Helm

```
helm repo add stable https://charts.helm.sh/stable
helm repo update
```

Ví dụ: Cài đặt biểu đồ

```
helm install my-release stable/nginx-ingress \
  --namespace ingress \
  --create-namespace \
  --set controller.replicaCount=2
```


Ví dụ: Liệt kê các bản phát hành đã cài đặt

```
helm list -A
```

Ví dụ: Nhận giá trị của một bản phát hành

```
helm get values my-release -n ingress
```

Ví dụ: Gỡ cài đặt một bản phát hành

```
helm uninstall my-release -n ingress
```

30.13 Các kịch bản khắc phục sự cố CKA hoàn chỉnh

Kỳ thi CKA luôn bao gồm các nhiệm vụ khắc phục sự cố. Đây là những mẫu phổ biến nhất:

Kịch bản 1: Khắc phục cụm bị hỏng - node NotReady

```
# Bước 1: Xác định vấn đề node
kubectl get nodes
# NAME          STATUS    ROLES    AGE   VERSION
# worker-1 NotReady <none> 5d v1.34.0 ← vấn đề!

# Bước 2: Kiểm tra điều kiện node
kubectl describe node worker-1 | grep -A 10 "Conditions:"

# Bước 3: SSH tới node
ssh worker-1

# Bước 4: Kiểm tra kubelet
sudo systemctl status kubelet
# • kubelet.service - kubelet: Tác nhân nút Kubernetes
#   Hoạt động: không thành công (Kết quả: mã thoát)
sudo journalctl -u kubelet -n 30 --no-pager

# Lỗi thường gặp 1: Dịch vụ kubelet chưa được bật
sudo systemctl enable kubelet
sudo systemctl start kubelet

# Lỗi thường gặp 2: Lỗi cấu hình kubelet
sudo journalctl -u kubelet | grep "error"
# "không tải được tệp cấu hình Kubelet /var/lib/kubelet/config.yaml"
# Tệp có thể bị thiếu hoặc bị hỏng - tạo lại bằng kubeadm:
sudo kubeadm init phase kubelet-config

# Lỗi thường gặp 3: Thời gian chạy của container không hoạt động
sudo systemctl status containerd
sudo systemctl start containerd
sudo systemctl restart kubelet
```

Tình huống 2: Sửa Deployment bị hỏng

```
# Tác vụ: "Ứng dụng web Deployment trong giao diện người dùng namespace không chạy. Hãy sửa nó."

# Bước 1: Kiểm tra trạng thái triển khai
kubectl get deployment web-app -n frontend
# NAME          READY   UP-TO-DATE   AVAILABLE   AGE
# ứng dụng web 0/3   0           0           10m

# Bước 2: Mô tả việc triển khai
kubectl describe deployment web-app -n frontend | tail -30

# Bước 3: Kiểm tra pods
kubectl get pods -n frontend -l app=web-app
# NAME                                READY   STATUS             RESTARTS   AGE
# web-app-7d8f9b6c4-abc12             0/1     ImagePullBackOff   0           10m

# Bước 4: Mô tả pod bị lỗi
kubectl describe pod web-app-7d8f9b6c4-abc12 -n frontend
# Sự kiện:
#   Cảnh báo không thành công 2m kubelet Không thể kéo image "nginx:1.99.99":
#     lỗi rpc: ...
#     404 Không tìm thấy

# Bước 5: Sửa image
kubectl set image deployment/web-app web=nginx:1.25 -n frontend
kubectl rollout status deployment/web-app -n frontend
```

Kịch bản 3: Phơi sáng Deployment

```
# Tác vụ: "Tạo Service để hiển thị máy chủ api triển khai trên cổng 8080"

# Tạo mệnh lệnh nhanh chóng
kubectl expose deployment api-server --port=8080 --target-port=8080 \
  --name=api-service -n backend

# Xác minh nó hoạt động
kubectl get svc api-service -n backend
kubectl get endpoints api-service -n backend # Nên có IP
kubectl run test --image=curlimages/curl --rm -it --restart=Never \
  -n backend -- curl http://api-service:8080/health
```

Kịch bản 4: Tạo RBAC cho người dùng

```
# Nhiệm vụ: "Người dùng dev-user sẽ có thể get/list pods trong namespace dev"

kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  -n dev

kubectl create rolebinding dev-user-pod-reader \
  --role=pod-reader \
  --user=dev-user \
  -n dev

# Xác minh:
kubectl auth can-i get pods -n dev --as=dev-user # vâng
kubectl auth can-i delete pods -n dev --as=dev-user # no
kubectl auth can-i get pods -n production --as=dev-user # no
```

30.14 Bài tập luyện CKA: các dạng nhiệm vụ thường gặp

Loại nhiệm vụ: sao lưu etcd (xuất hiện trong hầu hết các kỳ thi CKA)

```
# Đọc kỹ câu hỏi - nó sẽ cho bạn biết nơi lưu bản sao lưu
# Tác vụ điển hình: "Sao lưu etcd sang /opt/cluster-backup.db"

ETCDCTL_API=3 etcdctl snapshot save /opt/cluster-backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key

# Xác minh:
etcdctl snapshot status /opt/cluster-backup.db -w table
```

Loại nhiệm vụ: NetworkPolicy (nhiệm vụ CKA chung)

```
# Nhiệm vụ: "Từ chối tất cả lưu lượng truy cập vào namespace bị hạn chế,
#           nhưng cho phép lưu lượng truy cập từ giám sát namespace trên cổng 9090"

# Bước 1: Áp dụng mặc định từ chối tất cả
cat << 'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: restricted
spec:
  podSelector: {}
  policyTypes:
    - Ingress
EOF

# Bước 2: Cho phép giám sát namespace
cat << 'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: restricted
spec:
  podSelector: {}
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: monitoring
      ports:
        - port: 9090
EOF
```

Loại nhiệm vụ: Bảo trì nút (cồng + taint)

```
# Nhiệm vụ: "Nút worker-2 cần được bảo trì.
#           Xả hết nước, sau đó taint với bảo trì=true:NoSchedule"

# Bước 1: Cordon (ngăn chặn việc lên lịch mới)
kubectl cordon worker-2

# Bước 2: Xả (đuổi chạy pods)
kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data

# Bước 3: Làm bản để bảo trì
kubectl taint node worker-2 maintenance=true:NoSchedule

# Sau khi bảo trì, khôi phục:
kubectl taint node worker-2 maintenance=true:NoSchedule- # Xóa taint
kubectl uncordon worker-2
```

Loại nhiệm vụ: Tạo multi-container pod

```
# Nhiệm vụ: "Tạo một 'máy chủ web' pod với hai containers:
#           nginx:1.25 có tên là 'web' trên cổng 80 và
#           busybox:1.35 có tên là 'logger' chạy 'sleep 3600'"

# Sử dụng --dry-run để tạo mẫu, sau đó chỉnh sửa:
kubectl run web-server --image=nginx:1.25 --dry-run=client -o yaml > pod.yaml

# Chỉnh sửa pod.yaml để thêm container thứ hai và sửa cổng:

apiVersion: v1
kind: Pod
metadata:
  name: web-server
spec:
  containers:
  - name: web
    image: nginx:1.25
    ports:
    - containerPort: 80
  - name: logger
    image: busybox:1.35
    command: ["sleep", "3600"]
```

30.15 Chuẩn bị ngày thi và lỗi thường gặp

Thiết lập shell của bạn trước khi bắt đầu nhiệm vụ:

```
# 4 bí danh này giúp tiết kiệm thời gian đáng kể trong kỳ thi
alias k=kubectl
alias kn='kubectl -n'
alias kg='kubectl get'
alias kd='kubectl describe'
export do='--dry-run=client -o yaml' # Sử dụng: k chạy thử --image=nginx $do

# Bật tính năng tự động hoàn thành kubectl (phải được thiết lập trong bài kiểm tra)
source <(kubectl completion bash)
complete -F __start_kubectl k

# Đặt namespace hiện tại để tránh -n trên mọi lệnh
kubectl config set-context --current --namespace=<task-namespace>
```

Những lỗi thường gặp gây mất điểm:

```
# Sai lầm 1: Làm sai namespace
# Luôn kiểm tra: Chế độ xem cấu hình kubectl --minify | grep namespace
# Đặt namespace khi bắt đầu mỗi tác vụ

# Sai lầm 2: Quên --dry-run=client -o yaml đối với các tài nguyên phức tạp
# Luôn tạo YAML thay vì cố gắng viết nó từ đầu

# Sai lầm 3: Không xác minh sau khi hoàn thành nhiệm vụ
# Sau mỗi nhiệm vụ: kubectl lấy <resource> và kubectl mô tả <resource>
# Check that it's actually Running/Bound/Ready before moving on

# Sai lầm 4: Lãng phí thời gian vào một nhiệm vụ khó khăn
# Nếu bị kẹt >5 phút: gắn cờ nó, chuyển sang nhiệm vụ tiếp theo, quay lại
# 17 nhiệm vụ trong 2 giờ = ~7 phút mỗi nhiệm vụ

# Sai lầm 5: Không đọc bài đầy đủ
# Tác vụ có thể chỉ định: namespace, tên tài nguyên, nhãn, trường cụ thể
# Đọc toàn bộ nhiệm vụ trước khi gõ bất cứ điều gì

# Sai lầm 6: Sử dụng sai context/cluster
# Khi bắt đầu mỗi tác vụ: Cấu hình kubectl bối cảnh sử dụng <context-from-task>
```

Phụ lục A - Bảng tham khảo nhanh

Các lệnh thực thi container A.1

```
# Tương tác shell
kubectl exec -it <pod> -- /bin/bash
kubectl exec -it <pod> -- /bin/sh # Alpine/minimal images

# Nhắm mục tiêu container cụ thể
kubectl exec -it <pod> -c <container> -- bash

# Lệnh đơn không tương tác
kubectl exec <pod> -- <command>

# Giải thích vỏ cho pipes/redirects
kubectl exec <pod> -- sh -c 'cmd1 | cmd2'

# Thực thi cụ thể namespace
kubectl exec -it <pod> -n <ns> -- bash
```

Lệnh ghi nhật ký A.2

```
kubectl logs <pod> # Tất cả nhật ký
kubectl logs <pod> -c <container> # container cụ thể
kubectl logs -f <pod> # Stream/follow
kubectl logs <pod> --tail=100 # 100 dòng cuối cùng
kubectl logs <pod> --since=1h # 1 giờ qua
kubectl logs <pod> --since-time="2024-06-01T09:00Z" # Kể từ dấu thời gian
kubectl logs <pod> --previous # Nhật ký container bị lỗi
kubectl logs -l app=my-app --all-containers=true # Tất cả pods theo nhãn
```

Lệnh truyền tệp A.3

```
kubectl cp <pod>:/remote/path ./local/path # Container → cục bộ
kubectl cp ./local/path <pod>:/remote/path # Địa phương → container
kubectl cp <pod>:/path -c <container> ./path # container cụ thể
```

Lệnh kiểm tra A.4

```
kubectl describe pod <pod> # Bản tóm tắt dễ đọc của con người
kubectl get pod <pod> -o yaml # Thông số YAML đầy đủ
kubectl get pod <pod> -o json # JSON đầy đủ
kubectl get pod <pod> -o jsonpath='{.status.podIP}' # Trường cụ thể
kubectl get pods -A -o wide # Tất cả pods, tất cả namespaces
kubectl top pod <pod> --containers # Sử dụng tài nguyên
```

Chuyển tiếp cổng A.5

```
kubectl port-forward <pod> 8080:80 # Địa phương:Container
kubectl port-forward service/<svc> 8080:80 # Qua dịch vụ
kubectl port-forward deployment/<dep> 8080:80 # Thông qua triển khai
kubectl port-forward <pod> 8080:80 9090:9090 # Nhiều cổng
```

Bộ chứa thử nghiệm tạm thời A.6 (Tạo và tự động xóa)

```
# BusyBox tương tác shell
kubectl run test --image=busybox --rm -it --restart=Never -- sh

# Kiểm tra ping
kubectl run test --image=busybox --rm -it --restart=Never -- ping -c 4 <host>

# Kiểm tra bằng curl
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- curl -s <url>

# Bộ công cụ đầy đủ (netshoot)
kubectl run test --image=nicolaka/netshoot --rm -it --restart=Never -- bash

# namespace cụ thể
kubectl run test --image=busybox --rm -it --restart=Never -n <ns> -- sh

# Kiểm tra cổng TCP
kubectl run test --image=busybox --rm -it --restart=Never -- nc -zv <host> <port>

# Theo dõi lộ trình
kubectl run test --image=nicolaka/netshoot --rm -it --restart=Never -- traceroute <host>
```

Kiểm tra A.7 CoreDNS

```
# Kiểm tra CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Xem cấu hình CoreDNS
kubectl get configmap coredns -n kube-system -o yaml

# Nhật ký CoreDNS
kubectl logs -n kube-system -l k8s-app=kube-dns

# Kiểm tra phân giải DNS
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup kubernetes.default
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <svc>.<ns>.svc.cluster.local
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig <svc>.default.svc.cluster.local
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig +short <svc>
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig @<coredns-ip> <svc>

# Kiểm tra pod resolv.conf
kubectl exec <pod> -- cat /etc/resolv.conf

# Kiểm tra DNS bên ngoài
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup google.com
```

Bộ chứa gỡ lỗi tạm thời A.8 (Ổn định từ Kubernetes 1.25)

```
# Đưa phần gỡ lỗi container vào pod đang chạy
kubectl debug -it <pod> --image=nicolaka/netshoot --target=<container> -- bash

# Gỡ lỗi distroless pod
kubectl debug -it <pod> --image=busybox --target=<container> -- sh

# Tạo bản sao gỡ lỗi của pod
kubectl debug <pod> -it --copy-to=<debug-pod-name> --image=busybox -- sh
kubectl delete pod <debug-pod-name> # Dọn dẹp

# Gỡ lỗi node
kubectl debug node/<node-name> -it --image=nicolaka/netshoot
```

Phụ lục B - Tham chiếu tốc độ lệnh CKA 2026

Phụ lục này được thiết kế như một thẻ sửa đổi nhanh chóng cho tất cả các lệnh miền CKA.

Kiến trúc cụm B.1 & RBAC

```
# Quản lý bối cảnh
kubectl config get-contexts
kubectl config use-context <context>
kubectl config set-context --current --namespace=<ns>
kubectl config current-context

# Tạo RBAC
kubectl create role <name> --verb=<verbs> --resource=<resources> -n <ns>
kubectl create clusterrole <name> --verb=<verbs> --resource=<resources>
kubectl create rolebinding <name> --role=<role> --user=<user> -n <ns>
kubectl create rolebinding <name> --clusterrole=<cr> --user=<user> -n <ns>
kubectl create clusterrolebinding <name> --clusterrole=<cr> --user=<user>
kubectl create clusterrolebinding <name> --clusterrole=<cr> --serviceaccount=<ns>:<sa>

# Kiểm tra quyền
kubectl auth can-i <verb> <resource> --as=<user> -n <ns>
kubectl auth can-i --list --as=<user> -n <ns>

# ServiceAccount
kubectl create serviceaccount <name> -n <ns>

# etcd
ETCDCTL_API=3 etcdctl snapshot save /path/backup.db --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
etcdctl snapshot status /path/backup.db --write-out=table
```

Workload & Lập lịch B.2

```
# Deployments
kubectl create deployment <name> --image=<img> --replicas=<n>
kubectl scale deployment <name> --replicas=<n>
kubectl set image deployment/<name> <container>=<new-image>
kubectl rollout status deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout history deployment/<name>

# Pods (bắt buộc)
kubectl run <name> --image=<img>
kubectl run <name> --image=<img> --env="KEY=VALUE" --labels="k=v"
kubectl run <name> --image=<img> --requests='cpu=100m,memory=128Mi'
kubectl run <name> --image=<img> --limits='cpu=500m,memory=512Mi'

# Jobs và CronJobs
kubectl create job <name> --image=<img> -- <command>
kubectl create cronjob <name> --image=<img> --schedule="* * * * *" -- <command>

# ConfigMaps và Secrets
kubectl create configmap <name> --from-literal=key=val --from-file=<file>
kubectl create secret generic <name> --from-literal=key=val

# Lập lịch nút
kubectl label node <node> <key>=<val>
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>
kubectl taint nodes <node> <key>=<val>:<effect>
kubectl taint nodes <node> <key>=<val>:<effect>- # Xóa taint
```

B.3 Services & Mạng

```
# Services
kubectl expose deployment <name> --port=<port> --target-port=<tp> --type=<type>
kubectl expose pod <name> --port=<port>

# Kiểm tra điểm cuối
kubectl get endpoints <svc>

# Kiểm tra DNS
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <service>
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <svc>.<ns>.svc.cluster.local

# NetworkPolicy: liệt kê tất cả
kubectl get networkpolicies -A
```

Lưu trữ B.4

```
# PV và PVC
kubectl get pv
kubectl get pvc -A
kubectl describe pvc <name>

# Xác minh volume đã được gắn
kubectl exec <pod> -- df -h <mountpath>
kubectl exec <pod> -- mount | grep <mountpath>
```

B.5 Khắc phục sự cố

```
# Khắc phục sự cố Pod
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl logs <pod> -c <container>
kubectl get events -n <ns> --sort-by=.lastTimestamp

# Khắc phục sự cố nút (trên node)
systemctl status kubelet
systemctl restart kubelet
journalctl -u kubelet -n 100 --no-pager

# Kiểm tra tất cả pods không chạy trên toàn cụm
kubectl get pods -A | grep -v Running | grep -v Completed

# Nhận pod trên node cụ thể
kubectl get pods -A -o wide | grep <node-name>

# Sử dụng tài nguyên
kubectl top nodes
kubectl top pods -A --sort-by=memory
```

Phụ lục C - Tham chiếu mẫu Kustomize

C.1 Hoàn thành tham chiếu trường kustomization.yaml

```
# kustomization.yaml - Tất cả các trường phổ biến
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Tài nguyên: danh sách các tệp YAML hoặc các tài liệu tham khảo từ xa cần đưa vào
resources:
  - deployment.yaml
  - service.yaml
  - ../../base # Đường dẫn tương đối đến tùy chỉnh khác
  - github.com/org/repo//path?ref=v1.0.0 # Tham chiếu GitHub từ xa

# Thành phần: các mảnh tùy chỉnh có thể tái sử dụng
components:
  - ../../components/monitoring
  - ../../components/tls

# Namespace: ghi đè namespace trên tất cả tài nguyên
namespace: production

# Chuyển đổi tên
namePrefix: prod-
nameSuffix: -v2

# Các nhãn chung (cũng sửa đổi bộ chọn)
commonLabels:
  environment: production
  managed-by: kustomize

# Các chú thích chung (KHÔNG sửa đổi bộ chọn - an toàn hơn cho các bản cập nhật)
commonAnnotations:
  team: platform-engineering

# Biến đổi hình ảnh
images:
  - name: my-app
    newName: registry.company.com/my-app
    newTag: v1.5.2
  - name: sidecar
    newTag: stable
  - name: my-app
    digest: sha256:abcdef... # Ghim vào digest (không thay đổi)

# Các bản vá (hợp nhất chiến lược hoặc JSON 6902)
patches:
  - path: patch-replicas.yaml # Bản vá dựa trên tệp
  - path: patch-env.yaml
    target:
      kind: Deployment
      name: my-app
  - target: # Bản vá JSON 6902 nội tuyến
    kind: Deployment
    name: my-app
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 5

# Trình tạo ConfigMap
configMapGenerator:
  - name: app-config
    files:
      - config.properties
    literals:
      - LOG_LEVEL=info
    options:
      disableNameSuffixHash: false
```

```
# Trình tạo Secret
secretGenerator:
- name: app-secrets
  type: Opaque
  literals:
    - username=admin
  envs:
    - .env.production
  options:
    disableNameSuffixHash: true
```

C.2 Các mẫu lệnh Kustomize phổ biến

```
# Kết xuất và xem xét (luôn làm điều này trước khi áp dụng)
kubectl kustomize ./overlays/production | less

# Khác biệt với cụm trực tiếp
kubectl diff -k ./overlays/production

# Áp dụng lớp phủ
kubectl apply -k ./overlays/production

# Áp dụng với tính năng chạy thử phía máy chủ (xác thực dựa trên lược đồ API)
kubectl apply -k ./overlays/production --dry-run=server

# Xóa tất cả tài nguyên trong tùy chỉnh
kubectl delete -k ./overlays/production

# Lưu đầu ra được kết xuất cho tạo phẩm CI/CD
kubectl kustomize ./overlays/production > rendered-$(date +%Y%m%d-%H%M%S).yaml

# Áp dụng kết quả được hiển thị (hữu ích khi bạn muốn giữ lại tạo phẩm)
kubectl kustomize ./overlays/production | kubectl apply -f -

# Xem buổi giới thiệu sau khi áp dụng
kubectl apply -k ./overlays/production && \
  kubectl rollout status deployment/my-app -n production --timeout=300s
```

Phụ lục D - Tham khảo nhanh về lấp đầy khoảng trống CKA

Phụ lục này cung cấp tài liệu tham khảo lệnh cô đọng đặc biệt cho chín lĩnh vực chủ đề được xác định là thiếu hoặc mỏng trong tài liệu gốc.

Tham khảo nhanh cài đặt cụm D.1

```
# Vô hiệu hóa trao đổi
swapoff -a && sed -i ' / swap / s/^/#/' /etc/fstab

# Tải mô-đun hạt nhân
modprobe overlay && modprobe br_netfilter

# Cài đặt containerd + đặt SystemdCgroup=true
apt install containerd && containerd config default > /etc/containerd/config.toml
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# Cài đặt kubeadm/kubelet/kubect1
apt install -y kubelet kubeadm kubect1 && apt-mark hold kubelet kubeadm kubect1

# Ban đầu control plane
kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=<IP>

# Định cấu hình kubect1
mkdir -p ~/.kube && cp /etc/kubernetes/admin.conf ~/.kube/config

# Cài đặt CNI (Calico)
kubect1 apply -f https://docs.projectcalico.org/manifests/calico.yaml

# Tham gia worker
kubeadm join <CP_IP>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>

# Tạo lệnh tham gia mới (nếu mã thông báo hết hạn)
kubeadm token create --print-join-command

# Kiểm tra chứng chỉ
kubeadm certs check-expiration && kubeadm certs renew all
```

D.2 HA Control Plane Tham khảo nhanh

```
# Khởi tạo HA control plane
kubeadm init \
  --control-plane-endpoint "LB_DNS:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16

# Tham gia thêm control plane nodes
kubeadm join LB_DNS:6443 --token <t> \
  --discovery-token-ca-cert-hash sha256:<h> \
  --control-plane --certificate-key <k>

# Kiểm tra danh sách thành viên etcd
kubect1 exec -it etcd-cp1 -n kube-system -- sh -c \
  'ETCDCTL_API=3 etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key'
```

D.3 CRI/CNI/CSI Tham khảo nhanh

```
# CRI - kiểm tra phiên bản thời gian chạy trên node
kubectl get node <n> -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'

# crictl - tương tác trực tiếp với thời gian chạy
sudo crictl ps                # Running containers
sudo crictl pods              # Tất cả pods
sudo crictl logs <container-id> # Nhật ký container
sudo crictl pull nginx:1.25    # Kéo image

# CNI - kiểm tra plugin pods
kubectl get pods -n kube-system | grep -E "calico|flannel|cilium"

# CSI - kiểm tra trình điều khiển
kubectl get csidriver
kubectl get csistoragecapacities -A
kubectl get csinode
```

D.4 CRD và Tham khảo nhanh về Operator

```
# Danh sách CRDs
kubectl get crd
kubectl api-resources --api-group=mycompany.com

# Cài đặt CRD
kubectl apply -f crd.yaml

# Tạo phiên bản CR
kubectl apply -f custom-resource.yaml

# Liệt kê các trường hợp
kubectl get <plural-name> -A
kubectl describe <kind> <name>

# Xóa CRD (XÓA TẤT CẢ CÁC TRƯỜNG HỢP)
kubectl delete crd <name>
```

D.5 Gateway API Tham khảo nhanh

```
# Cài đặt Gateway API CRDs
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-install.yaml

# Kiểm tra tài nguyên
kubectl get gatewayclass
kubectl get gateway -A
kubectl get httproute -A

# Mô tả cách khắc phục sự cố
kubectl describe gateway my-gateway -n production
kubectl describe httproute my-route -n production
```

D.6 HPA Tham khảo nhanh

```
# Tạo HPA bắt buộc
kubectl autoscale deployment <name> --min=2 --max=10 --cpu-percent=50

# Xem HPA
kubectl get hpa
kubectl describe hpa <name>

# Xóa HPA
kubectl delete hpa <name>

# Tạo tải (kiểm tra HPA)
kubectl run load --image=busybox --rm -it --restart=Never -- \
  sh -c "while true; do wget -q -O- http://<svc>; done"

# Kiểm tra số liệu
kubectl top pod --sort-by=cpu
```

Tham khảo nhanh đầu dò D.7 và PDB

```
# Gỡ lỗi đầu dò
kubectl describe pod <n> | grep -A 10 "Liveness\|Readiness\|Startup"
kubectl get pod <n> -o jsonpath='{.status.containerStatuses[0].restartCount}'
kubectl logs <n> --previous

# Các lệnh PDB
kubectl create -f pdb.yaml
kubectl get pdb
kubectl describe pdb <name>
kubectl delete pdb <name>
```

Tham khảo nhanh về lưu trữ D.8

```
# StorageClass
kubectl get sc
kubectl describe sc <name>

# PV và PVC
kubectl get pv
kubectl get pvc -A
kubectl describe pvc <name>

# Mở rộng PVC
kubectl patch pvc <name> -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'

# Giải phóng PV bị kẹt (xóa yêu cầuRef)
kubectl patch pv <name> -p '{"spec":{"claimRef": null}}'

# Kiểm tra chính sách thu hồi
kubectl get pv -o custom-columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy
```

D.9 ConfigMap và Secret Tham khảo nhanh

```
# ConfigMap
kubectl create configmap <n> --from-literal=key=val --from-file=file.conf
kubectl get configmap <n> -o yaml
kubectl edit configmap <n>
kubectl delete configmap <n>

# Secret
kubectl create secret generic <n> --from-literal=user=admin --from-literal=pass=secret
kubectl create secret tls <n> --cert=tls.crt --key=tls.key
kubectl get secret <n> -o yaml
kubectl get secret <n> -o jsonpath='{.data.password}' | base64 --decode
kubectl delete secret <n>
```

Tài liệu tham khảo và đọc thêm

- Tài liệu chính thức của Kubernetes: <https://kubernetes.io/docs/>
- Chương trình giảng dạy CKA (CNCF): <https://github.com/cncf/curriculum>
- Thay đổi chương trình CKA (Tháng 2 năm 2025):
<https://training.linuxfoundation.org/certified-kubernetes-administrator-cka-program-changes/>
- Trang thi CKA nền tảng Linux: <https://training.linuxfoundation.org/certification/certified-kubernetes-administrator-cka/>
- Tài liệu chính thức của CoreDNS: <https://coredns.io/manual/toc/>
- Tài liệu chính thức của Kustomize: <https://kubectl.docs.kubernetes.io/references/kustomize/>
- Kustomize GitHub: <https://github.com/kubernetes-sigs/kustomize>
- Hình ảnh netshoot (GitHub): <https://github.com/nicolaka/netshoot>
- Bảng cheat kubectrl (Chính thức): <https://kubernetes.io/docs/reference/kubectrl/cheatsheet/>
- Đặc điểm kỹ thuật Kubernetes DNS: <https://github.com/kubernetes/dns/blob/master/docs/specification.md>
- Hướng dẫn về container tạm thời: <https://kubernetes.io/docs/concepts/workloads/pods/ephemeral-containers/>
- Tài liệu etcdctl: <https://etcd.io/docs/v3.5/op-guide/maintenance/>
- Hướng dẫn nâng cấp kubeadm: <https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>
- Hướng dẫn RBAC: <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- Hướng dẫn NetworkPolicy: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Gateway API Tài liệu: <https://gateway-api.sigs.k8s.io/>
- Tài liệu HPA: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- Tài liệu StorageClass: <https://kubernetes.io/docs/concepts/storage/storage-classes/>
- Tài liệu CRD: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>
- Hướng dẫn kubeadm HA: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/>
- Giao diện môi trường chạy container: <https://kubernetes.io/docs/concepts/architecture/cni/>
- Thông số kỹ thuật CNI: <https://github.com/containernetworking/cni>

Tài liệu này là hướng dẫn tham khảo kỹ thuật toàn diện bao gồm chương trình giảng dạy kỳ thi CKA 2026 hoàn chỉnh, tương tác Kubernetes container, kiểm tra tạm thời containers, chẩn đoán CoreDNS, loại bỏ và sửa đổi container, kubectrl Kustomize, cài đặt cụm từ đầu, HA Thiết lập control plane, giao diện mở rộng (CNI/CSI/CRI), CRDs và Operators, Gateway API, HPA tự động điều chỉnh quy mô, đầu dò tự phục hồi và PodDisruptionBudget, cung cấp bộ nhớ động với StorageClasses và một bản hoàn chỉnh Tham chiếu ConfigMap và Secrets.

Cập nhật kiến thức CKA cho Kubernetes 1.35

Mốc kiểm tra: 17/09/2026. Trang CKA và FAQ của Linux Foundation hiện công bố môi trường thi dùng **Kubernetes v1.35**. Repo này bám phiên bản thi, không tự chuyển sang bản Kubernetes mới nhất. Trước ngày thi, kiểm tra lại **FAQ chính thức**.

Phạm vi ôn tập và cách dùng bộ tài liệu

Lĩnh vực CKA	Tỷ trọng	Nội dung cần thực hành
Kiến trúc, cài đặt và cấu hình cụm	25%	RBAC, kubeadm, HA, nâng cấp, Helm, Kustomize, CRD và Operator
Workload và lập lịch	15%	Deployment, cấu hình ứng dụng, autoscaling, probe, tài nguyên và lập lịch
Service và mạng	20%	Kết nối Pod, Service, Gateway API, Ingress, NetworkPolicy, CoreDNS
Lưu trữ	10%	StorageClass, PV/PVC, chế độ truy cập và cấp phát volume
Xử lý sự cố	30%	Cụm và node, thành phần hệ thống, tài nguyên, log, mạng

Nguồn: [đề cương CKA của Linux Foundation](#). Một tính năng mới không mặc nhiên là nội dung chắc chắn xuất hiện trong đề thi. Các bài trong repo là bài tự luyện, không phải đề thi thật.

Đọc tài liệu Kubernetes đầy đủ để học nền tảng; dùng các hướng dẫn Gateway API, Ingress và CoreDNS để luyện thao tác. Bản tra cứu mạng Linux có nhiều lệnh ngoài phạm vi CKA: ưu tiên `ip`, `ss`, `curl`, `dig`, `nslookup`, `tcpdump` và cách đọc kết quả. Công cụ có sẵn phụ thuộc máy thực hành; trong kỳ thi, tuân theo chỉ dẫn của từng bài.

Node Linux: kiểm tra cgroup và runtime trước khi nâng cấp

Kubelet 1.35 mặc định đặt `failCgroupV1: true`. Node còn dùng cgroup v1 có thể không khởi động được kubelet. Chuẩn bị lab bằng cgroup v2; `SystemdCgroup = true` chỉ chọn trình điều khiển, không chuyển hệ điều hành từ cgroup v1 sang v2.

```
# Chạy trên từng node; kết quả mong đợi là cgroup2fs
stat -fc %T /sys/fs/cgroup

# Xác định phiên bản và trạng thái runtime
containerd --version
sudo crictl info
sudo systemctl status containerd kubelet --no-pager
sudo journalctl -u kubelet -n 100 --no-pager
```

`failCgroupV1: false` là cách tương thích tạm thời, không phải cấu hình khuyến nghị cho lab mới. Xem [KubeletConfiguration 1.35](#) và [cgroup v2](#).

Kubernetes 1.35 là nhánh cuối còn hỗ trợ containerd 1.x. Khi dùng containerd 2.x, tạo cấu hình bằng chính binary đã cài; đường dẫn plugin CRI khác 1.x. Hai phía kubelet/runtime phải thống nhất trình điều khiển cgroup. Lab này tắt swap để dùng cấu hình kubeadm mặc định; Kubernetes vẫn có hỗ trợ swap có điều kiện, nên không hiểu thành “Kubernetes tuyệt đối không hỗ trợ swap”. Nguồn: [container runtime](#) và [swap trên Linux](#).

Nâng cấp kubeadm từ 1.34 lên 1.35

Không bỏ qua phiên bản minor. Sao lưu etcd, kiểm tra cgroup/CNI/runtime, rồi nâng cấp từng control plane trước các worker. Chọn bản vá **1.35.x** có trong kho; nếu đề bài chỉ định phiên bản thì dùng đúng phiên bản đó. Không coi 1.35.0 là bản vá mới nhất.

Trên từng node cần nâng cấp, đổi kho APT sang 1.35 và xác định gói đích:

```
sudo install -d -m 755 /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key |
  sudo gpg --dearmor --yes -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.35/deb/ /' |
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
```

```
apt-cache madison kubeadm
```

```
# Ví dụ lấy bản 1.35.x cao nhất hiện có; có thể gán phiên bản đề bài yêu cầu.
K8S_PKG_VERSION=$(apt-cache madison kubeadm | awk '$3 ~ /^1\.35\./ {print $3}' | sort -V | tail -1)
test -n "$K8S_PKG_VERSION" || exit 1
K8S_TARGET="v${K8S_PKG_VERSION%-*}"
printf 'Gói: %s; Kubernetes: %s\n' "$K8S_PKG_VERSION" "$K8S_TARGET"

sudo apt-mark unhold kubeadm
sudo apt-get install -y "kubeadm=$K8S_PKG_VERSION"
sudo apt-mark hold kubeadm
```

Trên control plane đầu tiên:

```
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply "$K8S_TARGET"
```

Trên mỗi control plane còn lại hoặc worker, sau khi nâng cấp gói kubeadm:

```
sudo kubeadm upgrade node
```

Với **từng node**, dùng máy có kubeconfig quản trị để drain trước khi nâng kubelet. Thay tên bên dưới bằng tên node thật.

--delete-emptydir-data chỉ dùng khi đã chấp nhận mất dữ liệu emptyDir; nếu PDB chặn drain, kiểm tra nguyên nhân trước.

```
# Máy quản trị: mỗi lần chỉ bảo trì một node
NODE_NAME=worker-1
kubectl drain "$NODE_NAME" --ignore-daemonsets

# Trên node đang bảo trì: dùng cùng K8S_PKG_VERSION đã chọn ở trên
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y "kubelet=$K8S_PKG_VERSION" "kubectl=$K8S_PKG_VERSION"
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# Máy quản trị: xác minh node Ready rồi mở lại lập lịch
kubectl get nodes -o wide
kubectl uncordon "$NODE_NAME"
kubectl get pods -A
```

Nguồn: [quy trình nâng cấp kubeadm 1.35](#) và [chính sách chênh lệch phiên bản](#). Kubelet không được mới hơn kube-apiserver; kubectl được hỗ trợ trong phạm vi lệch một minor so với kube-apiserver, với giới hạn chặt hơn nếu các API server khác phiên bản.

Thay đổi CPU và bộ nhớ của Pod tại chỗ

In-place Pod resize ổn định trong 1.35. Có thể thay requests/limits CPU, bộ nhớ của container qua subresource /resize, không cần xoá Pod. resizePolicy quyết định việc phải khởi động lại container; thay đổi không bảo đảm luôn được áp dụng ngay nếu node thiếu tài nguyên. Không được đổi lớp QoS của Pod.

```
kubectl create namespace resize-lab
kubectl apply -n resize-lab -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: resize-demo
spec:
  containers:
  - name: app
    image: nginx:stable
    resources:
      requests:
        cpu: 100m
        memory: 64Mi
      limits:
        cpu: 500m
        memory: 128Mi
    resizePolicy:
      - resourceName: cpu
```



```

    restartPolicy: NotRequired
  - resourceName: memory
    restartPolicy: RestartContainer
EOF
kubectl wait -n resize-lab --for=condition=Ready pod/resize-demo --timeout=120s
kubectl patch pod resize-demo -n resize-lab --subresource=resize --type=merge \
  -p '{"spec":{"containers":[{"name":"app","resources":{"requests":{"cpu":"200m","memory":"64Mi"},"limits":{"cpu":"750m","memory":"128Mi"}}}]}}'
kubectl get pod resize-demo -n resize-lab -o yaml

```

Quan sát `status.conditions` (`PodResizePending`, `PodResizeInProgress`) và tài nguyên thực tế trong `status.containerStatuses`. Việc đổi image và việc resize là hai thao tác khác nhau. Danh sách container, tên container, `command`, `args` của Pod hiện hữu vẫn có các ràng buộc bất biến. Đổi template của Deployment vẫn tạo rollout; VPA là thành phần bổ sung, không tự xuất hiện khi nâng Kubernetes.

Nguồn: [hướng dẫn resize dành cho 1.35](#).

Service, EndpointSlice và kube-proxy

Khi Service không có backend, kiểm tra selector, nhãn Pod, readiness và EndpointSlice:

```

kubectl get svc web -n default -o yaml
kubectl get pods -n default --show-labels -o wide
kubectl get endpointslices -n default -l kubernetes.io/service-name=web -o yaml

```

API Endpoints đã bị đánh dấu deprecated từ 1.33; vẫn có thể gặp trong bài thực hành cũ nhưng nên đọc EndpointSlice. `trafficDistribution: PreferSameZone` và `PreferSameNode` ổn định trong 1.35. Đây là **ưu tiên** vị trí backend; khác với ràng buộc `internalTrafficPolicy: Local` hoặc `externalTrafficPolicy: Local`, có thể làm rơi lưu lượng khi không có endpoint cục bộ.

Nguồn: [Service 1.35](#) và [định tuyến Service](#).

Chế độ IPVS của kube-proxy bị deprecated trong 1.35 nhưng chưa bị xoá. Khi gặp cụm dùng IPVS vẫn phải biết chẩn đoán; với lab mới, chọn iptables hoặc nftables phù hợp kernel và CNI. Không đổi chế độ proxy trong kỳ thi nếu đề không yêu cầu.

Nguồn: [ghi chú Kubernetes 1.35](#).

Gateway API và Ingress: phân biệt API với controller

Gateway API là tập CRD và cần controller triển khai; nó không tự được cài khi nâng Kubernetes. Trước khi áp dụng ví dụ, kiểm tra tài nguyên và phiên bản mà cụm hỗ trợ:

```

kubectl api-resources --api-group=gateway.networking.k8s.io
kubectl get gatewayclass
kubectl get gateway,httproute -A
kubectl describe gateway web-gateway -n default

```

Ưu tiên luyện GatewayClass, Gateway, HTTPRoute, listener, parentRefs, backendRefs, allowedRoutes và ReferenceGrant. Kiểm tra các điều kiện Accepted, Programmed, ResolvedRefs đúng trên loại tài nguyên tương ứng. Phiên bản Gateway API và controller phải theo ma trận tương thích, không suy ra từ Kubernetes 1.35. Xem [tài liệu Gateway API](#) và [ma trận Envoy Gateway](#).

Ingress NGINX của cộng đồng đã kết thúc bảo trì từ tháng 3/2026. Các ví dụ annotation `nginx.ingress.kubernetes.io/*` trong repo được giữ để học cách đọc cấu hình cũ và xử lý bài có sẵn controller. Không dùng manifest cài Ingress NGINX cũ làm khuyến nghị cho cụm mới. API `networking.k8s.io/v1` Ingress vẫn tồn tại; dự án controller ngừng bảo trì không có nghĩa API Ingress bị xoá. Nguồn: [thông báo ngừng Ingress NGINX](#) và [API Ingress](#).

Sao lưu và khôi phục etcd

Với etcd hiện đại, dùng `etcdctl` để chụp snapshot của cụm đang chạy; dùng `etcdutl` để kiểm tra và khôi phục snapshot. Khôi phục là thao tác ngoại tuyến, không cần truyền chứng chỉ truy cập API etcd cho `etcdutl`.

```

sudo etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \

```

```
--cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
--key=/etc/kubernetes/pki/etcd/healthcheck-client.key \
snapshot save /backup/snapshot.db
sudo etcdctl snapshot status /backup/snapshot.db -w table

# Ví dụ cho lab một thành viên: dừng thành phần liên quan trước khi phục hồi.
# Thư mục đích phải mới; điều chỉnh tên/địa chỉ theo cấu hình cụm thực tế.
sudo etcdctl snapshot restore /backup/snapshot.db \
--data-dir=/var/lib/etcd-restored \
--name=controlplane1 \
--initial-cluster=controlplane1=https://10.10.10.11:2380 \
--initial-advertise-peer-urls=https://10.10.10.11:2380 \
--bump-revision=1000000000 --mark-compacted
```

Revision bump và đánh dấu compact giúp watcher/informer của Kubernetes bỏ cache cũ sau phục hồi. Với nhiều thành viên, dùng cùng snapshot và cấu hình membership nhất quán trên từng thành viên. Sau đó cập nhật `hostPath` thư mục dữ liệu trong static Pod manifest, kiểm tra quyền truy cập, khởi động lại thành phần và xác minh API/Pod. Không áp dụng cấu hình một thành viên vào cụm nhiều thành viên đang chạy. Nguồn: [khôi phục etcd 3.6](#).

Giới hạn của lab cài đặt trong repo

Lab hiện có **1 HAProxy + 2 control plane + 2 worker** để luyện cài đặt và join. Với stacked etcd, hai thành viên cần cả hai để có quorum; mất một thành viên là mất quorum. Một HAProxy cũng là điểm lỗi đơn. Đây chưa phải kiến trúc HA chịu lỗi cho production.

Để luyện HA chịu mất một control plane, bố trí ít nhất ba control plane/etcd và đầu vào API có dự phòng. Số worker không thay đổi quorum etcd. Nguồn: [topology HA kubeadm](#).

Kiến thức mở rộng: đọc đúng mức độ ổn định

Image volume ở mức beta và bật mặc định trong 1.35, nhưng còn phụ thuộc runtime hỗ trợ (chẳng hạn containerd 2.1+). Pod-level resources vẫn là beta trong 1.35; không nhầm với resize tài nguyên từng container đã ổn định. Xem [volume](#) và [quản lý tài nguyên](#). Không dành thời gian cài tính năng thử nghiệm trong bài thi nếu không được yêu cầu.

Danh sách tự kiểm tra trước ngày thi

- Kiểm tra phiên bản thi và tài liệu được phép truy cập trên trang Linux Foundation.
- Luôn dùng đúng context, node và namespace đề bài chỉ định.
- Tự tạo và sửa manifest với `kubectl explain`, `--dry-run=client -o yaml` và `kubectl diff` khi phù hợp.
- Luyện một vòng nâng cấp, drain/uncordon, sao lưu/khôi phục và xử lý node NotReady trên lab riêng.
- Chẩn đoán mạng theo đường đi: Pod → Service/EndpointSlice → DNS → NetworkPolicy → Gateway/Ingress.
- Xác minh trạng thái thực tế sau thao tác; không chỉ kiểm tra lệnh trả về thành công.